

My Project

Latest Version

Generated by Doxygen 1.14.0

1 Data Structure Index	1
1.1 Data Structures	1
2 File Index	3
2.1 File List	3
3 Data Structure Documentation	5
3.1 bytes2 Union Reference	5
3.1.1 Detailed Description	5
3.1.2 Field Documentation	5
3.1.2.1 bd	5
3.1.2.2 ld	5
3.2 C_data Struct Reference	6
3.2.1 Detailed Description	7
3.2.2 Field Documentation	7
3.2.2.1 bank_ah	7
3.2.2.2 bc_ror	7
3.2.2.3 bkwi	7
3.2.2.4 bkwo	7
3.2.2.5 bv_full_load	8
3.2.2.6 bv_noload	8
3.2.2.7 bv_one_load	8
3.2.2.8 bv_ror	8
3.2.2.9 c_bat	8
3.2.2.10 c_load	8
3.2.2.11 c_mppt	8
3.2.2.12 c_pv	8
3.2.2.13 calc	9
3.2.2.14 day	9
3.2.2.15 day_end	9
3.2.2.16 day_start	9
3.2.2.17 day_update	9
3.2.2.18 dupdate	9
3.2.2.19 dynamic_ah	9
3.2.2.20 dynamic_ah_adj	9
3.2.2.21 dynamic_ah_adj_daily	10
3.2.2.22 dynamic_ah_daily	10
3.2.2.23 esr	10
3.2.2.24 hist	10
3.2.2.25 invkw	10
3.2.2.26 load_i1	10
3.2.2.27 load_i2	10
3.2.2.28 loadah	10

3.2.2.29 loadkw	11
3.2.2.30 p_bat	11
3.2.2.31 p_inverter	11
3.2.2.32 p_load	11
3.2.2.33 p_mppt	11
3.2.2.34 p_pv	11
3.2.2.35 pv_ah	11
3.2.2.36 pvkw	11
3.2.2.37 runtime	12
3.2.2.38 soc	12
3.2.2.39 start_power	12
3.2.2.40 t_comp	12
3.2.2.41 update	12
3.2.2.42 updates	12
3.2.2.43 v_bat	12
3.2.2.44 v_bbat	12
3.2.2.45 v_cbus	13
3.2.2.46 v_cc	13
3.2.2.47 v_cmode	13
3.2.2.48 v_inverter	13
3.2.2.49 v_pv	13
3.2.2.50 v_sensor	13
3.2.2.51 v_temp	13
3.3 CRC_OBJ Struct Reference	14
3.3.1 Detailed Description	14
3.3.2 Field Documentation	14
3.3.2.1 dataWidth	14
3.3.2.2 polyWidth	14
3.3.2.3 seedDirection	14
3.4 D_data Struct Reference	15
3.4.1 Detailed Description	15
3.4.2 Field Documentation	15
3.4.2.1 cont	15
3.4.2.2 dac0	15
3.4.2.3 dac1	15
3.4.2.4 last_info	15
3.4.2.5 lcd	15
3.4.2.6 vterm	16
3.5 dac_buf_type Union Reference	16
3.5.1 Detailed Description	16
3.5.2 Field Documentation	16
3.5.2.1 bd	16

3.5.2.2 ld	17
3.5.2.3 map	17
3.6 H_data Struct Reference	17
3.6.1 Detailed Description	17
3.6.2 Field Documentation	17
3.6.2.1 calib_p	17
3.6.2.2 enter_p	17
3.6.2.3 h_state	18
3.6.2.4 hid_display	18
3.6.2.5 select_p	18
3.6.2.6 sequence	18
3.6.2.7 wait_calib	18
3.6.2.8 wait_enter	18
3.6.2.9 wait_select	18
3.7 hist_type Struct Reference	19
3.7.1 Detailed Description	19
3.7.2 Field Documentation	19
3.7.2.1 ah	19
3.7.2.2 bound_factor	19
3.7.2.3 bound_rate	20
3.7.2.4 bsoc	20
3.7.2.5 cef	20
3.7.2.6 cef_calc	20
3.7.2.7 cef_save	20
3.7.2.8 drate	20
3.7.2.9 esr	20
3.7.2.10 esrp	20
3.7.2.11 h	21
3.7.2.12 pclock	21
3.7.2.13 peukert	21
3.7.2.14 peukert_adj	21
3.7.2.15 pv_eff	21
3.7.2.16 rate	21
3.7.2.17 rest_factor	21
3.7.2.18 rest_rate	21
3.7.2.19 samplei	22
3.7.2.20 sampleo	22
3.7.2.21 tot_eff	22
3.7.2.22 ttg_t	22
3.7.2.23 udod	22
3.7.2.24 updates	22
3.7.2.25 version	22

3.8 R_data Struct Reference	23
3.8.1 Detailed Description	23
3.8.2 Field Documentation	23
3.8.2.1 c_scale_cal	23
3.8.2.2 c_zero_cal	24
3.8.2.3 checkmark	24
3.8.2.4 crc	24
3.8.2.5 done	24
3.8.2.6 H	24
3.8.2.7 hist_save	24
3.8.2.8 hist_update	24
3.8.2.9 max5322_cmd	24
3.8.2.10 n_offset	25
3.8.2.11 n_scalar	25
3.8.2.12 raw_adc	25
3.8.2.13 raw_dac	25
3.8.2.14 scan_index	25
3.8.2.15 scan_select	25
3.9 rbutton_type Struct Reference	25
3.9.1 Detailed Description	26
3.9.2 Field Documentation	26
3.9.2.1 count	26
3.9.2.2 sw	26
3.10 ringBufS_t Struct Reference	26
3.10.1 Detailed Description	26
3.10.2 Field Documentation	26
3.10.2.1 buf	26
3.10.2.2 count	27
3.10.2.3 head	27
3.10.2.4 tail	27
3.11 S_data Struct Reference	27
3.11.1 Detailed Description	28
3.11.2 Field Documentation	28
3.11.2.1 button	28
3.11.2.2 sw_bitmap	28
3.12 spi1_configuration_t Struct Reference	28
3.12.1 Detailed Description	28
3.12.2 Field Documentation	29
3.12.2.1 baud	29
3.12.2.2 con0	29
3.12.2.3 con1	29
3.12.2.4 con2	29

3.12.2.5 operation	29
3.13 spi_link_type Struct Reference	29
3.13.1 Detailed Description	30
3.13.2 Field Documentation	30
3.13.2.1 config	30
3.13.2.2 delay	30
3.13.2.3 int_count	30
3.13.2.4 LCD_DATA	30
3.13.2.5 LCD_TIMER	30
3.13.2.6 SPI_AUX	30
3.13.2.7 SPI_LCD	31
3.13.2.8 txbuf	31
3.14 terminal_type Struct Reference	31
3.14.1 Detailed Description	31
3.14.2 Field Documentation	31
3.14.2.1 ceid	31
3.14.2.2 cmdlen	32
3.14.2.3 help_temp	32
3.14.2.4 host_display_ack	32
3.14.2.5 info	32
3.14.2.6 log_num	32
3.14.2.7 log_seq	32
3.14.2.8 mcode	32
3.14.2.9 msgid	32
3.14.2.10 mparm	33
3.14.2.11 TID	33
3.15 uart1_status_t Union Reference	33
3.15.1 Detailed Description	33
3.15.2 Field Documentation	33
3.15.2.1 ferr	33
3.15.2.2 oerr	34
3.15.2.3 perr	34
3.15.2.4 reserved	34
3.15.2.5 status	34
3.16 uart2_status_t Union Reference	34
3.16.1 Detailed Description	34
3.16.2 Field Documentation	35
3.16.2.1 ferr	35
3.16.2.2 oerr	35
3.16.2.3 perr	35
3.16.2.4 reserved	35
3.16.2.5 status	35

3.17 V_data Struct Reference	35
3.17.1 Detailed Description	37
3.17.2 Field Documentation	37
3.17.2.1 abort	37
3.17.2.2 ac_off	37
3.17.2.3 ac_on	37
3.17.2.4 ac_time	37
3.17.2.5 adc_count	37
3.17.2.6 alarm	37
3.17.2.7 all_errors	38
3.17.2.8 blight	38
3.17.2.9 blight_off	38
3.17.2.10 boost_ticks	38
3.17.2.11 buf	38
3.17.2.12 calib	38
3.17.2.13 cc_active	38
3.17.2.14 cc_state	38
3.17.2.15 checksum_error	39
3.17.2.16 debug	39
3.17.2.17 enter	39
3.17.2.18 error	39
3.17.2.19 fixup	39
3.17.2.20 flipper	39
3.17.2.21 float_ticks	39
3.17.2.22 get_time_text	39
3.17.2.23 help	40
3.17.2.24 help_id	40
3.17.2.25 in_boost	40
3.17.2.26 in_float	40
3.17.2.27 info	40
3.17.2.28 lowint_count	40
3.17.2.29 mode_pwm	40
3.17.2.30 msg_error	40
3.17.2.31 msg_ret	41
3.17.2.32 r_checksum	41
3.17.2.33 rbuf	41
3.17.2.34 response	41
3.17.2.35 rx_count	41
3.17.2.36 screen	41
3.17.2.37 sensor_set	41
3.17.2.38 sequences	41
3.17.2.39 set_sequ	42

3.17.2.40 spi_count	42
3.17.2.41 stack	42
3.17.2.42 switch_count	42
3.17.2.43 sys_info	42
3.17.2.44 system_stable	42
3.17.2.45 t_checksum	42
3.17.2.46 testing	42
3.17.2.47 ticker	43
3.17.2.48 ticks	43
3.17.2.49 time_info	43
3.17.2.50 timerint_count	43
3.17.2.51 tx_count	43
3.17.2.52 ui_state	43
3.17.2.53 ui_sw	43
3.17.2.54 wdt_ticks	44
3.18 V_help Struct Reference	44
3.18.1 Detailed Description	44
3.18.2 Field Documentation	44
3.18.2.1 display	44
3.18.2.2 message	44
4 File Documentation	45
4.1 bsoc.c	45
4.2 bsoc.h	53
4.3 daq.c	54
4.4 daq.h	60
4.5 dio.c	62
4.6 dio.h	64
4.7 eadog.c	65
4.8 eadog.h	71
4.9 hid.c	72
4.10 hid.h	73
4.11 main.c	74
4.12 mbmc.c	84
4.13 mbmc.h	88
4.14 adcc.c	89
4.15 adcc.h	93
4.16 clkref.c	95
4.17 clkref.h	95
4.18 crc.c	96
4.19 crc.h	99
4.20 device_config.c	100

4.21 device_config.h	101
4.22 dma1.c	101
4.23 dma1.h	104
4.24 dma2.c	105
4.25 dma2.h	106
4.26 ext_int.c	107
4.27 ext_int.h	108
4.28 fvr.c	109
4.29 fvr.h	110
4.30 interrupt_manager.c	110
4.31 interrupt_manager.h	111
4.32 mcc.c	112
4.33 mcc.h	113
4.34 memory.c	114
4.35 memory.h	116
4.36 pin_manager.c	117
4.37 pin_manager.h	118
4.38 pwm8.c	128
4.39 pwm8.h	128
4.40 spi1.c	129
4.41 spi1.h	131
4.42 tmr2.c	131
4.43 tmr2.h	133
4.44 tmr3.c	137
4.45 tmr3.h	139
4.46 tmr5.c	140
4.47 tmr5.h	142
4.48 tmr6.c	143
4.49 tmr6.h	145
4.50 uart1.c	149
4.51 uart1.h	152
4.52 uart2.c	153
4.53 uart2.h	157
4.54 mconfig.c	158
4.55 mconfig.h	161
4.56 msg_text.h	162
4.57 mydisplay.h	162
4.58 ringbufs.c	163
4.59 ringbufs.h	164
4.60 tests.h	165
4.61 timers.c	166
4.62 timers.h	166

4.63 vconfig.h	167
Index	171

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

bytes2	5
C_data	6
CRC_OBJ	14
D_data	15
dac_buf_type	16
H_data	17
hist_type	19
R_data	23
rbutton_type	25
ringBufS_t	26
S_data	27
spi1_configuration_t	28
spi_link_type	29
terminal_type	31
uart1_status_t	33
uart2_status_t	34
V_data	35
V_help	44

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

bsoc.c	45
bsoc.h	53
daq.c	54
daq.h	60
dio.c	62
dio.h	64
eadog.c	65
eadog.h	71
hid.c	72
hid.h	73
main.c	74
mbmc.c	84
mbmc.h	88
mconfig.c	158
mconfig.h	161
msg_text.h	162
mydisplay.h	162
ringbufs.c	163
ringbufs.h	164
tests.h	165
timers.c	166
timers.h	166
vconfig.h	167
mcc_generated_files/adcc.c	89
mcc_generated_files/adcc.h	93
mcc_generated_files/clkref.c	95
mcc_generated_files/clkref.h	95
mcc_generated_files/crc.c	96
mcc_generated_files/crc.h	99
mcc_generated_files/device_config.c	100
mcc_generated_files/device_config.h	101
mcc_generated_files/dma1.c	101
mcc_generated_files/dma1.h	104
mcc_generated_files/dma2.c	105
mcc_generated_files/dma2.h	106

mcc_generated_files/ext_int.c	107
mcc_generated_files/ext_int.h	108
mcc_generated_files/fvr.c	109
mcc_generated_files/fvr.h	110
mcc_generated_files/interrupt_manager.c	110
mcc_generated_files/interrupt_manager.h	111
mcc_generated_files/mcc.c	112
mcc_generated_files/mcc.h	113
mcc_generated_files/memory.c	114
mcc_generated_files/memory.h	116
mcc_generated_files/pin_manager.c	117
mcc_generated_files/pin_manager.h	118
mcc_generated_files/pwm8.c	128
mcc_generated_files/pwm8.h	128
mcc_generated_files/spi1.c	129
mcc_generated_files/spi1.h	131
mcc_generated_files/tmr2.c	131
mcc_generated_files/tmr2.h	133
mcc_generated_files/tmr3.c	137
mcc_generated_files/tmr3.h	139
mcc_generated_files/tmr5.c	140
mcc_generated_files/tmr5.h	142
mcc_generated_files/tmr6.c	143
mcc_generated_files/tmr6.h	145
mcc_generated_files/uart1.c	149
mcc_generated_files/uart1.h	152
mcc_generated_files/uart2.c	153
mcc_generated_files/uart2.h	157

Chapter 3

Data Structure Documentation

3.1 bytes2 Union Reference

Data Fields

- uint16_t [ld](#)
- uint8_t [bd](#) [2]

3.1.1 Detailed Description

Definition at line [18](#) of file [daq.c](#).

3.1.2 Field Documentation

3.1.2.1 bd

```
uint8_t bd[2]
```

Definition at line [20](#) of file [daq.c](#).

3.1.2.2 ld

```
uint16_t ld
```

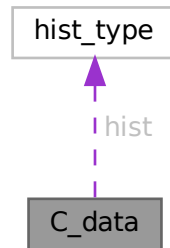
Definition at line [19](#) of file [daq.c](#).

The documentation for this union was generated from the following file:

- [daq.c](#)

3.2 C_data Struct Reference

Collaboration diagram for C_data:



Data Fields

- float [calc](#) [ADC_BUFFER_SIZE]
- float [c_load](#)
- float [c_bat](#)
- float [c_pv](#)
- float [c_mppt](#)
- float [v_cc](#)
- float [v_pv](#)
- float [v_bat](#)
- float [v_cbus](#)
- float [v_bbat](#)
- float [v_temp](#)
- float [v_inverter](#)
- float [v_sensor](#)
- float [v_cmode](#)
- float [bv_ror](#)
- float [bc_ror](#)
- float [p_load](#)
- float [p_inverter](#)
- float [p_pv](#)
- float [p_bat](#)
- float [p_mppt](#)
- float [start_power](#)
- float [t_comp](#)
- float [esr](#)
- float [bank_ah](#)
- float [dynamic_ah](#)
- float [pv_ah](#)
- float [loadah](#)
- float [dynamic_ah_adj](#)
- float [dynamic_ah_daily](#)
- float [dynamic_ah_adj_daily](#)
- float [bkwi](#)

- float [bkwo](#)
- float [pvkw](#)
- float [invkw](#)
- float [loadkw](#)
- uint16_t [runtime](#)
- uint16_t [soc](#)
- bool [update](#)
- [hist_type](#) [hist](#) [1]
- float [load_i1](#)
- float [load_i2](#)
- float [bv_noload](#)
- float [bv_one_load](#)
- float [bv_full_load](#)
- bool [day](#)
- bool [dupdate](#)
- time_t [day_start](#)
- time_t [day_end](#)
- time_t [day_update](#)
- time_t [updates](#)

3.2.1 Detailed Description

Definition at line 78 of file [mbmc.h](#).

3.2.2 Field Documentation

3.2.2.1 bank_ah

```
float bank_ah
```

Definition at line 83 of file [mbmc.h](#).

3.2.2.2 bc_ror

```
float bc_ror
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.3 bkwi

```
float bkwi
```

Definition at line 84 of file [mbmc.h](#).

3.2.2.4 bkwo

```
float bkwo
```

Definition at line 84 of file [mbmc.h](#).

3.2.2.5 `bv_full_load`

```
float bv_full_load
```

Definition at line 88 of file [mbmc.h](#).

3.2.2.6 `bv_noload`

```
float bv_noload
```

Definition at line 88 of file [mbmc.h](#).

3.2.2.7 `bv_one_load`

```
float bv_one_load
```

Definition at line 88 of file [mbmc.h](#).

3.2.2.8 `bv_ror`

```
float bv_ror
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.9 `c_bat`

```
float c_bat
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.10 `c_load`

```
float c_load
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.11 `c_mppt`

```
float c_mppt
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.12 `c_pv`

```
float c_pv
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.13 calc

```
float calc[ADC_BUFFER_SIZE]
```

Definition at line 79 of file [mbmc.h](#).

3.2.2.14 day

```
bool day
```

Definition at line 89 of file [mbmc.h](#).

3.2.2.15 day_end

```
time_t day_end
```

Definition at line 90 of file [mbmc.h](#).

3.2.2.16 day_start

```
time_t day_start
```

Definition at line 90 of file [mbmc.h](#).

3.2.2.17 day_update

```
time_t day_update
```

Definition at line 90 of file [mbmc.h](#).

3.2.2.18 dupdate

```
bool dupdate
```

Definition at line 89 of file [mbmc.h](#).

3.2.2.19 dynamic_ah

```
float dynamic_ah
```

Definition at line 83 of file [mbmc.h](#).

3.2.2.20 dynamic_ah_adj

```
float dynamic_ah_adj
```

Definition at line 83 of file [mbmc.h](#).

3.2.2.21 dynamic_ah_adj_daily

```
float dynamic_ah_adj_daily
```

Definition at line 83 of file [mbmc.h](#).

3.2.2.22 dynamic_ah_daily

```
float dynamic_ah_daily
```

Definition at line 83 of file [mbmc.h](#).

3.2.2.23 esr

```
float esr
```

Definition at line 82 of file [mbmc.h](#).

3.2.2.24 hist

```
hist_type hist[1]
```

Definition at line 87 of file [mbmc.h](#).

3.2.2.25 invkw

```
float invkw
```

Definition at line 84 of file [mbmc.h](#).

3.2.2.26 load_i1

```
float load_i1
```

Definition at line 88 of file [mbmc.h](#).

3.2.2.27 load_i2

```
float load_i2
```

Definition at line 88 of file [mbmc.h](#).

3.2.2.28 loadah

```
float loadah
```

Definition at line 83 of file [mbmc.h](#).

3.2.2.29 loadkw

```
float loadkw
```

Definition at line 84 of file [mbmc.h](#).

3.2.2.30 p_bat

```
float p_bat
```

Definition at line 81 of file [mbmc.h](#).

3.2.2.31 p_inverter

```
float p_inverter
```

Definition at line 81 of file [mbmc.h](#).

3.2.2.32 p_load

```
float p_load
```

Definition at line 81 of file [mbmc.h](#).

3.2.2.33 p_mppt

```
float p_mppt
```

Definition at line 81 of file [mbmc.h](#).

3.2.2.34 p_pv

```
float p_pv
```

Definition at line 81 of file [mbmc.h](#).

3.2.2.35 pv_ah

```
float pv_ah
```

Definition at line 83 of file [mbmc.h](#).

3.2.2.36 pvkw

```
float pvkw
```

Definition at line 84 of file [mbmc.h](#).

3.2.2.37 runtime

```
uint16_t runtime
```

Definition at line 85 of file [mbmc.h](#).

3.2.2.38 soc

```
uint16_t soc
```

Definition at line 85 of file [mbmc.h](#).

3.2.2.39 start_power

```
float start_power
```

Definition at line 81 of file [mbmc.h](#).

3.2.2.40 t_comp

```
float t_comp
```

Definition at line 82 of file [mbmc.h](#).

3.2.2.41 update

```
bool update
```

Definition at line 86 of file [mbmc.h](#).

3.2.2.42 updates

```
time_t updates
```

Definition at line 90 of file [mbmc.h](#).

3.2.2.43 v_bat

```
float v_bat
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.44 v_bbat

```
float v_bbat
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.45 v_cbus

```
float v_cbus
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.46 v_cc

```
float v_cc
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.47 v_cmode

```
float v_cmode
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.48 v_inverter

```
float v_inverter
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.49 v_pv

```
float v_pv
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.50 v_sensor

```
float v_sensor
```

Definition at line 80 of file [mbmc.h](#).

3.2.2.51 v_temp

```
float v_temp
```

Definition at line 80 of file [mbmc.h](#).

The documentation for this struct was generated from the following file:

- [mbmc.h](#)

3.3 CRC_OBJ Struct Reference

Data Fields

- `uint8_t` [dataWidth](#)
- `uint8_t` [polyWidth](#)
- `uint8_t` [seedDirection](#)

3.3.1 Detailed Description

CRC Generated Driver File

@Company Microchip Technology Inc.

@File Name [crc.c](#)

@Summary This is the generated driver implementation file for the CRC driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This source file provides APIs for CRC. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F57K42 Driver Version : 2.11 The generated drivers are tested against the following: Compiler : XC8 2.36 and above MPLAB : MPLAB X 6.00 Section: Included Files Section: Structures

Definition at line [58](#) of file [crc.c](#).

3.3.2 Field Documentation

3.3.2.1 dataWidth

```
uint8_t dataWidth
```

Definition at line [60](#) of file [crc.c](#).

3.3.2.2 polyWidth

```
uint8_t polyWidth
```

Definition at line [61](#) of file [crc.c](#).

3.3.2.3 seedDirection

```
uint8_t seedDirection
```

Definition at line [62](#) of file [crc.c](#).

The documentation for this struct was generated from the following file:

- `mcc_generated_files/crc.c`

3.4 D_data Struct Reference

Data Fields

- uint8_t [dac0](#): 8
- uint8_t [dac1](#): 4
- uint8_t [cont](#): 4
- char [lcd](#) [4][4][40]
- uint8_t [vterm](#): 1
- D_CODES [last_info](#)

3.4.1 Detailed Description

Definition at line [12](#) of file [daq.c](#).

3.4.2 Field Documentation

3.4.2.1 cont

```
uint8_t cont
```

Definition at line [15](#) of file [daq.c](#).

3.4.2.2 dac0

```
uint8_t dac0
```

Definition at line [13](#) of file [daq.c](#).

3.4.2.3 dac1

```
uint8_t dac1
```

Definition at line [14](#) of file [daq.c](#).

3.4.2.4 last_info

```
D_CODES last_info
```

Definition at line [45](#) of file [mydisplay.h](#).

3.4.2.5 lcd

```
char lcd[4][4][40]
```

Definition at line [43](#) of file [mydisplay.h](#).

3.4.2.6 vterm

```
uint8_t vterm
```

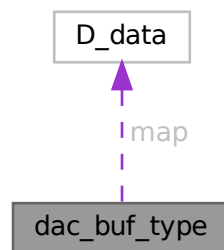
Definition at line 44 of file [mydisplay.h](#).

The documentation for this struct was generated from the following files:

- [daq.c](#)
- [mydisplay.h](#)

3.5 dac_buf_type Union Reference

Collaboration diagram for `dac_buf_type`:



Data Fields

- [uint16_t](#) `ld`
- [uint8_t](#) `bd` [2]
- [struct](#) [D_data](#) `map`

3.5.1 Detailed Description

Definition at line 23 of file [daq.c](#).

3.5.2 Field Documentation

3.5.2.1 bd

```
uint8_t bd[2]
```

Definition at line 25 of file [daq.c](#).

3.5.2.2 ld

```
uint16_t ld
```

Definition at line 24 of file [daq.c](#).

3.5.2.3 map

```
struct D_data map
```

Definition at line 26 of file [daq.c](#).

The documentation for this union was generated from the following file:

- [daq.c](#)

3.6 H_data Struct Reference

Data Fields

- H_CODES [hid_display](#)
- H_STATES [h_state](#)
- bool [wait_select](#)
- bool [select_p](#)
- bool [wait_enter](#)
- bool [enter_p](#)
- bool [wait_calib](#)
- bool [calib_p](#)
- uint8_t [sequence](#)

3.6.1 Detailed Description

Definition at line 54 of file [hid.h](#).

3.6.2 Field Documentation

3.6.2.1 calib_p

```
bool calib_p
```

Definition at line 57 of file [hid.h](#).

3.6.2.2 enter_p

```
bool enter_p
```

Definition at line 57 of file [hid.h](#).

3.6.2.3 h_state

H_STATES h_state

Definition at line 56 of file [hid.h](#).

3.6.2.4 hid_display

H_CODES hid_display

Definition at line 55 of file [hid.h](#).

3.6.2.5 select_p

bool select_p

Definition at line 57 of file [hid.h](#).

3.6.2.6 sequence

uint8_t sequence

Definition at line 58 of file [hid.h](#).

3.6.2.7 wait_calib

bool wait_calib

Definition at line 57 of file [hid.h](#).

3.6.2.8 wait_enter

bool wait_enter

Definition at line 57 of file [hid.h](#).

3.6.2.9 wait_select

bool wait_select

Definition at line 57 of file [hid.h](#).

The documentation for this struct was generated from the following file:

- [hid.h](#)

3.7 hist_type Struct Reference

Data Fields

- int32_t [pv_eff](#)
- int32_t [tot_eff](#)
- uint32_t [ttg_t](#)
- uint32_t [updates](#)
- time_t [pclock](#)
- float [peukert](#)
- float [cef](#)
- float [peukert_adj](#)
- float [cef_calc](#)
- float [cef_save](#)
- int16_t [h](#) [HPARAM_SIZE]
- uint16_t [rate](#)
- uint16_t [udod](#)
- uint16_t [bsoc](#)
- uint16_t [bound_rate](#)
- uint16_t [bound_factor](#)
- uint16_t [samplei](#)
- uint16_t [sampleo](#)
- uint16_t [ah](#)
- uint16_t [drate](#)
- uint16_t [esr](#)
- uint16_t [rest_rate](#)
- uint16_t [rest_factor](#)
- uint16_t [esrp](#)
- uint8_t [version](#)

3.7.1 Detailed Description

Definition at line 273 of file [vconfig.h](#).

3.7.2 Field Documentation

3.7.2.1 ah

uint16_t ah

Definition at line 279 of file [vconfig.h](#).

3.7.2.2 bound_factor

uint16_t bound_factor

Definition at line 279 of file [vconfig.h](#).

3.7.2.3 bound_rate

```
uint16_t bound_rate
```

Definition at line 279 of file [vconfig.h](#).

3.7.2.4 bsoc

```
uint16_t bsoc
```

Definition at line 279 of file [vconfig.h](#).

3.7.2.5 cef

```
float cef
```

Definition at line 277 of file [vconfig.h](#).

3.7.2.6 cef_calc

```
float cef_calc
```

Definition at line 277 of file [vconfig.h](#).

3.7.2.7 cef_save

```
float cef_save
```

Definition at line 277 of file [vconfig.h](#).

3.7.2.8 drate

```
uint16_t drate
```

Definition at line 279 of file [vconfig.h](#).

3.7.2.9 esr

```
uint16_t esr
```

Definition at line 279 of file [vconfig.h](#).

3.7.2.10 esrp

```
uint16_t esrp
```

Definition at line 279 of file [vconfig.h](#).

3.7.2.11 h

```
int16_t h[HPARAM_SIZE]
```

Definition at line 278 of file [vconfig.h](#).

3.7.2.12 pclock

```
time_t pclock
```

Definition at line 276 of file [vconfig.h](#).

3.7.2.13 peukert

```
float peukert
```

Definition at line 277 of file [vconfig.h](#).

3.7.2.14 peukert_adj

```
float peukert_adj
```

Definition at line 277 of file [vconfig.h](#).

3.7.2.15 pv_eff

```
int32_t pv_eff
```

Definition at line 274 of file [vconfig.h](#).

3.7.2.16 rate

```
uint16_t rate
```

Definition at line 279 of file [vconfig.h](#).

3.7.2.17 rest_factor

```
uint16_t rest_factor
```

Definition at line 279 of file [vconfig.h](#).

3.7.2.18 rest_rate

```
uint16_t rest_rate
```

Definition at line 279 of file [vconfig.h](#).

3.7.2.19 samplei

```
uint16_t samplei
```

Definition at line 279 of file [vconfig.h](#).

3.7.2.20 sampleo

```
uint16_t sampleo
```

Definition at line 279 of file [vconfig.h](#).

3.7.2.21 tot_eff

```
int32_t tot_eff
```

Definition at line 274 of file [vconfig.h](#).

3.7.2.22 ttg_t

```
uint32_t ttg_t
```

Definition at line 275 of file [vconfig.h](#).

3.7.2.23 udod

```
uint16_t udod
```

Definition at line 279 of file [vconfig.h](#).

3.7.2.24 updates

```
uint32_t updates
```

Definition at line 275 of file [vconfig.h](#).

3.7.2.25 version

```
uint8_t version
```

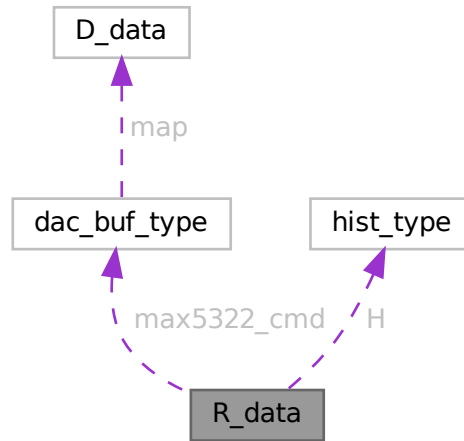
Definition at line 280 of file [vconfig.h](#).

The documentation for this struct was generated from the following file:

- [vconfig.h](#)

3.8 R_data Struct Reference

Collaboration diagram for R_data:



Data Fields

- `adc_result_t raw_adc` [ADC_BUFFER_SIZE]
- `adc_result_t raw_dac` [DAC_BUFFER_SIZE]
- `union dac_buf_type max5322_cmd`
- `int16_t n_offset` [NUM_C_SENSORS]
- `float n_scalar` [NUM_C_SENSORS]
- `uint8_t scan_index`
- `uint16_t scan_select`
- `bool done`
- `hist_type H`
- `uint8_t hist_save`: 1
- `uint8_t hist_update`: 1
- `uint8_t c_zero_cal`: 1
- `uint8_t c_scale_cal`: 1
- `uint16_t checkmark`
- `uint8_t crc`

3.8.1 Detailed Description

Definition at line 29 of file [daq.c](#).

3.8.2 Field Documentation

3.8.2.1 c_scale_cal

`uint8_t c_scale_cal`

Definition at line 42 of file [daq.c](#).

3.8.2.2 c_zero_cal

```
uint8_t c_zero_cal
```

Definition at line 41 of file [daq.c](#).

3.8.2.3 checkmark

```
uint16_t checkmark
```

Definition at line 43 of file [daq.c](#).

3.8.2.4 crc

```
uint8_t crc
```

Definition at line 44 of file [daq.c](#).

3.8.2.5 done

```
bool done
```

Definition at line 37 of file [daq.c](#).

3.8.2.6 H

```
hist_type H
```

Definition at line 38 of file [daq.c](#).

3.8.2.7 hist_save

```
uint8_t hist_save
```

Definition at line 39 of file [daq.c](#).

3.8.2.8 hist_update

```
uint8_t hist_update
```

Definition at line 40 of file [daq.c](#).

3.8.2.9 max5322_cmd

```
union dac_buf_type max5322_cmd
```

Definition at line 32 of file [daq.c](#).

3.8.2.10 n_offset

```
int16_t n_offset[NUM_C_SENSORS]
```

Definition at line 33 of file [daq.c](#).

3.8.2.11 n_scalar

```
float n_scalar[NUM_C_SENSORS]
```

Definition at line 34 of file [daq.c](#).

3.8.2.12 raw_adc

```
adc_result_t raw_adc[ADC_BUFFER_SIZE]
```

Definition at line 30 of file [daq.c](#).

3.8.2.13 raw_dac

```
adc_result_t raw_dac[DAC_BUFFER_SIZE]
```

Definition at line 31 of file [daq.c](#).

3.8.2.14 scan_index

```
uint8_t scan_index
```

Definition at line 35 of file [daq.c](#).

3.8.2.15 scan_select

```
uint16_t scan_select
```

Definition at line 36 of file [daq.c](#).

The documentation for this struct was generated from the following file:

- [daq.c](#)

3.9 rbutton_type Struct Reference

Data Fields

- SW_STATES [sw](#)
- uint32_t [count](#)

3.9.1 Detailed Description

Definition at line 40 of file [dio.h](#).

3.9.2 Field Documentation

3.9.2.1 count

```
uint32_t count
```

Definition at line 42 of file [dio.h](#).

3.9.2.2 sw

```
SW_STATES sw
```

Definition at line 41 of file [dio.h](#).

The documentation for this struct was generated from the following file:

- [dio.h](#)

3.10 ringBufS_t Struct Reference

Data Fields

- [uint8_t buf](#) [RBUF_SIZE]
- [uint8_t head](#)
- [uint8_t tail](#)
- [uint8_t count](#)

3.10.1 Detailed Description

Definition at line 19 of file [ringbufs.h](#).

3.10.2 Field Documentation

3.10.2.1 buf

```
uint8_t buf[RBUF_SIZE]
```

Definition at line 20 of file [ringbufs.h](#).

3.10.2.2 count

```
uint8_t count
```

Definition at line 23 of file [ringbufs.h](#).

3.10.2.3 head

```
uint8_t head
```

Definition at line 21 of file [ringbufs.h](#).

3.10.2.4 tail

```
uint8_t tail
```

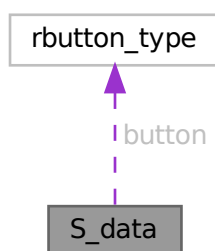
Definition at line 22 of file [ringbufs.h](#).

The documentation for this struct was generated from the following file:

- [ringbufs.h](#)

3.11 S_data Struct Reference

Collaboration diagram for S_data:



Data Fields

- volatile [rbutton_type](#) [button](#) [MAX_BUTTONS]
- volatile uint8_t [sw_bitmap](#)

3.11.1 Detailed Description

Definition at line 4 of file [dio.c](#).

3.11.2 Field Documentation

3.11.2.1 `button`

```
volatile rbutton\_type button[MAX_BUTTONS]
```

Definition at line 5 of file [dio.c](#).

3.11.2.2 `sw_bitmap`

```
volatile uint8_t sw_bitmap
```

Definition at line 6 of file [dio.c](#).

The documentation for this struct was generated from the following file:

- [dio.c](#)

3.12 `spi1_configuration_t` Struct Reference

Data Fields

- [uint8_t con0](#)
- [uint8_t con1](#)
- [uint8_t con2](#)
- [uint8_t baud](#)
- [uint8_t operation](#)

3.12.1 Detailed Description

SPI1 Generated Driver File

@Company Microchip Technology Inc.

@File Name [spi1.c](#)

@Summary This is the generated driver implementation file for the SPI1 driver using PIC10 / PIC12 / PIC16 / PIC18 MCUs

@Description This header file provides implementations for driver APIs for SPI1. Generation Information : Product Revision : PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8 Device : PIC18F57K42 Driver Version : 1.0.0 The generated drivers are tested against the following: Compiler : XC8 2.36 and above or later MPLAB : MPLAB X 6.00

Definition at line 50 of file [spi1.c](#).

3.12.2 Field Documentation

3.12.2.1 baud

`uint8_t baud`

Definition at line 54 of file [spi1.c](#).

3.12.2.2 con0

`uint8_t con0`

Definition at line 51 of file [spi1.c](#).

3.12.2.3 con1

`uint8_t con1`

Definition at line 52 of file [spi1.c](#).

3.12.2.4 con2

`uint8_t con2`

Definition at line 53 of file [spi1.c](#).

3.12.2.5 operation

`uint8_t operation`

Definition at line 55 of file [spi1.c](#).

The documentation for this struct was generated from the following file:

- [mcc_generated_files/spi1.c](#)

3.13 spi_link_type Struct Reference

Data Fields

- `uint8_t SPI_LCD`: 1
- `uint8_t SPI_AUX`: 1
- `uint8_t LCD_TIMER`: 1
- `volatile uint8_t LCD_DATA`: 1
- `uint16_t delay`
- `uint8_t config`
- `uint8_t * txbuf`
- `volatile int32_t int_count`

3.13.1 Detailed Description

Definition at line 183 of file [vconfig.h](#).

3.13.2 Field Documentation

3.13.2.1 config

```
uint8_t config
```

Definition at line 189 of file [vconfig.h](#).

3.13.2.2 delay

```
uint16_t delay
```

Definition at line 188 of file [vconfig.h](#).

3.13.2.3 int_count

```
volatile int32_t int_count
```

Definition at line 191 of file [vconfig.h](#).

3.13.2.4 LCD_DATA

```
volatile uint8_t LCD_DATA
```

Definition at line 187 of file [vconfig.h](#).

3.13.2.5 LCD_TIMER

```
uint8_t LCD_TIMER
```

Definition at line 186 of file [vconfig.h](#).

3.13.2.6 SPI_AUX

```
uint8_t SPI_AUX
```

Definition at line 185 of file [vconfig.h](#).

3.13.2.7 SPI_LCD

```
uint8_t SPI_LCD
```

Definition at line 184 of file [vconfig.h](#).

3.13.2.8 txbuf

```
uint8_t* txbuf
```

Definition at line 190 of file [vconfig.h](#).

The documentation for this struct was generated from the following file:

- [vconfig.h](#)

3.14 terminal_type Struct Reference

Data Fields

- `uint8_t` [msgid](#)
- `uint8_t` [TID](#)
- `uint8_t` [mcode](#)
- `uint8_t` [mparm](#)
- `uint8_t` [cmdlen](#)
- `uint8_t` [log_seq](#)
- `uint8_t` [host_display_ack](#): 1
- `D_CODES` [info](#)
- `D_CODES` [help_temp](#)
- `uint16_t` [ceid](#)
- `uint16_t` [log_num](#)

3.14.1 Detailed Description

Definition at line 201 of file [vconfig.h](#).

3.14.2 Field Documentation

3.14.2.1 ceid

```
uint16_t ceid
```

Definition at line 206 of file [vconfig.h](#).

3.14.2.2 cmdlen

`uint8_t cmdlen`

Definition at line 203 of file [vconfig.h](#).

3.14.2.3 help_temp

`D_CODES help_temp`

Definition at line 205 of file [vconfig.h](#).

3.14.2.4 host_display_ack

`uint8_t host_display_ack`

Definition at line 204 of file [vconfig.h](#).

3.14.2.5 info

`D_CODES info`

Definition at line 205 of file [vconfig.h](#).

3.14.2.6 log_num

`uint16_t log_num`

Definition at line 207 of file [vconfig.h](#).

3.14.2.7 log_seq

`uint8_t log_seq`

Definition at line 203 of file [vconfig.h](#).

3.14.2.8 mcode

`uint8_t mcode`

Definition at line 203 of file [vconfig.h](#).

3.14.2.9 mesgid

`uint8_t mesgid`

Definition at line 202 of file [vconfig.h](#).

3.14.2.10 mparm

```
uint8_t mparm
```

Definition at line 203 of file [vconfig.h](#).

3.14.2.11 TID

```
uint8_t TID
```

Definition at line 203 of file [vconfig.h](#).

The documentation for this struct was generated from the following file:

- [vconfig.h](#)

3.15 uart1_status_t Union Reference

```
#include <uart1.h>
```

Data Fields

- struct {
 - unsigned [perr](#): 1
 - unsigned [ferr](#): 1
 - unsigned [oerr](#): 1
 - unsigned [reserved](#): 5
- };
- uint8_t [status](#)

3.15.1 Detailed Description

Section: Data Type Definitions

Definition at line 74 of file [uart1.h](#).

3.15.2 Field Documentation

3.15.2.1 ferr

```
unsigned ferr
```

Definition at line 77 of file [uart1.h](#).

3.15.2.2 oerr

unsigned oerr

Definition at line 78 of file [uart1.h](#).

3.15.2.3 perr

unsigned perr

Definition at line 76 of file [uart1.h](#).

3.15.2.4 reserved

unsigned reserved

Definition at line 79 of file [uart1.h](#).

3.15.2.5 status

uint8_t status

Definition at line 81 of file [uart1.h](#).

The documentation for this union was generated from the following file:

- [mcc_generated_files/uart1.h](#)

3.16 uart2_status_t Union Reference

```
#include <uart2.h>
```

Data Fields

- struct {
 - unsigned [perr](#): 1
 - unsigned [ferr](#): 1
 - unsigned [oerr](#): 1
 - unsigned [reserved](#): 5
- };
- uint8_t [status](#)

3.16.1 Detailed Description

Section: Data Type Definitions

Definition at line 74 of file [uart2.h](#).

3.16.2 Field Documentation

3.16.2.1 ferr

unsigned ferr

Definition at line 77 of file [uart2.h](#).

3.16.2.2 oerr

unsigned oerr

Definition at line 78 of file [uart2.h](#).

3.16.2.3 perr

unsigned perr

Definition at line 76 of file [uart2.h](#).

3.16.2.4 reserved

unsigned reserved

Definition at line 79 of file [uart2.h](#).

3.16.2.5 status

uint8_t status

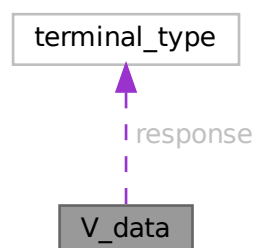
Definition at line 81 of file [uart2.h](#).

The documentation for this union was generated from the following file:

- [mcc_generated_files/uart2.h](#)

3.17 V_data Struct Reference

Collaboration diagram for V_data:



Data Fields

- UI_STATES [ui_state](#)
- char [buf](#) [64]
- char [info](#) [64]
- char [rbuf](#) [16]
- volatile time_t [ticks](#)
- volatile time_t [blight](#)
- volatile time_t [ac_time](#)
- volatile int8_t [testing](#)
- volatile uint8_t [sys_info](#): 1
- volatile uint8_t [ac_on](#): 1
- volatile uint8_t [ac_off](#): 1
- volatile uint8_t [time_info](#): 1
- volatile uint8_t [get_time_text](#): 1
- uint8_t [error](#)
- uint8_t [abort](#)
- uint8_t [msg_error](#)
- uint8_t [msg_ret](#)
- uint8_t [alarm](#)
- uint8_t [float_ticks](#)
- uint8_t [boost_ticks](#)
- uint8_t [wdt_ticks](#)
- UI_STATES [ui_sw](#)
- uint16_t [r_checksum](#)
- uint16_t [t_checksum](#)
- uint16_t [checksum_error](#)
- uint16_t [mode_pwm](#)
- uint16_t [sequences](#)
- uint16_t [all_errors](#)
- uint8_t [set_sequ](#): 1
- uint8_t [debug](#): 1
- uint8_t [help](#): 1
- uint8_t [stack](#): 3
- uint8_t [help_id](#): 2
- uint8_t [screen](#): 1
- [terminal_type](#) response
- volatile uint8_t [ticker](#)
- volatile uint8_t [cc_state](#)
- bool [flipper](#)
- bool [calib](#)
- bool [enter](#)
- bool [sensor_set](#)
- bool [fixup](#)
- bool [cc_active](#)
- volatile bool [system_stable](#)
- volatile bool [blight_off](#)
- volatile bool [in_float](#)
- volatile bool [in_boost](#)
- volatile uint32_t [lowint_count](#)
- volatile uint32_t [timerint_count](#)
- volatile uint32_t [tx_count](#)
- volatile uint32_t [rx_count](#)
- volatile uint32_t [adc_count](#)
- volatile uint32_t [spi_count](#)
- volatile uint32_t [switch_count](#)

3.17.1 Detailed Description

Definition at line 236 of file [vconfig.h](#).

3.17.2 Field Documentation

3.17.2.1 abort

```
uint8_t abort
```

Definition at line 242 of file [vconfig.h](#).

3.17.2.2 ac_off

```
volatile uint8_t ac_off
```

Definition at line 241 of file [vconfig.h](#).

3.17.2.3 ac_on

```
volatile uint8_t ac_on
```

Definition at line 241 of file [vconfig.h](#).

3.17.2.4 ac_time

```
volatile time_t ac_time
```

Definition at line 239 of file [vconfig.h](#).

3.17.2.5 adc_count

```
volatile uint32_t adc_count
```

Definition at line 250 of file [vconfig.h](#).

3.17.2.6 alarm

```
uint8_t alarm
```

Definition at line 242 of file [vconfig.h](#).

3.17.2.7 all_errors

```
uint16_t all_errors
```

Definition at line 244 of file [vconfig.h](#).

3.17.2.8 blight

```
volatile time_t blight
```

Definition at line 239 of file [vconfig.h](#).

3.17.2.9 blight_off

```
volatile bool blight_off
```

Definition at line 249 of file [vconfig.h](#).

3.17.2.10 boost_ticks

```
uint8_t boost_ticks
```

Definition at line 242 of file [vconfig.h](#).

3.17.2.11 buf

```
char buf[64]
```

Definition at line 238 of file [vconfig.h](#).

3.17.2.12 calib

```
bool calib
```

Definition at line 248 of file [vconfig.h](#).

3.17.2.13 cc_active

```
bool cc_active
```

Definition at line 248 of file [vconfig.h](#).

3.17.2.14 cc_state

```
volatile uint8_t cc_state
```

Definition at line 247 of file [vconfig.h](#).

3.17.2.15 checksum_error

```
uint16_t checksum_error
```

Definition at line 244 of file [vconfig.h](#).

3.17.2.16 debug

```
uint8_t debug
```

Definition at line 245 of file [vconfig.h](#).

3.17.2.17 enter

```
bool enter
```

Definition at line 248 of file [vconfig.h](#).

3.17.2.18 error

```
uint8_t error
```

Definition at line 242 of file [vconfig.h](#).

3.17.2.19 fixup

```
bool fixup
```

Definition at line 248 of file [vconfig.h](#).

3.17.2.20 flipper

```
bool flipper
```

Definition at line 248 of file [vconfig.h](#).

3.17.2.21 float_ticks

```
uint8_t float_ticks
```

Definition at line 242 of file [vconfig.h](#).

3.17.2.22 get_time_text

```
volatile uint8_t get_time_text
```

Definition at line 241 of file [vconfig.h](#).

3.17.2.23 help

`uint8_t help`

Definition at line 245 of file [vconfig.h](#).

3.17.2.24 help_id

`uint8_t help_id`

Definition at line 245 of file [vconfig.h](#).

3.17.2.25 in_boost

`volatile bool in_boost`

Definition at line 249 of file [vconfig.h](#).

3.17.2.26 in_float

`volatile bool in_float`

Definition at line 249 of file [vconfig.h](#).

3.17.2.27 info

`char info[64]`

Definition at line 238 of file [vconfig.h](#).

3.17.2.28 lowint_count

`volatile uint32_t lowint_count`

Definition at line 250 of file [vconfig.h](#).

3.17.2.29 mode_pwm

`uint16_t mode_pwm`

Definition at line 244 of file [vconfig.h](#).

3.17.2.30 msg_error

`uint8_t msg_error`

Definition at line 242 of file [vconfig.h](#).

3.17.2.31 msg_ret

```
uint8_t msg_ret
```

Definition at line 242 of file [vconfig.h](#).

3.17.2.32 r_checksum

```
uint16_t r_checksum
```

Definition at line 244 of file [vconfig.h](#).

3.17.2.33 rbuf

```
char rbuf[16]
```

Definition at line 238 of file [vconfig.h](#).

3.17.2.34 response

```
terminal_type response
```

Definition at line 246 of file [vconfig.h](#).

3.17.2.35 rx_count

```
volatile uint32_t rx_count
```

Definition at line 250 of file [vconfig.h](#).

3.17.2.36 screen

```
uint8_t screen
```

Definition at line 245 of file [vconfig.h](#).

3.17.2.37 sensor_set

```
bool sensor_set
```

Definition at line 248 of file [vconfig.h](#).

3.17.2.38 sequences

```
uint16_t sequences
```

Definition at line 244 of file [vconfig.h](#).

3.17.2.39 set_sequ

```
uint8_t set_sequ
```

Definition at line 245 of file [vconfig.h](#).

3.17.2.40 spi_count

```
volatile uint32_t spi_count
```

Definition at line 250 of file [vconfig.h](#).

3.17.2.41 stack

```
uint8_t stack
```

Definition at line 245 of file [vconfig.h](#).

3.17.2.42 switch_count

```
volatile uint32_t switch_count
```

Definition at line 250 of file [vconfig.h](#).

3.17.2.43 sys_info

```
volatile uint8_t sys_info
```

Definition at line 241 of file [vconfig.h](#).

3.17.2.44 system_stable

```
volatile bool system_stable
```

Definition at line 249 of file [vconfig.h](#).

3.17.2.45 t_checksum

```
uint16_t t_checksum
```

Definition at line 244 of file [vconfig.h](#).

3.17.2.46 testing

```
volatile int8_t testing
```

Definition at line 240 of file [vconfig.h](#).

3.17.2.47 ticker

```
volatile uint8_t ticker
```

Definition at line 247 of file [vconfig.h](#).

3.17.2.48 ticks

```
volatile time_t ticks
```

Definition at line 239 of file [vconfig.h](#).

3.17.2.49 time_info

```
volatile uint8_t time_info
```

Definition at line 241 of file [vconfig.h](#).

3.17.2.50 timerint_count

```
volatile uint32_t timerint_count
```

Definition at line 250 of file [vconfig.h](#).

3.17.2.51 tx_count

```
volatile uint32_t tx_count
```

Definition at line 250 of file [vconfig.h](#).

3.17.2.52 ui_state

```
UI_STATES ui_state
```

Definition at line 237 of file [vconfig.h](#).

3.17.2.53 ui_sw

```
UI_STATES ui_sw
```

Definition at line 243 of file [vconfig.h](#).

3.17.2.54 wdt_ticks

```
uint8_t wdt_ticks
```

Definition at line [242](#) of file [vconfig.h](#).

The documentation for this struct was generated from the following file:

- [vconfig.h](#)

3.18 V_help Struct Reference

Data Fields

- const char [message](#) [22]
- const char [display](#) [22]

3.18.1 Detailed Description

Definition at line [253](#) of file [vconfig.h](#).

3.18.2 Field Documentation

3.18.2.1 display

```
const char display[22]
```

Definition at line [254](#) of file [vconfig.h](#).

3.18.2.2 message

```
const char message[22]
```

Definition at line [254](#) of file [vconfig.h](#).

The documentation for this struct was generated from the following file:

- [vconfig.h](#)

Chapter 4

File Documentation

4.1 bsoc.c

```
00001 #include "bsoc.h"
00002 #include "mcc_generated_files/interrupt_manager.h"
00003
00004 const char *MQTT_ID = "DUMpload";
00005
00006 const uint32_t BVSOC_TABLE[BVSOC_SLOTS][2] = {
00007     23000, 5,
00008     23400, 10,
00009     23600, 20,
00010     24120, 25,
00011     24200, 30,
00012     24440, 40,
00013     24540, 45,
00014     24600, 50,
00015     24646, 53,
00016     24700, 55,
00017     24750, 57,
00018     24800, 60,
00019     24850, 63,
00020     24925, 67,
00021     25000, 70,
00022     25020, 72,
00023     25040, 75,
00024     25050, 80,
00025     25060, 85,
00026     25080, 90,
00027     25096, 92,
00028     25122, 93,
00029     25140, 95,
00030     25160, 97,
00031     25180, 100,
00032     26500, 98 // charging voltage guess
00033 };
00034
00035 const char infoline1[] = "    LOW Count    TOD Count    TX Count    RX Count    ADC Count    SPI Count
DIO Count";
00036 const char infoline2[] = "    DAILY AH    LO ESR    HI ESR    PV AH    BAT W Out    BAT W IN
REAL AH    F CYCLES    B CYCLES    UPDATES";
00037
00038 static uint32_t seq_log = 0;
00039
00040 #define MAX_GTI    650
00041 #define GTI_POWER    0 // start with zero power
00042 #define SPINNER_SPEED    200
00043 #define GTI_SPEED    1000
00044 #define Swrite    UART2_Write
00045 #define Sread    UART1_Read
00046 #define Sready    UART1_is_rx_ready
00047
00048 volatile int16_t gti_power = GTI_POWER;
00049 volatile bool flop = true;
00050 volatile uint8_t mqtt_r;
00051 volatile int16_t cmd_value = 0;
00052 time_t utc_cmd_value = 0;
00053
00054 uint8_t gti_sbuf[8] = {0x24, 0x56, 0x00, 0x21, 0x03, 0x99, 0x80, 0x6c};
00055 uint8_t gti_zero[8] = {0x24, 0x56, 0x00, 0x21, 0x00, 0x00, 0x80, 0x08};
```

```

00056
00057 uint8_t gti_checksum(uint8_t *, uint16_t);
00058
00059 /*
00060  * DC PWM diversion for direct PV power before MPPT
00061  */
00062 bool pv_diversion(bool kill)
00063 {
00064     static uint16_t pwm_val = 0;
00065     static bool pwm_active = false;
00066     bool ret = false;
00067
00068     if (kill) {
00069         pwm_val = 0;
00070         V.mode_pwm = 0;
00071         pwm_active = false;
00072         diversion_pwm_set(V.mode_pwm); // 10KHz PWM quick stop
00073         ret = false;
00074     } else {
00075         if (((cc_state(C.v_cmode) == M_FLOAT) || (cc_state(C.v_cmode) == M_BOOST)) && (C.calc[V_PV] >
DPWM_LOW_VOLTS)) {
00076             if (!pwm_active) {
00077                 pwm_active = true;
00078                 /*
00079                  * setup PWM start conditions
00080                  */
00081                 C.start_power = C.p_pv; // save for later possible display
00082             }
00083             if (C.p_pv < PWM_MAX_POWER) { // increase power
00084                 if (++pwm_val > DPWM_FULL)
00085                     pwm_val = DPWM_FULL; // max PWM limiter
00086             }
00087             if (C.p_pv > PWM_MAX_POWER) { // decrease power
00088                 if (V.mode_pwm)
00089                     V.mode_pwm--;
00090             }
00091         }
00092         V.mode_pwm = pwm_val;
00093         diversion_pwm_set(V.mode_pwm); // 10KHz PWM ramp up
00094         ret = true;
00095     } else {
00096         if (V.mode_pwm)
00097             V.mode_pwm--;
00098         diversion_pwm_set(V.mode_pwm); // 10KHz PWM ramp down
00099         pwm_val = 0;
00100         pwm_active = false;
00101         ret = false;
00102     }
00103 }
00104 }
00105
00106 return ret;
00107 }
00108
00109 /*
00110  * for GTI RS485 messages
00111  */
00112 uint8_t gti_checksum(uint8_t * gti_sbuf, uint16_t power)
00113 {
00114     uint16_t pu, pl, cs;
00115
00116     if (power > MAX_GTI) {
00117         power = MAX_GTI;
00118     }
00119
00120     pu = power >> 8;
00121     pl = power & 0xff;
00122     cs = 264 - pu - pl;
00123     if (cs >= 256) {
00124         cs = 8;
00125     }
00126
00127     gti_sbuf[4] = (uint8_t) pu;
00128     gti_sbuf[5] = (uint8_t) pl;
00129     gti_sbuf[7] = (uint8_t) cs;
00130     return (uint8_t) cs;
00131 }
00132
00133 /*
00134  * check for incoming data on the MQTT connection
00135  * use UART1 or the MQTT receive data port
00136  * calc_bsoc runs in interrupt low context and uses 16-bit results -> git_power from this
00137  * cmd_value is the buffer variable
00138  */
00139 void gti_cmds(void)
00140 {
00141     static uint8_t value[] = {0, 0, 0, 0}, vi = 0;

```

```

00142     static uint8_t utc_value[DEF_TIME_SIZE] = {0};
00143     static bool utc = false;
00144     uint8_t vcmd_size = sizeof(value);
00145     uint8_t utc_vcmd_size = DEF_TIME_SIZE - 1;
00146
00147     if (Sready()) {
00148         mgtt_r = Sread();
00149 #ifdef GTI_ECHO
00150         UART1_Write(mgtt_r); // debug echo
00151 #endif
00152
00153         switch (mgtt_r) {
00154             case '0':
00155             case '1':
00156             case '2':
00157             case '3':
00158             case '4':
00159             case '5':
00160             case '6':
00161             case '7':
00162             case '8':
00163             case '9':
00164                 if (!utc) { // process power cmds
00165                     if (vi < vcmd_size) {
00166                         value[vi++] = mgtt_r - 48; // ascii '0'
00167                     } else {
00168                     }
00169                 } else { // process UTC time from host cmds
00170                     if (vi < utc_vcmd_size) {
00171                         utc_value[vi++] = mgtt_r;
00172                     } else {
00173                     }
00174                 }
00175                 break;
00176             case 'T': // begin UTC value
00177                 utc = true;
00178                 break;
00179             case 't': // end UTC value
00180                 utc = false;
00181                 if (vi >= utc_vcmd_size) {
00182                     vi = 0;
00183                     utc_value[10] = 0;
00184                     utc_cmd_value = (time_t) atol((char *) utc_value);
00185
00186                     if (utc_cmd_value < DEF_TIME) {
00187                         utc_cmd_value = DEF_TIME;
00188                     }
00189                     set_time(utc_cmd_value);
00190                 }
00191                 break;
00192             case 'V': // begin power/utc value
00193                 vi = 0;
00194                 break;
00195             case 'X': // end power value
00196                 utc = false;
00197                 if (vi >= vcmd_size) {
00198                     vi = 0;
00199                     cmd_value = value[0]*1000 + value [1]*100 + value[2]*10 + value[3];
00200                     if (cmd_value > GTI_MAX) {
00201                         cmd_value = GTI_MAX;
00202                     }
00203                     if (cmd_value < 0) {
00204                         cmd_value = 0;
00205                     }
00206                     INTERRUPT_GlobalInterruptLowDisable(); // 16-bit atomic update
00207                     gti_power = cmd_value;
00208                     INTERRUPT_GlobalInterruptLowEnable();
00209                 }
00210                 break;
00211             case 'Z': // zero power
00212                 utc = false;
00213                 cmd_value = 0;
00214                 break;
00215             case '+': // incr power
00216                 utc = false;
00217                 cmd_value = gti_power + GTI_INCR;
00218                 if (cmd_value > GTI_MAX) {
00219                     cmd_value = GTI_MAX;
00220                 }
00221                 break;
00222             case '-': // decr power
00223                 utc = false;
00224                 cmd_value = gti_power - GTI_INCR;
00225                 if (cmd_value < 0) {
00226                     cmd_value = 0;
00227                 }
00228                 break;

```

```

00229         case 'I': // idle power
00230             utc = false;
00231             cmd_value = GTI_IDLE;
00232             break;
00233         case 'F': // normal operation
00234             utc = false;
00235             cmd_value = GTI_NORM;
00236             break;
00237         case 'M': // max unit rated power testing
00238             utc = false;
00239             cmd_value = GTI_MAX;
00240             break;
00241         case '#': // execute command symbol
00242             utc = false;
00243             INTERRUPT_GlobalInterruptLowDisable(); // 16-bit atomic update
00244             gti_power = cmd_value;
00245             INTERRUPT_GlobalInterruptLowEnable();
00246             break;
00247         default: // eat extra characters
00248             utc = false;
00249             while (Sready()) {
00250                 mqtt_r = Sread();
00251             }
00252             break;
00253     }
00254 }
00255 }
00256
00257 /*
00258  * low-pri interrupt ISR the runs every second for simple coulomb counting
00259  */
00260 void calc_bsoc(void)
00261 {
00262     uint8_t * log_ptr;
00263     uint32_t t_time;
00264     static uint8_t log_update_wait = 0;
00265     float adj = 1.0;
00266
00267     #ifdef DEBUG_BSOC1
00268         DEBUG1_SetHigh();
00269     #endif
00270     V.lowint_count++;
00271     /*
00272      * check for excess power and send to DC dump load
00273      */
00274     pv_diversion(false);
00275     /*
00276      * Charge Controller condition events
00277      */
00278     if (cc_state(C.v_cmode) == M_FLOAT) {
00279         if (!V.in_float && (++V.float_ticks > FLOAT_TIME))
00280             if (!V.in_float) {
00281                 V.in_float = true;
00282                 if (C.soc < 95) // SOC and AH corrections
00283                     C.dynamic_ah = C.bank_ah * 0.95f;
00284             }
00285     } else {
00286         V.float_ticks = 0;
00287     }
00288
00289     if (cc_state(C.v_cmode) == M_BOOST) {
00290         if (!V.in_boost && (++V.boost_ticks > FLOAT_TIME))
00291             if (!V.in_boost) { // SOC and AH corrections
00292                 V.in_boost = true;
00293                 if (C.soc < 80)
00294                     C.dynamic_ah = C.bank_ah * 0.80f;
00295             }
00296     } else {
00297         V.boost_ticks = 0;
00298     }
00299
00300     /*
00301      * Battery data calculations
00302      * had to move the current sensors for the correct PV, BAT and CC currents for this configuration
00303      */
00304     #ifdef FM80_FIXUPS
00305         C.dynamic_ah += (C.c_mppt / SSLICE); // Ah
00306         C.dynamic_ah_daily += (C.c_mppt / SSLICE); // Ah
00307         if (C.dynamic_ah > (C.bank_ah))
00308             C.dynamic_ah = C.bank_ah;
00309         if (C.dynamic_ah < 0.1)
00310             C.dynamic_ah = 0.1;
00311
00312         if (C.c_pv > 0.01)
00313             adj = C.hist[0].cef;
00314         if (C.c_pv < 0.01)
00315             adj = C.hist[0].peukert;

```

```

00316     C.dynamic_ah += ((C.c_pv * adj) / SSLICE); // Ah
00317
00318     C.dynamic_ah_adj = C.dynamic_ah; // need to add peukert factor here
00319     C.dynamic_ah_adj_daily = C.dynamic_ah_daily; // need to add peukert factor here
00320     if (C.dynamic_ah_adj > (C.bank_ah))
00321         C.dynamic_ah_adj = C.bank_ah;
00322     if (C.dynamic_ah_adj < 0.1)
00323         C.dynamic_ah_adj = 0.1;
00324
00325
00326     C.pv_ah += (C.c_pv / SSLICE);
00327     C.pvkw += (C.p_pv / SSLICE);
00328     C.invkw += (C.p_inverter / SSLICE);
00329     C.loadkw += (C.p_load / SSLICE);
00330     if (C.p_bat > 0.0)
00331         C.bkwi += (C.p_bat / SSLICE);
00332     if (C.p_bat < 0.0)
00333         C.bkwo += fabs(C.p_bat / SSLICE);
00334
00335     C.soc = ((uint16_t) ((C.dynamic_ah_adj / C.bank_ah)*100.0) + 1);
00336     if (C.soc > 100)
00337         C.soc = 100;
00338
00339     if (C.c_mppt < 0.0) {
00340         C.runtime = (uint16_t) (-(C.dynamic_ah_adj / C.c_mppt));
00341     } else {
00342         C.runtime = 99;
00343     }
00344     if (C.runtime > 99) {
00345         C.runtime = 99;
00346     }
00347 #else
00348     C.dynamic_ah += (C.c_bat / SSLICE); // Ah
00349     C.dynamic_ah_daily += (C.c_bat / SSLICE); // Ah
00350     if (C.dynamic_ah > (C.bank_ah))
00351         C.dynamic_ah = C.bank_ah;
00352     if (C.dynamic_ah < 0.1)
00353         C.dynamic_ah = 0.1;
00354
00355     if (C.c_bat > 0.01)
00356         adj = C.hist[0].cef;
00357     if (C.c_bat < 0.01)
00358         adj = C.hist[0].peukert;
00359     C.dynamic_ah += ((C.c_bat * adj) / SSLICE); // Ah
00360
00361     C.dynamic_ah_adj = C.dynamic_ah; // need to add peukert factor here
00362     C.dynamic_ah_adj_daily = C.dynamic_ah_daily; // need to add peukert factor here
00363     if (C.dynamic_ah_adj > (C.bank_ah))
00364         C.dynamic_ah_adj = C.bank_ah;
00365     if (C.dynamic_ah_adj < 0.1)
00366         C.dynamic_ah_adj = 0.1;
00367
00368
00369     C.pv_ah += (C.c_pv / SSLICE);
00370     C.pvkw += (C.p_pv / SSLICE);
00371     C.invkw += (C.p_inverter / SSLICE);
00372     C.loadkw += (C.p_load / SSLICE);
00373     if (C.p_bat > 0.0)
00374         C.bkwi += (C.p_bat / SSLICE);
00375     if (C.p_bat < 0.0)
00376         C.bkwo += fabs(C.p_bat / SSLICE);
00377
00378     C.soc = ((uint16_t) ((C.dynamic_ah_adj / C.bank_ah)*100.0) + 1);
00379     if (C.soc > 100)
00380         C.soc = 100;
00381
00382     if (C.c_bat < 0.0) {
00383         C.runtime = (uint16_t) (-(C.dynamic_ah_adj / C.c_bat));
00384     } else {
00385         C.runtime = 99;
00386     }
00387     if (C.runtime > 99) {
00388         C.runtime = 99;
00389     }
00390 #endif
00391
00392     /*
00393     * check event flags
00394     */
00395     if (V.time_info) { // set local time from remote server
00396         V.time_info = false;
00397         t_time = (uint32_t) atol(V.rbuf);
00398         if (t_time > DEF_TIME) {
00399             set_time(t_time);
00400         }
00401     }
00402

```

```

00403     /*
00404     * serial port log data updates using DMA
00405     */
00406     if (V.sys_info) {
00407         V.sys_info = false;
00408     } else {
00409         if (!log_update_wait++ && V.system_stable && !V.get_time_text) {
00410             log_ptr = port_data_dma_ptr();
00411             if (H.sequence == HID_AUX) {
00412                 }
00413             }
00414             /*
00415             * REMOVE THE CHIP PROGRAMMER, IT CHANGES THE ADC READINGS
00416             */
00417             /*
00418             * format data to JSON
00419             */
00420 #ifdef FM80_FIXUPS
00421     C.p_pv = C.v_pv * C.c_bat; // power from FM80 AC
00422     C.p_bat = C.v_bat * C.c_mppt; // Power to/from BATTERY
00423     C.p_mppt = C.v_bat * C.c_pv; // Power from Charge Controller
00424 #else
00425     C.p_pv = C.v_pv * C.c_pv; // power from FM80 AC
00426     C.p_bat = C.v_bat * C.c_bat; // Power to/from BATTERY
00427     C.p_mppt = C.v_bat * C.c_mppt; // Power from Charge Controller
00428 #endif
00429     snprintf((char *) log_ptr, max_port_data - 1, "{\r\n \"DLname\": \"%s MBMC K42\", \r\n
\"DLsequence\": %lu, \r\n \"DLccmode\": %u, \r\n \"DLgti\": %u, \r\n \"DLv_pv\": %7.2f, \r\n \"DLv_bat\":
%7.2f, \r\n \"DLc_pv\": %7.2f, \r\n \"DLc_mppt\": %7.2f, \r\n \"DLc_bat\": %7.2f, \r\n \"DLp_pv\":
%7.2f, \r\n \"DLp_mppt\": %7.2f, \r\n \"DLp_bat\": %7.2f, \r\n \"DLah_bat\": %7.2f, \r\n \"DLbuild_date\":
\"%s\", \r\n \"DLbuild_time\": \"%s\" \r\n } \r\n",
VER, seq_log++, V.cc_state, gti_power, C.v_pv, C.v_bat, C.c_pv, C.c_mppt, C.c_bat,
C.p_pv, C.p_mppt, C.p_bat, C.dynamic_ah, build_date, build_time);
00431
00432     StartTimer(TMR_DISPLAY, SOCDELAY); // sync the spi dma display updates to avoid memory
contention
00433 #ifdef DEBUG_JSON
00434     DEBUG2_Toggle();
00435 #endif
00436     send_port_data_dma(strlen((char*) log_ptr));
00437     /*
00438     * send power limiter commands to the GTI Inverter
00439     * uses serial port 2 4800 baud
00440     */
00441     gti_checksum(gti_sbuf, (uint16_t) gti_power);
00442     for (uint8_t i = 0; i < 8; i++) {
00443         if (flop) {
00444             Swrite(gti_sbuf[i]);
00445         } else {
00446             Swrite(gti_zero[i]);
00447         }
00448     }
00449 }
00450
00451     if (log_update_wait >= LOG_WAIT) {
00452         log_update_wait = 0;
00453     }
00454 }
00455
00456     C.update = false;
00457 #ifdef DEBUG_BSOC1
00458     DEBUG1_SetLow();
00459 #endif
00460 }
00461
00462 void init_bsoc(void)
00463 {
00464     /*
00465     * use raw battery voltage
00466     */
00467     C.soc = Volts_to_SOC((uint32_t) (conv_raw_result(V_BAT, CONV) * 1000.0f));
00468     C.dynamic_ah = C.bank_ah * (Volts_to_SOC((uint32_t) (conv_raw_result(V_BAT, CONV) * 1000.0f)) /
100.0f);
00469     if (V.cc_active) { // charge controller online FIXUP
00470         C.dynamic_ah += 10.0f;
00471         C.soc += 10;
00472     }
00473     C.dynamic_ah_adj = C.dynamic_ah;
00474     TMR3_SetInterruptHandler(calc_bsoc); // one second timer
00475 }
00476
00477 void start_bsoc(void)
00478 {
00479     // Enabling TMR3 interrupt.
00480     PIE6bits.TMR3IE = 1;
00481 #ifdef DEBUG_BSOC2
00482     DEBUG2_Toggle();

```

```

00483 #endif
00484 }
00485
00486 void stop_bsoc(void)
00487 {
00488     // Disabling TMR3 interrupt.
00489     PIE6bits.TMR3IE = 0;
00490 }
00491
00492 void reset_bsoc(const R_CODES rmode)
00493 {
00494     switch (rmode) {
00495     case R_CYCLE:
00496     default:
00497         C.pv_ah = 0.0;
00498         break;
00499     }
00500 }
00501
00502 /* Current is in 0.1A units */
00503 uint32_t peukert(uint16_t brate, float bcurrent, float peukert, int16_t bsoc)
00504 {
00505     float t1, t2, t3, t4, t5, t6, t7, t_time;
00506
00507     t1 = (float) bcurrent; // Load on battery in 0.1A units
00508     t1 /= 10.0; // convert back to A
00509     if (t1 < 1.0) t1 = 1.0; // inverter current floor is 1.0 amp
00510     t2 = (float) brate; // Battery capacity in 1.0A units
00511     t3 = (float) ((float) bsoc / 100.0); // State of Charge
00512     if (t3 > 0.0) t2 = t2 * t3; // only decrease battery Ah
00513     if (t2 < 1.0) t2 = 1.0; // Ah floor is 1.0
00514     t4 = t2 * (float) (pow((t2 / (float) 20.0), ((float) peukert - (float) 1.0)));
00515     t_time = pow((t2 / (float) 20.0), peukert); // factor at 20h rate
00516     t5 = pow(t1, peukert); // factor at load current rate
00517     t6 = t4 / t5; // runtime in hours at the current draw
00518     if (t6 > MAXRUNTIME) t6 = MAXRUNTIME; // max time is 4 days in hours
00519     if (t6 < MINRUNTIME) t6 = MINRUNTIME;
00520     t7 = t4 / t_time; // time at fixed 20h rate
00521
00522     C.hist[0].peukert_adj = t7 / t6; // Peukert factor to resize Ah usage, t7=runtime at the 20h
    current rate/t6=runtime at the inverter load current value
00523     if (C.hist[0].peukert_adj < Perk_ADJ_FL)
00524         C.hist[0].peukert_adj = Perk_ADJ_FL; // limit how LOW
00525     if (C.hist[0].peukert_adj > Perk_ADJ_FH)
00526         C.hist[0].peukert_adj = Perk_ADJ_FH; // limit how HIGH
00527
00528     t6 = t6 * 3600.0f; // convert to seconds for runtime at current discharge rate
00529     return(uint32_t) t6;
00530 }
00531
00532 /*
00533  * static SOC table
00534  */
00535 uint16_t Volts_to_SOC(const uint32_t cvoltage)
00536 {
00537     uint8_t slot;
00538     uint16_t soc = 0;
00539
00540     /*
00541      * walk up the table
00542      */
00543     for (slot = 0; slot < BVSOC_SLOTS; slot++) {
00544         if (cvoltage > BVSOC_TABLE[slot][0]) {
00545             soc = (uint16_t) BVSOC_TABLE[slot][1];
00546         }
00547     }
00548     return soc;
00549 }
00550
00551 /*
00552  * check battery ESR, returns positive ESR value when done,
00553  * a negative number code when running the sequence and
00554  * -1.0 when each FSM sequence is done
00555  * (fsm 'true' will init the state machine and return the init code)
00556  */
00557 float esr_check(const uint8_t fsm)
00558 {
00559     static uint8_t esr_state = 0;
00560     float ret = -1.0;
00561
00562     if (fsm) {
00563         esr_state = 0;
00564         ret = -10.0;
00565     } else {
00566         switch (esr_state) {
00567         case 0:

```

```

00569         StartTimer(TMR_ESR, 10000); // start the sequence timer
00570         esr_state++; // move to the next state of the FSM
00571         break;
00572     case 1:
00573         /*
00574          * set the load resistors to all off
00575          */
00576         set_load_relay_one(false);
00577         set_load_relay_two(false);
00578         if (TimerDone(TMR_ESR)) { // check for expired timer
00579             StartTimer(TMR_ESR, 10000); // done, restart the timer, complete sequence, return -1.0
00580         } else {
00581             ret = -2.0; // nope, return with a progress code
00582         }
00583         /*
00584          * save unloaded battery voltage
00585          */
00586         update_adc_result();
00587         C.bv_noload = conv_raw_result(V_BAT, CONV);
00588         esr_state++; // move to the next state of the FSM
00589         break;
00590     case 2:
00591         set_load_relay_one(true);
00592         if (TimerDone(TMR_ESR)) {
00593             StartTimer(TMR_ESR, 10000);
00594         } else {
00595             ret = -3.0;
00596         }
00597
00598         update_adc_result();
00599         C.bv_one_load = conv_raw_result(V_BAT, CONV);
00600         C.load_i1 = conv_raw_result(C_BATT, CONV); // get current
00601         esr_state++;
00602         break;
00603     case 3:
00604         set_load_relay_two(true);
00605         if (!TimerDone(TMR_ESR))
00606             ret = -4.0;
00607
00608         update_adc_result();
00609         C.bv_full_load = conv_raw_result(V_BAT, CONV);
00610         C.load_i2 = conv_raw_result(C_BATT, CONV); // get current
00611
00612         C.esr = fabs((C.bv_one_load - C.bv_full_load) / (C.load_i1 - C.load_i2)); // find internal
resistance causing voltage drop (sorta)
00613         set_load_relay_one(false);
00614         set_load_relay_two(false);
00615         esr_state = 0;
00616         ret = C.esr;
00617         break;
00618     default:
00619         break;
00620 }
00621 }
00622 return ret;
00623 }
00624
00625 /*
00626 * read analog signal from RS485 interface for charge controller status
00627 */
00628 uint8_t cc_state(float cc_signal)
00629 {
00630     uint8_t state = M_OFFLINE; // offline
00631
00632     if ((cc_signal >= 4.65) || (cc_signal < 0.75))
00633         return state; // dead signal range
00634
00635     if ((cc_signal >= 4.25) && (cc_signal < 4.65)) // 4.4
00636         return state; //offline
00637
00638     if ((cc_signal >= 3.75) && (cc_signal < 4.25)) // 4.0
00639         state = M_LIMIT;
00640     if ((cc_signal >= 3.25) && (cc_signal < 3.75)) // 3.5
00641         state = M_FLOAT;
00642     if ((cc_signal >= 2.75) && (cc_signal < 3.25)) // 3.0
00643         state = M_BOOST;
00644     if ((cc_signal >= 2.25) && (cc_signal < 2.75)) // 2.5
00645         state = M_EQUAL;
00646     if ((cc_signal >= 1.75) && (cc_signal < 2.25)) // 2.0
00647         state = M_MPPT;
00648     if ((cc_signal >= 1.25) && (cc_signal < 1.75)) // 1.5
00649         state = M_ACT;
00650     if ((cc_signal >= .75) && cc_signal < 1.25) // 1.0
00651         state = M_DEACT;
00652     return state;
00653 }

```


4.2 bsoc.h

```

00001 /* Microchip Technology Inc. and its subsidiaries. You may use this software
00002 * and any derivatives exclusively with Microchip products.
00003 *
00004 * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00005 * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
00006 * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
00007 * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
00008 * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
00009 *
00010 * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00011 * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00012 * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
00013 * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE
00014 * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
00015 * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
00016 * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
00017 *
00018 * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
00019 * TERMS.
00020 */
00021
00022 /*
00023 * File:
00024 * Author:
00025 * Comments:
00026 * Revision history:
00027 */
00028
00029 // This is a guard condition so that contents of this file are not included
00030 // more than once.
00031 #ifndef BSOC_H
00032 #define BSOC_H
00033
00034 #include <xc.h> // include processor files - each processor file is guarded.
00035 #include <math.h>
00036 #include <stdio.h>
00037 #include <string.h>
00038 #include "mcc_generated_files/tmr3.h"
00039 #include "vconfig.h"
00040 #include "mbmc.h"
00041 #include "eadog.h"
00042 #include "timers.h"
00043 #include "hid.h"
00044 #include "mconfig.h"
00045
00046 #define FM80_FIXUPS // correct current variables
00047
00048 #define MAXRUNTIME 95.0f
00049 #define MINRUNTIME 0.05f
00050 #define Perk_ADJ_FL 1.10f // If the peukert_adj is below this, adjust the Ah out by this Ah
00051 // adjustment factor
00052 #define Perk_ADJ_FH 5.00f // If the peukert_adj is above this, adjust the Ah out by this Ah
00053 // adjustment factor
00054 #define Temp_ADJ_FL 1.20f // Adjust the battery cap up or down 1C by this percentage from Temp_ZERO
00055 #define Temp_ZERO 2671 // Temperature for base battery rate capacity in tenths of C
00056 #define CEF_HSOC 0.45f // Charge Efficiency Factor, high SOC
00057 #define CEF_MSOC 0.70f // middle
00058 #define CEF_LSOC 0.90f // Charge Efficiency Factor, low SOC
00059 #define CEF_OVER 0.99f // CEF overflow in calc
00060 #define CEF_MAX 0.95f // Invalid charge factor above this, should be about the same as CEF_LOSC
00061 #define CEF_MIN 0.40f // Invalid charge factor below this
00062 #define CEF_RESET 0.80f // reset AH values below this point.
00063 #define CEF_DERATE 1.20f // Ahi derate factor to start dynamic CEF corrections
00064 #define SOC_DERATE 70 // derate CEF above this
00065 #define SOC_CEF 100 // Max state of charge to update dynamic charge efficiency factors
00066 #define HI_ESR 1500 // highest possible esr
00067
00068 #define BVSOC_SLOTS 26 // Battery to SOC data table slots
00069
00070 #define FLOAT_TIME 60 // cc in float mode delay time before flag set
00071
00072 /*
00073 * PV side diversion
00074 */
00075 #define DPWM_FULL 300
00076 #define DPWM_LOW_VOLTS 40.0f
00077 #define PV_MAX_POWER 300.0f
00078 #define PWM_MAX_POWER 250.0f
00079 #define PWM_WIN_POWER 10.0f
00080
00081 #define GTI_MAX 600 // 600
00082 #define GTI_IDLE 50
00083 #define GTI_INCR 100
00084 #define GTI_NORM 450 // 450

```

```

00083
00084 typedef enum {
00085     R_CYCLE = 0,
00086     R_TOTAL,
00087     R_PV,
00088     R_BAT,
00089 } R_CODES;
00090
00091 typedef enum {
00092     M_DEACT = 0,
00093     M_ACT = 1,
00094     M_MPPT = 2,
00095     M_EQUAL = 3,
00096     M_BOOST = 4,
00097     M_FLOAT = 5,
00098     M_LIMIT = 6,
00099     M_OFFLINE = 255,
00100 } M_CODES;
00101
00102 extern volatile C_data C;
00103 extern V_data V;
00104 extern H_data H;
00105 extern const char *build_date, *build_time;
00106
00107 void calc_bsoc(void);
00108 void init_bsoc(void);
00109 void start_bsoc(void);
00110 void stop_bsoc(void);
00111 void reset_bsoc(R_CODES);
00112 uint16_t Volts_to_SOC(uint32_t);
00113 uint32_t peukert(uint16_t, float, float, int16_t);
00114 float esr_check(uint8_t);
00115 uint8_t cc_state(float);
00116 bool pv_diversion(bool);
00117 void gti_cmds(void);
00118
00119 #endif /* BSOC_H */
00120

```

4.3 daq.c

```

00001 /*
00002  * 12-bit analog 64 sample average per channel on ports A and B
00003  * uses the compute burst average mode threshold interrupt to auto change
00004  * channels during interrupt after a repeat count
00005  * The end result is a ~13-bit ADC value
00006  */
00007
00008 #include <xc.h>
00009
00010 #include "daq.h"
00011
00012 typedef struct D_data {
00013     uint8_t dac0 : 8;
00014     uint8_t dac1 : 4;
00015     uint8_t cont : 4;
00016 } D_data;
00017
00018 union bytes2 {
00019     uint16_t ld;
00020     uint8_t bd[2];
00021 };
00022
00023 union dac_buf_type {
00024     uint16_t ld;
00025     uint8_t bd[2];
00026     struct D_data map;
00027 };
00028
00029 typedef struct R_data { // internal variables
00030     adc_result_t raw_adc[ADC_BUFFER_SIZE];
00031     adc_result_t raw_dac[DAC_BUFFER_SIZE];
00032     union dac_buf_type max5322_cmd;
00033     int16_t n_offset[NUM_C_SENSORS];
00034     float n_scalar[NUM_C_SENSORS];
00035     uint8_t scan_index;
00036     uint16_t scan_select;
00037     bool done;
00038     hist_type H;
00039     uint8_t hist_save : 1;
00040     uint8_t hist_update : 1;
00041     uint8_t c_zero_cal : 1;
00042     uint8_t c_scale_cal : 1;

```

```

00043     uint16_t checkmark;
00044     uint8_t crc;
00045 } R_data;
00046
00047 static volatile R_data R = {
00048     .done = false,
00049     .scan_index = 0,
00050 #ifdef BAT_100A
00051     .n_offset[A100B] = C_OFFSET100,
00052     .n_scalar[A100B] = C_A100,
00053 #else
00054     .n_offset[A200] = C_OFFSET200,
00055     .n_scalar[A200] = C_A200,
00056 #endif
00057     .n_offset[A100] = C_OFFSET100,
00058     .n_offset[A100M] = C_OFFSET100M,
00059     .n_scalar[A100] = C_A100,
00060     .n_scalar[A100M] = C_A100M,
00061     .raw_dac[DCHAN_A] = 0x0,
00062     .raw_dac[DCHAN_B] = 0x0,
00063     .checkmark = EE_CHECKMARK,
00064     .crc = TATE,
00065     .c_scale_cal = false,
00066     .c_zero_cal = false,
00067     .hist_save = false,
00068     .hist_update = false,
00069 };
00070
00071 static void adc_int_handler(void);
00072 static void adc_int_t_handler(void);
00073 static bool check_range(int16_t, int16_t, int16_t);
00074
00075 static R_data r_cal;
00076
00077 /*
00078  * start computed ADC results: 64 samples per average value per selected channel from
00079  * AN channel zero to LAST_ADC_CHAN
00080  * check_adc_scan returns true when sequence is complete
00081  */
00082 bool start_adc_scan(void)
00083 {
00084     bool ret;
00085     if (R.done) {
00086         ret = false;
00087     } else {
00088         R.scan_index = 0;
00089         R.scan_select = (uint16_t) ((ANSELB << 8) + ANSELA) & ADC_SCAN_CHAN; // skip digital pins PORT
00090         A and B
00091         ADCC_SetADIInterruptHandler(adc_int_handler);
00092         ADCC_SetADTIInterruptHandler(adc_int_t_handler);
00093         ADCC_DischargeSampleCapacitor(); // short ADC sample cap before channel sampling
00094         ADCC_StartConversion(R.scan_index & 0xf);
00095 #ifdef DEBUG_DAQ1
00096         DEBUG1_SetHigh();
00097 #endif
00098 #ifdef DEBUG_DAQ2
00099         DEBUG2_SetHigh();
00100 #else
00101         PIELbits.ADIE = 0;
00102 #endif
00103         ret = true;
00104     }
00105     return ret;
00106 }
00107
00108 /*
00109  * check scan done flag
00110  */
00111 bool check_adc_scan(void)
00112 {
00113     return R.done;
00114 }
00115
00116 /*
00117  * clear scan done flag
00118  */
00119 void clear_adc_scan(void)
00120 {
00121     R.done = false;
00122 }
00123
00124 /*
00125  * update the raw adc values
00126  */
00127 bool update_adc_result(void)
00128 {
00129     if (R.done) {

```

```

00129         clear_adc_scan();
00130         start_adc_scan();
00131         StartTimer(TMR_ADC, ADC_SCAN_SPEED);
00132         while (!TimerDone(TMR_ADC) && !check_adc_scan());
00133         return true;
00134     } else {
00135         return false;
00136     }
00137 }
00138
00139 /*
00140  * read average value of a channel after scan completion (done)
00141  */
00142 adc_result_t get_raw_result(const adcc_channel_t index)
00143 {
00144     static adc_result_t raw_result;
00145
00146     // stop ADC interrupt value updates
00147     PIE1bits.ADTIE = 0; // while getting raw values
00148     raw_result = R.raw_adc[index];
00149     // restore ADC enables
00150     PIE1bits.ADTIE = 1;
00151
00152     return raw_result;
00153 }
00154
00155 /*
00156  * turn ADC values into standard program values
00157  */
00158 float conv_raw_result(const adcc_channel_t chan, const adc_conv_t to_what)
00159 {
00160     switch (to_what) {
00161     case CONV:
00162         if (!(ADC_SCAN_CHAN » chan & 0x1))
00163             return NAN;
00164
00165         if (ADC_C_CHAN » chan & 0x1) { // current conversion
00166             if (ADC_C_CHAN_TYPE » chan & 0x1) {
00167 #ifdef BAT_100A
00168                 return((float) ((int16_t) get_raw_result(chan)) - R.n_offset[A100B]) *
00169                     R.n_scalar[A100B];
00170 #else
00171                 return((float) ((int16_t) get_raw_result(chan)) - R.n_offset[A200]) *
00172                     R.n_scalar[A200];
00173 #endif
00174             } else {
00175                 if (ADC_C_CHAN_MPPT » chan & 0x1) {
00176                     return((float) ((int16_t) get_raw_result(chan)) - R.n_offset[A100M]) *
00177                         R.n_scalar[A100M];
00178                 } else {
00179                     return((float) ((int16_t) get_raw_result(chan)) - R.n_offset[A100]) *
00180                         R.n_scalar[A100];
00181                 }
00182             }
00183         } else {
00184             if (ADC_T_CHAN » chan & 0x1) { // temp conversion
00185                 return 25.0; // filler until sensor is selected
00186             } else { // voltage conversion
00187                 if (ADC_V_CHAN_TYPE » chan & 0x01) {
00188                     return((float) get_raw_result(chan) * V_SCALE_H) / 1000.0f;
00189                 } else {
00190                     return((float) get_raw_result(chan) * V_SCALE) / 1000.0f;
00191                 }
00192             }
00193         }
00194     case O_CONV:
00195         if (ADC_C_CHAN » chan & 0x1 || ADC_T_CHAN » chan & 0x1)
00196             return((float) get_raw_result(chan) * C_SCALE) / 1000.0f;
00197
00198         if (ADC_V_CHAN_TYPE » chan & 0x01) {
00199             return((float) get_raw_result(chan) * V_SCALE_H) / 1000.0f;
00200         } else {
00201             return((float) get_raw_result(chan) * V_SCALE) / 1000.0f;
00202         }
00203     default:
00204         return 0.0f;
00205     }
00206     break;
00207 }
00208
00209 /*
00210  * ADC per conversion interrupt

```

```

00212  */
00213  static void adc_int_handler(void)
00214  {
00215  #ifdef DEBUG_DAQ2
00216      DEBUG2_Toggle();
00217  #endif
00218  }
00219
00220  /*
00221  * ADC per channel average interrupt
00222  */
00223  static void adc_int_t_handler(void)
00224  {
00225      /*
00226       * use the filter result buffer for raw adc data
00227       */
00228      R.raw_adc[R.scan_index] = ((adc_result_t) ((ADFLTRH « 8) + ADFLTRL));
00229      do {
00230          if (++R.scan_index > LAST_ADC_CHAN) {
00231              R.done = true;
00232          #ifdef DEBUG_DAQ1
00233              DEBUG1_SetLow();
00234          #endif
00235              return;
00236          } while (!((R.scan_select » R.scan_index) & 0x1)); // check for analog port bit
00238          ADCC_DischargeSampleCapacitor(); // short ADC sample cap before next channel sampling
00239          ADCC_StartConversion(R.scan_index & 0xf);
00240          V.adc_count++;
00241          #ifdef DEBUG_DAQ1
00242              DEBUG1_SetHigh();
00243          #endif
00244      }
00245
00246  /*
00247  * set == true, set dac spi params
00248  * set == false restore default spi params
00249  */
00250  void dac_spi_control(bool set)
00251  {
00252      static bool init = false;
00253      static uint8_t S0, S1, S2, SC, SB; // SPI device status backup
00254
00255      if (set) {
00256          SPI1CON0bits.EN = 0;
00257          if (!init) {
00258              init = true;
00259              S0 = SPI1CON0;
00260              S1 = SPI1CON1;
00261              S2 = SPI1CON2;
00262              SC = SPI1CLK;
00263              SB = SPI1BAUD;
00264          }
00265          /*
00266           * set DAC SPI mode, speed and fifo
00267           */
00268          // mode 1
00269          SPI1CON1 = 0x00;
00270          SPI1CON1bits.CKE = 1;
00271          SPI1CON1bits.CKP = 0;
00272          SPI1CON1bits.SMP = 0;
00273          // SSET disabled; RXR suspended if the Rx FIFO is full; TXR required for a transfer;
00274          SPI1CON2 = 0x03;
00275          // BAUD 0;
00276          SPI1BAUD = 0x0f; // 2MHz SCK
00277          // CLKSEL FOSC;
00278          SPI1CLK = 0x00;
00279          // BMODE every byte; LSBF MSb first; EN enabled; MST bus master;
00280          SPI1CON0 = 0x83;
00281          SPI1CON0bits.EN = 1;
00282      } else {
00283          if (init) {
00284              /*
00285               * restore default SPI mode
00286               */
00287              SPI1CON0bits.EN = 0;
00288              SPI1CON1 = S1;
00289              SPI1CON2 = S2;
00290              SPI1CLK = SC;
00291              SPI1BAUD = SB;
00292              SPI1CON0 = S0;
00293              SPI1CON0bits.EN = 1;
00294          }
00295      }
00296  }
00297
00298  void set_dac(void)

```

```

00299 {
00300     while (!SPI1STATUSbits.TXBE); // wait until TX buffer is empty
00301     CSB_SetHigh();
00302     CS_SDCARD_SetHigh();
00303     dac_spi_control(true);
00304     R.max5322_cmd.map.dac0 = R.raw_dac[DCHAN_A]&0xff;
00305     R.max5322_cmd.map.dac1 = (R.raw_dac[DCHAN_A] » 8) &0xf;
00306     R.max5322_cmd.map.cont = DAC_LOAD_A; // update DAC A @ registers
00307     DAC_CS0_SetLow();
00308     SPI1_ExchangeByte(R.max5322_cmd.bd[1]);
00309     SPI1_ExchangeByte(R.max5322_cmd.bd[0]);
00310     DAC_CS0_SetHigh();
00311     R.max5322_cmd.map.dac0 = R.raw_dac[DCHAN_B]&0xff;
00312     R.max5322_cmd.map.dac1 = (R.raw_dac[DCHAN_B] » 8) &0xf;
00313     R.max5322_cmd.map.cont = DAC_LOAD_B; // update DAC B @ registers
00314     DAC_CS0_SetLow();
00315     SPI1_ExchangeByte(R.max5322_cmd.bd[1]);
00316     SPI1_ExchangeByte(R.max5322_cmd.bd[0]);
00317     while (!SPI1STATUSbits.TXBE); // wait until TX buffer is empty
00318     DAC_CS0_SetHigh();
00319     dac_spi_control(false);
00320 }
00321
00322 static uint16_t convert_dac_raw(float voltage)
00323 {
00324     /*
00325      * check limits
00326      */
00327     if (voltage < 0.001f)
00328         voltage = 0.001f;
00329     if (voltage > 10.0f)
00330         voltage = 10.0f;
00331     /*
00332      * scale to DAC units
00333      */
00334     return(uint16_t) (voltage / DAC_SCALE);
00335 }
00336
00337 /*
00338 * 0.0 to 10.0 volts
00339 */
00340 uint16_t set_dac_a(const float voltage)
00341 {
00342     R.raw_dac[DCHAN_A] = convert_dac_raw(voltage);
00343     return R.raw_dac[DCHAN_A];
00344 }
00345
00346 /*
00347 * 0.0 to 10.0 volts
00348 */
00349 uint16_t set_dac_b(const float voltage)
00350 {
00351     R.raw_dac[DCHAN_B] = convert_dac_raw(voltage);
00352     return R.raw_dac[DCHAN_B];
00353 }
00354
00355 /*
00356 * sanity check values in a range
00357 */
00358 static bool check_range(const int16_t value, const int16_t window, const int16_t standard)
00359 {
00360     if (value > (standard + window))
00361         return false;
00362     if (value < (standard - window))
00363         return false;
00364     return true;
00365 }
00366
00367 /*
00368 * mode set to false for check values only, do not update
00369 */
00370 bool cal_current_zero(const bool mode, const int16_t cb, const int16_t cp, const int16_t cm)
00371 {
00372     if (!check_range(cb, ZERO_RANGE, C_CAL_ZERO))
00373         return false;
00374     if (!check_range(cp, ZERO_RANGE, C_CAL_ZERO))
00375         return false;
00376     if (!check_range(cm, ZERO_RANGE, C_CAL_ZERO))
00377         return false;
00378     if (!mode)
00379         return true;
00380     return true;
00381 }
00382
00383 #ifdef BAT_100A
00384
00385

```

```

00386     R.n_offset[A100B] = cb;
00387 #else
00388     R.n_offset[A200] = cb;
00389 #endif
00390     R.n_offset[A100] = cp;
00391     R.n_offset[A100M] = cm;
00392     R.c_zero_cal = true;
00393     return true;
00394 }
00395
00396 /*
00397  * update internal current scaling using a calibrated 10A value in both sensors
00398  */
00399 bool cal_current_10A(const bool mode, const int16_t cb, const int16_t cp, const float scaleb, const
float scalep)
00400 {
00401 #ifdef BAT_100A
00402     if (!check_range(cb, TEN_A_RANGE, C_CAL_A100))
00403 #else
00404     if (!check_range(cb, TEN_A_RANGE, C_CAL_A200))
00405 #endif
00406     return false;
00407
00408     if (!check_range(cp, TEN_A_RANGE, C_CAL_A100))
00409     return false;
00410
00411     if (!mode)
00412     return true;
00413
00414 #ifdef BAT_100A
00415     R.n_scalar[A100B] = scaleb;
00416 #else
00417     R.n_scalar[A200] = scaleb;
00418 #endif
00419     R.n_scalar[A100] = scalep;
00420     R.n_scalar[A100M] = scalep;
00421     R.c_scale_cal = true;
00422     return true;
00423 }
00424
00425 /*
00426  * read eeprom into temp program variable buffer
00427  */
00428 bool read_cal_data(void)
00429 {
00430     uint16_t x = 0, y;
00431     uint8_t *r_cal_ptr, crcVal_save;
00432
00433     y = sizeof(r_cal);
00434     r_cal_ptr = (uint8_t*) & r_cal;
00435     // Initialize the CRC module
00436     CRC_Initialize();
00437
00438     // Start CRC
00439     CRC_Start();
00440
00441     do {
00442         r_cal_ptr[x] = DATAEE_ReadByte(x);
00443     } while (++x < y);
00444     crcVal_save = r_cal.crc;
00445
00446     if (r_cal.checkmark == EE_CHECKMARK) {
00447         r_cal.crc = TATE; // standard CRC filler code
00448         x = 0;
00449
00450         do {
00451             CRC_8BitDataWrite(r_cal_ptr[x]);
00452             while (CRC_IsBusy());
00453         } while (++x < y);
00454         if (CRC_CalculatedResultGet(NORMAL, 0x00) != crcVal_save)
00455             return false;
00456     } else {
00457         return false;
00458     }
00459     r_cal.crc = crcVal_save; // reload actual eeprom CRC
00460     return true;
00461 }
00462
00463 /*
00464  * write program variables to eeprom
00465  */
00466 void write_cal_data(void)
00467 {
00468     uint16_t x = 0, y;
00469     uint8_t *r_cal_ptr, crcVal;
00470
00471     y = sizeof(R);

```

```

00472     R.crc = TATE;
00473     r_cal_ptr = (uint8_t*) & R;
00474     R.checkmark = EE_CHECKMARK;
00475     // Initialize the CRC module
00476     CRC_Initialize();
00477
00478     // Start CRC
00479     CRC_Start();
00480
00481     do {
00482         DATAEE_WriteByte(x, r_cal_ptr[x]);
00483         CRC_8BitDataWrite(r_cal_ptr[x]);
00484         while (CRC_IsBusy());
00485     } while (++x < y);
00486     // Read CRC check value
00487     crcVal = (uint8_t) CRC_CalculatedResultGet(NORMAL, 0x00);
00488     // store crc in EEPROM
00489     DATAEE_WriteByte(sizeof(R) - 1, crcVal);
00490 }
00491
00492 /*
00493  * copy program variable buffer into active variable
00494  */
00495 void update_cal_data(void)
00496 {
00497     R = r_cal;
00498     if (!R.c_zero_cal) {
00499 #ifdef BAT_100A
00500         R.n_offset[A100B] = C_OFFSET100;
00501 #else
00502         R.n_offset[A200] = C_OFFSET200;
00503 #endif
00504         R.n_offset[A100] = C_OFFSET100;
00505         R.n_offset[A100M] = C_OFFSET100M;
00506     }
00507
00508     if (!R.c_scale_cal) {
00509 #ifdef BAT_100A
00510         R.n_scalar[A100B] = C_A100;
00511 #else
00512         R.n_scalar[A200] = C_A200;
00513 #endif
00514         R.n_scalar[A100] = C_A100;
00515         R.n_scalar[A100M] = C_A100M;
00516     }
00517 }
00518
00519 /*
00520  * mode true: copy local hist data to operational history
00521  * mode false: copy operational history to local buffer
00522  */
00523 bool update_hist_data(const bool mode, volatile hist_type *hist)
00524 {
00525     if (hist == NULL)
00526         return false;
00527
00528     if (mode) {
00529         if (R.hist_save) {
00530             *hist = R.H;
00531             if (R.hist_update) { // we should have a valid history time to load
00532                 return true;
00533             }
00534         }
00535     } else {
00536         R.H = *hist;
00537         R.hist_save = true;
00538     }
00539     return false;
00540 }
00541
00542 void set_hist_flag(void)
00543 {
00544     R.hist_update = true;
00545 }

```

4.4 daq.h

```

00001 /* Microchip Technology Inc. and its subsidiaries. You may use this software
00002  * and any derivatives exclusively with Microchip products.
00003  *
00004  * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00005  * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
00006  * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A

```



```

00007  * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
00008  * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
00009  *
00010  * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00011  * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00012  * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
00013  * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE
00014  * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
00015  * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
00016  * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
00017  *
00018  * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
00019  * TERMS.
00020  */
00021
00022 /*
00023  * REMOVE THE CHIP PROGRAMMER, IT CHANGES THE ADC READINGS
00024  */
00025 /*
00026  * 12-bit analog 64 sample average per channel on ports A and B
00027  * uses the compute burst average mode threshold interrupt to auto change
00028  * channels during interrupt after a repeat count
00029  */
00030
00031 // This is a guard condition so that contents of this file are not included
00032 // more than once.
00033 #ifndef DAQ_H
00034 #define DAQ_H
00035
00036 #include <math.h>
00037 #include "mcc_generated_files/spi1.h"
00038 #include "mcc_generated_files/crc.h"
00039 #include "vconfig.h"
00040 #include "timers.h"
00041 #include "tests.h"
00042
00043 extern struct V_data V;
00044 /*
00045  * adc channel configuration
00046  */
00047 #define LAST_ADC_CHAN 0XD
00048 #define ADC_BUFFER_SIZE 16 // MUST BE AT LEAST 1
00049 #define ADC_SCAN_SPEED 200 // sample timer speed in ms
00050 #define ADC_SCAN_CHAN 0b0011110001110111 // convert these analog ports bitmap
00051
00052 /*
00053  * Normal or high volt channels (bit set)
00054  */
00055 #define ADC_V_CHAN_TYPE 0b0010000000100000
00056
00057 /*
00058  * current sensor configuration
00059  */
00060 // #define BAT_100A // USE the 100A honeywell sensor for battery current, reduces range but has better
// data within that range
00061 #define NUM_C_SENSORS 3
00062 #define ADC_C_CHAN 0b0000100000000001 // 5 volt hall current sensor bitmap
00063 #define ADC_C_CHAN_TYPE 0b0000000000000001 // 0 - 100A, 1=200A or battery 100A type if BAT_100A is
// defined
00064 #define ADC_C_CHAN_MPPT 0b0000100000000000 // 0 - 100A mppt channel
00065 /*
00066  * temp sensor configuration
00067  */
00068 #define NUM_T_SENSORS 0
00069 #define ADC_T_CHAN 0b0000000000000000 // 5 volt temp sensor bitmap
00070 #define ADC_T_CHAN_TYPE 0b0000000000000000 // type bitmap
00071
00072 /*
00073  * dac channel configuration
00074  */
00075 #define LAST_DAC_CHAN 0X1
00076 #define DAC_BUFFER_SIZE 2 // MUST BE AT LEAST 1
00077 #define DAC_LOAD_A 0b0100
00078 #define DAC_LOAD_B 0b0101
00079 #define DCHAN_A 0
00080 #define DCHAN_B 1
00081 #define DAC_SCALE 0.002442f // 10 vdc @ 12-bits resolution per bit
00082
00083 #define ZERO_RANGE 100
00084 #define TEN_A_RANGE 100
00085
00086 #define EE_CHECKMARK 0x1957
00087 #define TATE 0x42
00088
00089 /*
00090  * current sensor configuration
00091  */

```

```

00092 //define CAL_10A // set 10A scaling
00093 //define CAL_TWO // dupe the PV sensor results into the MPPT calibration table
00094
00095 /*
00096  * conversion constants
00097  */
00098 /*
00099  * REMOVE THE CHIP PROGRAMMER, IT CHANGES THE ADC READINGS
00100  */
00101 /*
00102  * 13-bit adc result ADRPT 64 samples and a ADCRS of 5 [0..8190] steps
00103  */
00104 #define C_SCALE      0.6250000f
00105 #define V_SCALE      4.1254125f
00106 #define V_SCALE_H    8.2000000f
00107
00108 #ifndef BAT_100A
00109 #define A200          0 // BATTERY,  adc line 0
00110 #endif
00111 #define A100B         0 // BATTERY,  adc line 0
00112 #define A100          1 // CV,        adc line 1
00113 #define A100M         2 // PV,        adc line 1
00114
00115 #ifndef BAT_100A
00116 #define C_A200        0.0361010f // BATTERY Amp scalar, second line [0], 0.0862000f, BO PLUG
00117 #endif
00118 #define C_A100B       0.0361010f // PV Amp scalar, second line [0]
00119 #define C_A100        0.0361010f // PV Amp scalar, first line [1]
00120 #define C_A100M       0.0862000f // MPPT Amp scalar, first line [2]
00121
00122 #ifndef BAT_100A
00123 #define C_OFFSET200 3965 // BATTERY sensor adc zero Amp counts, line 0,  BO sensor PLUG, orig value
3965
00124 #endif
00125 #define C_OFFSET100B 3934 // BATTERY sensor adc zero Amp counts, line 0
00126 #define C_OFFSET100 3995 // PV sensor adc zero Amp counts, line 1
00127 #define C_OFFSET100M 3939 // MPPT sensor adc zero Amp counts, line 2, Orig value 3934, adjust
this for DL bat current zero
00128
00129 /*
00130  * sanity check values for current calibration routines
00131  */
00132 #define C_CAL_ZERO 4000
00133 #ifndef BAT_100A
00134 #define C_CAL_A200 4150 // fixme with real values
00135 #endif
00136 #define C_CAL_A100 4240 // ..
00137
00138 #include <xc.h> // include processor files - each processor file is guarded.
00139 #include "mcc_generated_files/adcc.h"
00140 #include "mcc_generated_files/pin_manager.h"
00141 #include "mcc_generated_files/memory.h"
00142
00143 typedef enum {
00144     CONV, // auto system conversion
00145     O_CONV, // convert to voltage for calibration/offsets
00146 } adc_conv_t;
00147
00148 bool start_adc_scan(void);
00149 bool check_adc_scan(void);
00150 void clear_adc_scan(void);
00151 bool update_adc_result(void);
00152 adc_result_t get_raw_result(const adcc_channel_t);
00153 float conv_raw_result(const adcc_channel_t, const adc_conv_t);
00154 void dac_spi_control(bool);
00155 void set_dac(void);
00156 uint16_t set_dac_a(float);
00157 uint16_t set_dac_b(float);
00158 bool cal_current_zero(bool, int16_t, int16_t, int16_t);
00159 bool cal_current_10A(bool, int16_t, int16_t, float, float);
00160 bool read_cal_data(void);
00161 void write_cal_data(void);
00162 void update_cal_data(void);
00163 bool update_hist_data(bool, volatile hist_type*);
00164 void set_hist_flag(void);
00165 #endif
00166

```

4.5 dio.c

```

00001 #include "dio.h"
00002 #include "mcc_generated_files/ext_int.h"
00003

```

```

00004 typedef struct S_data { // switch control data structure
00005     volatile rbutton_type button[MAX_BUTTONS];
00006     volatile uint8_t sw_bitmap;
00007 } S_data;
00008
00009 static S_data S;
00010 extern V_data V;
00011
00012 static void switch_handler(void);
00013
00014 /*
00015  * return the current actual state of x switch
00016  * the actual current state of a switch is ONLY changed in the interrupt handler
00017  */
00018 SW_STATES get_switch(const uint8_t i)
00019 {
00020     if (i >= NUM_SWITCHES)
00021         return SW_OFF;
00022
00023     return S.button[i].sw;
00024 }
00025
00026 /*
00027  * return the bitmap state of x switch
00028  * the actual current state of a switch is ONLY changed in the interrupt handler
00029  */
00030 SW_STATES get_switch_bm(const uint8_t i)
00031 {
00032     if (i >= NUM_SWITCHES)
00033         return SW_OFF;
00034
00035     return (S.sw_bitmap >> i) & 0x01;
00036 }
00037
00038 /*
00039  * return the x switch structure
00040  * mainly for testing
00041  */
00042 rbutton_type get_switch_data(const uint8_t i)
00043 {
00044     if (i >= NUM_SWITCHES)
00045         return S.button[SNULL];
00046
00047     return S.button[i];
00048 }
00049
00050 /*
00051  * check the switch pressed bitmap for button presses, the actual current switch state might differ
00052  */
00053 uint8_t check_switches(void)
00054 {
00055     return S.sw_bitmap;
00056 }
00057
00058 /*
00059  * clear X switch pressed and time pressed data
00060  */
00061 void clear_switch(const uint8_t i)
00062 {
00063     if (i >= NUM_SWITCHES)
00064         return;
00065
00066     S.button[i].count = 0;
00067     S.sw_bitmap &= ~(1 << i); //clear switch pressed bit
00068 }
00069
00070 void switch_handler(void)
00071 {
00072     uint8_t i = 0, sw_value;
00073
00074     /*
00075      * enable the outputs for reading and reset MAX Change-of-State pin
00076      */
00077     MAX_EN1_SetLow(); // rev 1.1+ board
00078     V.blight = V.ticks + BL_TIME;
00079     V.blight_off = false;
00080
00081 #ifdef DEBUG_SWH1
00082     DEBUG1_SetHigh();
00083 #endif
00084
00085     // Nop for MAX chip output Propagation Delay after enable
00086     V.switch_count++;
00087     Nop();
00088     Nop();
00089     Nop();
00090     Nop();

```

```

00091     Nop();
00092     Nop();
00093     // start reading the various pic port input bits after the max chip is ready
00094     do {
00095         switch (i) {
00096             case SENTER:
00097                 sw_value = ENTER_B_GetValue();
00098                 break;
00099             case SSELECT:
00100                 sw_value = SELECT_B_GetValue();
00101                 break;
00102             case SCALIB:
00103                 sw_value = CALIB_GetValue();
00104                 break;
00105             case S1:
00106                 sw_value = 1;
00107                 break;
00108             case S0:
00109                 sw_value = 1;
00110                 break;
00111             default:
00112                 sw_value = 1;
00113                 break;
00114         }
00115
00116         // update actual current button state
00117         if (sw_value) {
00118             if (S.button[i].sw == SW_ON) {
00119                 S.button[i].sw = SW_OFF;
00120             }
00121         } else {
00122             if (S.button[i].sw == SW_OFF) {
00123                 S.button[i].sw = SW_ON;
00124                 S.button[i].count = V.timerint_count; // so we can check button SW_ON duration
00125                 S.sw_bitmap |= 1 << i; // set switch pressed bit
00126             }
00127         }
00128     } while (++i < NUM_SWITCHES);
00129
00130 #ifdef DEBUG_SWH1
00131     DEBUG1_SetLow();
00132 #endif
00133 #ifdef DEBUG_SWH2
00134     DEBUG2_Toggle();
00135 #endif
00136
00137     MAX_EN1_SetHigh(); // disable MAX output pins
00138 }
00139
00140 void start_switch_handler(void)
00141 {
00142     EXT_INT1_InterruptDisable();
00143     INT1_SetInterruptHandler(switch_handler);
00144     S.button[SNULL].sw = SW_INVALID; // set a error condition for invalid button number
00145     EXT_INT1_InterruptEnable();
00146 }

```

4.6 dio.h

```

00001 /* Microchip Technology Inc. and its subsidiaries. You may use this software
00002  * and any derivatives exclusively with Microchip products.
00003  *
00004  * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00005  * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
00006  * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
00007  * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
00008  * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
00009  *
00010  * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00011  * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00012  * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
00013  * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE
00014  * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
00015  * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
00016  * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
00017  *
00018  * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
00019  * TERMS.
00020  */
00021
00022 /*
00023  * File:
00024  * Author:

```

```

00025  * Comments:
00026  * Revision history:
00027  */
00028
00029 // This is a guard condition so that contents of this file are not included
00030 // more than once.
00031 #ifndef DIO_H
00032 #define DIO_H
00033
00034 #include <xc.h> // include processor files - each processor file is guarded.
00035 #include "vconfig.h"
00036 #include "tests.h"
00037
00038 #define NUM_SWITCHES      6
00039
00040 typedef struct rbutton_type {
00041     SW_STATES get_switch(const uint8_t);
00042     uint32_t count;
00043 } rbutton_type;
00044
00045 void start_switch_handler(void);
00046 SW_STATES get_switch(const uint8_t);
00047 SW_STATES get_switch_bm(const uint8_t);
00048 rbutton_type get_switch_data(uint8_t);
00049 uint8_t check_switches(void);
00050 void clear_switch(uint8_t);
00051
00052 #endif /* DIO_H */
00053

```

4.7 eadog.c

```

00001 #include <string.h>
00002 #include "vconfig.h"
00003 #include "eadog.h"
00004 #include "ringbufs.h"
00005 #include "tests.h"
00006 #include "mcc_generated_files/mcc.h"
00007
00008 #define max_strlen 21
00009 // #define max_port_data 1024
00010
00011 struct spi_link_type spi_link;
00012 static uint8_t port_data[max_port_data] = {255, 0, 255, 0, 255, 0, 255, 0, 255, 0, 255, 0,
255, 0};
00013
00014 static void send_lcd_cmd_long(uint8_t); // for display init only
00015 static void send_lcd_data(uint8_t);
00016 static void send_lcd_cmd(uint8_t);
00017 static void spi_byte(void);
00018
00019 /*
00020  * in DMA mode this is a ISR that runs when the source count is complete
00021  */
00022 void clear_lcd_done(void)
00023 {
00024     spi_link.LCD_DATA = false;
00025 }
00026
00027 void spi_rec_done(void)
00028 {
00029     #ifdef USE_LCD_DMA
00030     #endif
00031 }
00032
00033 /*
00034  * SPI buffer overrun ISR
00035  */
00036 static void spi_byte(void)
00037 {
00038 }
00039
00040 void wdtdelay(const uint32_t delay)
00041 {
00042     static uint32_t dcount;
00043
00044     for (dcount = 0; dcount <= delay; dcount++) { // delay a bit
00045         Nop();
00046         ClrWdt(); // reset the WDT timer
00047     };
00048 }
00049
00050 /*

```

```

00051  * Init the NHD-0420D3Z-NSW-BBW-V3 in 8-bit serial mode
00052  * channel 1 DMA
00053  */
00054 void init_display(void)
00055 {
00056     spi_link.txbuf = lcd_dma_buf;
00057
00058     #ifdef DEBUG_DISP2
00059         DLED2 = true;
00060     #endif
00061
00062     #ifndef USEMCC_SPI
00063         SPI1CON0bits.EN = 0;
00064         // SSET disabled; RXR suspended if the RxFIFO is full; TXR required for a transfer;
00065         SPI1CON2 = 0x03;
00066         // BMODE every byte; LSBF MSb first; EN enabled; MST bus master;
00067         SPI1CON0 = 0x83;
00068         SPI1CON0bits.EN = 1;
00069     #endif
00070
00071
00072     #ifdef USE_LCD_DMA
00073         DMA1_SetSCNTInterruptHandler(clear_lcd_done);
00074         DMA1_SetORInterruptHandler(spi_byte);
00075         DMA1_SetDMAPriority(2);
00076     #endif
00077
00078     wdtdelay(LCD_PWR_DELAY); // > 400ms power up delay
00079
00080     #ifdef USE_LCD_DMA
00081         SPI1INTFbits.SPI1TXUIF = 0;
00082         DMA1CON0bits.EN = 0;
00083         SPI1CON0bits.EN = 0;
00084         SPI1CON2 = 0x02; // Received data is not stored in the FIFO
00085         SPI1CON0bits.EN = 1;
00086         DMA1CON1bits.DMODE = 0;
00087         DMA1CON1bits.DSTP = 0;
00088         DMA1CON1bits.SMODE = 1;
00089         DMA1CON1bits.SMR = 0;
00090         DMA1CON1bits.SSTP = 1;
00091         DMA1SSA = (uint24_t) spi_link.txbuf;
00092         DMA1CON0bits.DGO = 0;
00093         DMA1CON0bits.EN = 1; /* enable DMA */
00094         SPI1INTFbits.SPI1TXUIF = 1;
00095         send_lcd_cmd_dma(LCD_CMD_BRI); // set back-light level
00096         send_lcd_data_dma(NHD_BL_MED);
00097         send_lcd_cmd_dma(LCD_CMD_CONT); // set display contrast
00098         send_lcd_data_dma(NHD_CONT);
00099         send_lcd_cmd_dma(LCD_CMD_ON); // display on
00100         send_lcd_cmd_dma(LCD_CMD_CLR); // clear screen
00101         wdtdelay(NHD_L_DELAY);
00102         DMA1_StopTransfer();
00103     #else
00104         SPI1CON0bits.EN = 0;
00105         // mode 3
00106         SPI1CON1 = 0x20;
00107         // SSET disabled; RXR suspended if the RxFIFO is full; TXR required for a transfer;
00108         SPI1CON2 = 0x03;
00109         // BAUD 0;
00110         SPI1BAUD = 0x04; // 50kHz SCK
00111         // CLKSEL MFINTOSC;
00112         SPI1CLK = 0x02;
00113         // BMODE every byte; LSBF MSb first; EN enabled; MST bus master;
00114         SPI1CON0 = 0x83;
00115         SPI1CON0bits.EN = 1;
00116         send_lcd_cmd(LCD_CMD_BRI); // set back-light level
00117         send_lcd_data(NHD_BL_LOW);
00118         send_lcd_cmd(LCD_CMD_CONT); // set display contrast
00119         send_lcd_data(NHD_CONT);
00120         send_lcd_cmd(LCD_CMD_OFF); // display on
00121         send_lcd_cmd(LCD_CMD_CLR); // clear screen
00122         wdtdelay(NHD_L_DELAY);
00123         DMA1_StopTransfer();
00124     #endif
00125
00126     #ifdef DEBUG_DISP2
00127         DLED2 = false;
00128     #endif
00129 }
00130
00131 /*
00132  * channel 2 DMA, serial port 1 transmit
00133  */
00134 void init_port_dma(void)
00135 {
00136     DMA2CON1bits.DMODE = 0;
00137     DMA2CON1bits.DSTP = 0;

```

```

00138     DMA2CON1bits.SMODE = 1;
00139     DMA2CON1bits.SMR = 0;
00140     DMA2CON1bits.SSTP = 1;
00141     DMA2CON0bits.SIRQEN = 0;
00142     DMA2DSA = 0x3DEA; // U1TXB SERIAL PORT 1
00143     DMA2SSA = (uint24_t) port_data;
00144     DMA2CON0bits.DGO = 0;
00145 }
00146
00147 #ifdef NHD
00148
00149 /*
00150  * R2 short on LCD NHD-0420D3Z-NSW-BBW-V3 board
00151  */
00152
00153 static void send_lcd_data(const uint8_t data)
00154 {
00155     CSB_SetLow();
00156     SPI1_ExchangeByte(data);
00157     wdtdelay(8);
00158 }
00159
00160 static void send_lcd_cmd(const uint8_t cmd)
00161 {
00162     CSB_SetLow();
00163     SPI1_ExchangeByte(NHD_CMD);
00164     wdtdelay(8);
00165     SPI1_ExchangeByte(cmd);
00166     wdtdelay(8);
00167 }
00168
00169 static void send_lcd_cmd_long(const uint8_t cmd)
00170 {
00171     CSB_SetLow();
00172     SPI1_ExchangeByte(NHD_CMD);
00173     wdtdelay(8);
00174     SPI1_ExchangeByte(cmd);
00175     wdtdelay(800);
00176 }
00177
00178 /*
00179  * CAN use DMA channel 1 for transfers
00180  */
00181 void eaDogM_WriteString(char *strPtr)
00182 {
00183     uint8_t len = (uint8_t) strlen(strPtr);
00184
00185     wait_lcd_done();
00186     wait_lcd_set();
00187     CSB_SetLow(); /* SPI select display */
00188     if (len > (uint8_t) max_strlen) {
00189         len = max_strlen;
00190     }
00191     memset(spi_link.txbuf, ' ', max_strlen + 1);
00192     memcpy(spi_link.txbuf, strPtr, len);
00193 #ifdef USE_LCD_DMA
00194     DMA1CON0bits.EN = 0; /* disable DMA to change source count */
00195     DMA1_SetSourceSize(len);
00196     DMA1_SetDestinationSize(1);
00197     DMA1CON0bits.EN = 1; /* enable DMA */
00198 #else
00199     SPI1_ExchangeBlock(spi_link.txbuf, len);
00200 #endif
00201     start_lcd(); // start DMA transfer
00202 }
00203
00204 /*
00205  * uses DMA channel 1 for transfers
00206  */
00207 void send_lcd_cmd_dma(const uint8_t strPtr)
00208 {
00209     send_lcd_data_dma(NHD_CMD); //prefix
00210     send_lcd_data_dma(strPtr); // cmd code
00211 }
00212
00213 /*
00214  * uses DMA channel 1 for transfers
00215  */
00216 void send_lcd_data_dma(const uint8_t strPtr)
00217 {
00218     wait_lcd_done();
00219     wait_lcd_set();
00220     CSB_SetLow(); /* SPI select display */
00221     spi_link.txbuf[0] = strPtr;
00222     DMA1CON0bits.EN = 0; /* disable DMA to change source count */
00223     DMA1_SetSourceSize(1);
00224     DMA1_SetDestinationSize(1);

```

```

00225     DMA1CON0bits.EN = 1; /* enable DMA */
00226     start_lcd(); // start DMA transfer
00227 }
00228
00229 /*
00230  * send three byte command string via DMA
00231  */
00232 void send_lcd_pos_dma(const uint8_t strPtr)
00233 {
00234     wait_lcd_done();
00235     wait_lcd_set();
00236     CSB_SetLow(); /* SPI select display */
00237     spi_link.txbuf[0] = NHD_CMD;
00238     spi_link.txbuf[1] = NHD_POS;
00239     spi_link.txbuf[2] = strPtr;
00240     DMA1CON0bits.EN = 0; /* disable DMA to change source count */
00241     DMA1_SetSourceSize(3);
00242     DMA1_SetDestinationSize(1);
00243     DMA1CON0bits.EN = 1; /* enable DMA */
00244     start_lcd(); // start DMA transfer
00245 }
00246
00247 void eaDogM_WriteStringAtPos(const uint8_t r, const uint8_t c, char *strPtr)
00248 {
00249     uint8_t row;
00250
00251     switch (r) {
00252     case LCD1:
00253         row = 0x40;
00254         break;
00255     case LCD2:
00256         row = 0x14;
00257         break;
00258     case LCD3:
00259         row = 0x54;
00260         break;
00261     case LCD0:
00262         row = 0x00;
00263         break;
00264     default:
00265         row = 0x00;
00266         break;
00267     }
00268
00269 #ifdef USE_LCD_DMA
00270     send_lcd_pos_dma(row + c);
00271     wdtdelay(NHD_S_DELAY); // display command processing delay
00272 #else
00273     send_lcd_cmd(0x45);
00274     send_lcd_data(row + c);
00275 #endif
00276     eaDogM_WriteString(strPtr);
00277 }
00278
00279 void eaDogM_WriteIntAtPos(uint8_t r, uint8_t c, uint8_t i)
00280 {
00281
00282 }
00283
00284 void eaDogM_SetPos(const uint8_t r, const uint8_t c)
00285 {
00286
00287 }
00288
00289 void eaDogM_ClearRow(const uint8_t r)
00290 {
00291
00292 }
00293
00294 void eaDogM_WriteByteToCGRAM(uint8_t ndx, uint8_t data)
00295 {
00296
00297 }
00298
00299 #else
00300
00301 /*
00302  * add short spi delay (default)
00303  */
00304 static void send_lcd_data(const uint8_t data)
00305 {
00306     RS_SetHigh();
00307     CSB_SetLow();
00308     SPI1_ExchangeByte(data);
00309     wdtdelay(8);
00310 }
00311

```



```

00312 /*
00313  * add inst spi delay
00314 */
00315 static void send_lcd_cmd(const uint8_t cmd)
00316 {
00317     RS_SetLow();
00318     CSB_SetLow();
00319     SPI1_ExchangeByte(cmd);
00320     wdtdelay(30);
00321     RS_SetHigh();
00322 }
00323
00324 /*
00325  * add clear/home spi delay
00326 */
00327 static void send_lcd_cmd_long(const uint8_t cmd)
00328 {
00329     RS_SetLow();
00330     CSB_SetLow();
00331     SPI1_ExchangeByte(cmd);
00332     wdtdelay(800);
00333     RS_SetHigh();
00334 }
00335
00336 void eaDogM_SetPos(const uint8_t r, const uint8_t c)
00337 {
00338     uint8_t cmdPos;
00339     cmdPos = (uint8_t) EADOGM_CMD_DDRAM_ADDR + (uint8_t) ((uint8_t) r * (uint8_t) EADOGM_COLSPAN) +
00340     (uint8_t) c;
00341     eaDogM_WriteCommand(cmdPos);
00342 }
00343 void eaDogM_ClearRow(const uint8_t r)
00344 {
00345     uint8_t i;
00346     eaDogM_SetPos(r, 0);
00347     for (i = 0; i < EADOGM_COLSPAN; i++) {
00348         eaDogM_WriteChr(' ');
00349     }
00350 }
00351
00352 /*
00353  * uses DMA channel 1 for transfers
00354 */
00355 void eaDogM_WriteString(char *strPtr)
00356 {
00357     wait_lcd_set();
00358     /* reset buffer for DMA */
00359     ringBufS_flush(spi_link.tx1a, false);
00360     CSB_SetLow(); /* SPI select display */
00361     if (strlen(strPtr) > max_strlen) strPtr[max_strlen] = 0; /* buffer overflow check */
00362     DMA1CON0bits.EN = 0; /* disable DMA to change source count */
00363     DMA1SSZ = strlen(strPtr);
00364     DMA1CON0bits.EN = 1; /* enable DMA */
00365     printf("%s", strPtr); /* testing copy method using STDIO redirect to buffer */
00366     start_lcd();
00367 #ifdef DISPLAY_SLOW
00368     wdtdelay(9000);
00369 #endif
00370 }
00371
00372 /*
00373  * uses DMA channel 1 for transfers
00374 */
00375 void send_lcd_cmd_dma(uint8_t strPtr)
00376 {
00377     wait_lcd_set();
00378     /* reset buffer for DMA */
00379     ringBufS_flush(spi_link.tx1a, false);
00380     RS_SetLow();
00381     CSB_SetLow(); /* SPI select display */
00382     DMA1CON0bits.EN = 0; /* disable DMA to change source count */
00383     DMA1SSZ = 1;
00384     DMA1CON0bits.EN = 1; /* enable DMA */
00385     printf("%c", strPtr); /* testing copy method using STDIO redirect to buffer */
00386     start_lcd();
00387     wait_lcd_done();
00388     RS_SetHigh();
00389 }
00390
00391 /*
00392  * uses DMA channel 1 for transfers
00393 */
00394 void send_lcd_data_dma(uint8_t strPtr)
00395 {
00396     wait_lcd_set();
00397     /* reset buffer for DMA */

```

```

00398     ringBufS_flush(spi_link.tx1a, false);
00399     RS_SetHigh();
00400     CSB_SetLow(); /* SPI select display */
00401     DMA1CON0bits.EN = 0; /* disable DMA to change source count */
00402     DMA1SSZ = 1;
00403     DMA1CON0bits.EN = 1; /* enable DMA */
00404     printf("%c", strPtr); // testing copy method using STDIO redirect to buffer
00405     start_lcd();
00406 }
00407
00408 void eaDogM_WriteStringAtPos(const uint8_t r, const uint8_t c, char *strPtr)
00409 {
00410     send_lcd_cmd_dma((EADOGM_CMD_DDRAM_ADDR + (r * EADOGM_COLSPAN) + c));
00411     eaDogM_WriteString(strPtr);
00412 }
00413
00414 void eaDogM_WriteIntAtPos(uint8_t r, uint8_t c, uint8_t i)
00415 {
00416     eaDogM_WriteCommand((EADOGM_CMD_DDRAM_ADDR + (r * EADOGM_COLSPAN) + c));
00417
00418     eaDogM_WriteChr(i / 10 + '0');
00419     eaDogM_WriteChr(i % 10 + '0');
00420 }
00421 }
00422
00423 // this writes a byte to the internal CGRAM (v2.02)
00424 // format for ndx: 00CCRRR = CCC = character 0 to 7, RRR = row 0 to 7
00425
00426 void eaDogM_WriteByteToCGRAM(uint8_t ndx, uint8_t data)
00427 {
00428     uint8_t cmd;
00429
00430     cmd = ndx & 0b00111111; // mask off upper to bits
00431     cmd = cmd | EADOGM_CMD_CGRAM_ADDR; // set bit cmd bits
00432
00433     eaDogM_WriteCommand(cmd);
00434     eaDogM_WriteChr(data);
00435
00436     // this is done to make sure we are back in data mode
00437     eaDogM_SetPos(0, 0);
00438 }
00439 #endif
00440
00441 void eaDogM_WriteCommand(const uint8_t cmd)
00442 {
00443     send_lcd_cmd_dma(cmd);
00444 }
00445
00446 void eaDogM_WriteChr(const int8_t value)
00447 {
00448     send_lcd_data_dma((uint8_t) value);
00449 }
00450
00451 /*
00452  * uses DMA channel 2 for transfers
00453  */
00454 void send_port_data_dma(uint16_t dsize)
00455 {
00456     if (dsize > max_port_data)
00457         dsize = max_port_data;
00458
00459     DMA2CON0bits.EN = 0; /* disable DMA to change source count */
00460     DMA2SSZ = dsize;
00461     DMA2DSZ = 1;
00462     DMA2CON0bits.EN = 1; /* enable DMA */
00463     DMA2CON0bits.DMA2SIRQEN = 1; /* start DMA trigger */
00464     V.tx_count += dsize;
00465 }
00466
00467 /*
00468  * return pointer to internal data buffer for DMA
00469  */
00470 uint8_t* port_data_dma_ptr(void)
00471 {
00472     return port_data;
00473 }
00474
00475 /*
00476  * Trigger the SPI DMA transfer to the LCD display
00477  */
00478 void start_lcd(void)
00479 {
00480     #ifdef USE_LCD_DMA
00481         DMA1_StartTransferWithTrigger();
00482     #endif
00483 }
00484 }

```

```

00485
00486 void wait_lcd_set(void)
00487 {
00488     spi_link.LCD_DATA = true;
00489 }
00490
00491 bool wait_lcd_check(void)
00492 {
00493     return spi_link.LCD_DATA;
00494 }
00495
00496 void wait_lcd_done(void)
00497 {
00498     #ifdef USE_LCD_DMA
00499         while (spi_link.LCD_DATA) {
00500             };
00501         while (!SPI1STATUSbits.TXBE) {
00502             };
00503         wdtdelay(180);
00504         CSB_SetHigh();
00505     #endif
00506 }
00507

```

4.8 eadog.h

```

00001 /*
00002  * File:    eadog.h
00003  * Author:  root
00004  *
00005  * Created on July 27, 2015, 2:05 PM
00006  */
00007
00008 /* Parts taken from:
00009  * Modified for C18 and PIC18F45K80 with SPI using ring buffers with interrupts
00010  *
00011  *         file: EA-DOGM_MIO.c
00012  *         version: 2.03
00013  *         description: Multi I/O driver for EA DOGM displays
00014  *         : Uses 8Bit, SPI HW or SPI SW (bitbang)
00015  *         written by : Michael Bradley (mbradley@mcu-labs.com)
00016  *         contributions: Imaginos (CCS forum), Emil Nad (8Bit testing)
00017  *         jgschmidt (CCS forum)
00018  */
00019
00020 #ifndef EADOG_H
00021 #define EADOG_H
00022
00023 #ifdef __cplusplus
00024 extern "C" {
00025 #endif
00026 #include "vconfig.h"
00027 #include "ringbufs.h"
00028
00029     extern struct V_data V;
00030
00031 // #define USEMCC_SPI
00032
00033 #define USE_LCD_DMA
00034
00035 #define NSB      5
00036 #define LSB      21
00037
00038 #define LCD_PWR_DELAY    350000
00039
00040 #define LCD_CMD_MASK     0x01
00041 #define LCD_CMD_SET     0x100
00042 #define LCD_CLEAR_HOME  0x04
00043 #define NHD_CMD          0xFE
00044 #define NHD_BL_OFF       1
00045 #define NHD_BL_LOW       2
00046 #define NHD_BL_MED       5
00047 #define NHD_BL_HIGH      8
00048
00049 #define LCD_CMD_MASK     0x01
00050 #define LCD_CMD_SET     0x100
00051 #define LCD_CLEAR_HOME  0x04
00052 #define LCD_CMD_ON      0x41
00053 #define LCD_CMD_OFF     0x42
00054 #define LCD_CMD_HOME     0x46
00055 #define LCD_CMD_CLR     0x51
00056 #define LCD_CMD_CONT     0x52
00057 #define LCD_CMD_BRI     0x53

```

```

00058 #define NHD_CMD      0xFE
00059 #define NHD_POS       0x45
00060 #define NHD_BL_OFF    1
00061 #define NHD_BL_LOW    2
00062 #define NHD_BL_MED    5
00063 #define NHD_BL_HIGH   8
00064 #define NHD_CONT      45
00065
00066 #define NHD_T_DELAY   8
00067 #define NHD_S_DELAY   200
00068 #define NHD_L_DELAY   800
00069
00070 #define LCD0          0
00071 #define LCD1          1
00072 #define LCD2          2
00073 #define LCD3          3
00074
00075 void wdtdelay(uint32_t);
00076 void init_display(void);
00077 void init_port_dma(void);
00078 void send_port_data_dma(uint16_t);
00079 uint8_t* port_data_dma_ptr(void);
00080 void send_lcd_data_dma(uint8_t);
00081 void send_lcd_cmd_dma(uint8_t);
00082 void start_lcd(void);
00083 void wait_lcd_set(void);
00084 bool wait_lcd_check(void);
00085 void wait_lcd_done(void);
00086 void eaDogM_WriteChr(int8_t);
00087 void eaDogM_WriteCommand(uint8_t);
00088 void eaDogM_SetPos(uint8_t, uint8_t);
00089 void eaDogM_ClearRow(uint8_t);
00090 void eaDogM_WriteString(char *);
00091 void eaDogM_WriteStringAtPos(uint8_t, uint8_t, char *);
00092 void eaDogM_WriteIntAtPos(uint8_t, uint8_t, uint8_t);
00093 void eaDogM_WriteByteToCGRAM(uint8_t, uint8_t);
00094
00095 void clear_lcd_done(void);
00096 void spi_rec_done(void);
00097
00098 #define eaDogM_Cls()          eaDogM_WriteCommand(EADOGM_CMD_CLR)
00099 #define eaDogM_CursorOn()    eaDogM_WriteCommand(EADOGM_CMD_CURSOR_ON)
00100 #define eaDogM_CursorOff()   eaDogM_WriteCommand(EADOGM_CMD_CURSOR_OFF)
00101 #define eaDogM_DisplayOn()   eaDogM_WriteCommand(EADOGM_CMD_DISPLAY_ON)
00102 #define eaDogM_DisplayOff()  eaDogM_WriteCommand(EADOGM_CMD_DISPLAY_OFF)
00103
00104 #ifdef __cplusplus
00105 }
00106 #endif
00107
00108 #endif /* EADOG_H */
00109

```

4.9 hid.c

```

00001 #include "hid.h"
00002
00003 H_data* hid_input(H_data* h)
00004 {
00005     if (get_switch_bm(SSELECT)) {
00006         h->wait_select = false;
00007         h->select_p = get_switch_bm(SSELECT);
00008         clear_switch(SSELECT);
00009     }
00010
00011     if (get_switch_bm(SENTER)) {
00012         h->wait_enter = false;
00013         h->enter_p = get_switch_bm(SENTER);
00014         clear_switch(SENTER);
00015     }
00016
00017     if (get_switch_bm(SCALIB)) {
00018         h->wait_calib = false;
00019         h->calib_p = get_switch_bm(SCALIB);
00020         clear_switch(SCALIB);
00021     }
00022     return h;
00023 }
00024
00025 H_data* hid_display(H_data* h)
00026 {
00027     hid_input(h);
00028

```

```

00029     switch (h->h_state) {
00030     case H_STATE_INIT:
00031         h->h_state = H_STATE_DISPLAY;
00032         h->hid_display = HID_MAIN;
00033         h->wait_enter = true;
00034         h->wait_select = true;
00035         h->wait_calib = true;
00036         h->select_p = SW_OFF;
00037         h->enter_p = SW_OFF;
00038         h->calib_p = SW_OFF;
00039         break;
00040     default: // need to look at this logic
00041     case H_STATE_DISPLAY:
00042         if (!h->wait_select && (h->select_p == SW_OFF)) {
00043             h->sequence = ++h->sequence & 0x3;
00044             if ((h->sequence == HID_AUX) && !get_switch(SCALIB))
00045                 h->sequence = (++h->sequence) & 0x3; // skip TRIM screen unless CALIB button is also
pressed
00046                 h->hid_display = h->sequence;
00047                 h->wait_select = true;
00048             }
00049             break;
00050         }
00051         return h;
00052     }
00053
00054 bool check_enter_button(H_data* h)
00055 {
00056     if (!h->wait_enter && (h->enter_p == SW_OFF)) {
00057         h->wait_enter = true;
00058         return true;
00059     }
00060     return false;
00061 }
00062
00063 bool check_calib_button(H_data* h)
00064 {
00065     if (!h->wait_calib && (h->calib_p == SW_OFF)) {
00066         h->wait_calib = true;
00067         return true;
00068     }
00069     return false;
00070 }
00071
00072 void clear_hid_pflags(H_data* h)
00073 {
00074     h->select_p = SW_OFF;
00075     h->enter_p = SW_OFF;
00076     h->calib_p = SW_OFF;
00077 }
00078
00079 void clear_hid_wflags(H_data* h)
00080 {
00081     h->wait_enter = true;
00082     h->wait_select = true;
00083     h->wait_calib = true;
00084 }

```

4.10 hid.h

```

00001 /* Microchip Technology Inc. and its subsidiaries. You may use this software
00002 * and any derivatives exclusively with Microchip products.
00003 *
00004 * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00005 * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
00006 * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
00007 * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
00008 * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
00009 *
00010 * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00011 * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00012 * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
00013 * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE
00014 * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
00015 * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
00016 * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
00017 *
00018 * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
00019 * TERMS.
00020 */
00021
00022 /*
00023 * File:

```

```

00024  * Author:
00025  * Comments:
00026  * Revision history:
00027  */
00028
00029 // This is a guard condition so that contents of this file are not included
00030 // more than once.
00031 #ifndef HID_H
00032 #define HID_H
00033
00034 #include <xc.h> // include processor files - each processor file is guarded.
00035 #include "dio.h"
00036
00037 typedef enum {
00038     HID_MAIN = 0,
00039     HID_PWR,
00040     HID_RUN,
00041     HID_AUX,
00042     HID_LAST,
00043 } H_CODES;
00044
00045 typedef enum {
00046     H_STATE_INIT = 0,
00047     H_STATE_DISPLAY,
00048     H_STATE_WAIT,
00049     H_STATE_LOG,
00050     H_STATE_COMM,
00051     H_STATE_ERROR
00052 } H_STATES;
00053
00054 typedef struct H_data { // control data structure
00055     H_CODES hid_display;
00056     H_STATES h_state;
00057     bool wait_select, select_p, wait_enter, enter_p, wait_calib, calib_p; // buttons pressed should
        clean
00058     uint8_t sequence;
00059 } H_data;
00060
00061 H_data* hid_input(H_data*);
00062 H_data* hid_display(H_data*);
00063 bool check_enter_button(H_data*);
00064 bool check_calib_button(H_data*);
00065 void clear_hid_pflags(H_data*);
00066 void clear_hid_wflags(H_data*);
00067
00068 #endif /* HID_H */
00069

```

4.11 main.c

```

00001
00020
00021 /*
00022     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00023
00024     Subject to your compliance with these terms, you may use Microchip software and any
00025     derivatives exclusively with Microchip products. It is your responsibility to comply with third
        party
00026     license terms applicable to your use of third party software (including open source software) that
        may accompany Microchip software.
00027
00028     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00029     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00030     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00031     FOR A PARTICULAR PURPOSE.
00032
00033     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00034     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00035     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00036     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00037     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00038     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00039     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00040     SOFTWARE.
00041 */
00042
00043
00044 /*
00045  * THIS IS the DUMPLoad version of this program with MQTT
00046  *
00047  * This program controls and monitors solar power battery arrays on a 24vdc system
00048  * MBMC uses a simple expert machine to try and keep current the energy stored in a bank of batteries
00049  * The main physics functions are contained in the noload_soc, update_cef, ChargeBatt and pick_batt
        functions

```

```

00050 * Much more work is needed to make the program flow logical but making it work first is the current
      effort
00051 * The network interface processor on a Linux machine will be able to analyze the collected data for
      better control at some future point
00052 *
00053 * standard program display units:
00054 * Voltage in Volts,
00055 * Current in Amps
00056 * Watts Power in Watts
00057 * Ah battery capacity Ah
00058 *
00059 * some important data structures
00060 * V: structure, global control structure
00061 * R: structure, real values from measurements
00062 * C: structure, calculated values from measurements or programs
00063 * D: structure, LCD control and buffers
00064 * H: structure, battery history parameters
00065 *
00066 * USART2      GTI power limiter command output
00067 * USART1      MBMC logger and command channel
00068 * timer 2 100us PWM clock, no interrupt
00069 * timer 3 one second housekeeping clock for battery state tracking, low priority interrupt
00070 * timer 5 one second timer, TOD interrupt
00071 * timer 6 500us software timer ticker and parse simple serial port #1 single character commands,
      interrupt
00072 *
00073 * dma1 spi transmit
00074 * dma2 tx1 transmit
00075 *
00076 * 10 analog channels are active
00077 * PORTA,PORTB analog inputs
00078 * ana0 battery current          200A hall sensor battery output to inverter 5v R1
00079 * ana1 PV input current        100A hall sensor input from PV array 5v R2
00080 * ana2 charge controller output voltage      R3
00081 * ana3 PLUS VREF              Using a external 4.095 volt reference IC from TI/BB
00082 * ana4 battery voltage        Voltage a primary inverter battery R4
00083 * ana5 pv voltage             R5 ~60 v max
00084 * ana6 modbus controller voltage      R6
00085 * anb2 backup battery voltage      R7
00086 * anb3 thermo_batt            thermistor input 10K at 25C 5v R8
00087 * anb4 inverter voltage        R9
00088 * anb5 light sensor voltage      R10 ~60 v max
00089 *
00090 * switch MAX debounce inputs INP, uses external interrupt 1
00091 * max pin  pic pin      connector pin  name      MBMC_IO board input connector pin
00092 * 1      rf0      spi 9      10
00093 * 2      rf1      spi 10     1
00094 * 3      rf2      PD 9      SELECT  2
00095 * 4      rf3      PD 8      ENTER   3
00096 * 5      rd3      PD 3      CALIB   4
00097 * 6      rd2      PD 2/misc 2 SW0 5
00098 * 7      misc 3      6
00099 * 8      misc 4      7
00100 *
00101 * 4x20 LCD status panel NHD-0420D3Z-NSW-BBW-V3 and led status lights .
00102 *
00103 * uln2802 port output pins OUTP
00104 * i/o port function
00105 * 0      RD4 load relay 1
00106 * 1      RD5 load relay 2
00107 * 2      RD6 PV/AC Charger DC charge-controller source relay
00108 * 4      RF4 spare
00109 *
00110 *
00111 * enclosure connections
00112 *
00113 * digital/power
00114 * 11 red      1 black outp1
00115 * 12 orange   2 white outp2
00116 * 13 blue     3 grey outp3
00117 * 14 black    4 purple outp4
00118 * 15 grey     5 blue outp5
00119 * 16 pink     6 green outp6
00120 * 17 white jp2 1 +5      7 yellow outp7
00121 * 18 black jp2 2 B      8 orange +24
00122 * 19 red jp2 3 A      9 red VSS
00123 * 20 green jp2 4 gnd    10 brown chassis ground
00124 * modbus board signals
00125 *
00126 * analog signal
00127 * 1 orange     2 black
00128 * 3 purple     4 yellow
00129 * 5 tan        6 blue modbus analog signal
00130 * 7 pink       8 brown
00131 * 9 grey       10 green
00132 * 11 white vss gnd     12 red vdd
00133 * 13 black +3.3      14 white 5.0 calibration vref

```

```

00134 * 15 black pwm vss      16 red pwm sig
00135 *
00136 * system variables float
00137 *
00138 */
00139
00140 #pragma warning disable 520
00141 #pragma warning disable 1498
00142 #pragma warning disable 1090
00143
00144 #ifndef __DEFINED_int24_t
00145 typedef signed long long int24_t;
00146 #define __DEFINED_int24_t
00147 #endif
00148
00149 #include <stdio.h>
00150 #include <string.h>
00151 #include "mcc_generated_files/mcc.h"
00152 #include "mcc_generated_files/uart1.h"
00153 #include "eadog.h"
00154 #include "timers.h"
00155 #include "mconfig.h"
00156 #include "mydisplay.h"
00157 #include "daq.h"
00158 #include "mbmc.h"
00159 #include "dio.h"
00160 #include "hid.h"
00161 #include "bsoc.h"
00162
00163
00164 V_data V = {
00165     .ticks = DEF_TIME,
00166     .ticker = 25, // timer start offset
00167     .checksum_error = 0,
00168     .all_errors = 0,
00169     .debug = false,
00170     .stack = false, // 0 no messages, 1-10 messages in queue
00171     .help_id = 0,
00172     .sequences = 0,
00173     .set_sequ = false,
00174     .lowint_count = 0,
00175     .timerint_count = 0,
00176     .calib = CALIB,
00177     .screen = 0,
00178     .system_stable = false,
00179     .fixup = FIXUPS,
00180     .blight_off = false,
00181     .blight = 0,
00182     .float_ticks = 0,
00183     .boost_ticks = 0,
00184     .in_float = false,
00185     .in_boost = false,
00186     .ac_time = 0,
00187     .rbuf = "1",
00188 };
00189
00190 H_data H = {
00191     .hid_display = HID_MAIN,
00192     .h_state = H_STATE_INIT,
00193     .wait_enter = true,
00194     .wait_select = true,
00195     .wait_calib = true,
00196     .sequence = 0,
00197 };
00198
00199 /*
00200 * FIXME STUFF
00201 */
00202
00203 volatile uint16_t tickCount[TMR_COUNT] = {0};
00204 volatile uint8_t mode_sw = false;
00205 volatile C_data C = {
00206     .bank_ah = BANKAH - DEGRADE,
00207     .day = true,
00208     .hist[0].version = HVER,
00209     .hist[0].cef = CEF,
00210     .hist[0].peukert = PEUKERT,
00211     .hist[0].updates = 0,
00212     .hist[0].h[10] = 0,
00213     .hist[0].h[9] = HI_ESR, // highest possible ESR
00214     .day_start = 0,
00215     .day_end = 0,
00216     .day_update = 0,
00217     .dupdate = false,
00218     .updates = 0,
00219     .dynamic_ah_daily = 0.0001f,
00220     .dynamic_ah_adj_daily = 0.0001f,

```



```

00221     .c_bat = 0.1f,
00222     .c_load = 0.1f,
00223     .c_mppt = 0.1f,
00224     .c_pv = 0.1f,
00225     .esr = 0.1f,
00226     .p_bat = 0.1f,
00227     .v_pv = 0.1f,
00228 };
00229
00230 struct tm *t_mbmc;
00231 time_t pclock;
00232
00233 static bool current_sensor_cal(void);
00234 static bool display_history(void);
00235 static void wdt_reset(void);
00236 static void clock_tick_isr(void);
00237
00238 const char *build_date = __DATE__, *build_time = __TIME__;
00239
00240 /*
00241  * Main application
00242  */
00243 void main(void)
00244 {
00245     bool t_out = STATUSbits.TO;
00246     uint32_t t_time;
00247
00248     UI_STATES mode; /* link configuration host/equipment/etc ... */
00249     uint8_t inp_index = 0, i = C_BATT, j = C_PV, k = V_CC, i_ror;
00250
00251     // Initialize the device
00252     SYSTEM_Initialize();
00253     TMR5_SetInterruptHandler(clock_tick_isr);
00254     TMR6_SetInterruptHandler(sw_timers_isr);
00255
00256     // Enable high priority global interrupts
00257     INTERRUPT_GlobalInterruptHighEnable();
00258
00259     // Enable low priority global interrupts.
00260     INTERRUPT_GlobalInterruptLowEnable();
00261
00262     V.ui_state = UI_STATE_INIT;
00263     // WWDT_SoftEnable();
00264
00265
00266     /*
00267      * RS-232 link I/O relay defaults to monitor/log mode with no power
00268      */
00269     WaitMs(300); // wait for mode switch to settle
00270     if (RB0_GetValue()) {
00271         mode = UI_STATE_HOST;
00272     } else {
00273         mode = UI_STATE_LOG;
00274     }
00275
00276
00277     V.mode_pwm = 0; // set diversion PWM to no power
00278
00279     diversion_pwm_set(V.mode_pwm); // 10KHz PWM
00280
00281     i_ror = U1RXB;
00282
00283     while (true) {
00284         switch (V.ui_state) {
00285             case UI_STATE_INIT:
00286                 /*
00287                  * DMA serial uart1 I/O testing
00288                  */
00289                 init_port_dma();
00290
00291                 init_display();
00292                 eaDogM_CursorOff();
00293 #ifdef USE_DAC
00294                 set_dac(); // set both channels to zero volts
00295 #endif
00296                 /*
00297                  * load the battery to reduce surface charge
00298                  */
00299                 set_load_relay_zero(true); // serial monitor relay on host board
00300                 set_load_relay_one(true);
00301                 set_load_relay_two(true);
00302
00303                 V.ui_state = mode;
00304                 srand(1957);
00305                 set_vterm(0); // set to buffer 0
00306                 sprintf(get_vterm_ptr(0, 0), " Dump Load Mon");
00307                 sprintf(get_vterm_ptr(1, 0), " Version %s", VER);

```

```

00308     sprintf(get_vterm_ptr(2, 0), " NSASPOOK          ");
00309     if (t_out) {
00310         sprintf(get_vterm_ptr(3, 0), "          ");
00311     } else {
00312         sprintf(get_vterm_ptr(3, 0), " WDT TIMEOUT REBOOT ");
00313     }
00314     sprintf(get_vterm_ptr(0, 1), "          ");
00315     sprintf(get_vterm_ptr(1, 1), "          ");
00316     sprintf(get_vterm_ptr(2, 1), "          ");
00317     sprintf(get_vterm_ptr(3, 1), "          ");
00318     sprintf(get_vterm_ptr(0, 2), "          ");
00319     sprintf(get_vterm_ptr(1, 2), "          ");
00320     sprintf(get_vterm_ptr(2, 2), "          ");
00321     sprintf(get_vterm_ptr(3, 2), "          ");
00322     update_lcd(0);
00323     WaitMs(100);
00324     if (V.get_time_text) {
00325         StartTimer(TMR_TEXT, TXIDELAY);
00326
00327         while (true) {
00328             wdt_reset();
00329             if (TimerDone(TMR_TEXT)) {
00330                 V.get_time_text = false;
00331                 V.time_info = false;
00332                 break;
00333             }
00334             if (V.time_info) {
00335                 V.time_info = false;
00336                 t_time = (uint32_t) atol(V.rbuf);
00337                 if (t_time > DEF_TIME) {
00338                     set_time(t_time);
00339                 }
00340             }
00341         }
00342     }
00343     StartTimer(TMR_DISPLAY, DDELAY);
00344     StartTimer(TMR_ADC, ADC_SCAN_SPEED);
00345     StartTimer(TMR_INFO, TDELAY);
00346     StartTimer(TMR_FLIPPER, DFLIP);
00347     StartTimer(TMR_HELPDIS, TDELAY);
00348
00349     start_adc_scan();
00350     start_switch_handler();
00351     WaitMs(1000);
00352     /*
00353     * check for quickly changing battery voltage
00354     * to stabilize as to get a better static SOC value
00355     */
00356     if (read_cal_data()) {
00357         update_cal_data();
00358         if (update_hist_data(true, &C.hist[0]) && (C.hist[0].pclock > time(NULL))) {
00359             set_time(C.hist[0].pclock); // load EEPROM time history
00360         }
00361     }
00362     #ifndef CLR_CYCLE_DATA
00363         C.hist[0].h[1] = 0;
00364         C.hist[0].h[11] = 0;
00365         C.hist[0].updates = 0;
00366     #endif
00367
00368     #ifndef CLR_BATTERY_DATA
00369         C.hist[0].h[0] = 0;
00370         C.hist[0].h[1] = 0;
00371         C.hist[0].h[2] = 0;
00372         C.hist[0].h[4] = 0;
00373         C.hist[0].h[5] = 0;
00374         C.hist[0].h[6] = 0;
00375         C.hist[0].h[9] = 0;
00376         C.hist[0].h[10] = 0;
00377         C.hist[0].h[11] = 0;
00378         C.hist[0].updates = 0;
00379     #endif
00380
00381     #ifndef CLR_PV_DATA
00382         C.hist[0].h[3] = 0;
00383         C.hist[0].h[12] = 0;
00384     #endif
00385
00386     sprintf(get_vterm_ptr(2, 0), "Read EEPROM DATA  ");
00387 } else {
00388     sprintf(get_vterm_ptr(2, 0), "Invalid EEPROM DATA ");
00389     write_cal_data();
00390 }
00391 update_lcd(0);
00392 clear_adc_scan();
00393 start_adc_scan();
00394 WaitMs(2000);

```

```

00395
00396     V.cc_state = cc_state(conv_raw_result(V_CMODE, CONV));
00397     if (V.cc_state && (V.cc_state != M_OFFLINE)) {
00398         V.cc_active = true;
00399     } else {
00400         V.cc_active = false;
00401     }
00402
00403 #ifndef SKIP_ESR
00404     i_ror = 1;
00405     do {
00406         calc_ror_data();
00407         sprintf(get_vterm_ptr(1, 0), "BV %2.4f CC%d", conv_raw_result(V_BAT,
CONV), V.cc_state);
00408         sprintf(get_vterm_ptr(2, 0), "S S%cC %d %2.4f", spinners(5, false), i_ror,
C.bv_ror);
00409         update_lcd(0);
00410         clear_adc_scan();
00411         start_adc_scan();
00412         wdt_reset();
00413         WaitMs(ROR_WAIT); // time between samples
00414
00415     } while ((i_ror++ < ROR_TIMES) && (C.bv_ror > ROR_LIMIT_LOW));
00416
00417     static_soc(); // defaults
00418     init_bsoc(); // system calculations
00419     set_load_relay_one(false);
00420     set_load_relay_two(false);
00421     sprintf(get_vterm_ptr(0, 0), "Static SOC %d", C.soc);
00422     sprintf(get_vterm_ptr(1, 0), "Battery Ah %3.2f", C.dynamic_ah);
00423     sprintf(get_vterm_ptr(2, 0), " ");
00424     update_lcd(0);
00425     WaitMs(200);
00426     sprintf(get_vterm_ptr(0, 0), "Battery ESR");
00427     sprintf(get_vterm_ptr(1, 0), "Calculation");
00428     sprintf(get_vterm_ptr(2, 0), "Check 30 seconds");
00429     update_lcd(0);
00430     uint16_t i_esr = 1;
00431     uint8_t shape = 0;
00432     float esr_temp;
00433     while ((esr_temp = esr_check(false)) < 0.0f) {
00434         WaitMs(110); // limit display updates
00435         shape = (uint8_t) fabs(esr_temp);
00436         sprintf(get_vterm_ptr(2, 0), "Checking %c %c", spinners(0, false),
spinners(shape, false));
00437         update_lcd(0);
00438
00439         if (i_esr++ > 1)
00440             break;
00441     };
00442     sprintf(get_vterm_ptr(0, 0), "ESR %2.6f", C.esr);
00443     sprintf(get_vterm_ptr(1, 0), "R1 %2.3f %3.4f", C.bv_one_load, C.load_i1);
00444     sprintf(get_vterm_ptr(2, 0), "R2 %2.3f %3.4f", C.bv_full_load, C.load_i2);
00445     update_lcd(0);
00446     WaitMs(400);
00447 #endif
00448     V.sensor_set = get_switch(SCALIB);
00449     WaitMs(1000);
00450     if (V.sensor_set && get_switch(SCALIB)) {
00451         current_sensor_cal();
00452         WaitMs(4000);
00453     }
00454     V.blight = time(NULL) + BL_TIME;
00455     V.sensor_set = false;
00456     V.system_stable = true;
00457     break;
00458 case UI_STATE_HOST:
00459     break;
00460 case UI_STATE_LOG:
00461     break;
00462 case UI_STATE_MON: // monitor
00463     // P.SYSTEM_STABLE = true;
00464     break;
00465 case UI_STATE_ERROR:
00466 default:
00467     V.ui_state = UI_STATE_INIT;
00468     break;
00469 }
00470
00471 /*
00472  * start the ADC scan for raw data
00473  */
00474 if (TimerDone(TMR_ADC) && check_adc_scan()) {
00475     /*
00476      * download the system data variables
00477      */
00478     convert_adc_data();

```

```

00479         /*
00480         * restart the conversion process
00481         */
00482         clear_adc_scan();
00483         start_adc_scan();
00484         StartTimer(TMR_ADC, ADC_SCAN_SPEED);
00485     }
00486
00487     if (TimerDone(TMR_DISPLAY)) { // limit update rate
00488         gti_cmds();
00489         if (TimerDone(TMR_HELPDIS)) {
00490             set_display_info(DIS_STR);
00491         }
00492         pclock = time(NULL);
00493         t_mbm = localtime(&pclock);
00494         sprintf(get_vterm_ptr(3, 0), "%s", asctime(t_mbm));
00495         sprintf(get_vterm_ptr(3, 1), "%s", asctime(t_mbm));
00496         calc_model_data();
00497         V.cc_state = cc_state(C.v_cmode);
00498         if (C.dupdate) {
00499             C.dupdate = false;
00500             set_hist_flag();
00501 #ifndef SKIP_ESR
00502             uint16_t i_esr = 1;
00503             esr_check(true); // init FSM
00504             while (esr_check(false) < 0.0f) {
00505                 WaitMs(110);
00506                 if (i_esr++ > 512)
00507                     break;
00508             };
00509 #endif
00510             load_hist_data(); // calculate history data
00511             update_hist_data(false, &C.hist[0]); // load EEPROM history buffer
00512             write_cal_data(); // save updated history to EEPROM
00513             sprintf(get_vterm_ptr(0, 0), "History Data Saved ");
00514             sprintf(get_vterm_ptr(1, 0), "To EEPROM");
00515             sprintf(get_vterm_ptr(2, 0), "Time %lu, %lu", time(NULL), C.hist[0].updates);
00516             update_lcd(0);
00517             WaitMs(2000);
00518         }
00519         /*
00520         * Display back-light control
00521         */
00522         if (set_back_light_off(false)) {
00523             send_lcd_cmd_dma(0x53); // back-light command
00524             send_lcd_data_dma(NHD_BL_MED); // light on
00525             wait_lcd_done();
00526         }
00527         if (set_back_light_off(true)) {
00528             send_lcd_cmd_dma(0x53);
00529             send_lcd_data_dma(NHD_BL_OFF); // light off
00530             wait_lcd_done();
00531         }
00532         /*
00533         * LCD display updates
00534         */
00535         hid_display(&H);
00536         switch (H.hid_display) {
00537             case HID_PWR:
00538                 V.calib = false;
00539                 sprintf(get_vterm_ptr(0, 0), "PV %c W %3.2f", (C.day) ? 'D' : ' ', C.p_pv);
00540                 sprintf(get_vterm_ptr(1, 0), "LOAD W %3.2f LA %2.2f", C.p_load, C.c_load);
00541                 sprintf(get_vterm_ptr(2, 0), "MPPT W %3.2f %d", C.p_mppt, V.cc_state);
00542
00543                 sprintf(get_vterm_ptr(0, 1), "PV %c WH %3.2f", (C.day) ? 'D' : ' ', C.pvkw);
00544                 sprintf(get_vterm_ptr(1, 1), "LOAD WH %3.2f", C.loadkw);
00545                 sprintf(get_vterm_ptr(2, 1), "INV WH %3.2f", C.invwk);
00546                 break;
00547             case HID_MAIN:
00548                 V.calib = false;
00549                 sprintf(get_vterm_ptr(0, 0), "PV %2.2f PA %2.2f", calc_fixups(C.calc[V_PV],
00550 WIDE_ZERO), calc_fixups(C.calc[C_BATT], WIDE_ZERO | NO_NEG));
00551                 sprintf(get_vterm_ptr(1, 0), "BV %2.2f BA %2.2f", calc_fixups(C.calc[V_BAT],
00552 WIDE_ZERO), C.calc[C_MPPT]);
00553                 sprintf(get_vterm_ptr(2, 0), "CV %2.2f CA %2.2f", calc_fixups(C.calc[V_CC],
00554 WIDE_ZERO), C.calc[C_PV]);
00555
00556                 sprintf(get_vterm_ptr(0, 1), "BAT IWH %4.1f", C.bkwi);
00557                 sprintf(get_vterm_ptr(1, 1), "BAT OWH %4.1f", C.bkwo);
00558                 sprintf(get_vterm_ptr(2, 1), "BAT TWH %4.1f", C.bkwi + C.bkwo);
00559                 break;
00560             case HID_RUN:
00561                 V.calib = false;
00562                 sprintf(get_vterm_ptr(0, 0), "BATT W %3.2f", C.p_bat);
00563                 sprintf(get_vterm_ptr(1, 0), "BAH T%3.2f D%3.2f", C.dynamic_ah,
00564 C.dynamic_ah_daily);
00565                 sprintf(get_vterm_ptr(2, 0), "S% c %d RUN %d V%2.2f", spinners(5, false),

```

```

    C.soc, C.runtime, C.calc[V_BAT]);
00562
00563     sprintf(get_vterm_ptr(0, 1), "ESR %2.6f", C.esr);
00564     sprintf(get_vterm_ptr(1, 1), "R1 %2.3f %3.4f", C.bv_one_load, C.load_i1);
00565     sprintf(get_vterm_ptr(2, 1), "R2 %2.3f %3.4f", C.bv_full_load, C.load_i2);
00566     break;
00567     case HID_AUX:
00568         if (!V.calib) { // clear buffer and start from zero
00569             lp_filter(0.0f, i, -1);
00570             lp_filter(0.0f, j, -1);
00571             lp_filter(0.0f, k, -1);
00572         }
00573         V.calib = true;
00574         sprintf(get_vterm_ptr(0, 0), "%d %2.4f, %d TRIM", get_raw_result(i), C.calc[i],
inp_index + 1);
00575         if (inp_index < 9) {
00576             sprintf(get_vterm_ptr(1, 0), "%d %2.4f, %d TRIM", get_raw_result(j),
C.calc[j], inp_index + 2);
00577             sprintf(get_vterm_ptr(2, 0), "%d %2.4f, %d TRIM", get_raw_result(k),
C.calc[k], inp_index + 3);
00578         } else {
00579             sprintf(get_vterm_ptr(1, 0), "%4d, %4d, %4lu", C.hist[0].h[11],
C.hist[0].h[1], C.hist[0].updates);
00580             sprintf(get_vterm_ptr(2, 0), "%4d, %4d, %4d", C.hist[0].h[0],
C.hist[0].h[9], C.hist[0].h[10]);
00581         }
00582         break;
00583     default:
00584         break;
00585 }
00586 clear_hid_pflags(&H);
00587
00588 StartTimer(TMR_DISPLAY, DDELAY);
00589 V.enter = check_enter_button(&H);
00590 if (V.enter && (H.hid_display != HID_AUX)) {
00591     V.enter = false;
00592     V.screen = ~V.screen;
00593     inp_index = 0;
00594 }
00595 set_vterm(V.screen);
00596 update_lcd(V.screen);
00597 display_history();
00598
00599 wait_lcd_done();
00600 #ifdef USE_DAC
00601     set_dac_a(3.333);
00602     set_dac_b(6.666);
00603     set_dac();
00604 #endif
00605 }
00606
00607 /*
00608  * show help display if button pressed
00609  */
00610 check_help(V.flipper);
00611
00612 if (V.enter) {
00613     V.enter = false;
00614     V.calib = false;
00615     inp_index += 3;
00616     if (inp_index > 9)
00617         inp_index = 0;
00618     switch (inp_index) {
00619     case 0:
00620         i = C_BATT;
00621         j = C_PV;
00622         k = V_CC;
00623         break;
00624     case 3:
00625         i = V_BAT;
00626         j = V_PV;
00627         k = V_CMODE;
00628         break;
00629     case 6:
00630         i = V_BBAT;
00631         j = C_MPPT;
00632         k = V_INVERTER;
00633         break;
00634     case 9:
00635         i = V_LIGHT_SENSOR;
00636         j = V_LIGHT_SENSOR;
00637         k = V_LIGHT_SENSOR;
00638         break;
00639     default:
00640         break;
00641     }
00642 };

```

```

00643
00644     /*
00645     * show command messages if flag is set for timer duration
00646     */
00647     if (V.set_sequ) {
00648         if (TimerDone(TMR_INFO)) {
00649             V.set_sequ = false;
00650             set_vterm(0);
00651             update_lcd(0);
00652         } else {
00653
00654             set_vterm(2);
00655             update_lcd(2);
00656         }
00657     }
00658     wdt_reset();
00659 }
00660 }
00661
00662 static bool current_sensor_cal(void)
00663 {
00664     uint8_t x = 0;
00665     uint32_t cb = 0, cp = 0, cm = 0;
00666
00667     sprintf(get_vterm_ptr(0, 0), "Battery and PV      ");
00668     sprintf(get_vterm_ptr(1, 0), "Sensor Zero      ");
00669     sprintf(get_vterm_ptr(2, 0), "Calibration      ");
00670     update_lcd(0);
00671     WaitMs(2000);
00672     do {
00673         if (++x > CAL_DELAY)
00674             return false;
00675         sprintf(get_vterm_ptr(2, 0), "Release button %c ", spinners(4, false));
00676         update_lcd(0);
00677         WaitMs(100);
00678     } while (get_switch(SCALIB));
00679
00680     x = 0;
00681     do {
00682         cb += get_raw_result(C_BATT); // get a average result
00683         cp += get_raw_result(C_PV);
00684         cm += get_raw_result(C_MPPT);
00685 #ifdef CAL_TWO
00686         cm = cp;
00687 #endif
00688         sprintf(get_vterm_ptr(0, 0), "Sensor Readings      ");
00689         sprintf(get_vterm_ptr(1, 0), " %d %d %d          ", get_raw_result(C_BATT),
get_raw_result(C_PV), get_raw_result(C_MPPT));
00690         sprintf(get_vterm_ptr(2, 0), "Stability clock %d ", x);
00691         update_lcd(0);
00692         clear_adc_scan();
00693         start_adc_scan();
00694         WaitMs(100);
00695     } while (++x < CAL_DELAY);
00696     cb = cb >> 6; // shift down for average result
00697     cp = cp >> 6;
00698     cm = cm >> 6;
00699
00700     if (cal_current_zero(false, (int16_t) cb, (int16_t) cp, (int16_t) cm)) {
00701         cal_current_zero(true, (int16_t) cb, (int16_t) cp, (int16_t) cm);
00702         sprintf(get_vterm_ptr(0, 0), "Battery and PV      ");
00703         sprintf(get_vterm_ptr(1, 0), " %d %d %d          ", cb, cp, cm);
00704         sprintf(get_vterm_ptr(2, 0), "Zero Cal Set      ");
00705         update_lcd(0);
00706         WaitMs(2000);
00707         write_cal_data();
00708     } else {
00709         sprintf(get_vterm_ptr(0, 0), "Battery and PV      ");
00710         sprintf(get_vterm_ptr(1, 0), " %d %d %d          ", cb, cp, cm);
00711         sprintf(get_vterm_ptr(2, 0), "Zero Out Of Range  ");
00712         update_lcd(0);
00713         WaitMs(2000);
00714         return false;
00715     }
00716
00717 #ifdef CAL_10A
00718     uint32_t cbz, cpz;
00719
00720     clear_switch(SCALIB);
00721     sprintf(get_vterm_ptr(0, 0), "Battery and PV      ");
00722     sprintf(get_vterm_ptr(1, 0), "10 Amp Sensor      ");
00723     sprintf(get_vterm_ptr(2, 0), "Calibration      ");
00724     update_lcd(0);
00725     WaitMs(2000);
00726     x = 0;
00727     do {
00728         if (++x > CAL_DELAY)

```

```

00729         return false;
00730         sprintf(get_vterm_ptr(2, 0), "Press button %c ", spinners(4, false));
00731         update_lcd(0);
00732         WaitMs(100);
00733     } while (!get_switch(SCALIB));
00734
00735     x = 0;
00736     cbz = cb;
00737     cpz = cp;
00738     cb = 0;
00739     cp = 0;
00740     do {
00741         cb += get_raw_result(C_BATT);
00742         cp += get_raw_result(C_PV);
00743         sprintf(get_vterm_ptr(0, 0), "Sensor Readings ");
00744         sprintf(get_vterm_ptr(1, 0), " %d %d ", get_raw_result(C_BATT),
get_raw_result(C_PV));
00745         sprintf(get_vterm_ptr(2, 0), "Stability clock %d ", x);
00746         update_lcd(0);
00747         clear_adc_scan();
00748         start_adc_scan();
00749         WaitMs(100);
00750     } while (++x < CAL_DELAY);
00751     cb = cb >> 6;
00752     cp = cp >> 6;
00753
00754     if (cal_current_10A(false, cb, cp, 0.0, 0.0)) {
00755         cal_current_10A(true, cb, cp, 10.0 / (float) (cb - cbz), 10.0 / (float) (cp - cpz));
00756         sprintf(get_vterm_ptr(0, 0), "Battery and PV ");
00757         sprintf(get_vterm_ptr(1, 0), " %f %f ", 10.0 / (float) ((cb - cbz) + 1), 10.0 /
(float) ((cp - cpz) + 1));
00758         sprintf(get_vterm_ptr(2, 0), "10A Cal Set ");
00759         update_lcd(0);
00760         WaitMs(5000);
00761         write_cal_data();
00762     } else {
00763         sprintf(get_vterm_ptr(0, 0), "Battery and PV ");
00764         sprintf(get_vterm_ptr(1, 0), " %ld %ld ", get_raw_result(C_BATT),
get_raw_result(C_PV));
00765         sprintf(get_vterm_ptr(2, 0), "10A Out Of Range ");
00766         update_lcd(0);
00767         WaitMs(2000);
00768
00769         return false;
00770     }
00771 #endif
00772
00773     return true;
00774 }
00775
00776 static bool display_history(void)
00777 {
00778     static uint8_t bwait = 0;
00779
00780     if (get_switch(SCALIB) && (++bwait > 5)) {
00781         pclock = time(NULL);
00782         t_mbm = localtime(&pclock);
00783         sprintf(get_vterm_ptr(0, 0), "%d %d %d %d ", C.hist[0].h[0], C.hist[0].h[1],
C.hist[0].h[2], C.hist[0].h[3]);
00784         sprintf(get_vterm_ptr(1, 0), "%d %d %d %d ", C.hist[0].h[4], C.hist[0].h[5],
C.hist[0].h[6], C.hist[0].h[7]);
00785         sprintf(get_vterm_ptr(2, 0), "%d %d %d %d ", C.hist[0].h[8], C.hist[0].h[9],
C.hist[0].h[10], C.hist[0].h[12]);
00786         sprintf(get_vterm_ptr(3, 0), "%s ", asctime(t_mbm));
00787         update_lcd(0);
00788         WaitMs(2000);
00789         bwait = 0;
00790         return true;
00791     } else {
00792         if (!get_switch(SCALIB))
00793             bwait = 0;
00794         return false;
00795     }
00796 }
00797
00798 static void wdt_reset(void)
00799 {
00800 #ifdef DEBUG_WDT1
00801     RELAYL1_SetHigh();
00802 #endif
00803 #ifdef DEBUG_WDT2
00804     RELAYL2_SetHigh();
00805 #endif
00806     CLRWD();
00807 }
00808
00809 static void clock_tick_isr(void)

```

```

00810 {
00811     SLED = (uint8_t) ~SLED;
00812     V.timerint_count++;
00813     V.ticks++;
00814     check_day_time();
00815     V.ticker = 0;
00816
00817     if (RELAYL1_PORT || RELAYL2_PORT) {
00818         if (++V.wdt_ticks > MAX_LOAD_TIME) {
00819             while (1) { // shutdown load relays and lockup for WDT to reboot
00820                 RELAYL1_SetLow();
00821                 DEBUG2_Toggle();
00822                 RELAYL2_SetLow();
00823                 DEBUG1_Toggle();
00824                 SLED = (uint8_t) ~SLED;
00825             };
00826         }
00827     } else {
00828         V.wdt_ticks = 0;
00829     }
00830 }

```

4.12 mbmc.c

```

00001 #include "mbmc.h"
00002 #include "mcc_generated_files/ext_int.h"
00003 #include "bsoc.h"
00004
00005 struct tm *t_mbmc; // don't use the xc8 clock function, time will be saved in the history structure
00006 volatile uint32_t utctime = 0; // utctime set from remote ntp server
00007
00008 /* spinner defines */
00009 #define MAX_SHAPES 6
00010 const char spin[MAX_SHAPES][20] = {
00011     "|||/--", // classic LCD version with no \ character
00012     "|||/--\\|\\|", // classic
00013     "OOOOOO--_", // eye blink
00014     "vv<<^>", // point spinner
00015     "...*x#x#XX| |--", // warp portal
00016     "..ooOOoo" // ball bouncer
00017 };
00018
00019 /*
00020  * floating point low pass filter,
00021  * slow/fast select, use (-1) to zero buffer channel and return new
00022  */
00023 float lp_filter(const float new, const uint8_t bn, const int8_t slow)
00024 {
00025     static float smooth[LP_BUFFER_SIZE];
00026     float lp_speed;
00027
00028     if (bn >= LP_BUFFER_SIZE) // buffer index check
00029         return new;
00030
00031     if (slow == (-1)) { // reset smooth buffer and return original value
00032         smooth[bn] = 0.0;
00033         return new;
00034     }
00035
00036     if (slow) { // some random filter cutoffs beta values
00037         lp_speed = 0.06;
00038     } else {
00039         lp_speed = 0.1;
00040     }
00041     // exponentially weighted moving average
00042     return smooth[bn] = smooth[bn] + ((new - smooth[bn]) * lp_speed);
00043 }
00044
00045 void convert_adc_data(void)
00046 {
00047     uint8_t i = 0;
00048
00049     do {
00050         if (V.calib) {
00051             C.calc[i] = lp_filter(conv_raw_result(i, O_CONV), i, true);
00052         } else {
00053             if (i == C_BATT || i == C_PV) { // smooth PWM effects
00054                 C.calc[i] = lp_filter(conv_raw_result(i, CONV), i, true);
00055             } else {
00056                 C.calc[i] = lp_filter(conv_raw_result(i, CONV), i, false);
00057             }
00058         }
00059     } while (++i < ADC_BUFFER_SIZE);

```



```

00060 }
00061
00062 /*
00063  * buffer model data from DAQ
00064  */
00065 void calc_model_data(void)
00066 {
00067     static float bvror = 0.0, bcror = 0.0;
00068     stop_bsoc();
00069
00070     if (!V.calib) {
00071         C.updates++;
00072         C.c_bat = C.calc[C_BATT];
00073         C.bc_ror = C.c_bat - bcror;
00074         bcror = C.c_bat;
00075         C.c_pv = C.calc[C_PV];
00076         C.c_mppt = C.calc[C_MPPT];
00077         C.v_bat = C.calc[V_BAT];
00078         C.v_sensor = C.calc[V_LIGHT_SENSOR];
00079         C.v_cmode = C.calc[V_CMODE];
00080         C.bv_ror = C.v_bat - bvror;
00081         bvror = C.v_bat;
00082         C.v_pv = C.calc[V_PV];
00083         C.v_cc = C.calc[V_CC];
00084         C.v_inverter = C.calc[V_INVERTER];
00085         C.c_load = lp_filter(C.calc[C_PV]+(-(C.calc[C_BATT])), 16, true);
00086         C.p_load = lp_filter(C.c_load * C.v_bat, 17, true);
00087         C.p_pv = lp_filter(C.c_pv * C.v_cc, 18, true);
00088         C.p_inverter = lp_filter((C.c_load * C.v_inverter) - STATIC_LOAD_POWER, 19, true);
00089         C.p_bat = lp_filter(C.c_bat * C.v_bat, 20, true);
00090         C.p_mppt = lp_filter(C.c_mppt * C.v_pv, 21, true);
00091         /*
00092          * calculation limits
00093          */
00094         if (C.p_inverter < 0.0)
00095             C.p_inverter = 0.0001;
00096     }
00097     start_bsoc();
00098     C.update = true;
00099 }
00100
00101 /*
00102  * find rate of change of battery voltage under load
00103  */
00104 void calc_ror_data(void)
00105 {
00106     static float bvror = 0.0, bcror = 0.0; // must remember prior values
00107
00108     C.bc_ror = fabs(conv_raw_result(C_BATT, CONV) - bcror);
00109     bcror = conv_raw_result(C_BATT, CONV);
00110     C.bv_ror = fabs(conv_raw_result(V_BAT, CONV) - bvror);
00111     if (C.bv_ror < ROR_LIMIT_NOISE) // skip noise values
00112         C.bv_ror = ROR_LIMIT_LOW + ROR_LIMIT_SET; // keep trying value
00113     bvror = conv_raw_result(V_BAT, CONV);
00114 }
00115
00116 /*
00117  * boot battery condition defaults
00118  */
00119 void static_soc(void)
00120 {
00121     C.runtime = 120;
00122     C.soc = 100;
00123     C.dynamic_ah = C.bank_ah;
00124     C.dynamic_ah_adj = C.dynamic_ah;
00125 }
00126
00127 void set_load_relay_zero(const bool mode)
00128 {
00129     RELAY0_LAT = mode;
00130 }
00131
00132 void set_load_relay_one(const bool mode)
00133 {
00134     RELAY1_LAT = mode;
00135 }
00136
00137 void set_load_relay_two(const bool mode)
00138 {
00139     RELAY2_LAT = mode;
00140 }
00141
00142 void set_ac_charger_relay(const bool mode)
00143 {
00144     AC_CHARGER_LAT = mode;
00145 }
00146

```

```

00147 bool get_ac_charger_relay(void)
00148 {
00149     return AC_CHARGER_LAT;
00150 }
00151
00152 /* Misc ACSII spinner character generator, stores position for each shape */
00153 char spinners(uint8_t shape, const uint8_t reset)
00154 {
00155     static uint8_t s[MAX_SHAPES];
00156     char c;
00157
00158     if (shape > (MAX_SHAPES - 1))
00159         shape = 0;
00160     if (reset)
00161         s[shape] = 0;
00162     c = spin[shape][s[shape]];
00163     if (++s[shape] >= strlen(spin[shape]))
00164         s[shape] = 0;
00165     return c;
00166 }
00167
00168 /*
00169  * should be called every second in the time keeper task
00170  * returns true at dusk or dawn switch-over
00171  * checks for EEPROM updates with 24hr history
00172  */
00173 bool check_day_time(void)
00174 {
00175     static uint8_t day_delay = 0, low_bat_delay = 0;
00176     float light;
00177
00178     light = conv_raw_result(V_LIGHT_SENSOR, CONV);
00179     /*
00180      * limit AC charger time per trigger
00181      */
00182     if ((V.ac_time++>MAX_AC_TIME) && get_ac_charger_relay()) {
00183         V.ac_time = 0;
00184         set_ac_charger_relay(false);
00185     }
00186
00187     if (V.ac_off) {
00188         set_ac_charger_relay(false);
00189         V.ac_time = 0;
00190         V.ac_off = false;
00191     }
00192
00193     if (V.ac_on) {
00194         set_ac_charger_relay(true);
00195         V.ac_time = 0;
00196         V.ac_on = false;
00197     }
00198
00199     /*
00200      * history eeprom update
00201      */
00202     if (C.day_update) {
00203         if (time(NULL) >= C.day_update) {
00204             C.day_update = 0; // set to trigger time to false
00205             C.dupdate = true; // trigger a EEPROM history write
00206             if (V.in_float) {
00207                 V.in_float = false;
00208                 V.float_ticks = 0;
00209                 C.hist[0].h[11]++; //increase the battery float cycles count
00210             }
00211             if (V.in_boost) {
00212                 V.in_boost = false;
00213                 V.boost_ticks = 0;
00214                 C.hist[0].h[1]++; //increase the battery boost cycles count
00215             }
00216         }
00217     }
00218
00219     if (V.blight) {
00220         if (time(NULL) >= V.blight) {
00221             V.blight = 0; // set to trigger time to false
00222             V.blight_off = true; // trigger a display back-light off command
00223         }
00224     }
00225
00226     /*
00227      * AC charger auto routines
00228      */
00229     if (V.system_stable) {
00230         // low battery volts, turn on charger
00231         if ((C.v_bat < BAT_LOW_VOLTS) && (C.v_bat > BAT_DEAD_VOLTS) && (++low_bat_delay > 10)) {
00232             V.ac_on = true;
00233             low_bat_delay = 0;

```

```

00234     } else {
00235         low_bat_delay = 0;
00236     }
00237
00238     if (!day_delay++) {
00239         if (C.soc > SOC_CRITICAL) {
00240             if (!C.day) {
00241                 if (light > DAWN_VOLTS) {
00242                     C.day = true;
00243                     C.day_start = time(NULL);
00244                     if (get_ac_charger_relay()) { // USE PV charging during the day
00245                         set_ac_charger_relay(false);
00246                         V.ac_time = 0;
00247                     }
00248                     return true;
00249                 }
00250             } else {
00251                 if (light < DUSK_VOLTS) {
00252                     C.day = false;
00253                     C.day_end = time(NULL);
00254                     C.day_update = C.day_end + DUPDATE; // set up trigger time
00255                     /*
00256                      * at low battery condition charge with AC at night
00257                      */
00258                     if ((C.soc < SOC_TOO_LOW)) {
00259                         set_ac_charger_relay(true);
00260                         V.ac_time = 0;
00261                     }
00262                     return true;
00263                 }
00264             }
00265         } else {
00266             if ((C.p_pv < C.p_inverter) && (light < BRIGHT_VOLTS)) { // see if PV can raise SOC
00267                 if (!get_ac_charger_relay()) { // USE AC charger if PV power will be lower
00268                     set_ac_charger_relay(true);
00269                     V.ac_time = 0;
00270                 }
00271             }
00272         }
00273     }
00274 }
00275 if (day_delay >= DAY_DELAY)
00276     day_delay = 0;
00277 return false;
00278 }
00279
00280 /*
00281  * calculate history data buffer from current program data
00282  */
00283 void load_hist_data(void)
00284 {
00285     int16_t esr_rescale;
00286
00287     stop_bsoc();
00288     esr_rescale = (int16_t) (C.esr * 1000.0);
00289     C.hist[0].updates++;
00290     C.hist[0].h[12] += (volatile int16_t) C.pvkw;
00291     if (esr_rescale > C.hist[0].h[10])
00292         C.hist[0].h[10] = esr_rescale;
00293     if (esr_rescale < C.hist[0].h[9])
00294         C.hist[0].h[9] = esr_rescale;
00295     C.hist[0].h[8] = (int16_t) (C.bv_one_load * 100.0f);
00296     C.hist[0].h[7] = (int16_t) (C.bv_full_load * 100.0f);
00297     C.hist[0].h[6] = (int16_t) C.dynamic_ah;
00298     C.hist[0].h[5] = (int16_t) C.bkwi;
00299     C.hist[0].h[4] = (int16_t) C.bkwo;
00300     C.hist[0].h[3] = (int16_t) C.pv_ah;
00301     C.hist[0].h[0] = (int16_t) C.dynamic_ah_daily;
00302     C.hist[0].pclock = time(NULL);
00303     C.dynamic_ah_daily = 0.0;
00304     C.dynamic_ah_adj_daily = 0.0;
00305     C.pvkw = 0.0;
00306     start_bsoc();
00307 }
00308
00309 /*
00310  * if t > 0, t is set to memory location of current_time variable
00311  */
00312 time_t time(time_t * t)
00313 {
00314     static time_t current_time;
00315     PIE8bits.TMR5IE = 0;
00316     current_time = V.ticks;
00317     PIE8bits.TMR5IE = 1;
00318     if (t) {
00319         t = &current_time;
00320     }

```

```

00321     return current_time;
00322 }
00323
00324 void set_time(const time_t t)
00325 {
00326     PIE8bits.TMR5IE = 0;
00327     V.ticks = t;
00328     PIE8bits.TMR5IE = 1;
00329 }
00330
00331 /*
00332  * cleanup mainly noise values near zero value
00333  */
00334 float calc_fixups(float data, FIX_CODES fixup)
00335 {
00336     if ((fixup & SKIP) || (!V.fixup))
00337         return data;
00338
00339     if (fixup & WIDE_ZERO) {
00340         if ((data < 0.05) && (data > -0.05))
00341             data = 0.0;
00342     }
00343     if (fixup & NO_NEG) {
00344         if (data < 0.0)
00345             data = 0.0;
00346     }
00347     return data;
00348 }

```

4.13 mbmc.h

```

00001 /* Microchip Technology Inc. and its subsidiaries. You may use this software
00002  * and any derivatives exclusively with Microchip products.
00003  *
00004  * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00005  * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
00006  * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
00007  * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
00008  * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
00009  *
00010  * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00011  * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00012  * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
00013  * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE
00014  * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
00015  * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
00016  * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
00017  *
00018  * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
00019  * TERMS.
00020  */
00021
00022 /*
00023  * File:
00024  * Author:
00025  * Comments:
00026  * Revision history:
00027  */
00028
00029 // This is a guard condition so that contents of this file are not included
00030 // more than once.
00031 #ifndef MBMC_H
00032 #define MBMC_H
00033
00034 #include <xc.h> // include processor files - each processor file is guarded.
00035 #include <time.h>
00036 #include "vconfig.h"
00037 #include "daq.h"
00038
00039 extern V_data V;
00040
00041 #define LOCALTIME_OFFSET    60ul*60ul*(-8ul)
00042 #define LP_BUFFER_SIZE    ADC_BUFFER_SIZE+8
00043 #define STATIC_LOAD_POWER    14.00f
00044 #define BANKAH    200.0f // baatery bank label Ah
00045 #define DEGRADE    25.0f // adjust to what the battery bank Ah really is
00046 #define SSLICE    3600.0f // samples per hour
00047
00048 #ifndef DEBUG_HIST1
00049 #define DUPDATE    600 // after dark update time offset
00050 #else
00051 #define DUPDATE    14400 // after dark update time offset
00052 #endif

```

```

00053
00054 #define BLOAD1      10.0f          // load resistor in ohms
00055 #define BLOAD2      2.5f           // ditto
00056 #define MAX_LOAD_TIME 75
00057
00058 #define ROR_LIMIT_LOW 0.0140f
00059 #define ROR_LIMIT_SET 0.0001f
00060 #define ROR_LIMIT_NOISE 0.0051f
00061
00062 #define ROR_WAIT      20
00063 #define ROR_TIMES     1
00064
00065 #define DAWN_VOLTS    22.0f
00066 #define DUSK_VOLTS    12.0f
00067 #define BRIGHT_VOLTS 50.0f
00068 #define BAT_LOW_VOLTS 21.5f
00069 #define BAT_DEAD_VOLTS 18.0f
00070 #define MAX_AC_TIME 60*60*4 // AC charger max runtime per trigger
00071 #define DAY_DELAY     240
00072 #define SOC_TOO_LOW 20
00073 #define SOC_CRITICAL 3
00074
00075 #define CEF           1.0f
00076 #define PEUKERT       1.0f
00077
00078 typedef struct C_data { // real calculated variables
00079     float calc[ADC_BUFFER_SIZE];
00080     float c_load, c_bat, c_pv, c_mppt, v_cc, v_pv, v_bat, v_cbus, v_bbat, v_temp, v_inverter,
00081     v_sensor, v_cmode, bv_ror, bc_ror;
00082     float p_load, p_inverter, p_pv, p_bat, p_mppt, start_power;
00083     float t_comp, esr;
00084     float bank_ah, dynamic_ah, pv_ah, loadah, dynamic_ah_adj, dynamic_ah_daily, dynamic_ah_adj_daily;
00085     float bkwi, bkwo, pvkw, invkw, loadkw;
00086     uint16_t runtime, soc;
00087     bool update;
00088     hist_type hist[1];
00089     float load_i1, load_i2, bv_noload, bv_one_load, bv_full_load;
00090     bool day, dupdate;
00091     time_t day_start, day_end, day_update, updates;
00092 } C_data;
00093
00094 typedef enum {
00095     WIDE_ZERO = 1,
00096     NO_NEG = 2,
00097     WIDE_NEG = 4,
00098     SKIP = 8,
00099 } FIX_CODES;
00100
00101 extern volatile C_data C;
00102
00103 float lp_filter(const float, const uint8_t, const int8_t);
00104 void convert_adc_data(void);
00105 void calc_model_data(void);
00106 void calc_ror_data(void);
00107 void static_soc(void);
00108 void set_load_relay_zero(const bool);
00109 void set_load_relay_one(const bool);
00110 void set_load_relay_two(const bool);
00111 void set_ac_charger_relay(const bool);
00112 bool get_ac_charger_relay(void);
00113 bool check_day_time(void);
00114 void load_hist_data(void);
00115 float calc_fixups(float data, FIX_CODES fixup);
00116
00117 char spinners(uint8_t, uint8_t);
00118 time_t time(time_t *);
00119 void set_time(const time_t t);
00120 #endif

```

4.14 adcc.c

```

00001
00002
00003 /*
00004  (c) 2018 Microchip Technology Inc. and its subsidiaries.
00005
00006  Subject to your compliance with these terms, you may use Microchip software and any
00007  derivatives exclusively with Microchip products. It is your responsibility to comply with third
00008  party
00009  license terms applicable to your use of third party software (including open source software) that
00010
00011  may accompany Microchip software.
00012
00013

```

```

00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "adcc.h"
00053 #include "interrupt_manager.h"
00054
00058 void (*ADCC_AD_I_InterruptHandler)(void);
00059 void (*ADCC_ADTI_InterruptHandler)(void);
00060
00064
00065 void ADCC_Initialize(void)
00066 {
00067     // set the ADCC to the options selected in the User Interface
00068     // ADLTH 0;
00069     ADLTHL = 0x00;
00070     // ADLTH 0;
00071     ADLTHH = 0x00;
00072     // ADUTH 0;
00073     ADUTHL = 0x00;
00074     // ADUTH 0;
00075     ADUTHH = 0x00;
00076     // ADSTPT 0;
00077     ADSTPTL = 0x00;
00078     // ADSTPT 0;
00079     ADSTPTH = 0x00;
00080     // ADACC 0;
00081     ADACCU = 0x00;
00082     // ADRPT 64;
00083     ADRPT = 0x40;
00084     // ADPCH ANA0;
00085     ADPCH = 0x00;
00086     // ADACQ 232;
00087     ADACQL = 0xE8;
00088     // ADACQ 3;
00089     ADACQH = 0x03;
00090     // ADCAP Additional uC disabled;
00091     ADCAP = 0x00;
00092     // ADPRE 0;
00093     ADPREL = 0x00;
00094     // ADPRE 0;
00095     ADPREH = 0x00;
00096     // ADDSEN disabled; ADGPOL digital_low; ADIPEN disabled; ADPPOL Vss;
00097     ADCON1 = 0x00;
00098     // ADCRS 5; ADMD Burst_average_mode; ADACLRE enabled; ADPSIS RES;
00099     ADCON2 = 0x5B;
00100     // ADCALC Actual result vs filtered value; ADTMD enabled; ADSOI ADGO not cleared;
00101     ADCON3 = 0x27;
00102     // ADMATH registers not updated;
00103     ADSTAT = 0x00;
00104     // ADNREF VSS; ADPREF external;
00105     ADREF = 0x02;
00106     // ADACT disabled;
00107     ADACT = 0x00;
00108     // ADCS FOSC/128;
00109     ADCLK = 0x3F;
00110     // ADGO stop; ADFM right; ADON enabled; ADCS FOSC/ADCLK; ADCONT disabled;
00111     ADCON0 = 0x84;
00112
00113     // Clear the ADC interrupt flag
00114     PIR1bits.ADIF = 0;
00115     // Enabling ADCC interrupt.
00116     PIE1bits.ADIE = 1;
00117
00118     ADCC_SetADIInterruptHandler(ADCC_DefaultInterruptHandler);
00119
00120     // Clear the ADC Threshold interrupt flag
00121     PIR1bits.ADTIF = 0;
00122     // Enabling ADCC threshold interrupt.
00123     PIE1bits.ADTIE = 1;
00124
00125     ADCC_SetADTIInterruptHandler(ADCC_DefaultInterruptHandler);
00126 }
00127

```

```

00128 void ADCC_StartConversion(adcc_channel_t channel)
00129 {
00130     // select the A/D channel
00131     ADPCH = channel;
00132
00133     // Turn on the ADC module
00134     ADCON0bits.ADON = 1;
00135
00136     // Start the conversion
00137     ADCON0bits.ADGO = 1;
00138 }
00139
00140 bool ADCC_IsConversionDone(void)
00141 {
00142     // Start the conversion
00143     return ((unsigned char) (!ADCON0bits.ADGO));
00144 }
00145
00146 adc_result_t ADCC_GetConversionResult(void)
00147 {
00148     // Return the result
00149     return ((adc_result_t) ((ADRESH « 8) + ADRESL));
00150 }
00151
00152 adc_result_t ADCC_GetSingleConversion(adcc_channel_t channel)
00153 {
00154     // select the A/D channel
00155     ADPCH = channel;
00156
00157     // Turn on the ADC module
00158     ADCON0bits.ADON = 1;
00159
00160     //Disable the continuous mode.
00161     ADCON0bits.ADCONT = 0;
00162
00163     // Start the conversion
00164     ADCON0bits.ADGO = 1;
00165
00166
00167     // Wait for the conversion to finish
00168     while (ADCON0bits.ADGO)
00169     {
00170     }
00171
00172
00173     // Conversion finished, return the result
00174     return ((adc_result_t) ((ADRESH « 8) + ADRESL));
00175 }
00176
00177 void ADCC_StopConversion(void)
00178 {
00179     //Reset the ADGO bit.
00180     ADCON0bits.ADGO = 0;
00181 }
00182
00183 void ADCC_SetStopOnInterrupt(void)
00184 {
00185     //Set the ADSOI bit.
00186     ADCON3bits.ADSOI = 1;
00187 }
00188
00189 void ADCC_DischargeSampleCapacitor(void)
00190 {
00191     //Set the ADC channel to AVss.
00192     ADPCH = 0x3b;
00193 }
00194
00195 void ADCC_LoadAcquisitionRegister(uint16_t acquisitionValue)
00196 {
00197     //Load the ADACQH and ADACQL registers.
00198     ADACQH = (uint8_t) (acquisitionValue « 8);
00199     ADACQL = (uint8_t) acquisitionValue;
00200 }
00201
00202 void ADCC_SetPrechargeTime(uint16_t prechargeTime)
00203 {
00204     //Load the ADPREH and ADPREL registers.
00205     ADPREH = (uint8_t) (prechargeTime « 8);
00206     ADPREL = (uint8_t) prechargeTime;
00207 }
00208
00209 void ADCC_SetRepeatCount(uint8_t repeatCount)
00210 {
00211     //Load the ADRPT register.
00212     ADRPT = repeatCount;
00213 }
00214

```

```

00215 uint8_t ADCC_GetCurrentCountofConversions(void)
00216 {
00217     //Return the contents of ADCNT register
00218     return ADCNT;
00219 }
00220
00221 void ADCC_ClearAccumulator(void)
00222 {
00223     //Reset the ADCON2bits.ADACLR bit.
00224     ADCON2bits.ADACLR = 1;
00225 }
00226
00227 uint24_t ADCC_GetAccumulatorValue(void)
00228 {
00229     //Return the contents of ADACCU, ADACCH and ADACCL registers
00230     return ((uint24_t)ADACCU « 16)+((uint24_t)ADACCH « 8) + ADACCL;
00231 }
00232
00233 bool ADCC_HasAccumulatorOverflowed(void)
00234 {
00235     //Return the status of ADSTATbits.ADAOV
00236     return ADSTATbits.ADAOV;
00237 }
00238
00239 uint16_t ADCC_GetFilterValue(void)
00240 {
00241     //Return the contents of ADFLTRH and ADFLTRL registers
00242     return ((uint16_t)(ADFLTRH « 8) + ADFLTRL);
00243 }
00244
00245 uint16_t ADCC_GetPreviousResult(void)
00246 {
00247     //Return the contents of ADPREVH and ADPREVL registers
00248     return ((uint16_t)(ADPREVH « 8) + ADPREVL);
00249 }
00250
00251 void ADCC_DefineSetPoint(uint16_t setPoint)
00252 {
00253     //Sets the ADSTPTH and ADSTPTL registers
00254     ADSTPTH = (uint8_t) (setPoint » 8);
00255     ADSTPTL = (uint8_t) setPoint;
00256 }
00257
00258 void ADCC_SetUpperThreshold(uint16_t upperThreshold)
00259 {
00260     //Sets the ADUTHH and ADUTHL registers
00261     ADUTHH = (uint8_t) (upperThreshold » 8);
00262     ADUTHL = (uint8_t) (upperThreshold);
00263 }
00264
00265 void ADCC_SetLowerThreshold(uint16_t lowerThreshold)
00266 {
00267     //Sets the ADLTHH and ADLTHL registers
00268     ADLTHH = (uint8_t) (lowerThreshold » 8);
00269     ADLTHL = (uint8_t) lowerThreshold;
00270 }
00271
00272 uint16_t ADCC_GetErrorCalculation(void)
00273 {
00274     //Return the contents of ADERRH and ADERRL registers
00275     return ((uint16_t)(ADERRH « 8) + ADERRL);
00276 }
00277
00278 void ADCC_EnableDoubleSampling(void)
00279 {
00280     //Sets the ADCON1bits.ADDSEN
00281     ADCON1bits.ADDSEN = 1;
00282 }
00283
00284 void ADCC_EnableContinuousConversion(void)
00285 {
00286     //Sets the ADCON0bits.ADCONT
00287     ADCON0bits.ADCONT = 1;
00288 }
00289
00290 void ADCC_DisableContinuousConversion(void)
00291 {
00292     //Resets the ADCON0bits.ADCONT
00293     ADCON0bits.ADCONT = 0;
00294 }
00295
00296 bool ADCC_HasErrorCrossedUpperThreshold(void)
00297 {
00298     //Returns the value of ADSTATbits.ADUTHR bit.
00299     return ADSTATbits.ADUTHR;
00300 }
00301

```



```

00302 bool ADCC_HasErrorCrossedLowerThreshold(void)
00303 {
00304     //Returns the value of ADSTATbits.ADLTHR bit.
00305     return ADSTATbits.ADLTHR;
00306 }
00307
00308 uint8_t ADCC_GetConversionStageStatus(void)
00309 {
00310     //Returns the contents of ADSTATbits.ADSTAT field.
00311     return ADSTATbits.ADSTAT;
00312 }
00313
00314 void __interrupt (irq(AD),base(8)) ADCC_ISR()
00315 {
00316     // Clear the ADCC interrupt flag
00317     PIR1bits.ADIF = 0;
00318
00319     if (ADCC_ADI_InterruptHandler)
00320         ADCC_ADI_InterruptHandler();
00321 }
00322
00323 void ADCC_SetADIInterruptHandler(void (* InterruptHandler)(void)){
00324     ADCC_ADI_InterruptHandler = InterruptHandler;
00325 }
00326
00327 void __interrupt (irq(ADT),base(8)) ADCC_ThresholdISR()
00328 {
00329     // Clear the ADCC Threshold interrupt flag
00330     PIR1bits.ADTIF = 0;
00331
00332     if (ADCC_ADTI_InterruptHandler)
00333         ADCC_ADTI_InterruptHandler();
00334 }
00335
00336 void ADCC_SetADTIInterruptHandler(void (* InterruptHandler)(void)){
00337     ADCC_ADTI_InterruptHandler = InterruptHandler;
00338 }
00339 void ADCC_DefaultInterruptHandler(void){
00340     // add your ADCC interrupt custom code
00341     // or set custom function using ADCC_SetADIInterruptHandler() or ADCC_SetADTIInterruptHandler()
00342 }

```

4.15 adcc.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047 #ifndef ADCC_H
00048 #define ADCC_H
00049
00053
00054 #include <xc.h>
00055 #include <stdint.h>
00056 #include <stdbool.h>
00057
00058 #ifdef __cplusplus // Provide C++ Compatibility
00059     extern "C" {
00060
00061

```

```
00062 #endif
00063
00067
00071
00072 typedef uint16_t adc_result_t;
00073 #ifndef uint24_t
00074 typedef __uint24 uint24_t;
00075 #endif
00076
00088
00089 typedef enum
00090 {
00091     C_BATT = 0x0,
00092     C_PV = 0x1,
00093     V_CC = 0x2,
00094     V_BAT = 0x4,
00095     V_PV = 0x5,
00096     V_CMODE = 0x6,
00097     V_BBAT = 0xA,
00098     C_MPPT = 0xB,
00099     V_INVERTER = 0xC,
00100     V_LIGHT_SENSOR = 0xD,
00101     channel_VSS = 0x3B,
00102     channel_Temp = 0x3C,
00103     channel_DAC1 = 0x3D,
00104     channel_FVR_Buffer1 = 0x3E,
00105     channel_FVR_Buffer2 = 0x3F
00106 } adcc_channel_t;
00107
00111
00140 void ADCC_Initialize(void);
00141
00169 void ADCC_StartConversion(adcc_channel_t channel);
00170
00199 bool ADCC_IsConversionDone(void);
00200
00231 adc_result_t ADCC_GetConversionResult(void);
00232
00262 adc_result_t ADCC_GetSingleConversion(adcc_channel_t channel);
00263
00287 void ADCC_StopConversion(void);
00288
00314 void ADCC_SetStopOnInterrupt(void);
00315
00339 void ADCC_DischargeSampleCapacitor(void);
00340
00365 void ADCC_LoadAcquisitionRegister(uint16_t);
00366
00391 void ADCC_SetPrechargeTime(uint16_t);
00392
00416 void ADCC_SetRepeatCount(uint8_t);
00417
00444 uint8_t ADCC_GetCurrentCountofConversions(void);
00445
00468 void ADCC_ClearAccumulator(void);
00469
00493 uint24_t ADCC_GetAccumulatorValue(void);
00494
00521 bool ADCC_HasAccumulatorOverflowed(void);
00522
00546 uint16_t ADCC_GetFilterValue(void);
00547
00574 uint16_t ADCC_GetPreviousResult(void);
00575
00600 void ADCC_DefineSetPoint(uint16_t);
00601
00626 void ADCC_SetUpperThreshold(uint16_t);
00627
00652 void ADCC_SetLowerThreshold(uint16_t);
00653
00679 uint16_t ADCC_GetErrorCalculation(void);
00680
00706 void ADCC_EnableDoubleSampling(void);
00707
00730 void ADCC_EnableContinuousConversion(void);
00731
00754 void ADCC_DisableContinuousConversion(void);
00755
00782 bool ADCC_HasErrorCrossedUpperThreshold(void);
00783
00810 bool ADCC_HasErrorCrossedLowerThreshold(void);
00811
00837 uint8_t ADCC_GetConversionStageStatus(void);
00838
00839
00840
00854 void ADCC_SetADIInterruptHandler(void (* InterruptHandler)(void));
```

```

00855
00856
00870 void ADCC_SetADTIInterruptHandler(void (* InterruptHandler)(void));
00871
00872
00889 void ADCC_DefaultInterruptHandler(void);
00890 #ifdef __cplusplus // Provide C++ Compatibility
00891     }
00892 }
00893 #endif
00894 #endif
00895
00896 #endif //ADCC_H
00900

```

4.16 clkref.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "clkref.h"
00053
00057
00058 void CLKREF_Initialize(void)
00059 {
00060     // CLK FOSC;
00061     CLKRCLK = 0x00;
00062     // DC 50% Duty Cycle; DIV BaseClock/16; CLKREN enabled;
00063     CLKRCON = 0x94;
00064 }

```

4.17 clkref.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND

```

```

00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef CLKREF_H
00048 #define CLKREF_H
00049
00053
00054 #include <stdbool.h>
00055 #include <stdint.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00060
00061 #endif
00062
00066
00092 void CLKREF_Initialize(void);
00093
00094 #ifdef __cplusplus // Provide C++ Compatibility
00095     }
00096
00097 #endif
00098 #endif // CLKREF_H
00104
00105

```

4.18 crc.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "crc.h"
00053
00057
00058 typedef struct
00059 {
00060     uint8_t dataWidth;
00061     uint8_t polyWidth;
00062     uint8_t seedDirection;
00063 }CRC_OBJ;
00064
00068
00069 static CRC_OBJ crcObj;
00070
00071
00075
00076 void CRC_Initialize(void)
00077 {
00078     uint16_t poly;

```

```

00079     uint16_t seed;
00080
00081     // SHIFTM shift left; CRCGO disabled; CRCEN enabled; ACCM data not augmented with 0's;
00082     CRCCON0 = 0x80;
00083     // DLEN 0; PLEN 7;
00084     CRCCON1 = (0 « 4) | (7);
00085     //
00086     CRCACCL = 0x00;
00087     //
00088     CRCACCH = 0x00;
00089     // MREG Program Flash Memory; SGO disabled; TRIGEN disabled; BURSTMD CRC request and Trigger; EN
disabled;
00090     SCANCON0 = 0x00;
00091     // LADR 0;
00092     SCANLADRU = 0x00;
00093     // LADR 0;
00094     SCANLADRH = 0x00;
00095     // LADR 0;
00096     SCANLADRL = 0x00;
00097     // HADR 63;
00098     SCANHADRU = 0x3F;
00099     // HADR 255;
00100     SCANHADRH = 0xFF;
00101     // HADR 255;
00102     SCANHADRL = 0xFF;
00103     // TSEL LFINTOSC;
00104     SCANTRIG = 0x00;
00105
00106     poly = 213;
00107     CRCXORH = poly » 8;
00108     CRCXORL = poly;
00109
00110     seed = 0;
00111     // CRCACC 0
00112     CRCACCH = seed » 8;
00113     CRCACCL = seed;
00114
00115     crcObj.dataWidth = (uint8_t)(CRCCON1bits.DLEN + 1);
00116     crcObj.polyWidth = (uint8_t)(CRCCON1bits.PLEN + 1);
00117 }
00118
00119 void CRC_Start(void)
00120 {
00121     // Start the serial shifter
00122     CRCCON0bits.CRCGO = 1;
00123 }
00124
00125 bool CRC_8BitDataWrite(uint8_t data)
00126 {
00127     if(!CRCCON0bits.FULL)
00128     {
00129         CRCDATL = data;
00130         return(1);
00131     }
00132     else
00133     {
00134         return(0);
00135     }
00136 }
00137
00138 static uint16_t CRC_ReverseValue(uint16_t crc)
00139 {
00140     uint16_t mask;
00141     uint16_t reverse;
00142
00143     mask = 1;
00144     mask «= crcObj.polyWidth - 1;
00145     reverse = 0;
00146
00147     while(crc)
00148     {
00149         if(crc & 0x01)
00150         {
00151             reverse |= mask;
00152         }
00153         mask »= 1;
00154         crc »= 1;
00155     }
00156     return reverse;
00157 }
00158
00159 uint16_t CRC_CalculatedResultGet(bool reverse, uint16_t xorValue)
00160 {
00161     uint16_t result,mask;
00162
00163     // augmentation logic
00164     CRCCON0bits.CRCGO = 0;

```

```

00165     PIR0bits.CRCIF = 0;
00166     CRCCON1bits.DLEN= CRCCON1bits.PLEN;
00167
00168     if(crcObj.polyWidth > 8)
00169     {
00170         CRCDATH = 0;
00171         CRCDATL = 0;
00172     }
00173     else
00174     {
00175         CRCDATL = 0;
00176     }
00177
00178     CRCCON0bits.CRCGO = 1;
00179     while(PIR0bits.CRCIF == 0);
00180     PIR0bits.CRCIF = 0;
00181
00182     CRCCON0bits.CRCGO = 0;
00183     CRCCON1bits.DLEN = (uint8_t)(crcObj.dataWidth - 1);
00184     // Read result
00185     result = ((uint16_t)CRCACCH « 8) |CRCACCL;
00186     if(crcObj.polyWidth < 16)
00187     {
00188         // Polynomial mask
00189         mask = (uint16_t)((1 « crcObj.polyWidth) - 1);
00190         result &= mask;
00191     }
00192
00193     if(reverse)
00194         result = CRC_ReverseValue(result);
00195
00196     result ^= xorValue;
00197
00198     return result;
00199 }
00200
00201 bool CRC_IsBusy(void)
00202 {
00203     // Is shifting in progress?
00204     return(CRCCON0bits.BUSY);
00205 }
00206
00207 void CRC_SCAN_StartScanner(void)
00208 {
00209     // Start the CRC serial shifter
00210     CRCCON0bits.CRCGO = 1;
00211
00212     // Start the scanner
00213     SCANCON0bits.SGO = 1;
00214 }
00215
00216 void CRC_SCAN_StopScanner(void)
00217 {
00218     // Stop the scanner
00219     SCANCON0bits.SGO = 0;
00220
00221     // Stop the CRC serial shifter
00222     CRCCON0bits.CRCGO = 0;
00223 }
00224
00225 void CRC_SCAN_SetAddressLimit(uint32_t startAddr, uint32_t endAddr)
00226 {
00227     // Load end address limit
00228     SCANHADRU = 0x3F & (endAddr » 16);
00229     SCANHADRH = 0xFF & (endAddr » 8);
00230     SCANHADRL = 0xFF & endAddr;
00231
00232     // Load start address limit
00233     SCANLADRU = 0x3F & (startAddr » 16);
00234     SCANLADRH = 0xFF & (startAddr » 8);
00235     SCANLADRL = 0xFF & startAddr;
00236 }
00237
00238
00239 bool CRC_SCAN_IsScannerBusy(void)
00240 {
00241     // Is scanner in progress?
00242     return(SCANCON0bits.BUSY);
00243 }
00244
00245 bool CRC_SCAN_HasScanCompleted(void)
00246 {
00247     // Has scanning completed?
00248     bool status = (unsigned char)(PIR0bits.SCANIF && PIR0bits.CRCIF);
00249     if(status)
00250     {
00251         PIR0bits.CRCIF = 0;

```

```

00252         PIRObits.SCANIF = 0;
00253     }
00254     return(status);
00255 }
00256
00257

```

4.19 crc.h

```

00001
00002
00003
00004 /*
00005     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00006
00007     Subject to your compliance with these terms, you may use Microchip software and any
00008     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00009     party
00010     license terms applicable to your use of third party software (including open source software) that
00011     may accompany Microchip software.
00012
00013     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00014     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00015     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00016     FOR A PARTICULAR PURPOSE.
00017
00018     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00019     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00020     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00021     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00022     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00023     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00024     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00025     SOFTWARE.
00026 */
00027
00028 #ifndef CRC_H
00029 #define CRC_H
00030
00031 #include <xc.h>
00032 #include <stdint.h>
00033 #include <stdbool.h>
00034
00035 // Provide C++ compatability
00036 #ifdef __cplusplus
00037 extern "C" {
00038 #endif
00039
00040 #define NORMAL 0
00041 #define REVERSE 1
00042
00043 void CRC_Initialize(void);
00044
00045 void CRC_Start(void);
00046
00047 bool CRC_8BitDataWrite(uint8_t data);
00048
00049 uint16_t CRC_CalculatedResultGet(bool reverse, uint16_t xorValue);
00050
00051 bool CRC_IsBusy(void);
00052
00053 void CRC_SCAN_StartScanner(void);
00054
00055 void CRC_SCAN_StopScanner(void);
00056
00057 void CRC_SCAN_SetAddressLimit(uint32_t startAddr, uint32_t endAddr);
00058
00059 bool CRC_SCAN_IsScannerBusy(void);
00060
00061 bool CRC_SCAN_HasScanCompleted(void);
00062
00063 #ifdef __cplusplus
00064 }
00065 #endif
00066
00067 #endif /* _CRC_H */

```

4.20 device_config.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 // Configuration bits: selected in the GUI
00048
00049 // CONFIG1L
00050 #pragma config FEXTOSC = ECH      // External Oscillator Selection->EC (external clock) above 8 MHz; PFM
    set to high power
00051 #pragma config RSTOSC = EXTOSC_4PLL    // Reset Oscillator Selection->EXTOSC with 4x PLL, with EXTOSC
    operating per FEXTOSC bits
00052
00053 // CONFIG1H
00054 #pragma config CLKOUTEN = OFF        // Clock out Enable bit->CLKOUT function is disabled
00055 #pragma config PR1WAY = OFF         // PRLOCKED One-Way Set Enable bit->PRLOCK bit can be set and cleared
    repeatedly
00056 #pragma config CSWEN = ON           // Clock Switch Enable bit->Writing to NOSC and NDIV is allowed
00057 #pragma config FCMEN = ON           // Fail-Safe Clock Monitor Enable bit->Fail-Safe Clock Monitor enabled
00058
00059 // CONFIG2L
00060 #pragma config MCLRE = EXTMCCLR     // MCLR Enable bit->If LVP = 0, MCLR pin is MCLR; If LVP = 1, RE3
    pin function is MCLR
00061 #pragma config PWRTS = PWRT_OFF     // Power-up timer selection bits->PWRT is disabled
00062 #pragma config MVECEEN = ON         // Multi-vector enable bit->Multi-vector enabled, Vector table used for
    interrupts
00063 #pragma config IVT1WAY = ON         // IVTLOCK bit One-way set enable bit->IVTLOCK bit can be cleared and
    set only once
00064 #pragma config LPBOREN = OFF        // Low Power BOR Enable bit->ULPBOR disabled
00065 #pragma config BOREN = SBORDIS      // Brown-out Reset Enable bits->Brown-out Reset enabled , SBOREN bit
    is ignored
00066
00067 // CONFIG2H
00068 #pragma config BORV = VBOR_2P45     // Brown-out Reset Voltage Selection bits->Brown-out Reset Voltage
    (VBOR) set to 2.45V
00069 #pragma config ZCD = OFF            // ZCD Disable bit->ZCD disabled. ZCD can be enabled by setting the ZCDSEN
    bit of ZCDCON
00070 #pragma config PPS1WAY = ON         // PPSLOCK bit One-Way Set Enable bit->PPSLOCK bit can be cleared and
    set only once; PPS registers remain locked after one clear/set cycle
00071 #pragma config STVREN = ON          // Stack Full/Underflow Reset Enable bit->Stack full/underflow will
    cause Reset
00072 #pragma config DEBUG = OFF          // Debugger Enable bit->Background debugger disabled
00073 #pragma config XINST = OFF          // Extended Instruction Set Enable bit->Extended Instruction Set and
    Indexed Addressing Mode disabled
00074
00075 // CONFIG3L
00076 #pragma config WDTCPSS = WDTCPSS_15 // WDT Period selection bits->Divider ratio 1:1048576
00077 #pragma config WDTE = OFF           // WDT operating mode->WDT Disabled; SWDTEN is ignored
00078
00079 // CONFIG3H
00080 #pragma config WDTCS = WDTCS_7      // WDT Window Select bits->>window always open (100%); software
    control; keyed access not required
00081 #pragma config WDTCCS = LFINTOSC     // WDT input clock selector->WDT reference clock is the 31.0 kHz
    LFINTOSC
00082
00083 // CONFIG4L
00084 #pragma config BBSIZE = BBSIZE_512 // Boot Block Size selection bits->Boot Block size is 512 words
00085 #pragma config BBEN = OFF           // Boot Block enable bit->Boot block disabled
00086 #pragma config SAFEN = OFF          // Storage Area Flash enable bit->SAF disabled
00087 #pragma config WRTAPP = OFF          // Application Block write protection bit->Application Block not write
    protected
00088

```



```

00089 // CONFIG4H
00090 #pragma config WRWB = OFF      // Boot Block Write Protection bit->Boot Block not write-protected
00091 #pragma config WRTC = OFF      // Configuration Register Write Protection bit->Configuration registers
                                // not write-protected
00092 #pragma config WRWD = OFF      // Data EEPROM Write Protection bit->Data EEPROM not write-protected
00093 #pragma config WRTSAF = OFF    // SAF Write protection bit->SAF not Write Protected
00094 #pragma config LVP = OFF       // Low Voltage Programming Enable bit->HV on MCLR/VPP must be used for
                                // programming
00095
00096 // CONFIG5L
00097 #pragma config CP = OFF        // PFM and Data EEPROM Code Protection bit->PFM and Data EEPROM code
                                // protection disabled

```

4.21 device_config.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef DEVICE_CONFIG_H
00048 #define DEVICE_CONFIG_H
00049
00050 #define _XTAL_FREQ 64000000
00051
00052 #endif /* DEVICE_CONFIG_H */

```

4.22 dma1.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046

```

```

00050
00051 #include <xc.h>
00052 #include "dma1.h"
00053
00054 void (*DMA1_SCNTI_InterruptHandler)(void);
00055 void (*DMA1_DCNTI_InterruptHandler)(void);
00056 void (*DMA1_AI_InterruptHandler)(void);
00057 void (*DMA1_ORI_InterruptHandler)(void);
00058
00062
00063 void DMA1_Initialize(void)
00064 {
00065     //Source Address : lcd_dma_buf
00066     DMA1SSA = (volatile __uint24) &lcd_dma_buf;
00067     //Destination Address : &SPI1TXB
00068     DMA1DSA = (volatile unsigned short) &SPI1TXB;
00069     //DMODE unchanged; DSTP not cleared; SMR GPR; SMODE incremented; SSTP cleared;
00070     DMA1CON1 = 0x03;
00071     //Source Message Size : 1
00072     DMA1SSZ = 1;
00073     //Destination Message Size : 1
00074     DMA1DSZ = 1;
00075     //Start Trigger : SIRQ SPI1TX;
00076     DMA1SIRQ = 0x15;
00077     //Abort Trigger : AIRQ None;
00078     DMA1AIRQ = 0x00;
00079
00080     // Clear Destination Count Interrupt Flag bit
00081     PIR2bits.DMA1DCNTIF = 0;
00082     // Clear Source Count Interrupt Flag bit
00083     PIR2bits.DMA1SCNTIF = 0;
00084     // Clear Abort Interrupt Flag bit
00085     PIR2bits.DMA1AIF = 0;
00086     // Clear Overrun Interrupt Flag bit
00087     PIR2bits.DMA1ORIF = 0;
00088
00089     PIE2bits.DMA1DCNTIE = 1;
00090     DMA1_SetDCNTIInterruptHandler(DMA1_DefaultInterruptHandler);
00091     PIE2bits.DMA1SCNTIE = 1;
00092     DMA1_SetSCNTIInterruptHandler(DMA1_DefaultInterruptHandler);
00093     PIE2bits.DMA1AIE = 1;
00094     DMA1_SetAIInterruptHandler(DMA1_DefaultInterruptHandler);
00095     PIE2bits.DMA1ORIE = 1;
00096     DMA1_SetORIInterruptHandler(DMA1_DefaultInterruptHandler);
00097
00098     //EN enabled; SIRQEN enabled; DGO not in progress; AIRQEN disabled;
00099     DMA1CON0 = 0xC0;
00100
00101 }
00102
00103 void DMA1_SelectSourceRegion(uint8_t region)
00104 {
00105     DMA1CON1bits.SMR = region;
00106 }
00107
00108 void DMA1_SetSourceAddress(uint24_t address)
00109 {
00110     DMA1SSA = address;
00111 }
00112
00113 void DMA1_SetDestinationAddress(uint16_t address)
00114 {
00115     DMA1DSA = address;
00116 }
00117
00118 void DMA1_SetSourceSize(uint16_t size)
00119 {
00120     DMA1SSZ = size;
00121 }
00122
00123 void DMA1_SetDestinationSize(uint16_t size)
00124 {
00125     DMA1DSZ = size;
00126 }
00127
00128 uint24_t DMA1_GetSourcePointer(void)
00129 {
00130     return DMA1SPTR;
00131 }
00132
00133 uint16_t DMA1_GetDestinationPointer(void)
00134 {
00135     return DMA1DPTR;
00136 }
00137
00138 void DMA1_SetStartTrigger(uint8_t sirq)
00139 {

```

```

00140     DMA1SIRQ = sirq;
00141 }
00142
00143 void DMA1_SetAbortTrigger(uint8_t airq)
00144 {
00145     DMA1AIRQ = airq;
00146 }
00147
00148 void DMA1_StartTransfer(void)
00149 {
00150     DMA1CON0bits.DGO = 1;
00151 }
00152
00153 void DMA1_StartTransferWithTrigger(void)
00154 {
00155     DMA1CON0bits.SIRQEN = 1;
00156 }
00157
00158 void DMA1_StopTransfer(void)
00159 {
00160     DMA1CON0bits.SIRQEN = 0;
00161     DMA1CON0bits.DGO = 0;
00162 }
00163
00164 void DMA1_SetDMAPriority(uint8_t priority)
00165 {
00166     // This function is dependant on the PR1WAY CONFIG bit
00167     PRLOCK = 0x55;
00168     PRLOCK = 0xAA;
00169     PRLOCKbits.PRLOCKED = 0;
00170     DMA1PR = priority;
00171     PRLOCK = 0x55;
00172     PRLOCK = 0xAA;
00173     PRLOCKbits.PRLOCKED = 1;
00174 }
00175
00176 void __interrupt (irq (IRQ_DMA1SCNT), base(8)) DMA1_DMASCNTI_ISR()
00177 {
00178     // Clear the source count interrupt flag
00179     PIR2bits.DMA1SCNTIF = 0;
00180
00181     if (DMA1_SCNTI_InterruptHandler)
00182         DMA1_SCNTI_InterruptHandler();
00183 }
00184
00185 void DMA1_SetSCNTIInterruptHandler(void (* InterruptHandler)(void))
00186 {
00187     DMA1_SCNTI_InterruptHandler = InterruptHandler;
00188 }
00189
00190 void __interrupt (irq (IRQ_DMA1DCNT), base(8)) DMA1_DMADCNTI_ISR()
00191 {
00192     // Clear the source count interrupt flag
00193     PIR2bits.DMA1DCNTIF = 0;
00194
00195     if (DMA1_DCNTI_InterruptHandler)
00196         DMA1_DCNTI_InterruptHandler();
00197 }
00198
00199 void DMA1_SetDCNTIInterruptHandler(void (* InterruptHandler)(void))
00200 {
00201     DMA1_DCNTI_InterruptHandler = InterruptHandler;
00202 }
00203
00204 void __interrupt (irq (IRQ_DMA1A), base(8)) DMA1_DMAAI_ISR()
00205 {
00206     // Clear the source count interrupt flag
00207     PIR2bits.DMA1AIF = 0;
00208
00209     if (DMA1_AI_InterruptHandler)
00210         DMA1_AI_InterruptHandler();
00211 }
00212
00213 void DMA1_SetAIInterruptHandler(void (* InterruptHandler)(void))
00214 {
00215     DMA1_AI_InterruptHandler = InterruptHandler;
00216 }
00217
00218 void __interrupt (irq (IRQ_DMA1OR), base(8)) DMA1_DMAORI_ISR()
00219 {
00220     // Clear the source count interrupt flag
00221     PIR2bits.DMA1ORIF = 0;
00222
00223     if (DMA1_ORI_InterruptHandler)
00224         DMA1_ORI_InterruptHandler();
00225 }
00226

```

```

00227 void DMA1_SetORIInterruptHandler(void (* InterruptHandler)(void))
00228 {
00229     DMA1_ORI_InterruptHandler = InterruptHandler;
00230 }
00231
00232 void DMA1_DefaultInterruptHandler(void){
00233     // add your DMA1 interrupt custom code
00234     // or set custom function using DMA1_SetSCNTIIInterruptHandler() /DMA1_SetDCNTIIInterruptHandler()
    /DMA1_SetAIIInterruptHandler() /DMA1_SetORIInterruptHandler()
00235 }

```

4.23 dma1.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
    party
00029     license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00032     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00033     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00034     FOR A PARTICULAR PURPOSE.
00035
00036     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00037     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00038     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00039     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00040     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00041     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00042     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00043     SOFTWARE.
00044 */
00045 */
00046
00047 #ifndef DMA1_H
00048 #define DMA1_H
00049
00053 #include <stdint.h>
00054
00055 uint8_t lcd_dma_buf[32];
00056
00063 void DMA1_Initialize(void);
00064
00070 void DMA1_SelectSourceRegion(uint8_t region);
00071
00077 void DMA1_SetSourceAddress(uint24_t address);
00078
00084 void DMA1_SetDestinationAddress(uint16_t address);
00085
00091 void DMA1_SetSourceSize(uint16_t size);
00092
00098 void DMA1_SetDestinationSize(uint16_t size);
00099
00105 uint24_t DMA1_GetSourcePointer(void);
00106
00112 uint16_t DMA1_GetDestinationPointer(void);
00113
00119 void DMA1_SetStartTrigger(uint8_t sirq);
00120
00126 void DMA1_SetAbortTrigger(uint8_t airq);
00127
00133 void DMA1_StartTransfer(void);
00134
00140 void DMA1_StartTransferWithTrigger(void);
00141
00147 void DMA1_StopTransfer(void);
00148
00154 void DMA1_SetDMAPriority(uint8_t priority);
00155
00161 void DMA1_SetSCNTIIInterruptHandler(void (* InterruptHandler)(void));
00162
00168 void DMA1_SetDCNTIIInterruptHandler(void (* InterruptHandler)(void));
00169
00175 void DMA1_SetAIIInterruptHandler(void (* InterruptHandler)(void));
00176
00182 void DMA1_SetORIInterruptHandler(void (* InterruptHandler)(void));
00188 void DMA1_DefaultInterruptHandler(void);
00189 #endif //DMA1_H

```

4.24 dma2.c

```

00001
00002
00003 /*
00004  (c) 2018 Microchip Technology Inc. and its subsidiaries.
00005
00006 Subject to your compliance with these terms, you may use Microchip software and any
00007 derivatives exclusively with Microchip products. It is your responsibility to comply with third
00008 party
00009 license terms applicable to your use of third party software (including open source software) that
00010
00011 may accompany Microchip software.
00012
00013 THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00014 EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00015 IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00016 FOR A PARTICULAR PURPOSE.
00017
00018 IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00019 INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00020 WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00021 HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00022 THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00023 CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00024 OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00025 SOFTWARE.
00026 */
00027
00028 #include <xc.h>
00029 #include "dma2.h"
00030
00031 void DMA2_Initialize(void)
00032 {
00033     //Source Address : uart1_dma_buf
00034     DMA2SSA = (volatile __uint24) &uart1_dma_buf;
00035     //Destination Address : &U1TXB
00036     DMA2DSA = (volatile unsigned short) &U1TXB;
00037     //DMODE unchanged; DSTP not cleared; SMR GPR; SMODE incremented; SSTP cleared;
00038     DMA2CON1 = 0x03;
00039     //Source Message Size : 1
00040     DMA2SSZ = 1;
00041     //Destination Message Size : 1
00042     DMA2DSZ = 1;
00043     //Start Trigger : SIRQ U1TX;
00044     DMA2SIRQ = 0x1C;
00045     //Abort Trigger : AIRQ None;
00046     DMA2AIRQ = 0x00;
00047
00048     // Clear Destination Count Interrupt Flag bit
00049     PIR5bits.DMA2DCNTIF = 0;
00050     // Clear Source Count Interrupt Flag bit
00051     PIR5bits.DMA2SCNTIF = 0;
00052     // Clear Abort Interrupt Flag bit
00053     PIR5bits.DMA2AIF = 0;
00054     // Clear Overrun Interrupt Flag bit
00055     PIR5bits.DMA2ORIF = 0;
00056
00057     PIE5bits.DMA2DCNTIE = 0;
00058     PIE5bits.DMA2SCNTIE = 0;
00059     PIE5bits.DMA2AIE = 0;
00060     PIE5bits.DMA2ORIE = 0;
00061
00062     //EN enabled; SIRQEN enabled; DGO not in progress; AIRQEN disabled;
00063     DMA2CON0 = 0xC0;
00064 }
00065
00066 void DMA2_SelectSourceRegion(uint8_t region)
00067 {
00068     DMA2CON1bits.SMR = region;
00069 }
00070
00071 void DMA2_SetSourceAddress(uint24_t address)
00072 {
00073     DMA2SSA = address;
00074 }
00075
00076 void DMA2_SetDestinationAddress(uint16_t address)
00077 {
00078     DMA2DSA = address;
00079 }
00080

```

```

00110 void DMA2_SetSourceSize(uint16_t size)
00111 {
00112     DMA2SSSZ= size;
00113 }
00114
00115 void DMA2_SetDestinationSize(uint16_t size)
00116 {
00117     DMA2DSZ= size;
00118 }
00119
00120 uint24_t DMA2_GetSourcePointer(void)
00121 {
00122     return DMA2SPTR;
00123 }
00124
00125 uint16_t DMA2_GetDestinationPointer(void)
00126 {
00127     return DMA2DPTR;
00128 }
00129
00130 void DMA2_SetStartTrigger(uint8_t sirq)
00131 {
00132     DMA2SIRQ = sirq;
00133 }
00134
00135 void DMA2_SetAbortTrigger(uint8_t airq)
00136 {
00137     DMA2AIRQ = airq;
00138 }
00139
00140 void DMA2_StartTransfer(void)
00141 {
00142     DMA2CON0bits.DGO = 1;
00143 }
00144
00145 void DMA2_StartTransferWithTrigger(void)
00146 {
00147     DMA2CON0bits.SIRQEN = 1;
00148 }
00149
00150 void DMA2_StopTransfer(void)
00151 {
00152     DMA2CON0bits.SIRQEN = 0;
00153     DMA2CON0bits.DGO = 0;
00154 }
00155
00156 void DMA2_SetDMAPriority(uint8_t priority)
00157 {
00158     // This function is dependant on the PR1WAY CONFIG bit
00159     PRLOCK = 0x55;
00160     PRLOCK = 0xAA;
00161     PRLOCKbits.PRLOCKED = 0;
00162     DMA2PR = priority;
00163     PRLOCK = 0x55;
00164     PRLOCK = 0xAA;
00165     PRLOCKbits.PRLOCKED = 1;
00166 }
00167

```

4.25 dma2.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
    party
00029     license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL

```

```

00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef DMA2_H
00048 #define DMA2_H
00049
00053 #include <stdint.h>
00054
00055 uint8_t uart1_dma_buf[32];
00056
00063 void DMA2_Initialize(void);
00064
00070 void DMA2_SelectSourceRegion(uint8_t region);
00071
00077 void DMA2_SetSourceAddress(uint24_t address);
00078
00084 void DMA2_SetDestinationAddress(uint16_t address);
00085
00091 void DMA2_SetSourceSize(uint16_t size);
00092
00098 void DMA2_SetDestinationSize(uint16_t size);
00099
00105 uint24_t DMA2_GetSourcePointer(void);
00106
00112 uint16_t DMA2_GetDestinationPointer(void);
00113
00119 void DMA2_SetStartTrigger(uint8_t sirq);
00120
00126 void DMA2_SetAbortTrigger(uint8_t airq);
00127
00133 void DMA2_StartTransfer(void);
00134
00140 void DMA2_StartTransferWithTrigger(void);
00141
00147 void DMA2_StopTransfer(void);
00148
00154 void DMA2_SetDMAPriority(uint8_t priority);
00155
00156
00157
00158
00159 #endif //DMA2_H

```

4.26 ext_int.c

```

00001
00023
00027 #include <xc.h>
00028 #include "ext_int.h"
00029 #include "interrupt_manager.h"
00030
00031 void (*INT0_InterruptHandler)(void);
00032 void (*INT1_InterruptHandler)(void);
00033
00034 void __interrupt(irq(INT0),base(8)) INT0_ISR()
00035 {
00036     EXT_INT0_InterruptFlagClear();
00037
00038     // Callback function gets called everytime this ISR executes
00039     INT0_CallBack();
00040 }
00041
00042
00043 void INT0_CallBack(void)
00044 {
00045     // Add your custom callback code here
00046     if(INT0_InterruptHandler)
00047     {
00048         INT0_InterruptHandler();
00049     }
00050 }
00051
00052 void INT0_SetInterruptHandler(void (* InterruptHandler)(void)){
00053     INT0_InterruptHandler = InterruptHandler;
00054 }
00055
00056 void INT0_DefaultInterruptHandler(void){
00057     // add your INT0 interrupt custom code
00058     // or set custom function using INT0_SetInterruptHandler()
00059 }
00060 void __interrupt(irq(INT1),base(8),low_priority) INT1_ISR()

```

```

00061 {
00062     EXT_INT1_InterruptFlagClear();
00063
00064     // Callback function gets called everytime this ISR executes
00065     INT1_CallBack();
00066 }
00067
00068
00069 void INT1_CallBack(void)
00070 {
00071     // Add your custom callback code here
00072     if(INT1_InterruptHandler)
00073     {
00074         INT1_InterruptHandler();
00075     }
00076 }
00077
00078 void INT1_SetInterruptHandler(void (* InterruptHandler)(void)){
00079     INT1_InterruptHandler = InterruptHandler;
00080 }
00081
00082 void INT1_DefaultInterruptHandler(void){
00083     // add your INT1 interrupt custom code
00084     // or set custom function using INT1_SetInterruptHandler()
00085 }
00086
00087 void EXT_INT_Initialize(void)
00088 {
00089
00090     // Clear the interrupt flag
00091     // Set the external interrupt edge detect
00092     EXT_INT0_InterruptFlagClear();
00093     EXT_INT0_fallingEdgeSet();
00094     // Set Default Interrupt Handler
00095     INT0_SetInterruptHandler(INT0_DefaultInterruptHandler);
00096     EXT_INT0_InterruptEnable();
00097
00098
00099     // Clear the interrupt flag
00100     // Set the external interrupt edge detect
00101     EXT_INT1_InterruptFlagClear();
00102     EXT_INT1_fallingEdgeSet();
00103     // Set Default Interrupt Handler
00104     INT1_SetInterruptHandler(INT1_DefaultInterruptHandler);
00105     EXT_INT1_InterruptEnable();
00106
00107 }
00108

```

4.27 ext_int.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef _EXT_INT_H
00048 #define _EXT_INT_H
00049
00053 #include <xc.h>
00054

```



```

00055 // Provide C++ Compatibility
00056 #ifdef __cplusplus
00057
00058 extern "C" {
00059
00060 #endif
00061
00093 #define EXT_INT0_InterruptFlagClear()      (PIR1bits.INT0IF = 0)
00094
00126 #define EXT_INT0_InterruptDisable()      (PIE1bits.INT0IE = 0)
00127
00157 #define EXT_INT0_InterruptEnable()      (PIE1bits.INT0IE = 1)
00158
00188 #define EXT_INT0_risingEdgeSet()      (INTCON0bits.INT0EDG = 1)
00189
00219 #define EXT_INT0_fallingEdgeSet()      (INTCON0bits.INT0EDG = 0)
00248 #define EXT_INT1_InterruptFlagClear()      (PIR5bits.INT1IF = 0)
00249
00281 #define EXT_INT1_InterruptDisable()      (PIE5bits.INT1IE = 0)
00282
00312 #define EXT_INT1_InterruptEnable()      (PIE5bits.INT1IE = 1)
00313
00343 #define EXT_INT1_risingEdgeSet()      (INTCON0bits.INT1EDG = 1)
00344
00374 #define EXT_INT1_fallingEdgeSet()      (INTCON0bits.INT1EDG = 0)
00406 void EXT_INT_Initialize(void);
00407
00411
00434 void INT0_CallBack(void);
00435
00457 void INT0_SetInterruptHandler(void (* InterruptHandler)(void));
00458
00481 extern void (*INT0_InterruptHandler)(void);
00482
00505 void INT0_DefaultInterruptHandler(void);
00506
00529 void INT1_CallBack(void);
00530
00552 void INT1_SetInterruptHandler(void (* InterruptHandler)(void));
00553
00576 extern void (*INT1_InterruptHandler)(void);
00577
00600 void INT1_DefaultInterruptHandler(void);
00601
00602 // Provide C++ Compatibility
00603 #ifdef __cplusplus
00604
00605 }
00606
00607 #endif
00608 #endif

```

4.28 fvr.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>

```

```

00052 #include "fvr.h"
00053
00057
00058 void FVR_Initialize(void)
00059 {
00060     // CDAFVR 4x; FVREN enabled; TSRNG Lo_range; ADFVR 4x; TSEN enabled;
00061     FVRCON = 0xAF;
00062 }
00063
00064 bool FVR_IsOutputReady(void)
00065 {
00066     return (FVRCONbits.FVRRDY);
00067 }
00071

```

4.29 fvr.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047 #ifndef FVR_H
00048 #define FVR_H
00049
00053
00054 #include <stdbool.h>
00055 #include <stdint.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00060
00061 #endif
00062
00066
00093 void FVR_Initialize(void);
00094
00127 bool FVR_IsOutputReady(void);
00128
00129 #ifdef __cplusplus // Provide C++ Compatibility
00130     }
00131 #endif
00132
00133 #endif
00134
00135 #endif // FVR_H
00139

```

4.30 interrupt_manager.c

```

00001
00025
00026 /*
00027     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00028

```

```

00029     Subject to your compliance with these terms, you may use Microchip software and any
00030     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00031 party
00031     license terms applicable to your use of third party software (including open source software) that
00032
00032     may accompany Microchip software.
00033
00034     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00035     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00036     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00037     FOR A PARTICULAR PURPOSE.
00038
00039     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00040     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00041     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00042     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00043     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00044     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00045     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00046     SOFTWARE.
00047 */
00048
00049 #include "interrupt_manager.h"
00050 #include "mcc.h"
00051
00052 void INTERRUPT_Initialize (void)
00053 {
00054     INTCON0bits.IPEN = 1;
00055
00056     bool state = (unsigned char)GIE;
00057     GIE = 0;
00058     IVTLOCK = 0x55;
00059     IVTLOCK = 0xAA;
00060     IVTLOCKbits.IVTLOCKED = 0x00; // unlock IVT
00061
00062     IVTBASEU = 0;
00063     IVTBASEH = 0;
00064     IVTBASEL = 8;
00065
00066     IVTLOCK = 0x55;
00067     IVTLOCK = 0xAA;
00068     IVTLOCKbits.IVTLOCKED = 0x01; // lock IVT
00069
00070     GIE = state;
00071
00072     // Assign peripheral interrupt priority vectors
00073     IPR2bits.DMA1AIP = 1;
00074     IPR2bits.DMA1DCNTIP = 1;
00075     IPR2bits.DMA1SCNTIP = 1;
00076     IPR2bits.DMA1ORIP = 1;
00077     IPR6bits.U2TXIP = 1;
00078     IPR6bits.U2RXIP = 1;
00079     IPR3bits.U1TXIP = 1;
00080     IPR3bits.U1RXIP = 1;
00081     IPR1bits.INT0IP = 1;
00082     IPR5bits.INT1IP = 0;
00083     IPR9bits.TMR6IP = 1;
00084     IPR8bits.TMR5IP = 0;
00085     IPR6bits.TMR3IP = 0;
00086     IPR1bits.ADIP = 1;
00087     IPR1bits.ADTIP = 1;
00088 }
00089
00090 void __interrupt (irq(default),base(8)) Default_ISR()
00091 {
00092 }
00093

```

4.31 interrupt_manager.h

```

00001
00025
00026 /*
00027     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00028
00029     Subject to your compliance with these terms, you may use Microchip software and any
00030     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00031 party
00031     license terms applicable to your use of third party software (including open source software) that
00032
00032     may accompany Microchip software.
00033
00034     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER

```

```

00035     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00036     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00037     FOR A PARTICULAR PURPOSE.
00038
00039     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00040     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00041     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00042     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00043     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00044     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00045     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00046     SOFTWARE.
00047 */
00048
00049 #ifndef INTERRUPT_MANAGER_H
00050 #define INTERRUPT_MANAGER_H
00051
00062 #define INTERRUPT_GlobalInterruptHighEnable() (INTCON0bits.GIEH = 1)
00063
00074 #define INTERRUPT_GlobalInterruptHighDisable() (INTCON0bits.GIEH = 0)
00075
00086 #define INTERRUPT_GlobalInterruptLowEnable() (INTCON0bits.GIEL = 1)
00087
00098 #define INTERRUPT_GlobalInterruptLowDisable() (INTCON0bits.GIEL = 0)
00109 void INTERRUPT_Initialize (void);
00110
00111 #endif // INTERRUPT_MANAGER_H

```

4.32 mcc.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #include "mcc.h"
00048
00049
00050 void SYSTEM_Initialize(void)
00051 {
00052     INTERRUPT_Initialize();
00053     PMD_Initialize();
00054     PIN_MANAGER_Initialize();
00055     OSCILLATOR_Initialize();
00056     FVR_Initialize();
00057     CRC_Initialize();
00058     DMA2_Initialize();
00059     ADCC_Initialize();
00060     DMA1_Initialize();
00061     TMR2_Initialize();
00062     PWM8_Initialize();
00063     EXT_INT_Initialize();
00064     TMR6_Initialize();
00065     TMR3_Initialize();
00066     TMR5_Initialize();
00067     UART1_Initialize();
00068     UART2_Initialize();
00069     SPI1_Initialize();
00070     CLKREF_Initialize();
00071     SystemArbiter_Initialize();
00072 }

```

```

00073
00074 void OSCILLATOR_Initialize(void)
00075 {
00076     // NOSC EXTOSC    with 4x PLL; NDIV 1;
00077     OSCCON1 = 0x20;
00078     // CSWHOLD may proceed; SOSCPWR Low power;
00079     OSCCON3 = 0x00;
00080     // MFOEN disabled; LFOEN disabled; ADOEN disabled; SOSSEN disabled; EXTOEN disabled; HFOEN
disabled;
00081     OSCEN = 0x00;
00082     // HFFRQ 4_MHz;
00083     OSCFRQ = 0x02;
00084     // TUN 0;
00085     OSTUNE = 0x00;
00086     // Wait for PLL to stabilize
00087     while(PLLR == 0)
00088     {
00089     }
00090 }
00091
00092 void PMD_Initialize(void)
00093 {
00094     // CLKRMD CLKR enabled; SYSCMD SYSCLK enabled; SCANMD SCANNER enabled; FVRMD FVR enabled; IOCMD
IOC enabled; CRCMD CRC enabled; HLVDMD HLVD enabled; NVMMMD NVM enabled;
00095     PMD0 = 0x00;
00096     // NCO1MD DDS(NCO1) enabled; TMR0MD TMR0 enabled; TMR1MD TMR1 enabled; TMR4MD TMR4 enabled; TMR5MD
TMR5 enabled; TMR2MD TMR2 enabled; TMR3MD TMR3 enabled; TMR6MD TMR6 enabled;
00097     PMD1 = 0x00;
00098     // ZCDMD ZCD enabled; DACMD DAC enabled; CMP1MD CMP1 enabled; ADCMD ADC enabled; CMP2MD CMP2
enabled;
00099     PMD2 = 0x00;
00100     // CCP2MD CCP2 enabled; CCP1MD CCP1 enabled; CCP4MD CCP4 enabled; CCP3MD CCP3 enabled; PWM6MD PWM6
enabled; PWM5MD PWM5 enabled; PWM8MD PWM8 enabled; PWM7MD PWM7 enabled;
00101     PMD3 = 0x00;
00102     // CWG3MD CWG3 enabled; CWG2MD CWG2 enabled; CWG1MD CWG1 enabled;
00103     PMD4 = 0x00;
00104     // U2MD UART2 enabled; U1MD UART1 enabled; SPI1MD SPI1 enabled; I2C2MD I2C2 enabled; I2C1MD I2C1
enabled;
00105     PMD5 = 0x00;
00106     // DSMMD DSM1 enabled; CLC3MD CLC3 enabled; CLC4MD CLC4 enabled; SMT1MD SMT1 enabled; CLC1MD CLC1
enabled; CLC2MD CLC2 enabled;
00107     PMD6 = 0x00;
00108     // DMA1MD DMA1 enabled; DMA2MD DMA2 enabled;
00109     PMD7 = 0x00;
00110 }
00111
00112
00113 void SystemArbiter_Initialize(void)
00114 {
00115     // This function is dependant on the PR1WAY CONFIG bit
00116     PRLOCK = 0x55;
00117     PRLOCK = 0xAA;
00118     PRLOCKbits.PRLOCKED = 1;
00119 }

```

4.33 mcc.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
party
00029     license terms applicable to your use of third party software (including open source software) that
may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */

```

```

00046
00047 #ifndef MCC_H
00048 #define MCC_H
00049 #include <xc.h>
00050 #include "device_config.h"
00051 #include "pin_manager.h"
00052 #include <stdint.h>
00053 #include <stdbool.h>
00054 #include <conio.h>
00055 #include "interrupt_manager.h"
00056 #include "dma1.h"
00057 #include "dma2.h"
00058 #include "crc.h"
00059 #include "adcc.h"
00060 #include "fvr.h"
00061 #include "tmr5.h"
00062 #include "tmr6.h"
00063 #include "pwm8.h"
00064 #include "tmr3.h"
00065 #include "tmr2.h"
00066 #include "memory.h"
00067 #include "ext_int.h"
00068 #include "uart2.h"
00069 #include "uart1.h"
00070 #include "spi1.h"
00071 #include "clkref.h"
00072
00073
00074
00086 void SYSTEM_Initialize(void);
00087
00099 void OSCILLATOR_Initialize(void);
00100
00112 void PMD_Initialize(void);
00113
00124 void SystemArbiter_Initialize(void);
00125
00126 #endif /* MCC_H */

```

4.34 memory.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "memory.h"
00053
00054
00058
00059 uint8_t FLASH_ReadByte(uint32_t flashAddr)
00060 {
00061     NVMCON1bits.NVMREG = 2;
00062     TBLPTRU = (uint8_t)((flashAddr & 0x00FF0000) » 16);
00063     TBLPTRH = (uint8_t)((flashAddr & 0x0000FF00)» 8);
00064     TBLPTRL = (uint8_t)(flashAddr & 0x000000FF);
00065
00066     asm("TBLRD");
00067

```

```

00068     return (TABLAT);
00069 }
00070
00071 uint16_t FLASH_ReadWord(uint32_t flashAddr)
00072 {
00073     return (((uint16_t)FLASH_ReadByte(flashAddr+1))<<8)|(FLASH_ReadByte(flashAddr)));
00074 }
00075
00076 void FLASH_WriteByte(uint32_t flashAddr, uint8_t *flashRdBufPtr, uint8_t byte)
00077 {
00078     uint32_t blockStartAddr = (uint32_t)(flashAddr & ((END_FLASH-1) ^ (ERASE_FLASH_BLOCKSIZE-1)));
00079     uint8_t offset = (uint8_t)(flashAddr & (ERASE_FLASH_BLOCKSIZE-1));
00080     uint8_t i;
00081
00082     // Entire row will be erased, read and save the existing data
00083     for (i=0; i<ERASE_FLASH_BLOCKSIZE; i++)
00084     {
00085         flashRdBufPtr[i] = FLASH_ReadByte((blockStartAddr+i));
00086     }
00087
00088     // Load byte at offset
00089     flashRdBufPtr[offset] = byte;
00090
00091     // Writes buffer contents to current block
00092     FLASH_WriteBlock(blockStartAddr, flashRdBufPtr);
00093 }
00094
00095 int8_t FLASH_WriteBlock(uint32_t writeAddr, uint8_t *flashWrBufPtr)
00096 {
00097     uint32_t blockStartAddr = (uint32_t)(writeAddr & ((END_FLASH-1) ^ (ERASE_FLASH_BLOCKSIZE-1)));
00098     uint8_t GIEBitValue = INTCON0bits.GIE;    // Save interrupt enable
00099     uint8_t i;
00100
00101     // Flash write must start at the beginning of a row
00102     if( writeAddr != blockStartAddr )
00103     {
00104         return -1;
00105     }
00106
00107     // Block erase sequence
00108     FLASH_EraseBlock(writeAddr);
00109
00110     // Block write sequence
00111     TBLPTRU = (uint8_t)((writeAddr & 0x00FF0000) >> 16);    // Load Table point register
00112     TBLPTRH = (uint8_t)((writeAddr & 0x0000FF00) >> 8);
00113     TBLPTL = (uint8_t)(writeAddr & 0x000000FF);
00114
00115     // Write block of data
00116     for (i=0; i<WRITE_FLASH_BLOCKSIZE; i++)
00117     {
00118         TABLAT = flashWrBufPtr[i];    // Load data byte
00119
00120         if (i == (WRITE_FLASH_BLOCKSIZE-1))
00121         {
00122             asm("TBLWT");
00123         }
00124         else
00125         {
00126             asm("TBLWTPOSTINC");
00127         }
00128     }
00129
00130     NVMCON1bits.NVMREG = 2;
00131     NVMCON1bits.WREN = 1;
00132     INTCON0bits.GIE = 0;    // Disable interrupts
00133     NVMCON2 = 0x55;
00134     NVMCON2 = 0xAA;
00135     NVMCON1bits.WR = 1;    // Start program
00136     INTCON0bits.GIE = GIEBitValue;    // Restore interrupt enable
00137     NVMCON1bits.WREN = 0;    // Disable writes to memory
00138
00139     return 0;
00140 }
00141
00142 void FLASH_EraseBlock(uint32_t baseAddr)
00143 {
00144     uint8_t GIEBitValue = INTCON0bits.GIE;    // Save interrupt enable
00145
00146     TBLPTRU = (uint8_t)((baseAddr & 0x00FF0000) >> 16);
00147     TBLPTRH = (uint8_t)((baseAddr & 0x0000FF00) >> 8);
00148     TBLPTL = (uint8_t)(baseAddr & 0x000000FF);
00149
00150     NVMCON1bits.NVMREG = 2;
00151     NVMCON1bits.WREN = 1;
00152     NVMCON1bits.FREE = 1;
00153     INTCON0bits.GIE = 0;    // Disable interrupts
00154     NVMCON2 = 0x55;

```

```

00155     NVMCON2 = 0xAA;
00156     NVMCON1bits.WR = 1; // Start program
00157     INTCON0bits.GIE = GIEBitValue; // Restore interrupt enable
00158     NVMCON1bits.WREN = 0; // Disable writes to memory
00159 }
00160
00164
00165 void DATAEE_WriteByte(uint16_t bAdd, uint8_t bData)
00166 {
00167     uint8_t GIEBitValue = INTCON0bits.GIE;
00168
00169     NVMADRH = (uint8_t)((bAdd >> 8) & 0x03);
00170     NVMADRL = (uint8_t)(bAdd & 0xFF);
00171     NVMDAT = bData;
00172     NVMCON1bits.NVMREG = 0;
00173     NVMCON1bits.WREN = 1;
00174     INTCON0bits.GIE = 0; // Disable interrupts
00175     NVMCON2 = 0x55;
00176     NVMCON2 = 0xAA;
00177     NVMCON1bits.WR = 1;
00178     // Wait for write to complete
00179     while (NVMCON1bits.WR)
00180     {
00181     }
00182
00183     NVMCON1bits.WREN = 0;
00184     INTCON0bits.GIE = GIEBitValue; // restore interrupt enable
00185 }
00186
00187 uint8_t DATAEE_ReadByte(uint16_t bAdd)
00188 {
00189     NVMADRH = (uint8_t)((bAdd >> 8) & 0x03);
00190     NVMADRL = (uint8_t)(bAdd & 0xFF);
00191     NVMCON1bits.NVMREG = 0;
00192     NVMCON1bits.RD = 1;
00193     NOP(); // NOPs may be required for latency at high frequencies
00194     NOP();
00195
00196     return (NVMDAT);
00197 }
00198
00199 void MEMORY_Tasks( void )
00200 {
00201     /* TODO : Add interrupt handling code */
00202     PIR0bits.NVMIF = 0;
00203 }

```

4.35 memory.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef MEMORY_H
00048 #define MEMORY_H
00049
00053
00054 #include <stdbool.h>
00055 #include <stdint.h>
00056

```



```

00057 #ifdef __cplusplus // Provide C++ Compatibility
00058
00059     extern "C" {
00060
00061 #endif
00062
00066 #define WRITE_FLASH_BLOCKSIZE    128
00068 #define ERASE_FLASH_BLOCKSIZE    128
00069 #define END_FLASH                0x020000
00070
00074
00099 uint8_t FLASH_ReadByte(uint32_t flashAddr);
00100
00125 uint16_t FLASH_ReadWord(uint32_t flashAddr);
00126
00157 void FLASH_WriteByte(uint32_t flashAddr, uint8_t *flashRdBufPtr, uint8_t byte);
00158
00193 int8_t FLASH_WriteBlock(uint32_t writeAddr, uint8_t *flashWrBufPtr);
00194
00218 void FLASH_EraseBlock(uint32_t baseAddr);
00219
00223
00249 void DATAEE_WriteByte(uint16_t bAdd, uint8_t bData);
00250
00275 uint8_t DATAEE_ReadByte(uint16_t bAdd);
00276
00277 void MEMORY_Tasks(void);
00278
00279 #ifdef __cplusplus // Provide C++ Compatibility
00280
00281     }
00282
00283 #endif
00284
00285 #endif // MEMORY_H
00289

```

4.36 pin_manager.c

```

00001
00025
00026 /*
00027     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00028
00029     Subject to your compliance with these terms, you may use Microchip software and any
00030     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00031     party
00032     license terms applicable to your use of third party software (including open source software) that
00033
00034     may accompany Microchip software.
00035
00036     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00037     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00038     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00039     FOR A PARTICULAR PURPOSE.
00040
00041     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00042     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00043     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00044     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00045     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00046     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00047     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00048     SOFTWARE.
00049 */
00048
00049 #include "pin_manager.h"
00050
00051
00052
00053
00054
00055 void PIN_MANAGER_Initialize(void)
00056 {
00060     LATE = 0x07;
00061     LATD = 0x00;
00062     LATA = 0x00;
00063     LATF = 0x23;
00064     LATB = 0x00;
00065     LATC = 0x07;
00066
00070     TRISE = 0x00;

```

```

00071     TRISF = 0x0C;
00072     TRISA = 0xFF;
00073     TRISB = 0xBF;
00074     TRISC = 0x90;
00075     TRISD = 0x0C;
00076
00080     ANSELD = 0x00;
00081     ANSELC = 0x00;
00082     ANSELB = 0x3C;
00083     ANSELE = 0x00;
00084     ANSELF = 0x00;
00085     ANSELA = 0xFF;
00086
00090     WPUD = 0x0C;
00091     WPUF = 0x0C;
00092     WPUE = 0x00;
00093     WPUB = 0x83;
00094     WPUA = 0x00;
00095     WPUC = 0x97;
00096
00100     RB1I2C = 0x00;
00101     RB2I2C = 0x00;
00102     RC3I2C = 0x00;
00103     RC4I2C = 0x00;
00104     RD0I2C = 0x00;
00105     RD1I2C = 0x00;
00106
00110     ODCONE = 0x00;
00111     ODCONF = 0x00;
00112     ODCONA = 0x00;
00113     ODCONB = 0x00;
00114     ODCONC = 0x00;
00115     ODCOND = 0x00;
00116
00120     SLRCONA = 0xFF;
00121     SLRCONB = 0xFF;
00122     SLRCONC = 0xFF;
00123     SLRCOND = 0xFF;
00124     SLRCONE = 0x07;
00125     SLRCONF = 0xFF;
00126
00130     INLVLA = 0xFF;
00131     INLVLB = 0xFF;
00132     INLVLC = 0xFF;
00133     INLVLD = 0xFF;
00134     INLVLE = 0x0F;
00135     INLVLF = 0xFF;
00136
00137
00138
00139
00140
00141
00142
00143
00144     U2RXPPS = 0x0F;    //RB7->UART2:RX2;
00145     SPI1SCKPPS = 0x13; //RC3->SPI1:SCK1;
00146     RB6PPS = 0x16;    //RB6->UART2:TX2;
00147     RC3PPS = 0x1E;    //RC3->SPI1:SCK1;
00148     INT0PPS = 0x08;   //RB0->EXT_INT:INT0;
00149     RD1PPS = 0x10;    //RD1->PWM8:PWM8;
00150     INT1PPS = 0x09;   //RB1->EXT_INT:INT1;
00151     RC5PPS = 0x1F;    //RC5->SPI1:SD01;
00152     RC6PPS = 0x13;    //RC6->UART1:TX1;
00153     U1RXPPS = 0x17;   //RC7->UART1:RX1;
00154     SPI1SDIPPS = 0x14; //RC4->SPI1:SDI1;
00155 }
00156
00157 void PIN_MANAGER_IOC(void)
00158 {
00159 }
00160

```

4.37 pin_manager.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
    party

```

```

00029     license terms applicable to your use of third party software (including open source software) that
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef PIN_MANAGER_H
00048 #define PIN_MANAGER_H
00049
00053
00054 #include <xc.h>
00055
00056 #define INPUT      1
00057 #define OUTPUT     0
00058
00059 #define HIGH       1
00060 #define LOW        0
00061
00062 #define ANALOG     1
00063 #define DIGITAL    0
00064
00065 #define PULL_UP_ENABLED      1
00066 #define PULL_UP_DISABLED    0
00067
00068 // get/set C_BATT aliases
00069 #define C_BATT_TRIS      TRISAbits.TRISA0
00070 #define C_BATT_LAT       LATAbits.LATA0
00071 #define C_BATT_PORT      PORTAbits.RA0
00072 #define C_BATT_WPU       WPUAbits.WPUA0
00073 #define C_BATT_OD        ODCONAbits.ODCA0
00074 #define C_BATT_ANS       ANSELAbits.ANSELA0
00075 #define C_BATT_SetHigh() do { LATAbits.LATA0 = 1; } while(0)
00076 #define C_BATT_SetLow()  do { LATAbits.LATA0 = 0; } while(0)
00077 #define C_BATT_Toggle()  do { LATAbits.LATA0 = ~LATAbits.LATA0; } while(0)
00078 #define C_BATT_GetValue() PORTAbits.RA0
00079 #define C_BATT_SetDigitalInput() do { TRISAbits.TRISA0 = 1; } while(0)
00080 #define C_BATT_SetDigitalOutput() do { TRISAbits.TRISA0 = 0; } while(0)
00081 #define C_BATT_SetPullup() do { WPUAbits.WPUA0 = 1; } while(0)
00082 #define C_BATT_ResetPullup() do { WPUAbits.WPUA0 = 0; } while(0)
00083 #define C_BATT_SetPushPull() do { ODCONAbits.ODCA0 = 0; } while(0)
00084 #define C_BATT_SetOpenDrain() do { ODCONAbits.ODCA0 = 1; } while(0)
00085 #define C_BATT_SetAnalogMode() do { ANSELAbits.ANSELA0 = 1; } while(0)
00086 #define C_BATT_SetDigitalMode() do { ANSELAbits.ANSELA0 = 0; } while(0)
00087
00088 // get/set C_PV aliases
00089 #define C_PV_TRIS      TRISAbits.TRISA1
00090 #define C_PV_LAT       LATAbits.LATA1
00091 #define C_PV_PORT      PORTAbits.RA1
00092 #define C_PV_WPU       WPUAbits.WPUA1
00093 #define C_PV_OD        ODCONAbits.ODCA1
00094 #define C_PV_ANS       ANSELAbits.ANSELA1
00095 #define C_PV_SetHigh() do { LATAbits.LATA1 = 1; } while(0)
00096 #define C_PV_SetLow()  do { LATAbits.LATA1 = 0; } while(0)
00097 #define C_PV_Toggle()  do { LATAbits.LATA1 = ~LATAbits.LATA1; } while(0)
00098 #define C_PV_GetValue() PORTAbits.RA1
00099 #define C_PV_SetDigitalInput() do { TRISAbits.TRISA1 = 1; } while(0)
00100 #define C_PV_SetDigitalOutput() do { TRISAbits.TRISA1 = 0; } while(0)
00101 #define C_PV_SetPullup() do { WPUAbits.WPUA1 = 1; } while(0)
00102 #define C_PV_ResetPullup() do { WPUAbits.WPUA1 = 0; } while(0)
00103 #define C_PV_SetPushPull() do { ODCONAbits.ODCA1 = 0; } while(0)
00104 #define C_PV_SetOpenDrain() do { ODCONAbits.ODCA1 = 1; } while(0)
00105 #define C_PV_SetAnalogMode() do { ANSELAbits.ANSELA1 = 1; } while(0)
00106 #define C_PV_SetDigitalMode() do { ANSELAbits.ANSELA1 = 0; } while(0)
00107
00108 // get/set V_CC aliases
00109 #define V_CC_TRIS      TRISAbits.TRISA2
00110 #define V_CC_LAT       LATAbits.LATA2
00111 #define V_CC_PORT      PORTAbits.RA2
00112 #define V_CC_WPU       WPUAbits.WPUA2
00113 #define V_CC_OD        ODCONAbits.ODCA2
00114 #define V_CC_ANS       ANSELAbits.ANSELA2
00115 #define V_CC_SetHigh() do { LATAbits.LATA2 = 1; } while(0)
00116 #define V_CC_SetLow()  do { LATAbits.LATA2 = 0; } while(0)
00117 #define V_CC_Toggle()  do { LATAbits.LATA2 = ~LATAbits.LATA2; } while(0)

```

```

00118 #define V_CC_GetValue() PORTAbits.RA2
00119 #define V_CC_SetDigitalInput() do { TRISAbits.TRISA2 = 1; } while(0)
00120 #define V_CC_SetDigitalOutput() do { TRISAbits.TRISA2 = 0; } while(0)
00121 #define V_CC_SetPullup() do { WPUAbits.WPUA2 = 1; } while(0)
00122 #define V_CC_ResetPullup() do { WPUAbits.WPUA2 = 0; } while(0)
00123 #define V_CC_SetPushPull() do { ODCONAbits.ODCA2 = 0; } while(0)
00124 #define V_CC_SetOpenDrain() do { ODCONAbits.ODCA2 = 1; } while(0)
00125 #define V_CC_SetAnalogMode() do { ANSELAbits.ANSELA2 = 1; } while(0)
00126 #define V_CC_SetDigitalMode() do { ANSELAbits.ANSELA2 = 0; } while(0)
00127
00128 // get/set RA3 procedures
00129 #define RA3_SetHigh() do { LATAbits.LATA3 = 1; } while(0)
00130 #define RA3_SetLow() do { LATAbits.LATA3 = 0; } while(0)
00131 #define RA3_Toggle() do { LATAbits.LATA3 = ~LATAbits.LATA3; } while(0)
00132 #define RA3_GetValue() PORTAbits.RA3
00133 #define RA3_SetDigitalInput() do { TRISAbits.TRISA3 = 1; } while(0)
00134 #define RA3_SetDigitalOutput() do { TRISAbits.TRISA3 = 0; } while(0)
00135 #define RA3_SetPullup() do { WPUAbits.WPUA3 = 1; } while(0)
00136 #define RA3_ResetPullup() do { WPUAbits.WPUA3 = 0; } while(0)
00137 #define RA3_SetAnalogMode() do { ANSELAbits.ANSELA3 = 1; } while(0)
00138 #define RA3_SetDigitalMode() do { ANSELAbits.ANSELA3 = 0; } while(0)
00139
00140 // get/set V_BAT aliases
00141 #define V_BAT_TRIS TRISAbits.TRISA4
00142 #define V_BAT_LAT LATAbits.LATA4
00143 #define V_BAT_PORT PORTAbits.RA4
00144 #define V_BAT_WPU WPUAbits.WPUA4
00145 #define V_BAT_OD ODCONAbits.ODCA4
00146 #define V_BAT_ANS ANSELAbits.ANSELA4
00147 #define V_BAT_SetHigh() do { LATAbits.LATA4 = 1; } while(0)
00148 #define V_BAT_SetLow() do { LATAbits.LATA4 = 0; } while(0)
00149 #define V_BAT_Toggle() do { LATAbits.LATA4 = ~LATAbits.LATA4; } while(0)
00150 #define V_BAT_GetValue() PORTAbits.RA4
00151 #define V_BAT_SetDigitalInput() do { TRISAbits.TRISA4 = 1; } while(0)
00152 #define V_BAT_SetDigitalOutput() do { TRISAbits.TRISA4 = 0; } while(0)
00153 #define V_BAT_SetPullup() do { WPUAbits.WPUA4 = 1; } while(0)
00154 #define V_BAT_ResetPullup() do { WPUAbits.WPUA4 = 0; } while(0)
00155 #define V_BAT_SetPushPull() do { ODCONAbits.ODCA4 = 0; } while(0)
00156 #define V_BAT_SetOpenDrain() do { ODCONAbits.ODCA4 = 1; } while(0)
00157 #define V_BAT_SetAnalogMode() do { ANSELAbits.ANSELA4 = 1; } while(0)
00158 #define V_BAT_SetDigitalMode() do { ANSELAbits.ANSELA4 = 0; } while(0)
00159
00160 // get/set V_PV aliases
00161 #define V_PV_TRIS TRISAbits.TRISA5
00162 #define V_PV_LAT LATAbits.LATA5
00163 #define V_PV_PORT PORTAbits.RA5
00164 #define V_PV_WPU WPUAbits.WPUA5
00165 #define V_PV_OD ODCONAbits.ODCA5
00166 #define V_PV_ANS ANSELAbits.ANSELA5
00167 #define V_PV_SetHigh() do { LATAbits.LATA5 = 1; } while(0)
00168 #define V_PV_SetLow() do { LATAbits.LATA5 = 0; } while(0)
00169 #define V_PV_Toggle() do { LATAbits.LATA5 = ~LATAbits.LATA5; } while(0)
00170 #define V_PV_GetValue() PORTAbits.RA5
00171 #define V_PV_SetDigitalInput() do { TRISAbits.TRISA5 = 1; } while(0)
00172 #define V_PV_SetDigitalOutput() do { TRISAbits.TRISA5 = 0; } while(0)
00173 #define V_PV_SetPullup() do { WPUAbits.WPUA5 = 1; } while(0)
00174 #define V_PV_ResetPullup() do { WPUAbits.WPUA5 = 0; } while(0)
00175 #define V_PV_SetPushPull() do { ODCONAbits.ODCA5 = 0; } while(0)
00176 #define V_PV_SetOpenDrain() do { ODCONAbits.ODCA5 = 1; } while(0)
00177 #define V_PV_SetAnalogMode() do { ANSELAbits.ANSELA5 = 1; } while(0)
00178 #define V_PV_SetDigitalMode() do { ANSELAbits.ANSELA5 = 0; } while(0)
00179
00180 // get/set V_CMODE aliases
00181 #define V_CMODE_TRIS TRISAbits.TRISA6
00182 #define V_CMODE_LAT LATAbits.LATA6
00183 #define V_CMODE_PORT PORTAbits.RA6
00184 #define V_CMODE_WPU WPUAbits.WPUA6
00185 #define V_CMODE_OD ODCONAbits.ODCA6
00186 #define V_CMODE_ANS ANSELAbits.ANSELA6
00187 #define V_CMODE_SetHigh() do { LATAbits.LATA6 = 1; } while(0)
00188 #define V_CMODE_SetLow() do { LATAbits.LATA6 = 0; } while(0)
00189 #define V_CMODE_Toggle() do { LATAbits.LATA6 = ~LATAbits.LATA6; } while(0)
00190 #define V_CMODE_GetValue() PORTAbits.RA6
00191 #define V_CMODE_SetDigitalInput() do { TRISAbits.TRISA6 = 1; } while(0)
00192 #define V_CMODE_SetDigitalOutput() do { TRISAbits.TRISA6 = 0; } while(0)
00193 #define V_CMODE_SetPullup() do { WPUAbits.WPUA6 = 1; } while(0)
00194 #define V_CMODE_ResetPullup() do { WPUAbits.WPUA6 = 0; } while(0)
00195 #define V_CMODE_SetPushPull() do { ODCONAbits.ODCA6 = 0; } while(0)
00196 #define V_CMODE_SetOpenDrain() do { ODCONAbits.ODCA6 = 1; } while(0)
00197 #define V_CMODE_SetAnalogMode() do { ANSELAbits.ANSELA6 = 1; } while(0)
00198 #define V_CMODE_SetDigitalMode() do { ANSELAbits.ANSELA6 = 0; } while(0)
00199
00200 // get/set RB0 procedures
00201 #define RB0_SetHigh() do { LATBbits.LATB0 = 1; } while(0)
00202 #define RB0_SetLow() do { LATBbits.LATB0 = 0; } while(0)
00203 #define RB0_Toggle() do { LATBbits.LATB0 = ~LATBbits.LATB0; } while(0)
00204 #define RB0_GetValue() PORTBbits.RB0

```

```

00205 #define RB0_SetDigitalInput() do { TRISBbits.TRISB0 = 1; } while(0)
00206 #define RB0_SetDigitalOutput() do { TRISBbits.TRISB0 = 0; } while(0)
00207 #define RB0_SetPullup() do { WPUBbits.WPUB0 = 1; } while(0)
00208 #define RB0_ResetPullup() do { WPUBbits.WPUB0 = 0; } while(0)
00209 #define RB0_SetAnalogMode() do { ANSELBbits.ANSELB0 = 1; } while(0)
00210 #define RB0_SetDigitalMode() do { ANSELBbits.ANSELB0 = 0; } while(0)
00211
00212 // get/set RB1 procedures
00213 #define RB1_SetHigh() do { LATBbits.LATB1 = 1; } while(0)
00214 #define RB1_SetLow() do { LATBbits.LATB1 = 0; } while(0)
00215 #define RB1_Toggle() do { LATBbits.LATB1 = ~LATBbits.LATB1; } while(0)
00216 #define RB1_GetValue() PORTBbits.RB1
00217 #define RB1_SetDigitalInput() do { TRISBbits.TRISB1 = 1; } while(0)
00218 #define RB1_SetDigitalOutput() do { TRISBbits.TRISB1 = 0; } while(0)
00219 #define RB1_SetPullup() do { WPUBbits.WPUB1 = 1; } while(0)
00220 #define RB1_ResetPullup() do { WPUBbits.WPUB1 = 0; } while(0)
00221 #define RB1_SetAnalogMode() do { ANSELBbits.ANSELB1 = 1; } while(0)
00222 #define RB1_SetDigitalMode() do { ANSELBbits.ANSELB1 = 0; } while(0)
00223
00224 // get/set V_BBAT aliases
00225 #define V_BBAT_TRIS TRISBbits.TRISB2
00226 #define V_BBAT_LAT LATBbits.LATB2
00227 #define V_BBAT_PORT PORTBbits.RB2
00228 #define V_BBAT_WPU WPUBbits.WPUB2
00229 #define V_BBAT_OD ODCONBbits.ODCB2
00230 #define V_BBAT_ANS ANSELBbits.ANSELB2
00231 #define V_BBAT_SetHigh() do { LATBbits.LATB2 = 1; } while(0)
00232 #define V_BBAT_SetLow() do { LATBbits.LATB2 = 0; } while(0)
00233 #define V_BBAT_Toggle() do { LATBbits.LATB2 = ~LATBbits.LATB2; } while(0)
00234 #define V_BBAT_GetValue() PORTBbits.RB2
00235 #define V_BBAT_SetDigitalInput() do { TRISBbits.TRISB2 = 1; } while(0)
00236 #define V_BBAT_SetDigitalOutput() do { TRISBbits.TRISB2 = 0; } while(0)
00237 #define V_BBAT_SetPullup() do { WPUBbits.WPUB2 = 1; } while(0)
00238 #define V_BBAT_ResetPullup() do { WPUBbits.WPUB2 = 0; } while(0)
00239 #define V_BBAT_SetPushPull() do { ODCONBbits.ODCB2 = 0; } while(0)
00240 #define V_BBAT_SetOpenDrain() do { ODCONBbits.ODCB2 = 1; } while(0)
00241 #define V_BBAT_SetAnalogMode() do { ANSELBbits.ANSELB2 = 1; } while(0)
00242 #define V_BBAT_SetDigitalMode() do { ANSELBbits.ANSELB2 = 0; } while(0)
00243
00244 // get/set C_MPPT aliases
00245 #define C_MPPT_TRIS TRISBbits.TRISB3
00246 #define C_MPPT_LAT LATBbits.LATB3
00247 #define C_MPPT_PORT PORTBbits.RB3
00248 #define C_MPPT_WPU WPUBbits.WPUB3
00249 #define C_MPPT_OD ODCONBbits.ODCB3
00250 #define C_MPPT_ANS ANSELBbits.ANSELB3
00251 #define C_MPPT_SetHigh() do { LATBbits.LATB3 = 1; } while(0)
00252 #define C_MPPT_SetLow() do { LATBbits.LATB3 = 0; } while(0)
00253 #define C_MPPT_Toggle() do { LATBbits.LATB3 = ~LATBbits.LATB3; } while(0)
00254 #define C_MPPT_GetValue() PORTBbits.RB3
00255 #define C_MPPT_SetDigitalInput() do { TRISBbits.TRISB3 = 1; } while(0)
00256 #define C_MPPT_SetDigitalOutput() do { TRISBbits.TRISB3 = 0; } while(0)
00257 #define C_MPPT_SetPullup() do { WPUBbits.WPUB3 = 1; } while(0)
00258 #define C_MPPT_ResetPullup() do { WPUBbits.WPUB3 = 0; } while(0)
00259 #define C_MPPT_SetPushPull() do { ODCONBbits.ODCB3 = 0; } while(0)
00260 #define C_MPPT_SetOpenDrain() do { ODCONBbits.ODCB3 = 1; } while(0)
00261 #define C_MPPT_SetAnalogMode() do { ANSELBbits.ANSELB3 = 1; } while(0)
00262 #define C_MPPT_SetDigitalMode() do { ANSELBbits.ANSELB3 = 0; } while(0)
00263
00264 // get/set V_INVERTER aliases
00265 #define V_INVERTER_TRIS TRISBbits.TRISB4
00266 #define V_INVERTER_LAT LATBbits.LATB4
00267 #define V_INVERTER_PORT PORTBbits.RB4
00268 #define V_INVERTER_WPU WPUBbits.WPUB4
00269 #define V_INVERTER_OD ODCONBbits.ODCB4
00270 #define V_INVERTER_ANS ANSELBbits.ANSELB4
00271 #define V_INVERTER_SetHigh() do { LATBbits.LATB4 = 1; } while(0)
00272 #define V_INVERTER_SetLow() do { LATBbits.LATB4 = 0; } while(0)
00273 #define V_INVERTER_Toggle() do { LATBbits.LATB4 = ~LATBbits.LATB4; } while(0)
00274 #define V_INVERTER_GetValue() PORTBbits.RB4
00275 #define V_INVERTER_SetDigitalInput() do { TRISBbits.TRISB4 = 1; } while(0)
00276 #define V_INVERTER_SetDigitalOutput() do { TRISBbits.TRISB4 = 0; } while(0)
00277 #define V_INVERTER_SetPullup() do { WPUBbits.WPUB4 = 1; } while(0)
00278 #define V_INVERTER_ResetPullup() do { WPUBbits.WPUB4 = 0; } while(0)
00279 #define V_INVERTER_SetPushPull() do { ODCONBbits.ODCB4 = 0; } while(0)
00280 #define V_INVERTER_SetOpenDrain() do { ODCONBbits.ODCB4 = 1; } while(0)
00281 #define V_INVERTER_SetAnalogMode() do { ANSELBbits.ANSELB4 = 1; } while(0)
00282 #define V_INVERTER_SetDigitalMode() do { ANSELBbits.ANSELB4 = 0; } while(0)
00283
00284 // get/set V_LIGHT_SENSOR aliases
00285 #define V_LIGHT_SENSOR_TRIS TRISBbits.TRISB5
00286 #define V_LIGHT_SENSOR_LAT LATBbits.LATB5
00287 #define V_LIGHT_SENSOR_PORT PORTBbits.RB5
00288 #define V_LIGHT_SENSOR_WPU WPUBbits.WPUB5
00289 #define V_LIGHT_SENSOR_OD ODCONBbits.ODCB5
00290 #define V_LIGHT_SENSOR_ANS ANSELBbits.ANSELB5
00291 #define V_LIGHT_SENSOR_SetHigh() do { LATBbits.LATB5 = 1; } while(0)

```

```

00292 #define V_LIGHT_SENSOR_SetLow() do { LATBbits.LATB5 = 0; } while(0)
00293 #define V_LIGHT_SENSOR_Toggle() do { LATBbits.LATB5 = ~LATBbits.LATB5; } while(0)
00294 #define V_LIGHT_SENSOR_GetValue() PORTBbits.RB5
00295 #define V_LIGHT_SENSOR_SetDigitalInput() do { TRISBbits.TRISB5 = 1; } while(0)
00296 #define V_LIGHT_SENSOR_SetDigitalOutput() do { TRISBbits.TRISB5 = 0; } while(0)
00297 #define V_LIGHT_SENSOR_SetPullup() do { WPUBbits.WPUB5 = 1; } while(0)
00298 #define V_LIGHT_SENSOR_ResetPullup() do { WPUBbits.WPUB5 = 0; } while(0)
00299 #define V_LIGHT_SENSOR_SetPushPull() do { ODCONBbits.ODCB5 = 0; } while(0)
00300 #define V_LIGHT_SENSOR_SetOpenDrain() do { ODCONBbits.ODCB5 = 1; } while(0)
00301 #define V_LIGHT_SENSOR_SetAnalogMode() do { ANSELBbits.ANSELB5 = 1; } while(0)
00302 #define V_LIGHT_SENSOR_SetDigitalMode() do { ANSELBbits.ANSELB5 = 0; } while(0)
00303
00304 // get/set RB6 procedures
00305 #define RB6_SetHigh() do { LATBbits.LATB6 = 1; } while(0)
00306 #define RB6_SetLow() do { LATBbits.LATB6 = 0; } while(0)
00307 #define RB6_Toggle() do { LATBbits.LATB6 = ~LATBbits.LATB6; } while(0)
00308 #define RB6_GetValue() PORTBbits.RB6
00309 #define RB6_SetDigitalInput() do { TRISBbits.TRISB6 = 1; } while(0)
00310 #define RB6_SetDigitalOutput() do { TRISBbits.TRISB6 = 0; } while(0)
00311 #define RB6_SetPullup() do { WPUBbits.WPUB6 = 1; } while(0)
00312 #define RB6_ResetPullup() do { WPUBbits.WPUB6 = 0; } while(0)
00313 #define RB6_SetAnalogMode() do { ANSELBbits.ANSELB6 = 1; } while(0)
00314 #define RB6_SetDigitalMode() do { ANSELBbits.ANSELB6 = 0; } while(0)
00315
00316 // get/set RB7 procedures
00317 #define RB7_SetHigh() do { LATBbits.LATB7 = 1; } while(0)
00318 #define RB7_SetLow() do { LATBbits.LATB7 = 0; } while(0)
00319 #define RB7_Toggle() do { LATBbits.LATB7 = ~LATBbits.LATB7; } while(0)
00320 #define RB7_GetValue() PORTBbits.RB7
00321 #define RB7_SetDigitalInput() do { TRISBbits.TRISB7 = 1; } while(0)
00322 #define RB7_SetDigitalOutput() do { TRISBbits.TRISB7 = 0; } while(0)
00323 #define RB7_SetPullup() do { WPUBbits.WPUB7 = 1; } while(0)
00324 #define RB7_ResetPullup() do { WPUBbits.WPUB7 = 0; } while(0)
00325 #define RB7_SetAnalogMode() do { ANSELBbits.ANSELB7 = 1; } while(0)
00326 #define RB7_SetDigitalMode() do { ANSELBbits.ANSELB7 = 0; } while(0)
00327
00328 // get/set CS_SDCARD aliases
00329 #define CS_SDCARD_TRIS TRISCbits.TRISC0
00330 #define CS_SDCARD_LAT LATCbits.LATC0
00331 #define CS_SDCARD_PORT PORTCbits.RC0
00332 #define CS_SDCARD_WPU WPUCbits.WPUC0
00333 #define CS_SDCARD_OD ODCONCbits.ODCC0
00334 #define CS_SDCARD_ANS ANSELCbits.ANSELC0
00335 #define CS_SDCARD_SetHigh() do { LATCbits.LATC0 = 1; } while(0)
00336 #define CS_SDCARD_SetLow() do { LATCbits.LATC0 = 0; } while(0)
00337 #define CS_SDCARD_Toggle() do { LATCbits.LATC0 = ~LATCbits.LATC0; } while(0)
00338 #define CS_SDCARD_GetValue() PORTCbits.RC0
00339 #define CS_SDCARD_SetDigitalInput() do { TRISCbits.TRISC0 = 1; } while(0)
00340 #define CS_SDCARD_SetDigitalOutput() do { TRISCbits.TRISC0 = 0; } while(0)
00341 #define CS_SDCARD_SetPullup() do { WPUCbits.WPUC0 = 1; } while(0)
00342 #define CS_SDCARD_ResetPullup() do { WPUCbits.WPUC0 = 0; } while(0)
00343 #define CS_SDCARD_SetPushPull() do { ODCONCbits.ODCC0 = 0; } while(0)
00344 #define CS_SDCARD_SetOpenDrain() do { ODCONCbits.ODCC0 = 1; } while(0)
00345 #define CS_SDCARD_SetAnalogMode() do { ANSELCbits.ANSELC0 = 1; } while(0)
00346 #define CS_SDCARD_SetDigitalMode() do { ANSELCbits.ANSELC0 = 0; } while(0)
00347
00348 // get/set RS aliases
00349 #define RS_TRIS TRISCbits.TRISC1
00350 #define RS_LAT LATCbits.LATC1
00351 #define RS_PORT PORTCbits.RC1
00352 #define RS_WPU WPUCbits.WPUC1
00353 #define RS_OD ODCONCbits.ODCC1
00354 #define RS_ANS ANSELCbits.ANSELC1
00355 #define RS_SetHigh() do { LATCbits.LATC1 = 1; } while(0)
00356 #define RS_SetLow() do { LATCbits.LATC1 = 0; } while(0)
00357 #define RS_Toggle() do { LATCbits.LATC1 = ~LATCbits.LATC1; } while(0)
00358 #define RS_GetValue() PORTCbits.RC1
00359 #define RS_SetDigitalInput() do { TRISCbits.TRISC1 = 1; } while(0)
00360 #define RS_SetDigitalOutput() do { TRISCbits.TRISC1 = 0; } while(0)
00361 #define RS_SetPullup() do { WPUCbits.WPUC1 = 1; } while(0)
00362 #define RS_ResetPullup() do { WPUCbits.WPUC1 = 0; } while(0)
00363 #define RS_SetPushPull() do { ODCONCbits.ODCC1 = 0; } while(0)
00364 #define RS_SetOpenDrain() do { ODCONCbits.ODCC1 = 1; } while(0)
00365 #define RS_SetAnalogMode() do { ANSELCbits.ANSELC1 = 1; } while(0)
00366 #define RS_SetDigitalMode() do { ANSELCbits.ANSELC1 = 0; } while(0)
00367
00368 // get/set CSB aliases
00369 #define CSB_TRIS TRISCbits.TRISC2
00370 #define CSB_LAT LATCbits.LATC2
00371 #define CSB_PORT PORTCbits.RC2
00372 #define CSB_WPU WPUCbits.WPUC2
00373 #define CSB_OD ODCONCbits.ODCC2
00374 #define CSB_ANS ANSELCbits.ANSELC2
00375 #define CSB_SetHigh() do { LATCbits.LATC2 = 1; } while(0)
00376 #define CSB_SetLow() do { LATCbits.LATC2 = 0; } while(0)
00377 #define CSB_Toggle() do { LATCbits.LATC2 = ~LATCbits.LATC2; } while(0)
00378 #define CSB_GetValue() PORTCbits.RC2

```



```

00379 #define CSB_SetDigitalInput() do { TRISCbits.TRISC2 = 1; } while(0)
00380 #define CSB_SetDigitalOutput() do { TRISCbits.TRISC2 = 0; } while(0)
00381 #define CSB_SetPullup() do { WPUCbits.WPUC2 = 1; } while(0)
00382 #define CSB_ResetPullup() do { WPUCbits.WPUC2 = 0; } while(0)
00383 #define CSB_SetPushPull() do { ODCONCbits.ODCC2 = 0; } while(0)
00384 #define CSB_SetOpenDrain() do { ODCONCbits.ODCC2 = 1; } while(0)
00385 #define CSB_SetAnalogMode() do { ANSELcbits.ANSELc2 = 1; } while(0)
00386 #define CSB_SetDigitalMode() do { ANSELcbits.ANSELc2 = 0; } while(0)
00387
00388 // get/set RC3 procedures
00389 #define RC3_SetHigh() do { LATCbits.LATC3 = 1; } while(0)
00390 #define RC3_SetLow() do { LATCbits.LATC3 = 0; } while(0)
00391 #define RC3_Toggle() do { LATCbits.LATC3 = ~LATCbits.LATC3; } while(0)
00392 #define RC3_GetValue() PORTCbits.RC3
00393 #define RC3_SetDigitalInput() do { TRISCbits.TRISC3 = 1; } while(0)
00394 #define RC3_SetDigitalOutput() do { TRISCbits.TRISC3 = 0; } while(0)
00395 #define RC3_SetPullup() do { WPUCbits.WPUC3 = 1; } while(0)
00396 #define RC3_ResetPullup() do { WPUCbits.WPUC3 = 0; } while(0)
00397 #define RC3_SetAnalogMode() do { ANSELcbits.ANSELc3 = 1; } while(0)
00398 #define RC3_SetDigitalMode() do { ANSELcbits.ANSELc3 = 0; } while(0)
00399
00400 // get/set RC4 procedures
00401 #define RC4_SetHigh() do { LATCbits.LATC4 = 1; } while(0)
00402 #define RC4_SetLow() do { LATCbits.LATC4 = 0; } while(0)
00403 #define RC4_Toggle() do { LATCbits.LATC4 = ~LATCbits.LATC4; } while(0)
00404 #define RC4_GetValue() PORTCbits.RC4
00405 #define RC4_SetDigitalInput() do { TRISCbits.TRISC4 = 1; } while(0)
00406 #define RC4_SetDigitalOutput() do { TRISCbits.TRISC4 = 0; } while(0)
00407 #define RC4_SetPullup() do { WPUCbits.WPUC4 = 1; } while(0)
00408 #define RC4_ResetPullup() do { WPUCbits.WPUC4 = 0; } while(0)
00409 #define RC4_SetAnalogMode() do { ANSELcbits.ANSELc4 = 1; } while(0)
00410 #define RC4_SetDigitalMode() do { ANSELcbits.ANSELc4 = 0; } while(0)
00411
00412 // get/set RC5 procedures
00413 #define RC5_SetHigh() do { LATCbits.LATC5 = 1; } while(0)
00414 #define RC5_SetLow() do { LATCbits.LATC5 = 0; } while(0)
00415 #define RC5_Toggle() do { LATCbits.LATC5 = ~LATCbits.LATC5; } while(0)
00416 #define RC5_GetValue() PORTCbits.RC5
00417 #define RC5_SetDigitalInput() do { TRISCbits.TRISC5 = 1; } while(0)
00418 #define RC5_SetDigitalOutput() do { TRISCbits.TRISC5 = 0; } while(0)
00419 #define RC5_SetPullup() do { WPUCbits.WPUC5 = 1; } while(0)
00420 #define RC5_ResetPullup() do { WPUCbits.WPUC5 = 0; } while(0)
00421 #define RC5_SetAnalogMode() do { ANSELcbits.ANSELc5 = 1; } while(0)
00422 #define RC5_SetDigitalMode() do { ANSELcbits.ANSELc5 = 0; } while(0)
00423
00424 // get/set RC6 procedures
00425 #define RC6_SetHigh() do { LATCbits.LATC6 = 1; } while(0)
00426 #define RC6_SetLow() do { LATCbits.LATC6 = 0; } while(0)
00427 #define RC6_Toggle() do { LATCbits.LATC6 = ~LATCbits.LATC6; } while(0)
00428 #define RC6_GetValue() PORTCbits.RC6
00429 #define RC6_SetDigitalInput() do { TRISCbits.TRISC6 = 1; } while(0)
00430 #define RC6_SetDigitalOutput() do { TRISCbits.TRISC6 = 0; } while(0)
00431 #define RC6_SetPullup() do { WPUCbits.WPUC6 = 1; } while(0)
00432 #define RC6_ResetPullup() do { WPUCbits.WPUC6 = 0; } while(0)
00433 #define RC6_SetAnalogMode() do { ANSELcbits.ANSELc6 = 1; } while(0)
00434 #define RC6_SetDigitalMode() do { ANSELcbits.ANSELc6 = 0; } while(0)
00435
00436 // get/set RC7 procedures
00437 #define RC7_SetHigh() do { LATCbits.LATC7 = 1; } while(0)
00438 #define RC7_SetLow() do { LATCbits.LATC7 = 0; } while(0)
00439 #define RC7_Toggle() do { LATCbits.LATC7 = ~LATCbits.LATC7; } while(0)
00440 #define RC7_GetValue() PORTCbits.RC7
00441 #define RC7_SetDigitalInput() do { TRISCbits.TRISC7 = 1; } while(0)
00442 #define RC7_SetDigitalOutput() do { TRISCbits.TRISC7 = 0; } while(0)
00443 #define RC7_SetPullup() do { WPUCbits.WPUC7 = 1; } while(0)
00444 #define RC7_ResetPullup() do { WPUCbits.WPUC7 = 0; } while(0)
00445 #define RC7_SetAnalogMode() do { ANSELcbits.ANSELc7 = 1; } while(0)
00446 #define RC7_SetDigitalMode() do { ANSELcbits.ANSELc7 = 0; } while(0)
00447
00448 // get/set RELAY0 aliases
00449 #define RELAY0_TRIS TRISDbits.TRISD0
00450 #define RELAY0_LAT LATDbits.LATD0
00451 #define RELAY0_PORT PORTDbits.RD0
00452 #define RELAY0_WPU WPUDbits.WPUD0
00453 #define RELAY0_OD ODCONDbits.ODCD0
00454 #define RELAY0_ANS ANSELDbits.ANSELd0
00455 #define RELAY0_SetHigh() do { LATDbits.LATD0 = 1; } while(0)
00456 #define RELAY0_SetLow() do { LATDbits.LATD0 = 0; } while(0)
00457 #define RELAY0_Toggle() do { LATDbits.LATD0 = ~LATDbits.LATD0; } while(0)
00458 #define RELAY0_GetValue() PORTDbits.RD0
00459 #define RELAY0_SetDigitalInput() do { TRISDbits.TRISD0 = 1; } while(0)
00460 #define RELAY0_SetDigitalOutput() do { TRISDbits.TRISD0 = 0; } while(0)
00461 #define RELAY0_SetPullup() do { WPUDbits.WPUD0 = 1; } while(0)
00462 #define RELAY0_ResetPullup() do { WPUDbits.WPUD0 = 0; } while(0)
00463 #define RELAY0_SetPushPull() do { ODCONDbits.ODCD0 = 0; } while(0)
00464 #define RELAY0_SetOpenDrain() do { ODCONDbits.ODCD0 = 1; } while(0)
00465 #define RELAY0_SetAnalogMode() do { ANSELDbits.ANSELd0 = 1; } while(0)

```

```

00466 #define RELAY0_SetDigitalMode()      do { ANSELDbits.ANSELD0 = 0; } while(0)
00467
00468 // get/set RD1 procedures
00469 #define RD1_SetHigh()                  do { LATDbits.LATD1 = 1; } while(0)
00470 #define RD1_SetLow()                   do { LATDbits.LATD1 = 0; } while(0)
00471 #define RD1_Toggle()                   do { LATDbits.LATD1 = ~LATDbits.LATD1; } while(0)
00472 #define RD1_GetValue()                 PORTDbits.RD1
00473 #define RD1_SetDigitalInput()          do { TRISDbits.TRISD1 = 1; } while(0)
00474 #define RD1_SetDigitalOutput()         do { TRISDbits.TRISD1 = 0; } while(0)
00475 #define RD1_SetPullup()                do { WPUDBits.WPUD1 = 1; } while(0)
00476 #define RD1_ResetPullup()              do { WPUDBits.WPUD1 = 0; } while(0)
00477 #define RD1_SetAnalogMode()            do { ANSELDbits.ANSELD1 = 1; } while(0)
00478 #define RD1_SetDigitalMode()           do { ANSELDbits.ANSELD1 = 0; } while(0)
00479
00480 // get/set SW0 aliases
00481 #define SW0_TRIS                       TRISDbits.TRISD2
00482 #define SW0_LAT                         LATDbits.LATD2
00483 #define SW0_PORT                       PORTDbits.RD2
00484 #define SW0_WPU                        WPUDBits.WPUD2
00485 #define SW0_OD                         ODCONDbits.ODCD2
00486 #define SW0_ANS                        ANSELDbits.ANSELD2
00487 #define SW0_SetHigh()                  do { LATDbits.LATD2 = 1; } while(0)
00488 #define SW0_SetLow()                   do { LATDbits.LATD2 = 0; } while(0)
00489 #define SW0_Toggle()                   do { LATDbits.LATD2 = ~LATDbits.LATD2; } while(0)
00490 #define SW0_GetValue()                 PORTDbits.RD2
00491 #define SW0_SetDigitalInput()          do { TRISDbits.TRISD2 = 1; } while(0)
00492 #define SW0_SetDigitalOutput()         do { TRISDbits.TRISD2 = 0; } while(0)
00493 #define SW0_SetPullup()                do { WPUDBits.WPUD2 = 1; } while(0)
00494 #define SW0_ResetPullup()              do { WPUDBits.WPUD2 = 0; } while(0)
00495 #define SW0_SetPushPull()              do { ODCONDbits.ODCD2 = 0; } while(0)
00496 #define SW0_SetOpenDrain()             do { ODCONDbits.ODCD2 = 1; } while(0)
00497 #define SW0_SetAnalogMode()            do { ANSELDbits.ANSELD2 = 1; } while(0)
00498 #define SW0_SetDigitalMode()           do { ANSELDbits.ANSELD2 = 0; } while(0)
00499
00500 // get/set CALIB aliases
00501 #define CALIB_TRIS                     TRISDbits.TRISD3
00502 #define CALIB_LAT                      LATDbits.LATD3
00503 #define CALIB_PORT                     PORTDbits.RD3
00504 #define CALIB_WPU                      WPUDBits.WPUD3
00505 #define CALIB_OD                       ODCONDbits.ODCD3
00506 #define CALIB_ANS                      ANSELDbits.ANSELD3
00507 #define CALIB_SetHigh()                 do { LATDbits.LATD3 = 1; } while(0)
00508 #define CALIB_SetLow()                  do { LATDbits.LATD3 = 0; } while(0)
00509 #define CALIB_Toggle()                  do { LATDbits.LATD3 = ~LATDbits.LATD3; } while(0)
00510 #define CALIB_GetValue()                PORTDbits.RD3
00511 #define CALIB_SetDigitalInput()         do { TRISDbits.TRISD3 = 1; } while(0)
00512 #define CALIB_SetDigitalOutput()        do { TRISDbits.TRISD3 = 0; } while(0)
00513 #define CALIB_SetPullup()               do { WPUDBits.WPUD3 = 1; } while(0)
00514 #define CALIB_ResetPullup()             do { WPUDBits.WPUD3 = 0; } while(0)
00515 #define CALIB_SetPushPull()             do { ODCONDbits.ODCD3 = 0; } while(0)
00516 #define CALIB_SetOpenDrain()            do { ODCONDbits.ODCD3 = 1; } while(0)
00517 #define CALIB_SetAnalogMode()           do { ANSELDbits.ANSELD3 = 1; } while(0)
00518 #define CALIB_SetDigitalMode()          do { ANSELDbits.ANSELD3 = 0; } while(0)
00519
00520 // get/set RELAYL1 aliases
00521 #define RELAYL1_TRIS                   TRISDbits.TRISD4
00522 #define RELAYL1_LAT                    LATDbits.LATD4
00523 #define RELAYL1_PORT                   PORTDbits.RD4
00524 #define RELAYL1_WPU                    WPUDBits.WPUD4
00525 #define RELAYL1_OD                     ODCONDbits.ODCD4
00526 #define RELAYL1_ANS                    ANSELDbits.ANSELD4
00527 #define RELAYL1_SetHigh()               do { LATDbits.LATD4 = 1; } while(0)
00528 #define RELAYL1_SetLow()                do { LATDbits.LATD4 = 0; } while(0)
00529 #define RELAYL1_Toggle()                do { LATDbits.LATD4 = ~LATDbits.LATD4; } while(0)
00530 #define RELAYL1_GetValue()              PORTDbits.RD4
00531 #define RELAYL1_SetDigitalInput()        do { TRISDbits.TRISD4 = 1; } while(0)
00532 #define RELAYL1_SetDigitalOutput()       do { TRISDbits.TRISD4 = 0; } while(0)
00533 #define RELAYL1_SetPullup()             do { WPUDBits.WPUD4 = 1; } while(0)
00534 #define RELAYL1_ResetPullup()           do { WPUDBits.WPUD4 = 0; } while(0)
00535 #define RELAYL1_SetPushPull()           do { ODCONDbits.ODCD4 = 0; } while(0)
00536 #define RELAYL1_SetOpenDrain()          do { ODCONDbits.ODCD4 = 1; } while(0)
00537 #define RELAYL1_SetAnalogMode()         do { ANSELDbits.ANSELD4 = 1; } while(0)
00538 #define RELAYL1_SetDigitalMode()         do { ANSELDbits.ANSELD4 = 0; } while(0)
00539
00540 // get/set RELAYL2 aliases
00541 #define RELAYL2_TRIS                   TRISDbits.TRISD5
00542 #define RELAYL2_LAT                    LATDbits.LATD5
00543 #define RELAYL2_PORT                   PORTDbits.RD5
00544 #define RELAYL2_WPU                    WPUDBits.WPUD5
00545 #define RELAYL2_OD                     ODCONDbits.ODCD5
00546 #define RELAYL2_ANS                    ANSELDbits.ANSELD5
00547 #define RELAYL2_SetHigh()               do { LATDbits.LATD5 = 1; } while(0)
00548 #define RELAYL2_SetLow()                do { LATDbits.LATD5 = 0; } while(0)
00549 #define RELAYL2_Toggle()                do { LATDbits.LATD5 = ~LATDbits.LATD5; } while(0)
00550 #define RELAYL2_GetValue()              PORTDbits.RD5
00551 #define RELAYL2_SetDigitalInput()        do { TRISDbits.TRISD5 = 1; } while(0)
00552 #define RELAYL2_SetDigitalOutput()       do { TRISDbits.TRISD5 = 0; } while(0)

```



```

00553 #define RELAYL2_SetPullup() do { WPUDbits.WPUD5 = 1; } while(0)
00554 #define RELAYL2_ResetPullup() do { WPUDbits.WPUD5 = 0; } while(0)
00555 #define RELAYL2_SetPushPull() do { ODCONDbits.ODCD5 = 0; } while(0)
00556 #define RELAYL2_SetOpenDrain() do { ODCONDbits.ODCD5 = 1; } while(0)
00557 #define RELAYL2_SetAnalogMode() do { ANSELDbits.ANSELD5 = 1; } while(0)
00558 #define RELAYL2_SetDigitalMode() do { ANSELDbits.ANSELD5 = 0; } while(0)
00559
00560 // get/set AC_CHARGER aliases
00561 #define AC_CHARGER_TRIS TRISDbits.TRISD6
00562 #define AC_CHARGER_LAT LATDbits.LATD6
00563 #define AC_CHARGER_PORT PORTDbits.RD6
00564 #define AC_CHARGER_WPU WPUDbits.WPUD6
00565 #define AC_CHARGER_OD ODCONDbits.ODCD6
00566 #define AC_CHARGER_ANS ANSELDbits.ANSELD6
00567 #define AC_CHARGER_SetHigh() do { LATDbits.LATD6 = 1; } while(0)
00568 #define AC_CHARGER_SetLow() do { LATDbits.LATD6 = 0; } while(0)
00569 #define AC_CHARGER_Toggle() do { LATDbits.LATD6 = ~LATDbits.LATD6; } while(0)
00570 #define AC_CHARGER_GetValue() PORTDbits.RD6
00571 #define AC_CHARGER_SetDigitalInput() do { TRISDbits.TRISD6 = 1; } while(0)
00572 #define AC_CHARGER_SetDigitalOutput() do { TRISDbits.TRISD6 = 0; } while(0)
00573 #define AC_CHARGER_SetPullup() do { WPUDbits.WPUD6 = 1; } while(0)
00574 #define AC_CHARGER_ResetPullup() do { WPUDbits.WPUD6 = 0; } while(0)
00575 #define AC_CHARGER_SetPushPull() do { ODCONDbits.ODCD6 = 0; } while(0)
00576 #define AC_CHARGER_SetOpenDrain() do { ODCONDbits.ODCD6 = 1; } while(0)
00577 #define AC_CHARGER_SetAnalogMode() do { ANSELDbits.ANSELD6 = 1; } while(0)
00578 #define AC_CHARGER_SetDigitalMode() do { ANSELDbits.ANSELD6 = 0; } while(0)
00579
00580 // get/set OUTP_3 aliases
00581 #define OUTP_3_TRIS TRISDbits.TRISD7
00582 #define OUTP_3_LAT LATDbits.LATD7
00583 #define OUTP_3_PORT PORTDbits.RD7
00584 #define OUTP_3_WPU WPUDbits.WPUD7
00585 #define OUTP_3_OD ODCONDbits.ODCD7
00586 #define OUTP_3_ANS ANSELDbits.ANSELD7
00587 #define OUTP_3_SetHigh() do { LATDbits.LATD7 = 1; } while(0)
00588 #define OUTP_3_SetLow() do { LATDbits.LATD7 = 0; } while(0)
00589 #define OUTP_3_Toggle() do { LATDbits.LATD7 = ~LATDbits.LATD7; } while(0)
00590 #define OUTP_3_GetValue() PORTDbits.RD7
00591 #define OUTP_3_SetDigitalInput() do { TRISDbits.TRISD7 = 1; } while(0)
00592 #define OUTP_3_SetDigitalOutput() do { TRISDbits.TRISD7 = 0; } while(0)
00593 #define OUTP_3_SetPullup() do { WPUDbits.WPUD7 = 1; } while(0)
00594 #define OUTP_3_ResetPullup() do { WPUDbits.WPUD7 = 0; } while(0)
00595 #define OUTP_3_SetPushPull() do { ODCONDbits.ODCD7 = 0; } while(0)
00596 #define OUTP_3_SetOpenDrain() do { ODCONDbits.ODCD7 = 1; } while(0)
00597 #define OUTP_3_SetAnalogMode() do { ANSELDbits.ANSELD7 = 1; } while(0)
00598 #define OUTP_3_SetDigitalMode() do { ANSELDbits.ANSELD7 = 0; } while(0)
00599
00600 // get/set LED0 aliases
00601 #define LED0_TRIS TRISEbits.TRISE0
00602 #define LED0_LAT LATEbits.LATE0
00603 #define LED0_PORT PORTEbits.RE0
00604 #define LED0_WPU WPUEbits.WPUE0
00605 #define LED0_OD ODCONEbits.ODCE0
00606 #define LED0_ANS ANSELEbits.ANSELE0
00607 #define LED0_SetHigh() do { LATEbits.LATE0 = 1; } while(0)
00608 #define LED0_SetLow() do { LATEbits.LATE0 = 0; } while(0)
00609 #define LED0_Toggle() do { LATEbits.LATE0 = ~LATEbits.LATE0; } while(0)
00610 #define LED0_GetValue() PORTEbits.RE0
00611 #define LED0_SetDigitalInput() do { TRISEbits.TRISE0 = 1; } while(0)
00612 #define LED0_SetDigitalOutput() do { TRISEbits.TRISE0 = 0; } while(0)
00613 #define LED0_SetPullup() do { WPUEbits.WPUE0 = 1; } while(0)
00614 #define LED0_ResetPullup() do { WPUEbits.WPUE0 = 0; } while(0)
00615 #define LED0_SetPushPull() do { ODCONEbits.ODCE0 = 0; } while(0)
00616 #define LED0_SetOpenDrain() do { ODCONEbits.ODCE0 = 1; } while(0)
00617 #define LED0_SetAnalogMode() do { ANSELEbits.ANSELE0 = 1; } while(0)
00618 #define LED0_SetDigitalMode() do { ANSELEbits.ANSELE0 = 0; } while(0)
00619
00620 // get/set DEBUG1 aliases
00621 #define DEBUG1_TRIS TRISEbits.TRISE1
00622 #define DEBUG1_LAT LATEbits.LATE1
00623 #define DEBUG1_PORT PORTEbits.RE1
00624 #define DEBUG1_WPU WPUEbits.WPUE1
00625 #define DEBUG1_OD ODCONEbits.ODCE1
00626 #define DEBUG1_ANS ANSELEbits.ANSELE1
00627 #define DEBUG1_SetHigh() do { LATEbits.LATE1 = 1; } while(0)
00628 #define DEBUG1_SetLow() do { LATEbits.LATE1 = 0; } while(0)
00629 #define DEBUG1_Toggle() do { LATEbits.LATE1 = ~LATEbits.LATE1; } while(0)
00630 #define DEBUG1_GetValue() PORTEbits.RE1
00631 #define DEBUG1_SetDigitalInput() do { TRISEbits.TRISE1 = 1; } while(0)
00632 #define DEBUG1_SetDigitalOutput() do { TRISEbits.TRISE1 = 0; } while(0)
00633 #define DEBUG1_SetPullup() do { WPUEbits.WPUE1 = 1; } while(0)
00634 #define DEBUG1_ResetPullup() do { WPUEbits.WPUE1 = 0; } while(0)
00635 #define DEBUG1_SetPushPull() do { ODCONEbits.ODCE1 = 0; } while(0)
00636 #define DEBUG1_SetOpenDrain() do { ODCONEbits.ODCE1 = 1; } while(0)
00637 #define DEBUG1_SetAnalogMode() do { ANSELEbits.ANSELE1 = 1; } while(0)
00638 #define DEBUG1_SetDigitalMode() do { ANSELEbits.ANSELE1 = 0; } while(0)
00639

```

```

00640 // get/set DEBUG2 aliases
00641 #define DEBUG2_TRIS          TRISEbits.TRISE2
00642 #define DEBUG2_LAT          LATEbits.LATE2
00643 #define DEBUG2_PORT        PORTEbits.RE2
00644 #define DEBUG2_WPU        WPUEbits.WPUE2
00645 #define DEBUG2_OD          ODCONEbits.ODCE2
00646 #define DEBUG2_ANS        ANSELEbits.ANSELE2
00647 #define DEBUG2_SetHigh()    do { LATEbits.LATE2 = 1; } while(0)
00648 #define DEBUG2_SetLow()     do { LATEbits.LATE2 = 0; } while(0)
00649 #define DEBUG2_Toggle()     do { LATEbits.LATE2 = ~LATEbits.LATE2; } while(0)
00650 #define DEBUG2_GetValue()   PORTEbits.RE2
00651 #define DEBUG2_SetDigitalInput() do { TRISEbits.TRISE2 = 1; } while(0)
00652 #define DEBUG2_SetDigitalOutput() do { TRISEbits.TRISE2 = 0; } while(0)
00653 #define DEBUG2_SetPullup()  do { WPUEbits.WPUE2 = 1; } while(0)
00654 #define DEBUG2_ResetPullup() do { WPUEbits.WPUE2 = 0; } while(0)
00655 #define DEBUG2_SetPushPull() do { ODCONEbits.ODCE2 = 0; } while(0)
00656 #define DEBUG2_SetOpenDrain() do { ODCONEbits.ODCE2 = 1; } while(0)
00657 #define DEBUG2_SetAnalogMode() do { ANSELEbits.ANSELE2 = 1; } while(0)
00658 #define DEBUG2_SetDigitalMode() do { ANSELEbits.ANSELE2 = 0; } while(0)
00659
00660 // get/set DAC_CS0 aliases
00661 #define DAC_CS0_TRIS        TRISFbits.TRISF0
00662 #define DAC_CS0_LAT        LATFbits.LATF0
00663 #define DAC_CS0_PORT        PORTFbits.RF0
00664 #define DAC_CS0_WPU        WPUFbits.WPUF0
00665 #define DAC_CS0_OD          ODCONFbits.ODCF0
00666 #define DAC_CS0_ANS        ANSELFbits.ANSELF0
00667 #define DAC_CS0_SetHigh()  do { LATFbits.LATF0 = 1; } while(0)
00668 #define DAC_CS0_SetLow()   do { LATFbits.LATF0 = 0; } while(0)
00669 #define DAC_CS0_Toggle()   do { LATFbits.LATF0 = ~LATFbits.LATF0; } while(0)
00670 #define DAC_CS0_GetValue() PORTFbits.RF0
00671 #define DAC_CS0_SetDigitalInput() do { TRISFbits.TRISF0 = 1; } while(0)
00672 #define DAC_CS0_SetDigitalOutput() do { TRISFbits.TRISF0 = 0; } while(0)
00673 #define DAC_CS0_SetPullup() do { WPUFbits.WPUF0 = 1; } while(0)
00674 #define DAC_CS0_ResetPullup() do { WPUFbits.WPUF0 = 0; } while(0)
00675 #define DAC_CS0_SetPushPull() do { ODCONFbits.ODCF0 = 0; } while(0)
00676 #define DAC_CS0_SetOpenDrain() do { ODCONFbits.ODCF0 = 1; } while(0)
00677 #define DAC_CS0_SetAnalogMode() do { ANSELFbits.ANSELF0 = 1; } while(0)
00678 #define DAC_CS0_SetDigitalMode() do { ANSELFbits.ANSELF0 = 0; } while(0)
00679
00680 // get/set IO_RF1 aliases
00681 #define IO_RF1_TRIS        TRISFbits.TRISF1
00682 #define IO_RF1_LAT        LATFbits.LATF1
00683 #define IO_RF1_PORT        PORTFbits.RF1
00684 #define IO_RF1_WPU        WPUFbits.WPUF1
00685 #define IO_RF1_OD          ODCONFbits.ODCF1
00686 #define IO_RF1_ANS        ANSELFbits.ANSELF1
00687 #define IO_RF1_SetHigh()  do { LATFbits.LATF1 = 1; } while(0)
00688 #define IO_RF1_SetLow()   do { LATFbits.LATF1 = 0; } while(0)
00689 #define IO_RF1_Toggle()   do { LATFbits.LATF1 = ~LATFbits.LATF1; } while(0)
00690 #define IO_RF1_GetValue() PORTFbits.RF1
00691 #define IO_RF1_SetDigitalInput() do { TRISFbits.TRISF1 = 1; } while(0)
00692 #define IO_RF1_SetDigitalOutput() do { TRISFbits.TRISF1 = 0; } while(0)
00693 #define IO_RF1_SetPullup() do { WPUFbits.WPUF1 = 1; } while(0)
00694 #define IO_RF1_ResetPullup() do { WPUFbits.WPUF1 = 0; } while(0)
00695 #define IO_RF1_SetPushPull() do { ODCONFbits.ODCF1 = 0; } while(0)
00696 #define IO_RF1_SetOpenDrain() do { ODCONFbits.ODCF1 = 1; } while(0)
00697 #define IO_RF1_SetAnalogMode() do { ANSELFbits.ANSELF1 = 1; } while(0)
00698 #define IO_RF1_SetDigitalMode() do { ANSELFbits.ANSELF1 = 0; } while(0)
00699
00700 // get/set SELECT_B aliases
00701 #define SELECT_B_TRIS      TRISFbits.TRISF2
00702 #define SELECT_B_LAT      LATFbits.LATF2
00703 #define SELECT_B_PORT      PORTFbits.RF2
00704 #define SELECT_B_WPU      WPUFbits.WPUF2
00705 #define SELECT_B_OD        ODCONFbits.ODCF2
00706 #define SELECT_B_ANS      ANSELFbits.ANSELF2
00707 #define SELECT_B_SetHigh() do { LATFbits.LATF2 = 1; } while(0)
00708 #define SELECT_B_SetLow()  do { LATFbits.LATF2 = 0; } while(0)
00709 #define SELECT_B_Toggle()  do { LATFbits.LATF2 = ~LATFbits.LATF2; } while(0)
00710 #define SELECT_B_GetValue() PORTFbits.RF2
00711 #define SELECT_B_SetDigitalInput() do { TRISFbits.TRISF2 = 1; } while(0)
00712 #define SELECT_B_SetDigitalOutput() do { TRISFbits.TRISF2 = 0; } while(0)
00713 #define SELECT_B_SetPullup() do { WPUFbits.WPUF2 = 1; } while(0)
00714 #define SELECT_B_ResetPullup() do { WPUFbits.WPUF2 = 0; } while(0)
00715 #define SELECT_B_SetPushPull() do { ODCONFbits.ODCF2 = 0; } while(0)
00716 #define SELECT_B_SetOpenDrain() do { ODCONFbits.ODCF2 = 1; } while(0)
00717 #define SELECT_B_SetAnalogMode() do { ANSELFbits.ANSELF2 = 1; } while(0)
00718 #define SELECT_B_SetDigitalMode() do { ANSELFbits.ANSELF2 = 0; } while(0)
00719
00720 // get/set ENTER_B aliases
00721 #define ENTER_B_TRIS      TRISFbits.TRISF3
00722 #define ENTER_B_LAT      LATFbits.LATF3
00723 #define ENTER_B_PORT      PORTFbits.RF3
00724 #define ENTER_B_WPU      WPUFbits.WPUF3
00725 #define ENTER_B_OD        ODCONFbits.ODCF3
00726 #define ENTER_B_ANS      ANSELFbits.ANSELF3

```

```

00727 #define ENTER_B_SetHigh() do { LATFbits.LATF3 = 1; } while(0)
00728 #define ENTER_B_SetLow() do { LATFbits.LATF3 = 0; } while(0)
00729 #define ENTER_B_Toggle() do { LATFbits.LATF3 = ~LATFbits.LATF3; } while(0)
00730 #define ENTER_B_GetValue() PORTFbits.RF3
00731 #define ENTER_B_SetDigitalInput() do { TRISFbits.TRISF3 = 1; } while(0)
00732 #define ENTER_B_SetDigitalOutput() do { TRISFbits.TRISF3 = 0; } while(0)
00733 #define ENTER_B_SetPullup() do { WPUFbits.WPUF3 = 1; } while(0)
00734 #define ENTER_B_ResetPullup() do { WPUFbits.WPUF3 = 0; } while(0)
00735 #define ENTER_B_SetPushPull() do { ODCONFbits.ODCF3 = 0; } while(0)
00736 #define ENTER_B_SetOpenDrain() do { ODCONFbits.ODCF3 = 1; } while(0)
00737 #define ENTER_B_SetAnalogMode() do { ANSELFbits.ANSELF3 = 1; } while(0)
00738 #define ENTER_B_SetDigitalMode() do { ANSELFbits.ANSELF3 = 0; } while(0)
00739
00740 // get/set IO_RF4 aliases
00741 #define IO_RF4_TRIS TRISFbits.TRISF4
00742 #define IO_RF4_LAT LATFbits.LATF4
00743 #define IO_RF4_PORT PORTFbits.RF4
00744 #define IO_RF4_WPU WPUFbits.WPUF4
00745 #define IO_RF4_OD ODCONFbits.ODCF4
00746 #define IO_RF4_ANS ANSELFbits.ANSELF4
00747 #define IO_RF4_SetHigh() do { LATFbits.LATF4 = 1; } while(0)
00748 #define IO_RF4_SetLow() do { LATFbits.LATF4 = 0; } while(0)
00749 #define IO_RF4_Toggle() do { LATFbits.LATF4 = ~LATFbits.LATF4; } while(0)
00750 #define IO_RF4_GetValue() PORTFbits.RF4
00751 #define IO_RF4_SetDigitalInput() do { TRISFbits.TRISF4 = 1; } while(0)
00752 #define IO_RF4_SetDigitalOutput() do { TRISFbits.TRISF4 = 0; } while(0)
00753 #define IO_RF4_SetPullup() do { WPUFbits.WPUF4 = 1; } while(0)
00754 #define IO_RF4_ResetPullup() do { WPUFbits.WPUF4 = 0; } while(0)
00755 #define IO_RF4_SetPushPull() do { ODCONFbits.ODCF4 = 0; } while(0)
00756 #define IO_RF4_SetOpenDrain() do { ODCONFbits.ODCF4 = 1; } while(0)
00757 #define IO_RF4_SetAnalogMode() do { ANSELFbits.ANSELF4 = 1; } while(0)
00758 #define IO_RF4_SetDigitalMode() do { ANSELFbits.ANSELF4 = 0; } while(0)
00759
00760 // get/set MAX_EN1 aliases
00761 #define MAX_EN1_TRIS TRISFbits.TRISF5
00762 #define MAX_EN1_LAT LATFbits.LATF5
00763 #define MAX_EN1_PORT PORTFbits.RF5
00764 #define MAX_EN1_WPU WPUFbits.WPUF5
00765 #define MAX_EN1_OD ODCONFbits.ODCF5
00766 #define MAX_EN1_ANS ANSELFbits.ANSELF5
00767 #define MAX_EN1_SetHigh() do { LATFbits.LATF5 = 1; } while(0)
00768 #define MAX_EN1_SetLow() do { LATFbits.LATF5 = 0; } while(0)
00769 #define MAX_EN1_Toggle() do { LATFbits.LATF5 = ~LATFbits.LATF5; } while(0)
00770 #define MAX_EN1_GetValue() PORTFbits.RF5
00771 #define MAX_EN1_SetDigitalInput() do { TRISFbits.TRISF5 = 1; } while(0)
00772 #define MAX_EN1_SetDigitalOutput() do { TRISFbits.TRISF5 = 0; } while(0)
00773 #define MAX_EN1_SetPullup() do { WPUFbits.WPUF5 = 1; } while(0)
00774 #define MAX_EN1_ResetPullup() do { WPUFbits.WPUF5 = 0; } while(0)
00775 #define MAX_EN1_SetPushPull() do { ODCONFbits.ODCF5 = 0; } while(0)
00776 #define MAX_EN1_SetOpenDrain() do { ODCONFbits.ODCF5 = 1; } while(0)
00777 #define MAX_EN1_SetAnalogMode() do { ANSELFbits.ANSELF5 = 1; } while(0)
00778 #define MAX_EN1_SetDigitalMode() do { ANSELFbits.ANSELF5 = 0; } while(0)
00779
00780 // get/set IO_RF6 aliases
00781 #define IO_RF6_TRIS TRISFbits.TRISF6
00782 #define IO_RF6_LAT LATFbits.LATF6
00783 #define IO_RF6_PORT PORTFbits.RF6
00784 #define IO_RF6_WPU WPUFbits.WPUF6
00785 #define IO_RF6_OD ODCONFbits.ODCF6
00786 #define IO_RF6_ANS ANSELFbits.ANSELF6
00787 #define IO_RF6_SetHigh() do { LATFbits.LATF6 = 1; } while(0)
00788 #define IO_RF6_SetLow() do { LATFbits.LATF6 = 0; } while(0)
00789 #define IO_RF6_Toggle() do { LATFbits.LATF6 = ~LATFbits.LATF6; } while(0)
00790 #define IO_RF6_GetValue() PORTFbits.RF6
00791 #define IO_RF6_SetDigitalInput() do { TRISFbits.TRISF6 = 1; } while(0)
00792 #define IO_RF6_SetDigitalOutput() do { TRISFbits.TRISF6 = 0; } while(0)
00793 #define IO_RF6_SetPullup() do { WPUFbits.WPUF6 = 1; } while(0)
00794 #define IO_RF6_ResetPullup() do { WPUFbits.WPUF6 = 0; } while(0)
00795 #define IO_RF6_SetPushPull() do { ODCONFbits.ODCF6 = 0; } while(0)
00796 #define IO_RF6_SetOpenDrain() do { ODCONFbits.ODCF6 = 1; } while(0)
00797 #define IO_RF6_SetAnalogMode() do { ANSELFbits.ANSELF6 = 1; } while(0)
00798 #define IO_RF6_SetDigitalMode() do { ANSELFbits.ANSELF6 = 0; } while(0)
00799
00800 // get/set IO_RF7 aliases
00801 #define IO_RF7_TRIS TRISFbits.TRISF7
00802 #define IO_RF7_LAT LATFbits.LATF7
00803 #define IO_RF7_PORT PORTFbits.RF7
00804 #define IO_RF7_WPU WPUFbits.WPUF7
00805 #define IO_RF7_OD ODCONFbits.ODCF7
00806 #define IO_RF7_ANS ANSELFbits.ANSELF7
00807 #define IO_RF7_SetHigh() do { LATFbits.LATF7 = 1; } while(0)
00808 #define IO_RF7_SetLow() do { LATFbits.LATF7 = 0; } while(0)
00809 #define IO_RF7_Toggle() do { LATFbits.LATF7 = ~LATFbits.LATF7; } while(0)
00810 #define IO_RF7_GetValue() PORTFbits.RF7
00811 #define IO_RF7_SetDigitalInput() do { TRISFbits.TRISF7 = 1; } while(0)
00812 #define IO_RF7_SetDigitalOutput() do { TRISFbits.TRISF7 = 0; } while(0)
00813 #define IO_RF7_SetPullup() do { WPUFbits.WPUF7 = 1; } while(0)

```

```

00814 #define IO_RF7_ResetPullup()      do { WPUFbits.WPUF7 = 0; } while(0)
00815 #define IO_RF7_SetPushPull()      do { ODCONFbits.ODCF7 = 0; } while(0)
00816 #define IO_RF7_SetOpenDrain()     do { ODCONFbits.ODCF7 = 1; } while(0)
00817 #define IO_RF7_SetAnalogMode()     do { ANSELFbits.ANSELF7 = 1; } while(0)
00818 #define IO_RF7_SetDigitalMode()    do { ANSELFbits.ANSELF7 = 0; } while(0)
00819
00830 void PIN_MANAGER_Initialize (void);
00831
00832
00833
00834
00835 #endif // PIN_MANAGER_H

```

4.38 pwm8.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
party
00029     license terms applicable to your use of third party software (including open source software) that
may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "pwm8.h"
00053
00057
00058 void PWM8_Initialize(void)
00059 {
00060     // Set the PWM to the options selected in the PIC10 / PIC12 / PIC16 / PIC18 MCUs.
00061     // PWM8POL active_hi; PWM8EN enabled;
00062     PWM8CON = 0x80;
00063
00064     // DC 0;
00065     PWM8DCH = 0x00;
00066
00067     // DC 1;
00068     PWM8DCL = 0x40;
00069
00070     // Select timer
00071     CCPTMRS1bits.P8TSEL = 1;
00072 }
00073
00074 void PWM8_LoadDutyValue(uint16_t dutyValue)
00075 {
00076     // Writing to 8 MSBs of PWM duty cycle in PWMDCH register
00077     PWM8DCH = (dutyValue & 0x03FC)»2;
00078
00079     // Writing to 2 LSBs of PWM duty cycle in PWMDCL register
00080     PWM8DCL = (dutyValue & 0x0003)«6;
00081 }

```

4.39 pwm8.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.

```

```

00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
party
00029     license terms applicable to your use of third party software (including open source software) that
may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef PWM8_H
00048     #define PWM8_H
00049
00050
00053
00054     #include <xc.h>
00055     #include <stdint.h>
00056
00057     #ifdef __cplusplus    // Provide C++ Compatibility
00058
00059         extern "C" {
00060
00061     #endif
00062
00066
00067     #define PWM8_INITIALIZE_DUTY_VALUE    1
00068
00072
00102     void PWM8_Initialize(void);
00103
00129     void PWM8_LoadDutyValue(uint16_t dutyValue);
00130
00131
00132     #ifdef __cplusplus    // Provide C++ Compatibility
00133
00134         }
00135
00136     #endif
00137
00138 #endif //PWM8_H

```

4.40 spi1.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
party
00029     license terms applicable to your use of third party software (including open source software) that
may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #include "spi1.h"

```

```

00048 #include <xc.h>
00049
00050 typedef struct {
00051     uint8_t con0;
00052     uint8_t con1;
00053     uint8_t con2;
00054     uint8_t baud;
00055     uint8_t operation;
00056 } spil_configuration_t;
00057
00058
00059 //con0 == SPIxCON0, con1 == SPIxCON1, con2 == SPIxCON2, baud == SPIxBAUD, operation == Master/Slave
00060 static const spil_configuration_t spil_configuration[] = {
00061     { 0x2, 0x20, 0x0, 0x5, 0 }
00062 };
00063
00064 void SPI1_Initialize(void)
00065 {
00066     //EN enabled; LSBF MSb first; MST bus master; BMODE every byte;
00067     SPI1CON0 = 0x02;
00068     //SMP Middle; CKE Idle to active; CKP Idle:High, Active:Low; FST disabled; SSP active high; SDIP
    active high; SDOP active high;
00069     SPI1CON1 = 0x20;
00070     //SSET disabled; TXR required for a transfer; RXR suspended if the RxFIFO is full;
00071     SPI1CON2 = 0x00;
00072     //CLKSEL MFINTOSC;
00073     SPI1CLK = 0x02;
00074     //BAUD 5;
00075     SPI1BAUD = 0x05;
00076     TRISCbits.TRISC3 = 0;
00077 }
00078
00079 bool SPI1_Open(spil_modes_t spilUniqueConfiguration)
00080 {
00081     if(!SPI1CON0bits.EN)
00082     {
00083         SPI1CON0 = spil_configuration[spilUniqueConfiguration].con0;
00084         SPI1CON1 = spil_configuration[spilUniqueConfiguration].con1;
00085         SPI1CON2 = spil_configuration[spilUniqueConfiguration].con2 | (_SPI1CON2_SPI1RXR_MASK |
    _SPI1CON2_SPI1TXR_MASK);
00086         SPI1BAUD = spil_configuration[spilUniqueConfiguration].baud;
00087         TRISCbits.TRISC3 = spil_configuration[spilUniqueConfiguration].operation;
00088         SPI1CON0bits.EN = 1;
00089         return true;
00090     }
00091     return false;
00092 }
00093
00094 void SPI1_Close(void)
00095 {
00096     SPI1CON0bits.EN = 0;
00097 }
00098
00099 uint8_t SPI1_ExchangeByte(uint8_t data)
00100 {
00101     SPI1TCNTL = 1;
00102     SPI1TXB = data;
00103     while(!PIR2bits.SPI1RXIF);
00104     return SPI1RXB;
00105 }
00106
00107 void SPI1_ExchangeBlock(void *block, size_t blockSize)
00108 {
00109     uint8_t *data = block;
00110     while(blockSize--)
00111     {
00112         SPI1TCNTL = 1;
00113         SPI1TXB = *data;
00114         while(!PIR2bits.SPI1RXIF);
00115         *data++ = SPI1RXB;
00116     }
00117 }
00118
00119 // Half Duplex SPI Functions
00120 void SPI1_WriteBlock(void *block, size_t blockSize)
00121 {
00122     uint8_t *data = block;
00123     while(blockSize--)
00124     {
00125         SPI1_ExchangeByte(*data++);
00126     }
00127 }
00128
00129 void SPI1_ReadBlock(void *block, size_t blockSize)
00130 {
00131     uint8_t *data = block;
00132     while(blockSize--)

```

```

00133     {
00134         *data++ = SPI1_ExchangeByte(0);
00135     }
00136 }
00137
00138 void SPI1_WriteByte(uint8_t byte)
00139 {
00140     SPI1TXB = byte;
00141 }
00142
00143 uint8_t SPI1_ReadByte(void)
00144 {
00145     return SPI1RXB;
00146 }

```

4.41 spi1.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047 #ifndef SPI1_MASTER_H
00048 #define SPI1_MASTER_H
00049
00053
00054 #include <stdio.h>
00055 #include <stdint.h>
00056 #include <stdbool.h>
00057
00058 /* SPI interfaces */
00059 typedef enum {
00060     SPI1_DEFAULT
00061 } spil_modes_t;
00062
00063 void SPI1_Initialize(void);
00064 bool SPI1_Open(spil_modes_t spilUniqueConfiguration);
00065 void SPI1_Close(void);
00066 uint8_t SPI1_ExchangeByte(uint8_t data);
00067 void SPI1_ExchangeBlock(void *block, size_t blockSize);
00068 void SPI1_WriteBlock(void *block, size_t blockSize);
00069 void SPI1_ReadBlock(void *block, size_t blockSize);
00070 void SPI1_WriteByte(uint8_t byte);
00071 uint8_t SPI1_ReadByte(void);
00072
00073 #endif //SPI1_H

```

4.42 tmr2.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any

```

```

00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that

00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "tmr2.h"
00053
00057
00061
00062 void TMR2_Initialize(void)
00063 {
00064     // Set TMR2 to the options selected in the User Interface
00065
00066     // T2CS FOSC/4;
00067     T2CLKCON = 0x01;
00068
00069     // T2PSYNC Not Synchronized; T2MODE Software control; T2CKPOL Rising Edge; T2CKSYNC Not
    Synchronized;
00070     T2HLT = 0x00;
00071
00072     // T2RSEL T2CKIPPS pin;
00073     T2RST = 0x00;
00074
00075     // PR2 99;
00076     T2PR = 0x63;
00077
00078     // TMR2 0;
00079     T2TMR = 0x00;
00080
00081     // Clearing IF flag.
00082     PIR4bits.TMR2IF = 0;
00083
00084     // T2CKPS 1:16; T2OUTPS 1:1; TMR2ON on;
00085     T2CON = 0xC0;
00086 }
00087
00088 void TMR2_ModeSet(TMR2_HLT_MODE mode)
00089 {
00090     // Configure different types HLT mode
00091     T2HLTbits.MODE = mode;
00092 }
00093
00094 void TMR2_ExtResetSourceSet(TMR2_HLT_EXT_RESET_SOURCE reset)
00095 {
00096     //Configure different types of HLT external reset source
00097     T2RSTbits.RSEL = reset;
00098 }
00099
00100 void TMR2_Start(void)
00101 {
00102     // Start the Timer by writing to TMRxON bit
00103     T2CONbits.TMR2ON = 1;
00104 }
00105
00106 void TMR2_StartTimer(void)
00107 {
00108     TMR2_Start();
00109 }
00110
00111 void TMR2_Stop(void)
00112 {
00113     // Stop the Timer by writing to TMRxON bit
00114     T2CONbits.TMR2ON = 0;
00115 }
00116
00117 void TMR2_StopTimer(void)
00118 {
00119     TMR2_Stop();
00120 }

```



```

00121
00122 uint8_t TMR2_Counter8BitGet(void)
00123 {
00124     uint8_t readVal;
00125
00126     readVal = TMR2;
00127
00128     return readVal;
00129 }
00130
00131 uint8_t TMR2_ReadTimer(void)
00132 {
00133     return TMR2_Counter8BitGet();
00134 }
00135
00136 void TMR2_Counter8BitSet(uint8_t timerVal)
00137 {
00138     // Write to the Timer2 register
00139     TMR2 = timerVal;
00140 }
00141
00142 void TMR2_WriteTimer(uint8_t timerVal)
00143 {
00144     TMR2_Counter8BitSet(timerVal);
00145 }
00146
00147 void TMR2_Period8BitSet(uint8_t periodVal)
00148 {
00149     PR2 = periodVal;
00150 }
00151
00152 void TMR2_LoadPeriodRegister(uint8_t periodVal)
00153 {
00154     TMR2_Period8BitSet(periodVal);
00155 }
00156
00157 bool TMR2_HasOverflowOccured(void)
00158 {
00159     // check if overflow has occurred by checking the TMRIF bit
00160     bool status = PIR4bits.TMR2IF;
00161     if(status)
00162     {
00163         // Clearing IF flag.
00164         PIR4bits.TMR2IF = 0;
00165     }
00166     return status;
00167 }

```

4.43 tmr2.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
    party
00029     license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00032     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00033     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00034     FOR A PARTICULAR PURPOSE.
00035
00036     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00037     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00038     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00039     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00040     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00041     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00042     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00043     SOFTWARE.
00044 */
00045 */
00046
00047 #ifndef TMR2_H
00048 #define TMR2_H
00049
00053
00054 #include <stdint.h>
00055 #include <stdbool.h>

```

```

00056
00057 #ifndef __cplusplus // Provide C++ Compatibility
00058
00059     extern "C" {
00060
00061 #endif
00062
00066
00078
00079 typedef enum
00080 {
00081
00082     /* Roll-over Pulse mode clears the TMRx upon TMRx = PRx, then continue running.
00083     ON bit must be set and is not affected by Resets
00084     */
00085
00086     /* Roll-over Pulse mode indicates that Timer starts
00087     immediately upon ON = 1 (Software Control)
00088     */
00089     TMR2_ROP_STARTS_TMRON,
00090
00091     /* Roll-over Pulse mode indicates that the Timer starts
00092     when ON = 1 and TMRx_ers = 1. Stops when TMRx_ers = 0
00093     */
00094     TMR2_ROP_STARTS_TMRON_ERSHIGH,
00095
00096     /* Roll-over Pulse mode indicates that the Timer starts
00097     when ON = 1 and TMRx_ers = 0. Stops when TMRx_ers = 1
00098     */
00099     TMR2_ROP_STARTS_TMRON_ERSLOW,
00100
00101     /* Roll-over Pulse mode indicates that the Timer resets
00102     upon rising or falling edge of TMRx_ers
00103     */
00104     TMR2_ROP_RESETS_ERSBOTHEDGE,
00105
00106     /* Roll-over Pulse mode indicates that the Timer resets
00107     upon rising edge of TMRx_ers
00108     */
00109     TMR2_ROP_RESETS_ERSRISINGEDGE,
00110
00111     /* Roll-over Pulse mode indicates that the Timer resets
00112     upon falling edge of TMRx_ers
00113     */
00114     TMR2_ROP_RESETS_ERSFALLINGEDGE,
00115
00116     /* Roll-over Pulse mode indicates that the Timer resets
00117     upon TMRx_ers = 0
00118     */
00119     TMR2_ROP_RESETS_ERSLOW,
00120
00121     /* Roll-over Pulse mode indicates that the Timer resets
00122     upon TMRx_ers = 1
00123     */
00124     TMR2_ROP_RESETS_ERSHIGH,
00125
00126     /*In all One-Shot mode the timer resets and the ON bit is
00127     cleared when the timer value matches the PRx period
00128     value. The ON bit must be set by software to start
00129     another timer cycle.
00130     */
00131
00132     /* One shot mode indicates that the Timer starts
00133     immediately upon ON = 1 (Software Control)
00134     */
00135     TMR2_OS_STARTS_TMRON,
00136
00137     /* One shot mode indicates that the Timer starts
00138     when a rising edge is detected on the TMRx_ers
00139     */
00140     TMR2_OS_STARTS_ERSRISINGEDGE ,
00141
00142     /* One shot mode indicates that the Timer starts
00143     when a falling edge is detected on the TMRx_ers
00144     */
00145     TMR2_OS_STARTS_ERSFALLINGEDGE ,
00146
00147     /* One shot mode indicates that the Timer starts
00148     when either a rising or falling edge is detected on TMRx_ers
00149     */
00150     TMR2_OS_STARTS_ERSBOTHEDGE,
00151
00152     /* One shot mode indicates that the Timer starts
00153     upon first TMRx_ers rising edge and resets on all
00154     subsequent TMRx_ers rising edges
00155     */
00156     TMR2_OS_STARTS_ERSFIRSTRISINGEDGE,

```

```

00157
00158 /* One shot mode indicates that the Timer starts
00159 upon first TMRx_ers falling edge and restarts on all
00160 subsequent TMRx_ers falling edges
00161 */
00162 TMR2_OS_STARTS_ERSTIRSTFALLINGEDGE,
00163
00164 /* One shot mode indicates that the Timer starts
00165 when a rising edge is detected on the TMRx_ers,
00166 resets upon TMRx_ers = 0
00167 */
00168 TMR2_OS_STARTS_ERSRISINGEDGEDETECT,
00169 /* One shot mode indicates that the Timer starts
00170 when a falling edge is detected on the TMRx_ers,
00171 resets upon TMRx_ers = 1
00172 */
00173 TMR2_OS_STARTS_ERSFALLINGEDGEDETECT,
00174
00175 /* One shot mode indicates that the Timer starts
00176 when a TMRx_ers = 1, ON = 1 and resets upon TMRx_ers = 0
00177 */
00178 TMR2_OS_STARTS_TMRON_ERSHIGH = 0x16,
00179
00180 /* One shot mode indicates that the Timer starts
00181 when a TMRx_ers = 0, ON = 1 and resets upon TMRx_ers = 1
00182 */
00183 TMR2_OS_STARTS_TMRON_ERLOW = 0x17,
00184
00185 /*In all Mono-Stable mode the ON bit must be initially set, but
00186 not cleared upon the TMRx = PRx, and the timer will start upon
00187 an TMRx_ers start event following TMRx = PRx.*/
00188
00189 /* Mono Stable mode indicates that the Timer starts
00190 when a rising edge is detected on the TMRx_ers and ON = 1
00191 */
00192 TMR2_MS_STARTS_TMRON_ERSRISINGEDGEDETECT = 0x11,
00193
00194 /* Mono Stable mode indicates that the Timer starts
00195 when a falling edge is detected on the TMRx_ers and ON = 1
00196 */
00197 TMR2_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT = 0x12,
00198
00199 /* Mono Stable mode indicates that the Timer starts
00200 when either a rising or falling edge is detected on TMRx_ers
00201 and ON = 1
00202 */
00203 TMR2_MS_STARTS_TMRON_ERSEBOTHEDGE = 0x13
00204
00205 } TMR2_HLT_MODE;
00206
00219
00220 typedef enum
00221 {
00222     /* T2INPPS is the Timer external reset source
00223     */
00224     TMR2_T2INPPS,
00225
00226     /* Reserved enum cannot be used
00227     */
00228     TMR2_RESERVED,
00229
00230     /* Timer4 Postscale is the Timer external reset source
00231     */
00232     TMR2_T4POSTSCALED,
00233
00234     /* Timer6 Postscale is the Timer external reset source
00235     */
00236     TMR2_T6POSTSCALED,
00237
00238     /* CCP1_OUT is the Timer external reset source
00239     */
00240     TMR2_CCP1_OUT,
00241
00242     /* CCP2_OUT is the Timer external reset source
00243     */
00244     TMR2_CCP2_OUT,
00245
00246     /* CCP3_OUT is the Timer external reset source
00247     */
00248     TMR2_CCP3_OUT,
00249
00250     /* CCP4_OUT is the Timer external reset source
00251     */
00252     TMR2_CCP4_OUT,
00253
00254     /* PWM5_out is the Timer external reset source
00255     */

```

```
00256     TMR2_PWM5_OUT,
00257
00258     /* PWM6_out is the Timer external reset source
00259     */
00260     TMR2_PWM6_OUT,
00261
00262     /* PWM7_out is the Timer external reset source
00263     */
00264     TMR2_PWM7_OUT,
00265
00266     /* PWM8_out is the Timer external reset source
00267     */
00268     TMR2_PWM8_OUT,
00269
00270     /* Reserved enum cannot be used
00271     */
00272     TMR2_RESERVED_2,
00273
00274     /* Reserved enum cannot be used
00275     */
00276     TMR2_RESERVED_3,
00277
00278     /* C1_OUT_SYNC is the Timer external reset source
00279     */
00280     TMR2_C1_OUT_SYNC,
00281
00282     /* C2_OUT_SYNC is the Timer external reset source
00283     */
00284     TMR2_C2_OUT_SYNC,
00285
00286     /* ZCD_Output is the Timer external reset source
00287     */
00288     TMR2_ZCD_OUTPUT,
00289
00290     /* CLC1_out is the Timer external reset source
00291     */
00292     TMR2_CLC1_OUT,
00293
00294     /* CLC2_out is the Timer external reset source
00295     */
00296     TMR2_CLC2_OUT,
00297
00298     /* CLC3_out is the Timer external reset source
00299     */
00300     TMR2_CLC3_OUT,
00301
00302     /* CLC4_out is the Timer external reset source
00303     */
00304     TMR2_CLC4_OUT,
00305
00306     /* UART1_rx_edge is the Timer external reset source
00307     */
00308     TMR2_UART1_RX_EDGE,
00309
00310     /* UART1_tx_edge is the Timer external reset source
00311     */
00312     TMR2_UART1_TX_EDGE,
00313
00314     /* UART2_rx_edge is the Timer external reset source
00315     */
00316     TMR2_UART2_RX_EDGE,
00317
00318     /* UART2_tx_edge is the Timer external reset source
00319     */
00320     TMR2_UART2_TX_EDGE
00321
00322 } TMR2_HLT_EXT_RESET_SOURCE;
00323
00324
00325
00329
00333
00365 void TMR2_Initialize(void);
00366
00401 void TMR2_ModeSet (TMR2_HLT_MODE mode);
00402
00436 void TMR2_ExtResetSourceSet (TMR2_HLT_EXT_RESET_SOURCE reset);
00437
00465 void TMR2_Start(void);
00466
00494 void TMR2_StartTimer(void);
00495
00526 void TMR2_Stop(void);
00527
00558 void TMR2_StopTimer(void);
00559
00593 uint8_t TMR2_Counter8BitGet(void);
```

```

00594
00628 uint8_t TMR2_ReadTimer(void);
00629
00667 void TMR2_Counter8BitSet(uint8_t timerVal);
00668
00706 void TMR2_WriteTimer(uint8_t timerVal);
00707
00758 void TMR2_Period8BitSet(uint8_t periodVal);
00759
00810 void TMR2_LoadPeriodRegister(uint8_t periodVal);
00811
00848 bool TMR2_HasOverflowOccured(void);
00849
00850 #ifdef __cplusplus // Provide C++ Compatibility
00851
00852     }
00853
00854 #endif
00855
00856 #endif // TMR2_H

```

4.44 tmr3.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047
00050
00051 #include <xc.h>
00052 #include "tmr3.h"
00053 #include "interrupt_manager.h"
00054
00055 volatile uint16_t timer3ReloadVal;
00056 void (*TMR3_InterruptHandler)(void);
00057
00060
00064
00065 void TMR3_Initialize(void)
00066 {
00067     //Set the Timer to the options selected in the GUI
00068
00069     //T3GE disabled; T3GTM disabled; T3GPOL low; T3GGO done; T3GSPM disabled;
00070     T3GCON = 0x00;
00071
00072     //GSS T3G_pin;
00073     T3GATE = 0x00;
00074
00075     //CS FOSC/4;
00076     T3CLK = 0x01;
00077
00078     //TMR3H 99;
00079     TMR3H = 0x63;
00080
00081     //TMR3L 192;
00082     TMR3L = 0xC0;
00083
00084     // Clearing IF flag before enabling the interrupt.
00085     PIR6bits.TMR3IF = 0;
00086
00087     // Load the TMR value to reload variable
00088     timer3ReloadVal=(uint16_t)((TMR3H << 8) | TMR3L);

```

```

00089
00090 // Enabling TMR3 interrupt.
00091 PIE6bits.TMR3IE = 1;
00092
00093 // Set Default Interrupt Handler
00094 TMR3_SetInterruptHandler(TMR3_DefaultInterruptHandler);
00095
00096 // CKPS 1:2; NOT_SYNC synchronize; TMR3ON enabled; T3RD16 disabled;
00097 T3CON = 0x11;
00098 }
00099
00100 void TMR3_StartTimer(void)
00101 {
00102     // Start the Timer by writing to TMRxON bit
00103     T3CONbits.TMR3ON = 1;
00104 }
00105
00106 void TMR3_StopTimer(void)
00107 {
00108     // Stop the Timer by writing to TMRxON bit
00109     T3CONbits.TMR3ON = 0;
00110 }
00111
00112 uint16_t TMR3_ReadTimer(void)
00113 {
00114     uint16_t readVal;
00115     uint8_t readValHigh;
00116     uint8_t readValLow;
00117
00118     T3CONbits.T3RD16 = 1;
00119
00120     readValLow = TMR3L;
00121     readValHigh = TMR3H;
00122
00123     readVal = ((uint16_t)readValHigh « 8) | readValLow;
00124
00125     return readVal;
00126 }
00127
00128 void TMR3_WriteTimer(uint16_t timerVal)
00129 {
00130     if (T3CONbits.NOT_SYNC == 1)
00131     {
00132         // Stop the Timer by writing to TMRxON bit
00133         T3CONbits.TMR3ON = 0;
00134
00135         // Write to the Timer3 register
00136         TMR3H = (uint8_t)(timerVal » 8);
00137         TMR3L = (uint8_t)timerVal;
00138
00139         // Start the Timer after writing to the register
00140         T3CONbits.TMR3ON = 1;
00141     }
00142     else
00143     {
00144         // Write to the Timer3 register
00145         TMR3H = (uint8_t)(timerVal » 8);
00146         TMR3L = (uint8_t)timerVal;
00147     }
00148 }
00149
00150 void TMR3_Reload(void)
00151 {
00152     TMR3_WriteTimer(timer3ReloadVal);
00153 }
00154
00155 void TMR3_StartSinglePulseAcquisition(void)
00156 {
00157     T3GCONbits.T3GGO = 1;
00158 }
00159
00160 uint8_t TMR3_CheckGateValueStatus(void)
00161 {
00162     return (T3GCONbits.T3GVAL);
00163 }
00164
00165 void __interrupt (irq(TMR3),base(8),low_priority) TMR3_ISR()
00166 {
00167     static volatile unsigned int CountCallBack = 0;
00168
00169     // Clear the TMR3 interrupt flag
00170     PIE6bits.TMR3IF = 0;
00171     TMR3_WriteTimer(timer3ReloadVal);
00172
00173     // callback function - called every 200th pass
00174     if (++CountCallBack >= TMR3_INTERRUPT_TICKER_FACTOR)
00175     {

```

```

00176         // ticker function call
00177         TMR3_CallBack();
00178
00179         // reset ticker counter
00180         CountCallBack = 0;
00181     }
00182 }
00183
00184 void TMR3_CallBack(void)
00185 {
00186     // Add your custom callback code here
00187     if(TMR3_InterruptHandler)
00188     {
00189         TMR3_InterruptHandler();
00190     }
00191 }
00192
00193 void TMR3_SetInterruptHandler(void (* InterruptHandler)(void)){
00194     TMR3_InterruptHandler = InterruptHandler;
00195 }
00196
00197 void TMR3_DefaultInterruptHandler(void){
00198     // add your TMR3 interrupt custom code
00199     // or set custom function using TMR3_SetInterruptHandler()
00200 }
00201

```

4.45 tmr3.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef TMR3_H
00048 #define TMR3_H
00049
00053
00054 #include <stdbool.h>
00055 #include <stdint.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00060
00061 #endif
00062
00063 #define TMR3_INTERRUPT_TICKER_FACTOR    200
00064
00068
00101 void TMR3_Initialize(void);
00102
00130 void TMR3_StartTimer(void);
00131
00162 void TMR3_StopTimer(void);
00163
00197 uint16_t TMR3_ReadTimer(void);
00198
00236 void TMR3_WriteTimer(uint16_t timerVal);
00237
00272 void TMR3_Reload(void);

```

```

00273
00311 void TMR3_StartSinglePulseAcquisition(void);
00312
00350 uint8_t TMR3_CheckGateValueStatus(void);
00351
00368 void TMR3_CallBack(void);
00369
00386 void TMR3_SetInterruptHandler(void (* InterruptHandler)(void));
00387
00404 extern void (*TMR3_InterruptHandler)(void);
00405
00422 void TMR3_DefaultInterruptHandler(void);
00423
00424 #ifdef __cplusplus // Provide C++ Compatibility
00425
00426     }
00427
00428 #endif
00429
00430 #endif // TMR3_H

```

4.46 tmr5.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00029     license terms applicable to your use of third party software (including open source software) that
00030     may accompany Microchip software.
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050
00051 #include <xc.h>
00052 #include "tmr5.h"
00053 #include "interrupt_manager.h"
00054
00058 volatile uint16_t timer5ReloadVal;
00059 void (*TMR5_InterruptHandler)(void);
00060
00064
00065 void TMR5_Initialize(void)
00066 {
00067     //Set the Timer to the options selected in the GUI
00068
00069     //T5GE disabled; T5GTM disabled; T5GPOL low; T5GGO done; T5GSPM disabled;
00070     T5GCON = 0x00;
00071
00072     //GSS T5G_pin;
00073     T5GATE = 0x00;
00074
00075     //CS FOSC/4;
00076     T5CLK = 0x01;
00077
00078     //TMR5H 99;
00079     TMR5H = 0x63;
00080
00081     //TMR5L 192;
00082     TMR5L = 0xC0;
00083
00084     // Clearing IF flag before enabling the interrupt.
00085     PIR8bits.TMR5IF = 0;
00086
00087     // Load the TMR value to reload variable
00088     timer5ReloadVal = (uint16_t) ((TMR5H << 8) | TMR5L);

```



```

00089
00090 // Enabling TMR5 interrupt.
00091 PIE8bits.TMR5IE = 1;
00092
00093 // Set Default Interrupt Handler
00094 TMR5_SetInterruptHandler(TMR5_DefaultInterruptHandler);
00095
00096 // CKPS 1:2; NOT_SYNC synchronize; TMR5ON enabled; T5RD16 disabled;
00097 T5CON = 0x11;
00098 }
00099
00100 void TMR5_StartTimer(void)
00101 {
00102     // Start the Timer by writing to TMRxON bit
00103     T5CONbits.TMR5ON = 1;
00104 }
00105
00106 void TMR5_StopTimer(void)
00107 {
00108     // Stop the Timer by writing to TMRxON bit
00109     T5CONbits.TMR5ON = 0;
00110 }
00111
00112 uint16_t TMR5_ReadTimer(void)
00113 {
00114     uint16_t readVal;
00115     uint8_t readValHigh;
00116     uint8_t readValLow;
00117
00118     T5CONbits.T5RD16 = 1;
00119
00120     readValLow = TMR5L;
00121     readValHigh = TMR5H;
00122
00123     readVal = ((uint16_t) readValHigh « 8) | readValLow;
00124
00125     return readVal;
00126 }
00127
00128 void TMR5_WriteTimer(uint16_t timerVal)
00129 {
00130     if (T5CONbits.NOT_SYNC == 1) {
00131         // Stop the Timer by writing to TMRxON bit
00132         T5CONbits.TMR5ON = 0;
00133
00134         // Write to the Timer5 register
00135         TMR5H = (uint8_t) (timerVal » 8);
00136         TMR5L = (uint8_t) timerVal;
00137
00138         // Start the Timer after writing to the register
00139         T5CONbits.TMR5ON = 1;
00140     } else {
00141         // Write to the Timer5 register
00142         TMR5H = (uint8_t) (timerVal » 8);
00143         TMR5L = (uint8_t) timerVal;
00144     }
00145 }
00146
00147 void TMR5_Reload(void)
00148 {
00149     TMR5_WriteTimer(timer5ReloadVal);
00150 }
00151
00152 void TMR5_StartSinglePulseAcquisition(void)
00153 {
00154     T5GCONbits.T5GGO = 1;
00155 }
00156
00157 uint8_t TMR5_CheckGateValueStatus(void)
00158 {
00159     return (T5GCONbits.T5GVAL);
00160 }
00161
00162 void __interrupt(irq(TMR5), base(8), low_priority) TMR5_ISR()
00163 {
00164     static volatile unsigned int CountCallBack = 0;
00165
00166     // Clear the TMR5 interrupt flag
00167     PIR8bits.TMR5IF = 0;
00168     TMR5_WriteTimer(timer5ReloadVal);
00169
00170     // callback function - called every 200th pass
00171     if (++CountCallBack >= TMR5_INTERRUPT_TICKER_FACTOR) {
00172         // ticker function call
00173         TMR5_CallBack();
00174
00175         // reset ticker counter

```

```

00176         CountCallBack = 0;
00177     }
00178 }
00179
00180 void TMR5_CallBack(void)
00181 {
00182     // Add your custom callback code here
00183     if (TMR5_InterruptHandler) {
00184         TMR5_InterruptHandler();
00185     }
00186 }
00187
00188 void TMR5_SetInterruptHandler(void (* InterruptHandler)(void))
00189 {
00190     TMR5_InterruptHandler = InterruptHandler;
00191 }
00192
00193 void TMR5_DefaultInterruptHandler(void)
00194 {
00195     // add your TMR5 interrupt custom code
00196     // or set custom function using TMR5_SetInterruptHandler()
00197 }
00198

```

4.47 tmr5.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047 #ifndef TMR5_H
00048 #define TMR5_H
00049
00053
00054 #include <stdbool.h>
00055 #include <stdint.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059
00061 #endif
00062
00063 #define TMR5_INTERRUPT_TICKER_FACTOR    200
00064
00068
00101 void TMR5_Initialize(void);
00102
00130 void TMR5_StartTimer(void);
00131
00162 void TMR5_StopTimer(void);
00163
00197 uint16_t TMR5_ReadTimer(void);
00198
00236 void TMR5_WriteTimer(uint16_t timerVal);
00237
00272 void TMR5_Reload(void);
00273
00311 void TMR5_StartSinglePulseAcquisition(void);
00312

```

```

00350 uint8_t TMR5_CheckGateValueStatus(void);
00351
00368 void TMR5_CallBack(void);
00369
00386 void TMR5_SetInterruptHandler(void (* InterruptHandler)(void));
00387
00404 extern void (*TMR5_InterruptHandler)(void);
00405
00422 void TMR5_DefaultInterruptHandler(void);
00423
00424 #ifdef __cplusplus // Provide C++ Compatibility
00425
00426     }
00427
00428 #endif
00429
00430 #endif // TMR5_H

```

4.48 tmr6.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047
00050
00051 #include <xc.h>
00052 #include "tmr6.h"
00053 #include "interrupt_manager.h"
00054
00058
00059 void (*TMR6_InterruptHandler)(void);
00060
00064
00065 void TMR6_Initialize(void)
00066 {
00067     // Set TMR6 to the options selected in the User Interface
00068
00069     // T6CS FOSC/4;
00070     T6CLKCON = 0x01;
00071
00072     // T6PSYNC Not Synchronized; T6MODE Software control; T6CKPOL Rising Edge; T6CKSYNC Not
00073     Synchronized;
00074     T6HLT = 0x00;
00075
00076     // T6RSEL T6CKIPPS pin;
00077     T6RST = 0x00;
00078
00079     // PR6 124;
00080     T6PR = 0x7C;
00081
00082     // TMR6 0;
00083     T6TMR = 0x00;
00084
00085     // Clearing IF flag before enabling the interrupt.
00086     PIR9bits.TMR6IF = 0;
00087
00088     // Enabling TMR6 interrupt.
00089     PIE9bits.TMR6IE = 1;
00090
00091     // Set Default Interrupt Handler

```

```
00091     TMR6_SetInterruptHandler(TMR6_DefaultInterruptHandler);
00092
00093     // T6CKPS 1:64; T6OUTPS 1:1; TMR6ON on;
00094     T6CON = 0xE0;
00095 }
00096
00097 void TMR6_ModeSet(TMR6_HLT_MODE mode)
00098 {
00099     // Configure different types HLT mode
00100     T6HLTbits.MODE = mode;
00101 }
00102
00103 void TMR6_ExtResetSourceSet(TMR6_HLT_EXT_RESET_SOURCE reset)
00104 {
00105     //Configure different types of HLT external reset source
00106     T6RSTbits.RSEL = reset;
00107 }
00108
00109 void TMR6_Start(void)
00110 {
00111     // Start the Timer by writing to TMRxON bit
00112     T6CONbits.TMR6ON = 1;
00113 }
00114
00115 void TMR6_StartTimer(void)
00116 {
00117     TMR6_Start();
00118 }
00119
00120 void TMR6_Stop(void)
00121 {
00122     // Stop the Timer by writing to TMRxON bit
00123     T6CONbits.TMR6ON = 0;
00124 }
00125
00126 void TMR6_StopTimer(void)
00127 {
00128     TMR6_Stop();
00129 }
00130
00131 uint8_t TMR6_Counter8BitGet(void)
00132 {
00133     uint8_t readVal;
00134     readVal = TMR6;
00135     return readVal;
00136 }
00137
00138
00139
00140 uint8_t TMR6_ReadTimer(void)
00141 {
00142     return TMR6_Counter8BitGet();
00143 }
00144
00145 void TMR6_Counter8BitSet(uint8_t timerVal)
00146 {
00147     // Write to the Timer6 register
00148     TMR6 = timerVal;
00149 }
00150
00151 void TMR6_WriteTimer(uint8_t timerVal)
00152 {
00153     TMR6_Counter8BitSet(timerVal);
00154 }
00155
00156 void TMR6_Period8BitSet(uint8_t periodVal)
00157 {
00158     PR6 = periodVal;
00159 }
00160
00161 void TMR6_LoadPeriodRegister(uint8_t periodVal)
00162 {
00163     TMR6_Period8BitSet(periodVal);
00164 }
00165
00166 void __interrupt(irq(TMR6),base(8)) TMR6_ISR()
00167 {
00168     // clear the TMR6 interrupt flag
00169     PIR9bits.TMR6IF = 0;
00170     // ticker function call;
00171     // ticker is 1 -> Callback function gets called everytime this ISR executes
00172     TMR6_CallBack();
00173 }
00174
00175 void TMR6_CallBack(void)
```

```

00178 {
00179     // Add your custom callback code here
00180     // this code executes every TMR6_INTERRUPT_TICKER_FACTOR periods of TMR6
00181     if(TMR6_InterruptHandler)
00182     {
00183         TMR6_InterruptHandler();
00184     }
00185 }
00186
00187 void TMR6_SetInterruptHandler(void (* InterruptHandler)(void)){
00188     TMR6_InterruptHandler = InterruptHandler;
00189 }
00190
00191 void TMR6_DefaultInterruptHandler(void){
00192     // add your TMR6 interrupt custom code
00193     // or set custom function using TMR6_SetInterruptHandler()
00194 }
00195

```

4.49 tmr6.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
00030     license terms applicable to your use of third party software (including open source software) that
00031     may accompany Microchip software.
00032
00033     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00034     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00035     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00036     FOR A PARTICULAR PURPOSE.
00037
00038     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00039     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00040     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00041     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00042     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00043     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00044     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00045     SOFTWARE.
00046 */
00047 #ifndef TMR6_H
00048 #define TMR6_H
00049
00053
00054 #include <stdint.h>
00055 #include <stdbool.h>
00056
00057 #ifdef __cplusplus // Provide C++ Compatibility
00058     extern "C" {
00059 #endif
00060
00061 #endif
00062
00066
00078
00079 typedef enum
00080 {
00081
00082     /* Roll-over Pulse mode clears the TMRx upon TMRx = PRx, then continue running.
00083     ON bit must be set and is not affected by Resets
00084     */
00085
00086     /* Roll-over Pulse mode indicates that Timer starts
00087     immediately upon ON = 1 (Software Control)
00088     */
00089     TMR6_ROP_STARTS_TMRON,
00090
00091     /* Roll-over Pulse mode indicates that the Timer starts
00092     when ON = 1 and TMRx_ers = 1. Stops when TMRx_ers = 0
00093     */
00094     TMR6_ROP_STARTS_TMRON_ERSHIGH,
00095
00096     /* Roll-over Pulse mode indicates that the Timer starts
00097     when ON = 1 and TMRx_ers = 0. Stops when TMRx_ers = 1
00098     */

```

```

00099   TMR6_ROP_STARTS_TMRON_ERLOW,
00100
00101   /* Roll-over Pulse mode indicates that the Timer resets
00102   upon rising or falling edge of TMRx_ers
00103   */
00104   TMR6_ROP_RESETS_ERBOTHEDGE,
00105
00106   /* Roll-over Pulse mode indicates that the Timer resets
00107   upon rising edge of TMRx_ers
00108   */
00109   TMR6_ROP_RESETS_ERSRISINGEDGE,
00110
00111   /* Roll-over Pulse mode indicates that the Timer resets
00112   upon falling edge of TMRx_ers
00113   */
00114   TMR6_ROP_RESETS_ERFALLINGEDGE,
00115
00116   /* Roll-over Pulse mode indicates that the Timer resets
00117   upon TMRx_ers = 0
00118   */
00119   TMR6_ROP_RESETS_ERLOW,
00120
00121   /* Roll-over Pulse mode indicates that the Timer resets
00122   upon TMRx_ers = 1
00123   */
00124   TMR6_ROP_RESETS_ERSHIGH,
00125
00126   /*In all One-Shot mode the timer resets and the ON bit is
00127   cleared when the timer value matches the PRx period
00128   value. The ON bit must be set by software to start
00129   another timer cycle.
00130   */
00131
00132   /* One shot mode indicates that the Timer starts
00133   immediately upon ON = 1 (Software Control)
00134   */
00135   TMR6_OS_STARTS_TMRON,
00136
00137   /* One shot mode indicates that the Timer starts
00138   when a rising edge is detected on the TMRx_ers
00139   */
00140   TMR6_OS_STARTS_ERSRISINGEDGE ,
00141
00142   /* One shot mode indicates that the Timer starts
00143   when a falling edge is detected on the TMRx_ers
00144   */
00145   TMR6_OS_STARTS_ERFALLINGEDGE ,
00146
00147   /* One shot mode indicates that the Timer starts
00148   when either a rising or falling edge is detected on TMRx_ers
00149   */
00150   TMR6_OS_STARTS_ERBOTHEDGE,
00151
00152   /* One shot mode indicates that the Timer starts
00153   upon first TMRx_ers rising edge and resets on all
00154   subsequent TMRx_ers rising edges
00155   */
00156   TMR6_OS_STARTS_ERFIRSTRISINGEDGE,
00157
00158   /* One shot mode indicates that the Timer starts
00159   upon first TMRx_ers falling edge and restarts on all
00160   subsequent TMRx_ers falling edges
00161   */
00162   TMR6_OS_STARTS_ERFIRSTFALLINGEDGE,
00163
00164   /* One shot mode indicates that the Timer starts
00165   when a rising edge is detected on the TMRx_ers,
00166   resets upon TMRx_ers = 0
00167   */
00168   TMR6_OS_STARTS_ERSRISINGEDGEDETECT,
00169   /* One shot mode indicates that the Timer starts
00170   when a falling edge is detected on the TMRx_ers,
00171   resets upon TMRx_ers = 1
00172   */
00173   TMR6_OS_STARTS_ERFALLINGEDGEDETECT,
00174
00175   /* One shot mode indicates that the Timer starts
00176   when a TMRx_ers = 1,ON =1 and resets upon TMRx_ers =0
00177   */
00178   TMR6_OS_STARTS_TMRON_ERSHIGH = 0x16,
00179
00180   /* One shot mode indicates that the Timer starts
00181   when a TMRx_ers = 0,ON = 1 and resets upon TMRx_ers =1
00182   */
00183   TMR6_OS_STARTS_TMRON_ERLOW = 0x17,
00184
00185   /*In all Mono-Stable mode the ON bit must be initially set,but

```

```

00186     not cleared upon the TMRx = PRx, and the timer will start upon
00187     an TMRx_ers start event following TMRx = PRx.*/
00188
00189     /* Mono Stable mode indicates that the Timer starts
00190     when a rising edge is detected on the TMRx_ers and ON = 1
00191     */
00192     TMR6_MS_STARTS_TMRON_ERSRISINGEDGEDETECT = 0x11,
00193
00194     /* Mono Stable mode indicates that the Timer starts
00195     when a falling edge is detected on the TMRx_ers and ON = 1
00196     */
00197     TMR6_MS_STARTS_TMRON_ERSFALLINGEDGEDETECT = 0x12,
00198
00199     /* Mono Stable mode indicates that the Timer starts
00200     when either a rising or falling edge is detected on TMRx_ers
00201     and ON = 1
00202     */
00203     TMR6_MS_STARTS_TMRON_ERSBOTHEDGE = 0x13
00204
00205 } TMR6_HLT_MODE;
00206
00219
00220 typedef enum
00221 {
00222     /* T6INPPS is the Timer external reset source
00223     */
00224     TMR6_T6INPPS,
00225
00226     /* Timer2 Postscale is the Timer external reset source
00227     */
00228     TMR6_T2POSTSCALED,
00229
00230     /* Timer4 Postscale is the Timer external reset source
00231     */
00232     TMR6_T4POSTSCALED,
00233
00234     /* Reserved enum cannot be used
00235     */
00236     TMR6_RESERVED,
00237
00238     /* CCP1_OUT is the Timer external reset source
00239     */
00240     TMR6_CCP1_OUT,
00241
00242     /* CCP2_OUT is the Timer external reset source
00243     */
00244     TMR6_CCP2_OUT,
00245
00246     /* CCP3_OUT is the Timer external reset source
00247     */
00248     TMR6_CCP3_OUT,
00249
00250     /* CCP4_OUT is the Timer external reset source
00251     */
00252     TMR6_CCP4_OUT,
00253
00254     /* PWM5_out is the Timer external reset source
00255     */
00256     TMR6_PWM5_OUT,
00257
00258     /* PWM6_out is the Timer external reset source
00259     */
00260     TMR6_PWM6_OUT,
00261
00262     /* PWM7_out is the Timer external reset source
00263     */
00264     TMR6_PWM7_OUT,
00265
00266     /* PWM8_out is the Timer external reset source
00267     */
00268     TMR6_PWM8_OUT,
00269
00270     /* Reserved enum cannot be used
00271     */
00272     TMR6_RESERVED_2,
00273
00274     /* Reserved enum cannot be used
00275     */
00276     TMR6_RESERVED_3,
00277
00278     /* C1_OUT_SYNC is the Timer external reset source
00279     */
00280     TMR6_C1_OUT_SYNC,
00281
00282     /* C2_OUT_SYNC is the Timer external reset source
00283     */
00284     TMR6_C2_OUT_SYNC,

```

```
00285
00286     /* ZCD_Output is the Timer external reset source
00287     */
00288     TMR6_ZCD_OUTPUT,
00289
00290     /* CLC1_out is the Timer external reset source
00291     */
00292     TMR6_CLC1_OUT,
00293
00294     /* CLC2_out is the Timer external reset source
00295     */
00296     TMR6_CLC2_OUT,
00297
00298     /* CLC3_out is the Timer external reset source
00299     */
00300     TMR6_CLC3_OUT,
00301
00302     /* CLC4_out is the Timer external reset source
00303     */
00304     TMR6_CLC4_OUT,
00305
00306     /* UART1_rx_edge is the Timer external reset source
00307     */
00308     TMR6_UART1_RX_EDGE,
00309
00310     /* UART1_tx_edge is the Timer external reset source
00311     */
00312     TMR6_UART1_TX_EDGE,
00313
00314     /* UART2_rx_edge is the Timer external reset source
00315     */
00316     TMR6_UART2_RX_EDGE,
00317
00318     /* UART2_tx_edge is the Timer external reset source
00319     */
00320     TMR6_UART2_TX_EDGE
00321
00322 } TMR6_HLT_EXT_RESET_SOURCE;
00323
00324
00325
00329
00333
00365 void TMR6_Initialize(void);
00366
00401 void TMR6_ModeSet(TMR6_HLT_MODE mode);
00402
00436 void TMR6_ExtResetSourceSet(TMR6_HLT_EXT_RESET_SOURCE reset);
00437
00465 void TMR6_Start(void);
00466
00494 void TMR6_StartTimer(void);
00495
00526 void TMR6_Stop(void);
00527
00558 void TMR6_StopTimer(void);
00559
00593 uint8_t TMR6_Counter8BitGet(void);
00594
00628 uint8_t TMR6_ReadTimer(void);
00629
00667 void TMR6_Counter8BitSet(uint8_t timerVal);
00668
00706 void TMR6_WriteTimer(uint8_t timerVal);
00707
00758 void TMR6_Period8BitSet(uint8_t periodVal);
00759
00810 void TMR6_LoadPeriodRegister(uint8_t periodVal);
00811
00812
00829 void TMR6_CallBack(void);
00846 void TMR6_SetInterruptHandler(void (* InterruptHandler)(void));
00847
00864 extern void (*TMR6_InterruptHandler)(void);
00865
00882 void TMR6_DefaultInterruptHandler(void);
00883
00884
00885 #ifdef __cplusplus // Provide C++ Compatibility
00886
00887     }
00888
00889 #endif
00890
00891 #endif // TMR6_H
```


4.50 uart1.c

```

00001
00002
00003 /*
00004  (c) 2018 Microchip Technology Inc. and its subsidiaries.
00005
00006 Subject to your compliance with these terms, you may use Microchip software and any
00007 derivatives exclusively with Microchip products. It is your responsibility to comply with third
00008 party
00009 license terms applicable to your use of third party software (including open source software) that
00010
00011 may accompany Microchip software.
00012
00013 THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00014 EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00015 IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00016 FOR A PARTICULAR PURPOSE.
00017
00018 IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00019 INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00020 WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00021 HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00022 THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00023 CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00024 OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00025 SOFTWARE.
00026 */
00027
00028 #include <xc.h>
00029 #include "uart1.h"
00030 #include "interrupt_manager.h"
00031
00032 #define UART1_TX_BUFFER_SIZE 255
00033 #define UART1_RX_BUFFER_SIZE 255
00034
00035 static volatile uint8_t uart1TxHead = 0;
00036 static volatile uint8_t uart1TxTail = 0;
00037 static volatile uint8_t uart1TxBuffer[UART1_TX_BUFFER_SIZE];
00038 volatile uint8_t uart1TxBufferRemaining;
00039
00040 static volatile uint8_t uart1RxHead = 0;
00041 static volatile uint8_t uart1RxTail = 0;
00042 static volatile uint8_t uart1RxBuffer[UART1_RX_BUFFER_SIZE];
00043 static volatile uart1_status_t uart1RxStatusBuffer[UART1_RX_BUFFER_SIZE];
00044 volatile uint8_t uart1RxCount;
00045 static volatile uart1_status_t uart1RxLastError;
00046
00047 void (*UART1_FramingErrorHandler)(void);
00048 void (*UART1_OverrunErrorHandler)(void);
00049 void (*UART1_ErrorHandler)(void);
00050
00051 void UART1_DefaultFramingErrorHandler(void);
00052 void UART1_DefaultOverrunErrorHandler(void);
00053 void UART1_DefaultErrorHandler(void);
00054
00055 void UART1_Initialize(void)
00056 {
00057     // Disable interrupts before changing states
00058     PIE3bits.U1RXIE = 0;
00059     UART1_SetRxInterruptHandler(UART1_Receive_ISR);
00060     PIE3bits.U1TXIE = 0;
00061     UART1_SetTxInterruptHandler(UART1_Transmit_ISR);
00062
00063     // Set the UART1 module to the options selected in the user interface.
00064
00065     // P1L 0;
00066     U1P1L = 0x00;
00067
00068     // P1H 0;
00069     U1P1H = 0x00;
00070
00071     // P2L 0;
00072     U1P2L = 0x00;
00073
00074     // P2H 0;
00075     U1P2H = 0x00;
00076
00077     // P3L 0;
00078     U1P3L = 0x00;
00079
00080     // P3H 0;
00081     U1P3H = 0x00;
00082
00083     // BRGS high speed; MODE Asynchronous 8-bit mode; RXEN enabled; TXEN enabled; ABDEN disabled;

```

```

00116     U1CON0 = 0xB0;
00117
00118     // RXBIMD Set RXBKIF on rising RX input; BRKOVr disabled; WUE disabled; SENDB disabled; ON
enabled;
00119     U1CON1 = 0x80;
00120
00121     // TXPOL not inverted; FLO off; COEN Checksum Mode 0; RXPOL not inverted; RUNOVF RX input shifter
stops all activity; STP Transmit 1Stop bit, receiver verifies first Stop bit;
00122     U1CON2 = 0x00;
00123
00124     // BRGL 138;
00125     U1BRGL = 0x8A;
00126
00127     // BRGH 0;
00128     U1BRGH = 0x00;
00129
00130     // STPMD in middle of first Stop bit; TXWRE No error;
00131     U1FIFO = 0x00;
00132
00133     // ABDIF Auto-baud not enabled or not complete; WUIF WUE not enabled by software; ABDIE disabled;
00134     U1UIR = 0x00;
00135
00136     // ABDOVF Not overflowed; TXCIF 0; RXBKIF No Break detected; RXFOIF not overflowed; CERIF No
Checksum error;
00137     U1ERRIR = 0x00;
00138
00139     // TXCIE disabled; FERIC disabled; TXMTIE disabled; ABDOVE disabled; CERIE disabled; RXFOIE
disabled; PERIE disabled; RXBKIE disabled;
00140     U1ERRIE = 0x00;
00141
00142
00143     UART1_SetFramingErrorHandler(UART1_DefaultFramingErrorHandler);
00144     UART1_SetOverrunErrorHandler(UART1_DefaultOverrunErrorHandler);
00145     UART1_SetErrorHandler(UART1_DefaultErrorHandler);
00146
00147     uart1RxLastError.status = 0;
00148
00149     // initializing the driver state
00150     uart1TxHead = 0;
00151     uart1TxTail = 0;
00152     uart1TxBufferRemaining = sizeof(uart1TxBuffer);
00153     uart1RxHead = 0;
00154     uart1RxTail = 0;
00155     uart1RxCount = 0;
00156
00157     // enable receive interrupt
00158     PIE3bits.U1RXIE = 1;
00159 }
00160
00161 bool UART1_is_rx_ready(void)
00162 {
00163     return (uart1RxCount ? true : false);
00164 }
00165
00166 bool UART1_is_tx_ready(void)
00167 {
00168     return (uart1TxBufferRemaining ? true : false);
00169 }
00170
00171 bool UART1_is_tx_done(void)
00172 {
00173     return U1ERRIRbits.TXMTIF;
00174 }
00175
00176 uart1_status_t UART1_get_last_status(void) {
00177     return uart1RxLastError;
00178 }
00179
00180 uint8_t UART1_Read(void)
00181 {
00182     uint8_t readValue = 0;
00183
00184     while(0 == uart1RxCount)
00185     {
00186     }
00187
00188     uart1RxLastError = uart1RxStatusBuffer[uart1RxTail];
00189
00190     readValue = uart1RxBuffer[uart1RxTail++];
00191     if(sizeof(uart1RxBuffer) <= uart1RxTail)
00192     {
00193         uart1RxTail = 0;
00194     }
00195     PIE3bits.U1RXIE = 0;
00196     uart1RxCount--;
00197     PIE3bits.U1RXIE = 1;
00198

```

```

00199     return readValue;
00200 }
00201
00202 void UART1_Write(uint8_t txData)
00203 {
00204     while(0 == uart1TxBufferRemaining)
00205     {
00206     }
00207
00208     if(0 == PIE3bits.U1TXIE)
00209     {
00210         U1TXB = txData;
00211     }
00212     else
00213     {
00214         PIE3bits.U1TXIE = 0;
00215         uart1TxBuffer[uart1TxHead++] = txData;
00216         if(sizeof(uart1TxBuffer) <= uart1TxHead)
00217         {
00218             uart1TxHead = 0;
00219         }
00220         uart1TxBufferRemaining--;
00221     }
00222     PIE3bits.U1TXIE = 1;
00223 }
00224
00225 void __interrupt (irq(U1TX),base(8)) UART1_tx_vect_isr()
00226 {
00227     if(UART1_TxInterruptHandler)
00228     {
00229         UART1_TxInterruptHandler();
00230     }
00231 }
00232
00233 void __interrupt (irq(U1RX),base(8)) UART1_rx_vect_isr()
00234 {
00235     if(UART1_RxInterruptHandler)
00236     {
00237         UART1_RxInterruptHandler();
00238     }
00239 }
00240
00241
00242
00243 void UART1_Transmit_ISR(void)
00244 {
00245     // use this default transmit interrupt handler code
00246     if(sizeof(uart1TxBuffer) > uart1TxBufferRemaining)
00247     {
00248         U1TXB = uart1TxBuffer[uart1TxTail++];
00249         if(sizeof(uart1TxBuffer) <= uart1TxTail)
00250         {
00251             uart1TxTail = 0;
00252         }
00253         uart1TxBufferRemaining++;
00254     }
00255     else
00256     {
00257         PIE3bits.U1TXIE = 0;
00258     }
00259
00260     // or set custom function using UART1_SetTxInterruptHandler()
00261 }
00262
00263 void UART1_Receive_ISR(void)
00264 {
00265     // use this default receive interrupt handler code
00266     uart1RxStatusBuffer[uart1RxHead].status = 0;
00267
00268     if(U1ERRIRbits.FERIF){
00269         uart1RxStatusBuffer[uart1RxHead].ferr = 1;
00270         UART1_FramingErrorHandler();
00271     }
00272
00273     if(U1ERRIRbits.RXFOIF){
00274         uart1RxStatusBuffer[uart1RxHead].oerr = 1;
00275         UART1_OverrunErrorHandler();
00276     }
00277
00278     if(uart1RxStatusBuffer[uart1RxHead].status){
00279         UART1_ErrorHandler();
00280     } else {
00281         UART1_RxDataHandler();
00282     }
00283
00284     // or set custom function using UART1_SetRxInterruptHandler()
00285 }

```

```

00286
00287 void UART1_RxDataHandler(void){
00288     // use this default receive interrupt handler code
00289     uart1RxBuffer[uart1RxHead++] = U1RXB;
00290     if(sizeof(uart1RxBuffer) <= uart1RxHead)
00291     {
00292         uart1RxHead = 0;
00293     }
00294     uart1RxCount++;
00295 }
00296
00297 void UART1_DefaultFramingErrorHandler(void){}
00298
00299 void UART1_DefaultOverrunErrorHandler(void){}
00300
00301 void UART1_DefaultErrorHandler(void){
00302     UART1_RxDataHandler();
00303 }
00304
00305 void UART1_SetFramingErrorHandler(void (* interruptHandler)(void)){
00306     UART1_FramingErrorHandler = interruptHandler;
00307 }
00308
00309 void UART1_SetOverrunErrorHandler(void (* interruptHandler)(void)){
00310     UART1_OverrunErrorHandler = interruptHandler;
00311 }
00312
00313 void UART1_SetErrorHandler(void (* interruptHandler)(void)){
00314     UART1_ErrorHandler = interruptHandler;
00315 }
00316
00317
00318
00319 void UART1_SetRxInterruptHandler(void (* InterruptHandler)(void)){
00320     UART1_RxInterruptHandler = InterruptHandler;
00321 }
00322
00323 void UART1_SetTxInterruptHandler(void (* InterruptHandler)(void)){
00324     UART1_TxInterruptHandler = InterruptHandler;
00325 }
00326
00327

```

4.51 uart1.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef UART1_H
00048 #define UART1_H
00049
00053
00054 #include <xc.h>
00055 #include <stdbool.h>
00056 #include <stdint.h>
00057
00058 #ifdef __cplusplus // Provide C++ Compatibility
00059     extern "C" {
00060

```

```

00061
00062 #endif
00063
00067
00068 #define UART1_DataReady (UART1_is_rx_ready())
00069
00073
00074 typedef union {
00075     struct {
00076         unsigned perr : 1;
00077         unsigned ferr : 1;
00078         unsigned oerr : 1;
00079         unsigned reserved : 5;
00080     };
00081     uint8_t status;
00082 }uart1_status_t;
00083
00087 extern volatile uint8_t uart1TxBufferRemaining;
00088 extern volatile uint8_t uart1RxCount;
00089
00093
00115 void UART1_Initialize(void);
00116
00163 bool UART1_is_rx_ready(void);
00164
00211 bool UART1_is_tx_ready(void);
00212
00258 bool UART1_is_tx_done(void);
00259
00306 uart1_status_t UART1_get_last_status(void);
00307
00355 uint8_t UART1_Read(void);
00356
00380 void UART1_Write(uint8_t txData);
00381
00401 void UART1_Transmit_ISR(void);
00402
00422 void UART1_Receive_ISR(void);
00423
00443 void UART1_RxDataHandler(void);
00444
00461 void UART1_SetFramingErrorHandler(void (* interruptHandler)(void));
00462
00479 void UART1_SetOverrunErrorHandler(void (* interruptHandler)(void));
00480
00497 void UART1_SetErrorHandler(void (* interruptHandler)(void));
00498
00499
00500
00517 void (*UART1_RxInterruptHandler)(void);
00518
00535 void (*UART1_TxInterruptHandler)(void);
00536
00537
00538
00555 void UART1_SetRxInterruptHandler(void (* InterruptHandler)(void));
00556
00573 void UART1_SetTxInterruptHandler(void (* InterruptHandler)(void));
00574
00575
00576
00577 #ifdef __cplusplus // Provide C++ Compatibility
00578     }
00579 }
00580
00581 #endif
00582
00583 #endif // UART1_H

```

4.52 uart2.c

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031

```

```

00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00050 #include <xc.h>
00051 #include "uart2.h"
00052 #include "interrupt_manager.h"
00053
00057 #define UART2_TX_BUFFER_SIZE 255
00058 #define UART2_RX_BUFFER_SIZE 255
00059
00063
00064 static volatile uint8_t uart2TxHead = 0;
00065 static volatile uint8_t uart2TxTail = 0;
00066 static volatile uint8_t uart2TxBuffer[UART2_TX_BUFFER_SIZE];
00067 volatile uint8_t uart2TxBufferRemaining;
00068
00069 static volatile uint8_t uart2RxHead = 0;
00070 static volatile uint8_t uart2RxTail = 0;
00071 static volatile uint8_t uart2RxBuffer[UART2_RX_BUFFER_SIZE];
00072 static volatile uart2_status_t uart2RxStatusBuffer[UART2_RX_BUFFER_SIZE];
00073 volatile uint8_t uart2RxCount;
00074 static volatile uart2_status_t uart2RxLastError;
00075
00079 void (*UART2_FramingErrorHandler)(void);
00080 void (*UART2_OverrunErrorHandler)(void);
00081 void (*UART2_ErrorHandler)(void);
00082
00083 void UART2_DefaultFramingErrorHandler(void);
00084 void UART2_DefaultOverrunErrorHandler(void);
00085 void UART2_DefaultErrorHandler(void);
00086
00087 void UART2_Initialize(void)
00088 {
00089     // Disable interrupts before changing states
00090     PIE6bits.U2RXIE = 0;
00091     UART2_SetRxInterruptHandler(UART2_Receive_ISR);
00092     PIE6bits.U2TXIE = 0;
00093     UART2_SetTxInterruptHandler(UART2_Transmit_ISR);
00094
00095     // Set the UART2 module to the options selected in the user interface.
00096
00097     // P1L 0;
00098     U2P1L = 0x00;
00099
00100     // P2L 0;
00101     U2P2L = 0x00;
00102
00103     // P3L 0;
00104     U2P3L = 0x00;
00105
00106     // BRGS high speed; MODE Asynchronous 8-bit mode; RXEN enabled; TXEN enabled; ABDEN disabled;
00107     U2CON0 = 0xB0;
00108
00109     // RXBIMD Set RXBKIF on rising RX input; BRKOVr disabled; WUE disabled; SENDB disabled; ON
    enabled;
00110     U2CON1 = 0x80;
00111
00112     // TXPOL not inverted; FLO off; RXPOL not inverted; RUNOVF RX input shifter stops all activity;
    STP Transmit 2 Stop bits, receiver verifies first Stop bit;
00113     U2CON2 = 0x30;
00114
00115     // BRGL 4;
00116     U2BRGL = 0x04;
00117
00118     // BRGH 13;
00119     U2BRGH = 0x0D;
00120
00121     // STPMD in middle of first Stop bit; TXWRE No error;
00122     U2FIFO = 0x00;
00123
00124     // ABDIF Auto-baud not enabled or not complete; WUIF WUE not enabled by software; ABDIE disabled;
00125     U2UIR = 0x00;
00126
00127     // ABDOVF Not overflowed; TXCIF 0; RXBKIF No Break detected; RXFOIF not overflowed; CERIF No
    Checksum error;

```

```

00128     U2ERRIR = 0x00;
00129
00130     // TXCIE disabled; FERRIE disabled; TXMTIE disabled; ABDOVE disabled; CERIE disabled; RXFOIE
disabled; PERIE disabled; RXBKIE disabled;
00131     U2ERRIE = 0x00;
00132
00133
00134     UART2_SetFramingErrorHandler(UART2_DefaultFramingErrorHandler);
00135     UART2_SetOverrunErrorHandler(UART2_DefaultOverrunErrorHandler);
00136     UART2_SetErrorHandler(UART2_DefaultErrorHandler);
00137
00138     uart2RxLastError.status = 0;
00139
00140     // initializing the driver state
00141     uart2TxHead = 0;
00142     uart2TxTail = 0;
00143     uart2TxBufferRemaining = sizeof(uart2TxBuffer);
00144     uart2RxHead = 0;
00145     uart2RxTail = 0;
00146     uart2RxCount = 0;
00147
00148     // enable receive interrupt
00149     PIE6bits.U2RXIE = 1;
00150 }
00151
00152 bool UART2_is_rx_ready(void)
00153 {
00154     return (uart2RxCount ? true : false);
00155 }
00156
00157 bool UART2_is_tx_ready(void)
00158 {
00159     return (uart2TxBufferRemaining ? true : false);
00160 }
00161
00162 bool UART2_is_tx_done(void)
00163 {
00164     return U2ERRIRbits.TXMTIF;
00165 }
00166
00167 uart2_status_t UART2_get_last_status(void) {
00168     return uart2RxLastError;
00169 }
00170
00171 uint8_t UART2_Read(void)
00172 {
00173     uint8_t readValue = 0;
00174
00175     while(0 == uart2RxCount)
00176     {
00177     }
00178
00179     uart2RxLastError = uart2RxStatusBuffer[uart2RxTail];
00180
00181     readValue = uart2RxBuffer[uart2RxTail++];
00182     if(sizeof(uart2RxBuffer) <= uart2RxTail)
00183     {
00184         uart2RxTail = 0;
00185     }
00186     PIE6bits.U2RXIE = 0;
00187     uart2RxCount--;
00188     PIE6bits.U2RXIE = 1;
00189
00190     return readValue;
00191 }
00192
00193 void UART2_Write(uint8_t txData)
00194 {
00195     while(0 == uart2TxBufferRemaining)
00196     {
00197     }
00198
00199     if(0 == PIE6bits.U2TXIE)
00200     {
00201         U2TXB = txData;
00202     }
00203     else
00204     {
00205         PIE6bits.U2TXIE = 0;
00206         uart2TxBuffer[uart2TxHead++] = txData;
00207         if(sizeof(uart2TxBuffer) <= uart2TxHead)
00208         {
00209             uart2TxHead = 0;
00210         }
00211         uart2TxBufferRemaining--;
00212     }
00213     PIE6bits.U2TXIE = 1;

```

```

00214 }
00215
00216 void __interrupt (irq(U2TX),base(8)) UART2_tx_vect_isr()
00217 {
00218     if(UART2_TxInterruptHandler)
00219     {
00220         UART2_TxInterruptHandler();
00221     }
00222 }
00223
00224 void __interrupt (irq(U2RX),base(8)) UART2_rx_vect_isr()
00225 {
00226     if(UART2_RxInterruptHandler)
00227     {
00228         UART2_RxInterruptHandler();
00229     }
00230 }
00231
00232
00233
00234 void UART2_Transmit_ISR(void)
00235 {
00236     // use this default transmit interrupt handler code
00237     if(sizeof(uart2TxBuffer) > uart2TxBufferRemaining)
00238     {
00239         U2TXB = uart2TxBuffer[uart2TxTail++];
00240         if(sizeof(uart2TxBuffer) <= uart2TxTail)
00241         {
00242             uart2TxTail = 0;
00243         }
00244         uart2TxBufferRemaining++;
00245     }
00246     else
00247     {
00248         PIE6bits.U2TXIE = 0;
00249     }
00250
00251     // or set custom function using UART2_SetTxInterruptHandler()
00252 }
00253
00254 void UART2_Receive_ISR(void)
00255 {
00256     // use this default receive interrupt handler code
00257     uart2RxStatusBuffer[uart2RxHead].status = 0;
00258
00259     if(U2ERRIRbits.FERIF) {
00260         uart2RxStatusBuffer[uart2RxHead].ferr = 1;
00261         UART2_FramingErrorHandler();
00262     }
00263
00264     if(U2ERRIRbits.RXFOIF) {
00265         uart2RxStatusBuffer[uart2RxHead].oerr = 1;
00266         UART2_OverrunErrorHandler();
00267     }
00268
00269     if(uart2RxStatusBuffer[uart2RxHead].status) {
00270         UART2_ErrorHandler();
00271     } else {
00272         UART2_RxDataHandler();
00273     }
00274
00275     // or set custom function using UART2_SetRxInterruptHandler()
00276 }
00277
00278 void UART2_RxDataHandler(void) {
00279     // use this default receive interrupt handler code
00280     uart2RxBuffer[uart2RxHead++] = U2RXB;
00281     if(sizeof(uart2RxBuffer) <= uart2RxHead)
00282     {
00283         uart2RxHead = 0;
00284     }
00285     uart2RxCount++;
00286 }
00287
00288 void UART2_DefaultFramingErrorHandler(void) {}
00289
00290 void UART2_DefaultOverrunErrorHandler(void) {}
00291
00292 void UART2_DefaultErrorHandler(void) {
00293     UART2_RxDataHandler();
00294 }
00295
00296 void UART2_SetFramingErrorHandler(void (* interruptHandler)(void)) {
00297     UART2_FramingErrorHandler = interruptHandler;
00298 }
00299
00300 void UART2_SetOverrunErrorHandler(void (* interruptHandler)(void)) {

```



```

00301     UART2_OverrunErrorHandler = interruptHandler;
00302 }
00303
00304 void UART2_SetErrorHandler(void (* interruptHandler)(void)){
00305     UART2_ErrorHandler = interruptHandler;
00306 }
00307
00308
00309
00310 void UART2_SetRxInterruptHandler(void (* InterruptHandler)(void)){
00311     UART2_RxInterruptHandler = InterruptHandler;
00312 }
00313
00314 void UART2_SetTxInterruptHandler(void (* InterruptHandler)(void)){
00315     UART2_TxInterruptHandler = InterruptHandler;
00316 }
00317
00318

```

4.53 uart2.h

```

00001
00023
00024 /*
00025     (c) 2018 Microchip Technology Inc. and its subsidiaries.
00026
00027     Subject to your compliance with these terms, you may use Microchip software and any
00028     derivatives exclusively with Microchip products. It is your responsibility to comply with third
00029     party
    license terms applicable to your use of third party software (including open source software) that
    may accompany Microchip software.
00030
00031
00032     THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00033     EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
00034     IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
00035     FOR A PARTICULAR PURPOSE.
00036
00037     IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00038     INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00039     WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
00040     HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
00041     THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
00042     CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
00043     OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
00044     SOFTWARE.
00045 */
00046
00047 #ifndef UART2_H
00048 #define UART2_H
00049
00053
00054 #include <xc.h>
00055 #include <stdbool.h>
00056 #include <stdint.h>
00057
00058 #ifdef __cplusplus // Provide C++ Compatibility
00059
00060     extern "C" {
00061
00062 #endif
00063
00067
00068 #define UART2_DataReady    (UART2_is_rx_ready())
00069
00073
00074 typedef union {
00075     struct {
00076         unsigned perr : 1;
00077         unsigned ferr : 1;
00078         unsigned oerr : 1;
00079         unsigned reserved : 5;
00080     };
00081     uint8_t status;
00082 }uart2_status_t;
00083
00087 extern volatile uint8_t uart2TxBufferRemaining;
00088 extern volatile uint8_t uart2RxCount;
00089
00093
00115 void UART2_Initialize(void);
00116
00163 bool UART2_is_rx_ready(void);

```

```

00164
00211 bool UART2_is_tx_ready(void);
00212
00258 bool UART2_is_tx_done(void);
00259
00306 uart2_status_t UART2_get_last_status(void);
00307
00355 uint8_t UART2_Read(void);
00356
00380 void UART2_Write(uint8_t txData);
00381
00401 void UART2_Transmit_ISR(void);
00402
00422 void UART2_Receive_ISR(void);
00423
00443 void UART2_RxDataHandler(void);
00444
00461 void UART2_SetFramingErrorHandler(void (* interruptHandler)(void));
00462
00479 void UART2_SetOverrunErrorHandler(void (* interruptHandler)(void));
00480
00497 void UART2_SetErrorHandler(void (* interruptHandler)(void));
00498
00499
00500
00517 void (*UART2_RxInterruptHandler)(void);
00518
00535 void (*UART2_TxInterruptHandler)(void);
00536
00537
00538
00555 void UART2_SetRxInterruptHandler(void (* InterruptHandler)(void));
00556
00573 void UART2_SetTxInterruptHandler(void (* InterruptHandler)(void));
00574
00575
00576
00577 #ifdef __cplusplus // Provide C++ Compatibility
00578     }
00579 }
00580 #endif
00581 #endif
00582
00583 #endif // UART2_H

```

4.54 mconfig.c

```

00001 #include "mconfig.h"
00002 #include "mydisplay.h"
00003 #include "msg_text.h"
00004 #include "bsoc.h"
00005
00006 /*
00007  * LCD data buffer for virtual display for four vterms
00008  */
00009 static D_data D = {' '};
00010
00011 /*
00012  * hardware specific routines
00013  */
00014 void diversion_pwm_set(const uint16_t level)
00015 {
00016     PWM8_LoadDutyValue(level);
00017 }
00018
00019 void diversion_pwm_full(void)
00020 {
00021     PWM8_LoadDutyValue(DPWM_FULL);
00022 }
00023
00024 /*
00025  * mode button help mode select
00026  */
00027 bool help_button(void)
00028 {
00029     if (!RB0_GetValue()) { // debounce and delay for button press
00030         V.help = true;
00031         if (TimerDone(TMR_HELP))
00032             return true;
00033     }
00034
00035     if (!V.help)
00036         StartTimer(TMR_HELP, BDELAY);

```

```

00037
00038     return false;
00039 }
00040
00041 bool check_help(const bool flipper)
00042 {
00043     bool estatus = false;
00044     /*
00045      * show help display
00046      */
00047     if (help_button() && display_info() != DIS_HELP) {
00048         StartTimer(TMR_FLIPPER, DFLIP);
00049         if (V.debug)
00050             vterm_dump();
00051
00052         set_vterm(1);
00053         set_temp_display_help(display_info());
00054         set_display_info(DIS_HELP);
00055         if (flipper) {
00056             sprintf(get_vterm_ptr(0, 1), "HELP %s", build_date);
00057         } else {
00058             sprintf(get_vterm_ptr(0, 1), "HELP %s", build_time);
00059         }
00060         sprintf(get_vterm_ptr(1, 1), "%s", T[V.help_id].display);
00061         sprintf(get_vterm_ptr(2, 1), "%s", T[V.help_id].message);
00062         V.help_id++; // cycle help text messages to LCD
00063         StartTimer(TMR_HELDPDIS, TDELAY);
00064         StartTimer(TMR_INFO, TDELAY);
00065         update_lcd(1);
00066         estatus = true;
00067     } else {
00068         if (TimerDone(TMR_HELDPDIS)) {
00069             set_vterm(0);
00070             V.help = false;
00071             set_display_info(display_help());
00072             if (TimerDone(TMR_FLIPPER)) {
00073                 V.flipper = !V.flipper;
00074                 StartTimer(TMR_FLIPPER, DFLIP);
00075             }
00076         }
00077     }
00078     return estatus;
00079 }
00080
00081 /*
00082  * write character data to vterm
00083  */
00084 uint8_t update_lcd(uint8_t vterm)
00085 {
00086     vterm = vterm & 0x03;
00087
00088     if (D.vterm >= 1 && vterm == 0)
00089         return D.vterm;
00090
00091     D.lcd[vterm][0][MAX_LINE] = 0;
00092     D.lcd[vterm][1][MAX_LINE] = 0;
00093     D.lcd[vterm][2][MAX_LINE] = 0;
00094     D.lcd[vterm][3][MAX_LINE] = 0;
00095     wait_lcd_done();
00096     eaDogM_WriteStringAtPos(0, 0, D.lcd[vterm][0]);
00097     wait_lcd_done();
00098     eaDogM_WriteStringAtPos(1, 0, D.lcd[vterm][1]);
00099     wait_lcd_done();
00100     eaDogM_WriteStringAtPos(2, 0, D.lcd[vterm][2]);
00101     wait_lcd_done();
00102     eaDogM_WriteStringAtPos(3, 0, D.lcd[vterm][3]);
00103     return D.vterm;
00104 }
00105
00106 /*
00107  * set terminal window to 0..3
00108  */
00109 uint8_t set_vterm(const uint8_t vterm)
00110 {
00111     D.vterm = vterm & 0x03;
00112     return D.vterm;
00113 }
00114
00115 /*
00116  * return pointer to vterm line buffer
00117  */
00118 char * get_vterm_ptr(const uint8_t line, const uint8_t vterm)
00119 {
00120     return D.lcd[vterm & 0x03][line & 0x03];
00121 }
00122
00123 void vterm_dump(void)

```

```

00124 {
00125     sprintf(V.buf, "vterm %x:%x  ", (uint16_t) get_vterm_ptr(0, 0), (uint16_t) get_vterm_ptr(0, 1));
00126     wait_lcd_done();
00127     eaDogM_WriteStringAtPos(0, 0, V.buf);
00128     sprintf(V.buf, "vterm %x:%x  ", (uint16_t) get_vterm_ptr(1, 0), (uint16_t) get_vterm_ptr(1, 1));
00129     wait_lcd_done();
00130     eaDogM_WriteStringAtPos(1, 0, V.buf);
00131     sprintf(V.buf, "vterm %x:%x  ", (uint16_t) get_vterm_ptr(2, 0), (uint16_t) get_vterm_ptr(2, 1));
00132     wait_lcd_done();
00133     eaDogM_WriteStringAtPos(2, 0, V.buf);
00134     sprintf(V.buf, "vterm %x:%x  ", (uint16_t) get_vterm_ptr(3, 0), (uint16_t) get_vterm_ptr(3, 1));
00135     wait_lcd_done();
00136     eaDogM_WriteStringAtPos(3, 0, V.buf);
00137     WaitMs(3000);
00138 }
00139
00140 void vterm_sequence(void)
00141 {
00142     sprintf(get_vterm_ptr(2, 2), " Mesg %d Stack %d          ", V.msg_error, V.stack);
00143     switch (V.response.info) {
00144     case DIS_HELP:
00145         sprintf(get_vterm_ptr(0, 1), "HELP %s          ", build_date);
00146         sprintf(get_vterm_ptr(1, 1), "DISPLAY %s      ", build_time);
00147         break;
00148     case DIS_CLEAR:
00149     default:
00150         sprintf(get_vterm_ptr(0, 0), "                ");
00151         sprintf(get_vterm_ptr(1, 0), "                ");
00152         sprintf(get_vterm_ptr(0, 2), "                ");
00153         sprintf(get_vterm_ptr(1, 2), "                ");
00154         break;
00155     }
00156 }
00157
00158 /*
00159  * update possible command messages
00160  */
00161 void MyeaDogM_WriteStringAtPos(const uint8_t r, const uint8_t c, char *strPtr)
00162 {
00163
00164     wait_lcd_done();
00165     if (V.response.info == DIS_STR) {
00166         eaDogM_WriteStringAtPos(r, c, strPtr);
00167     } else {
00168         if (V.response.info == DIS_HELP) {
00169             sprintf(get_vterm_ptr(2, 1), "%s", V.info);
00170             update_lcd(1);
00171         } else {
00172             if ((V.response.info != DIS_STR)) {
00173                 update_lcd(2);
00174             } else {
00175                 sprintf(get_vterm_ptr(2, 0), "%s", V.info);
00176                 update_lcd(0);
00177             }
00178         }
00179
00180         if ((V.response.info != DIS_STR) && TimerDone(TMR_INFO))
00181             V.response.info = DIS_STR;
00182     }
00183
00184     /*
00185      * this is for possible message flipping with the HELP button
00186      */
00187     if (D.last_info == DIS_HELP && V.response.info != DIS_HELP) {
00188         // show some stuff, maybe
00189     }
00190
00191     D.last_info = V.response.info;
00192     // DLED = false;
00193 }
00194
00195 inline D_CODES display_info(void)
00196 {
00197     return V.response.info;
00198 }
00199
00200 inline D_CODES display_help(void)
00201 {
00202     return V.response.help_temp;
00203 }
00204
00205 D_CODES set_display_info(const D_CODES new_response_info)
00206 {
00207     static D_CODES old_info = DIS_STR;
00208
00209     if (TimerDone(TMR_INFO)) {
00210         old_info = V.response.info;

```

```

00211     V.response.info = new_response_info;
00212 }
00213 return old_info;
00214 }
00215
00216 D_CODES set_temp_display_help(const D_CODES new_response_info)
00217 {
00218     static D_CODES old_info;
00219
00220     old_info = V.response.help_temp;
00221     V.response.help_temp = new_response_info;
00222     return old_info;
00223 }
00224
00225 bool set_back_light_off(bool mode)
00226 {
00227     static bool blight_on = true;
00228     bool send_cmd = false;
00229
00230     if (mode) {
00231         if (V.blight_off) { // background 10min trigger
00232             if (blight_on) { // check back-light state
00233                 blight_on = false; // update tracking state to OFF
00234                 send_cmd = true; // send trigger return code
00235             }
00236         }
00237     } else {
00238         if (!V.blight_off) {
00239             if (!blight_on) { // check back-light state
00240                 V.blight_off = false; // clear timer trigger
00241                 blight_on = true; // update tracking state to ON
00242                 V.blight = time(NULL) + BL_TIME; // reset background 10min trigger
00243                 send_cmd = true; // send trigger return code
00244             }
00245         }
00246     }
00247     return send_cmd;
00248 }

```

4.55 mconfig.h

```

00001 /* Microchip Technology Inc. and its subsidiaries. You may use this software
00002 * and any derivatives exclusively with Microchip products.
00003 *
00004 * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
00005 * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
00006 * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
00007 * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
00008 * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
00009 *
00010 * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00011 * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00012 * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
00013 * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE
00014 * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
00015 * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
00016 * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
00017 *
00018 * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
00019 * TERMS.
00020 */
00021
00022 /*
00023 * File:
00024 * Author:
00025 * Comments:
00026 * Revision history:
00027 */
00028
00029 // This is a guard condition so that contents of this file are not included
00030 // more than once.
00031 #ifndef MCONFIG_H
00032 #define MCONFIG_H
00033
00034
00035 #include "mcc_generated_files/mcc.h"
00036 #include "vconfig.h"
00037
00038 extern const char *build_date, *build_time;
00039 extern V_data V;
00040
00041 void diversion_pwm_full(void);
00042 void diversion_pwm_set(uint16_t);

```

```

00043 bool set_back_light_off(bool);
00044
00045 #endif
00046

```

4.56 msg_text.h

```

00001 /*
00002  * File:   msg_text.h
00003  * Author: root
00004  *
00005  * Created on July 15, 2019, 7:32 AM
00006  */
00007
00008 #ifndef MSG_TEXT_H
00009 #define MSG_TEXT_H
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014 #include "mconfig.h"
00015 #include "mydisplay.h"
00016
00017     typedef enum {
00018         display_message = 0,
00019         display_online,
00020         display_comm,
00021     } DISPLAY_TYPES;
00022
00023     const V_help T[] = {
00024     {
00025         .message = "commands 1",
00026         .display = "displays 1",
00027     },
00028     {
00029         .message = "commands 2",
00030         .display = "displays 2",
00031     },
00032     {
00033         .message = "commands 3",
00034         .display = "displays 3",
00035     },
00036     {
00037         .message = "commands 4",
00038         .display = "displays 4",
00039     },
00040     };
00041
00042 #ifdef __cplusplus
00043 }
00044 #endif
00045
00046 #endif /* MSG_TEXT_H */
00047

```

4.57 mydisplay.h

```

00001 /* Microchip Technology Inc. and its subsidiaries.  You may use this software
00002  * and any derivatives exclusively with Microchip products.
00003  *
00004  * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS".  NO WARRANTIES, WHETHER
00005  * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
00006  * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
00007  * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
00008  * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
00009  *
00010  * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00011  * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00012  * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
00013  * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE.  TO THE
00014  * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
00015  * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
00016  * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
00017  *
00018  * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
00019  * TERMS.
00020  */
00021

```

```

00022 /*
00023  * File:
00024  * Author:
00025  * Comments:
00026  * Revision history:
00027  */
00028
00029 // This is a guard condition so that contents of this file are not included
00030 // more than once.
00031 #ifndef XC_HEADER_TEMPLATE_H
00032 #define XC_HEADER_TEMPLATE_H
00033
00034 #include <xc.h> // include processor files - each processor file is guarded.
00035 #include <stdio.h>
00036 #include <string.h>
00037
00038 #include "vconfig.h"
00039 #include "eadog.h"
00040 #include "timers.h"
00041
00042 typedef struct D_data { // vterm control data structure
00043     char lcd[4][4][40];
00044     uint8_t vterm : 1;
00045     D_CODES last_info;
00046 } D_data;
00047
00048 void MyeaDogM_WriteStringAtPos(const uint8_t, const uint8_t, char *);
00049 uint8_t update_lcd(uint8_t);
00050 uint8_t set_vterm(const uint8_t);
00051 char * get_vterm_ptr(const uint8_t, const uint8_t);
00052 void vterm_dump(void);
00053 void vterm_sequence(void);
00054 inline D_CODES display_info(void);
00055 inline D_CODES display_help(void);
00056 bool help_button(void);
00057 bool check_help(const bool);
00058 D_CODES set_display_info(const D_CODES);
00059 D_CODES set_temp_display_help(const D_CODES);
00060
00061
00062 #ifdef __cplusplus
00063 extern "C" {
00064 #endif /* __cplusplus */
00065
00066     // TODO If C++ is being used, regular C code needs function names to have C
00067     // linkage so the functions can be used by the c code.
00068
00069 #ifdef __cplusplus
00070 }
00071 #endif /* __cplusplus */
00072
00073 #endif /* XC_HEADER_TEMPLATE_H */
00074

```

4.58 ringbufs.c

```

00001 #include <string.h>
00002 #include "ringbufs.h"
00003
00004 /*
00005  * general ring buffer fuctions from the internet
00006  */
00007 uint8_t modulo_inc(const uint8_t value, const uint8_t modulus)
00008 {
00009     uint8_t my_value = value + 1;
00010     if (my_value >= modulus) {
00011         my_value = 0;
00012     }
00013     return my_value;
00014 }
00015
00016 uint8_t modulo_dec(const uint8_t value, const uint8_t modulus)
00017 {
00018     uint8_t my_value = (0 == value) ? (modulus - 1) : (value - 1);
00019     return my_value;
00020 }
00021
00022 void ringBufS_init(volatile ringBufS_t *_this)
00023 {
00024     /*****
00025      The following clears:
00026      -> buf
00027      -> head

```

```

00028         -> tail
00029         -> count
00030         and sets head = tail
00031     ***/
00032     memset((void*) _this, 0, sizeof(*_this));
00033 }
00034
00035 int8_t ringBufS_empty(ringBufS_t *_this)
00036 {
00037     return(0 == _this->count);
00038 }
00039
00040 int8_t ringBufS_full(ringBufS_t *_this)
00041 {
00042     return(_this->count >= RBUF_SIZE);
00043 }
00044
00045 uint8_t ringBufS_get(ringBufS_t *_this)
00046 {
00047     uint8_t c;
00048     if (_this->count > 0) {
00049         c = _this->buf[_this->tail];
00050         _this->tail = modulo_inc(_this->tail, RBUF_SIZE);
00051         --_this->count;
00052     } else {
00053         c = 0; // return null with empty buffer
00054     }
00055     return(c);
00056 }
00057
00058 void ringBufS_put(ringBufS_t *_this, const uint8_t c)
00059 {
00060     if (_this->count < RBUF_SIZE) {
00061         _this->buf[_this->head] = c;
00062         _this->head = modulo_inc(_this->head, RBUF_SIZE);
00063         ++_this->count;
00064     }
00065 }
00066
00067 void ringBufS_put_dma(ringBufS_t *_this, const uint8_t c)
00068 {
00069     if (_this->count < RBUF_SIZE) {
00070         _this->buf[_this->head] = c;
00071         ++_this->head;
00072         ++_this->count;
00073     }
00074 }
00075
00076 void ringBufS_put_dma_cpy(ringBufS_t *_this, const char *ptr, const uint8_t len)
00077 {
00078     if (_this->count < RBUF_SIZE) {
00079         memcpy(_this->buf, ptr, len);
00080         _this->head = len;
00081         _this->count = len;
00082     }
00083 }
00084
00085 void ringBufS_flush(ringBufS_t *_this, const int8_t clearBuffer)
00086 {
00087     _this->count = 0;
00088     _this->head = 0;
00089     _this->tail = 0;
00090     if (clearBuffer) {
00091         memset(_this->buf, 0, sizeof(_this->buf));
00092     }
00093 }
00094

```

4.59 ringbufs.h

```

00001 /*
00002  * File:   ringbufs.h
00003  * Author: rootswitcDC circuit breakerh debounce icdebounce icfuse type T
00004  *
00005  * Created on July 25, 2015, 2:03 PM
00006  */
00007
00008 #ifndef RINGBUFS_H
00009 #define RINGBUFS_H
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif

```



```

00014
00015 #include "vconfig.h"
00016
00017 #define RBUF_SIZE    64
00018
00019     typedef struct ringBufS_t {
00020         uint8_t buf[RBUF_SIZE];
00021         uint8_t head;
00022         uint8_t tail;
00023         uint8_t count;
00024     } ringBufS_t;
00025
00026     void ringBufS_init(volatile ringBufS_t *_this);
00027     int8_t ringBufS_empty(ringBufS_t *_this);
00028     int8_t ringBufS_full(ringBufS_t *_this);
00029     uint8_t ringBufS_get(ringBufS_t *_this);
00030     void ringBufS_put(ringBufS_t *_this, const uint8_t c);
00031     void ringBufS_put_dma(ringBufS_t *_this, const uint8_t c);
00032     void ringBufS_put_dma_cpy(ringBufS_t *, const char *, const uint8_t);
00033     void ringBufS_flush(ringBufS_t *_this, const int8_t clearBuffer);
00034
00035
00036 #ifdef __cplusplus
00037 }
00038 #endif
00039
00040 #endif /* RINGBUFS_H */
00041

```

4.60 tests.h

```

00001 /* Microchip Technology Inc. and its subsidiaries.  You may use this software
00002  * and any derivatives exclusively with Microchip products.
00003  *
00004  * THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS".  NO WARRANTIES, WHETHER
00005  * EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED
00006  * WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A
00007  * PARTICULAR PURPOSE, OR ITS INTERACTION WITH MICROCHIP PRODUCTS, COMBINATION
00008  * WITH ANY OTHER PRODUCTS, OR USE IN ANY APPLICATION.
00009  *
00010  * IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
00011  * INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
00012  * WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS
00013  * BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE.  TO THE
00014  * FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS
00015  * IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT OF FEES, IF
00016  * ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
00017  *
00018  * MICROCHIP PROVIDES THIS SOFTWARE CONDITIONALLY UPON YOUR ACCEPTANCE OF THESE
00019  * TERMS.
00020  */
00021
00022 /*
00023  * File:
00024  * Author:
00025  * Comments:
00026  * Revision history:
00027  */
00028
00029 // This is a guard condition so that contents of this file are not included
00030 // more than once.
00031 #ifndef TESTS_H
00032 #define TESTS_H
00033
00034 #include <xc.h> // include processor files - each processor file is guarded.
00035 #include "mcc_generated_files/pin_manager.h"
00036
00037 #define DEBUG_MBMC // global application debug switch
00038 #define DLED1     DEBUG1_LAT
00039 #define DLED2     DEBUG2_LAT
00040
00041 #ifdef DEBUG_MBMC
00042 #define DEBUG_JSON
00043 //#define DEBUG_DAQ1 // daq debug switches : SCANNER
00044 //#define DEBUG_DAQ2 // ADC conversion
00045 //#define DEBUG_SWH1 // switches
00046 //#define DEBUG_SWH2
00047 //#define DEBUG_BSOC1 // battery run calculations
00048 //#define DEBUG_BSOC2
00049 //#define DEBUG_HIST1
00050 //#define DEBUG_TMR3 // timers
00051 //#define DEBUG_TRM5
00052 //#define DEBUG_DISP1 // LCD Display

```

```

00053 //define DEBUG_DISP2
00054 //define DEBUG_WDT1 // WDT
00055 //define DEBUG_WDT2
00056 #endif
00057
00058 #define CALIB    false
00059 #define FIXUPS   true
00060 //define CLR_CYCLE_DATA
00061 //define CLR_BATTERY_DATA
00062 //define CLR_PV_DATA
00063 #endif /* XC_HEADER_TEMPLATE_H */
00064

```

4.61 timers.c

```

00001 #include <xc.h>
00002 #include <stdint.h>
00003 #include <stdbool.h>
00004 #include "vconfig.h"
00005 #include "timers.h"
00006 #include "mcc_generated_files/uart1.h"
00007
00008 extern volatile uint16_t tickCount[TMR_COUNT];
00009
00010
00011 //*****
00012 // Start one of the software timers
00013 void StartTimer(const uint8_t timer, const uint16_t count)
00014 {
00015     tickCount[timer] = count << 1; //Interrupt is every 500us but StartTimer() takes multiple of 1ms so
    multiply by 2
00016 }
00017
00018 //*****
00019 // Check if one of the software software timers has timed out
00020
00021 bool TimerDone(const uint8_t timer)
00022 {
00023     bool ret = false;
00024     ClrWdt(); // reset the WDT timer
00025     if (tickCount[timer] == (uint16_t) 0) { //Check if counted down to zero
00026         ret = true; //then return true
00027     }
00028     return ret; //return status
00029 }
00030
00031 //*****
00032 // Simple delay for n milliseconds (blocking)
00033
00034 void WaitMs(const uint16_t numMilliseconds)
00035 {
00036     StartTimer(TMR_INTERNAL, numMilliseconds); //Start software timer and wait for it to count down
00037     while (!TimerDone(TMR_INTERNAL)) {
00038         Nop();
00039         Nop();
00040         Nop();
00041         Nop();
00042         ClrWdt(); // reset the WDT timer
00043         // Idle();
00044     } //Enter idle mode to reduce power while waiting
00045 } // (timer interrupt will wake part from idle)
00046
00047 void sw_timers_isr(void)
00048 {
00049     static uint8_t i;
00050
00051     //Decrement each software timer
00052     for (i = 0; i < TMR_COUNT; i++) {
00053         if (tickCount[i] != 0) {
00054             tickCount[i]--;
00055         }
00056     }
00057 }

```

4.62 timers.h

```

00001 #ifndef TIMERS_H

```

```

00002 #define TIMERS_H
00003
00004 #include <stdint.h>
00005 #include <stdbool.h>
00006
00007 //Software timers - use these to refer to timers rather than
00008 //integers. Add more as needed.
00009 //NOTE: 32767 ms max delay
00010
00011 enum APP_TIMERS {
00012     TMR_INTERNAL = 0, //Used in timers.c - do not remove or use elsewhere
00013     TMR_ADC,
00014     TMR_T2,
00015     TMR_T3,
00016     TMR_T4,
00017     TMR_MC_TX,
00018     TMR_HBIO,
00019     TMR_INFO,
00020     TMR_HELP,
00021     TMR_HELPDIS,
00022     TMR_DISPLAY,
00023     TMR_FLIPPER,
00024     TMR_ESR,
00025     TMR_TEXT,
00026     TMR_SPIN,
00027     TMR_GTI,
00028     //
00029     //(Add timers here as needed)
00030     //
00031     TMR_COUNT //number of timers - always keep at end of enum!
00032 };
00033
00034 void StartTimer(const uint8_t timer, const uint16_t count);
00035 bool TimerDone(const uint8_t timer);
00036 void WaitMs(const uint16_t numMilliseconds);
00037 void sw_timers_isr(void);
00038
00039 #endif //TIMERS_H

```

4.63 vconfig.h

```

00001 /* * File:    vconfig.h
00002  * Author: root
00003  *
00004  * Created on February 19, 2019, 4:41 PM
00005  */
00006
00007 #ifndef VCONFIG_H
00008 #define VCONFIG_H
00009
00010 #include "mcc_generated_files/adcc.h"
00011 #include "mcc_generated_files/ext_int.h"
00012
00013 #ifdef __cplusplus
00014 extern "C" {
00015 #endif
00016
00017 #include <xc.h>
00018 #include <time.h>
00019 #include "mcc_generated_files/spi1.h"
00020 #include "mcc_generated_files/pin_manager.h"
00021 #include "ringbufs.h"
00022
00023 #define VER "2.20"
00024
00025 /*
00026  * REMOVE THE CHIP PROGRAMMER, IT CHANGES THE ADC READINGS
00027  */
00028
00029 /*
00030  * 0.1 MBMC new version for one 24vdc battery string for the 57K42
00031  * 0.2 start to configure the hardware for mbmc duty
00032  * 0.3 configure for mbmc_io board
00033  * 0.4 ADC setup
00034  * 0.5 remove gemsecs coding
00035  * 0.6 adc scanning by threshold interrupt using burst average mode
00036  * 0.7 system data conversion from ADC data
00037  * 0.8 basic current and voltage tracking
00038  * 0.9 auto convert bitmaps
00039  * 0.91 start adding 32 utc time code
00040  * 0.94 switch input code
00041  * 0.95 MAX6818 interrupt coding
00042  * 0.97 need hardware fix on mbmc_io board to repair bypassed resistors R20 and R18
00043  * 0.98 last of the hardware bugs fixed

```

```

00043      * 0.99 power load calculations
00044      * 1.0+ mainly working code base
00045      * 1.07 move MAX6818 enable pin for board rev 1.1
00046      * 1.08 NHD 4x20 display
00047      * 1.16 update logging format
00048      * 1.17 current sensor calibration functions
00049      * 1.18 EEPROM routines
00050      * 1.19 8-bit crc for EEPROM change detection
00051      * 1.20 optional 10A calibration
00052      * 1.21 history tracking in EEPROM
00053      * 1.21 history update in EEPROM
00054      * 1.23 line 4 display issues with history display functions, temp fix is to set MAX_LINE to 19
00055      * need to work on this for a correct fix. review LCD_addressing.pdf in host_board folder
00056      * 1.24 fix LCD problems, was trying to send a 0x00 using printf
00057      * 1.25 load time_t date from saved EEPROM history if valid
00058      * 1.29 fine tuning the ADC configuration
00059      * 1.31 add third hall-sensor for actual PV current and CC current for MPPT VI conversion tracking
00060      * 1.32 data number fixups for display and calculations
00061      * 1.33 reorder logging data format
00062      * 1.34 CC mode to voltage decoding routines
00063      * 1.35 AC charger limits and float cycle counts for H[11]
00064      * 1.36 Charge controller op mode FIXUPS
00065      * 1.37 HID button response fixes, use bitmap data for stored button press info in hid.c
00066      * 1.38 UART 1 serial control parser. 'V' AC charger ON, 'v' AC charger off, '#' Show system
information
00067      * 1.39 Tweak one second interrupt times for better precision
00068      * 1.40 SOC and AH corrections at FLOAT and BOOST triggers from charge controller
00069      * 1.41 add daily ah tracking
00070      * 1.42 fix daily tracking
00071      * 1.43 add option using BAT_100A in daq.h to select Honeywell sensor for battery current sensor
00072      * 1.44 DC dump load from PV functions
00073      * 1.45 WDT hang protection
00074      * 1.46 server time updates
00075      * 1.47 Battery low voltage checks for charger activation
00076      * 1.48 add third current sensor zero calibration
00077      * 1.49 restructure and optimizations
00078      * 1.50 current sensor options in daq.h: CAL_10A CAL_TWO
00079      * 1.51 change ADC to FRC clock source
00080      * 1.52 back to ext clock for ADC with increased time for hold sample
00081      * 1.53 fix lcd display line limits
00082      * 1.56 connection updates and structure fixes
00083      * 2.00 FM80 dump load system monitor
00084      * 2.01 swap around the voltage and current sensors offsets and scalars for the FM80 system
configuration
00085      * 2.02 move variables on the LCD display code to match the actual measurements
00086      * 2.03 add json power variables, change MQTT data update to 1 second
00087      * 2.04 general code cleanup
00088      * 2.05 GTI power control and monitoring using UART2
00089      * 2.08 GTI command set, serial UART2
00090      * 2.09 Fix UART1 receive bugs
00091      * 2.10 Alpha version
00092      * 2.11 typo corrections
00093      * 2.12 cleanup GTI parser code
00094      * 2.13 add GTI power value parsing with V start value, [0..9][3], X execute value
00095      * 2.14 misc cleanup
00096      * 2.15 more cleanups
00097      * 2.16 add charge controller mode to MQTT variables, fix CCMODE voltage ranges
00098      * 2.17 add UTC time setting from MQTT
00099      * 2.18 doc updates and minor fixes
00100      * 2.19 current and power fixups for FM80 coding to JSON
00101      * 2.20 update to use xc8 3.0
00102      */
00103
00104      /*
00105      * REMOVE THE CHIP PROGRAMMER, IT CHANGES THE ADC READINGS
00106      */
00107      //#define TESTING
00108      //#define DISPLAY_SLOW
00109      #define DEF_TIME      1721693000
00110      #define DEF_TIME_SIZE  11
00111
00112      #define SLED          LED0_LAT
00113
00114      #define EADOGM_CMD_CLR      1
00115      #define EADOGM_CMD_CURSOR_ON      0b000001111
00116      #define EADOGM_CMD_CURSOR_OFF     0b000001100
00117      #define EADOGM_CMD_DISPLAY_ON     0b000001100
00118      #define EADOGM_CMD_DISPLAY_OFF    0b000001000
00119      #define EADOGM_CMD_DDRAM_ADDR     0b100000000
00120      #define EADOGM_CMD_CGRAM_ADDR     0b010000000
00121      #define EADOGM_CMD_SELECT_R0      0b000011000
00122      #define EADOGM_CMD_SELECT_R1      0b000010000
00123      #define EADOGM_CMD_SET_TABLE2     0b001010100
00124      #define EADOGM_COLSPAN            16
00125
00126      #define NHD          // 4x20 SPI display
00127

```

```

00128 #define T1 500
00129 #define T2 3000
00130 #define T3 5000
00131 #define T4 5000
00132 #define HBTS 5000 // short ping delay
00133 #define HBTL 30000 // ping delay
00134 #define RTY 3
00135 #define ERROR_CHECKSUM 31500
00136 #define ERROR_COMM 31500
00137
00138 #define DEFAULT_TID 1
00139 #define TDELAY 3000
00140 #define LDELAY 1000
00141 #define SDELAY 500
00142 #define BDELAY 300
00143 #define DDELAY 125 // display update spacing 125
00144 #define TXTDELAY 10000 // unix time from server timeout
00145 #define SOCDELAY 250 // sync SPI and UART dma 250
00146 #define DFLIP 1500 // display info flipping spacing
00147
00148 #define LOG_WAIT 1 // data logging interval in seconds
00149 #define D_CODE '5' // data logging line code version
00150 #define L_CODE '*' // info logging line code
00151
00152 #define CAL_DELAY 64
00153 // #define SKIP_ESR
00154
00155 /*
00156  * offsets in bytes
00157  */
00158
00159 #define TX_RESERVE 59
00160 #ifdef NHD
00161 #define MAX_LINE 20
00162 #else
00163 #define MAX_LINE 16
00164 #endif
00165 #define MAX_BUTTONS 8
00166
00167 #define HPARAM_SIZE 13
00168 #define HVER 1
00169
00170 #define BL_TIME 600
00171
00172 #define CC_DEACT 74 // 1.10
00173 #define CC_ACT 100 // 1.43
00174 #define CC_MPPT 120 // 1.75
00175 #define CC_EQUAL 140 // 2.10
00176 #define CC_BOOST 160 // 2.45
00177 #define CC_FLOAT 180 // 2.82
00178 #define CC_LIMIT 200 // 3.20
00179 #define CC_OFFLINE 240 // 4.00
00180
00181 #define max_port_data 1024
00182
00183 struct spi_link_type { // internal SPI state table
00184     uint8_t SPI_LCD : 1;
00185     uint8_t SPI_AUX : 1;
00186     uint8_t LCD_TIMER : 1;
00187     volatile uint8_t LCD_DATA : 1;
00188     uint16_t delay;
00189     uint8_t config;
00190     uint8_t * txbuf;
00191     volatile int32_t int_count;
00192 };
00193
00194 typedef enum {
00195     DIS_STR = 0,
00196     DIS_HELP,
00197     DIS_ERR,
00198     DIS_CLEAR,
00199 } D_CODES;
00200
00201 typedef struct terminal_type {
00202     uint8_t msgid;
00203     uint8_t TID, mcode, mparm, cmdlen, log_seq;
00204     uint8_t host_display_ack : 1;
00205     D_CODES info, help_temp;
00206     uint16_t ceid;
00207     uint16_t log_num;
00208 } terminal_type;
00209
00210 typedef enum {
00211     UI_STATE_INIT = 0,
00212     UI_STATE_HOST,
00213     UI_STATE_DEBUG,
00214     UI_STATE_LOG,

```

```

00215         UI_STATE_MON,
00216         UI_STATE_ERROR
00217     } UI_STATES;
00218
00219     typedef enum {
00220         SW_OFF = 0,
00221         SW_ON,
00222         SW_INVALID,
00223     } SW_STATES;
00224
00225     typedef enum {
00226         S0 = 0,
00227         S1 = 1,
00228         SSELECT = 2,
00229         SENTER = 3,
00230         SCALIB = 4,
00231         S5,
00232         S6,
00233         SNULL,
00234     } SW_NAMES;
00235
00236     typedef struct V_data { // control data structure
00237         UI_STATES ui_state;
00238         char buf[64], info[64], rbuf[16];
00239         volatile time_t ticks, blight, ac_time;
00240         volatile int8_t testing;
00241         volatile uint8_t sys_info : 1, ac_on : 1, ac_off : 1, time_info : 1, get_time_text : 1;
00242         uint8_t error, abort, msg_error, msg_ret, alarm, float_ticks, boost_ticks, wdt_ticks;
00243         UI_STATES ui_sw;
00244         uint16_t r_checksum, t_checksum, checksum_error, mode_pwm, sequences, all_errors;
00245         uint8_t set_sequ : 1, debug : 1, help : 1, stack : 3, help_id : 2, screen : 1;
00246         terminal_type response;
00247         volatile uint8_t ticker, cc_state;
00248         bool flipper, calib, enter, sensor_set, fixup, cc_active;
00249         volatile bool system_stable, blight_off, in_float, in_boost;
00250         volatile uint32_t lowint_count, timerint_count, tx_count, rx_count, adc_count, spi_count,
switch_count;
00251     } V_data;
00252
00253     typedef struct V_help {
00254         const char message[22], display[22];
00255     } V_help;
00256
00257     /*      hist[x].h[x]
00258     *      h0  Real Ah usage this cycle
00259     *      h1  Total charge cycles into at least boost
00260     *      h2  Current lowest discharge,
00261     *      h3  PV ah total
00262     *      h4  Batt W out
00263     *      h5  Batt W in
00264     *      h6  Real Ah usage
00265     *      h7  Batt Voltage full load
00266     *      h8  Batt Voltage one load
00267     *      h9  Lowest ESR
00268     *      h10 Highest ESR
00269     *      h11 Total charge cycles into float
00270     *      h12 Total kWh in
00271     */
00272
00273     typedef struct hist_type {
00274         int32_t pv_eff, tot_eff; // pv generation eff factor, total system eff factor
00275         uint32_t ttg_t, updates;
00276         time_t pclock;
00277         float peukert, cef, peukert_adj, cef_calc, cef_save;
00278         int16_t h[HPARAM_SIZE]; // h[6]=cumulative battery Ah cc and inv (real),h[0]=cumulative
battery Ah cc and inv (p_adj)
00279         uint16_t rate, udod, bsoc, bound_rate, bound_factor, samplei, sampleo, ah, drate, esr,
rest_rate, rest_factor, esrp;
00280         uint8_t version;
00281     } hist_type;
00282
00283 #ifdef __cplusplus
00284 }
00285 #endif
00286
00287 #endif /* VCONFIG_H */
00288

```

Index

- abort
 - V_data, [37](#)
- ac_off
 - V_data, [37](#)
- ac_on
 - V_data, [37](#)
- ac_time
 - V_data, [37](#)
- adc_count
 - V_data, [37](#)
- ah
 - hist_type, [19](#)
- alarm
 - V_data, [37](#)
- all_errors
 - V_data, [37](#)
- bank_ah
 - C_data, [7](#)
- baud
 - spi1_configuration_t, [29](#)
- bc_ror
 - C_data, [7](#)
- bd
 - bytes2, [5](#)
 - dac_buf_type, [16](#)
- bkwi
 - C_data, [7](#)
- bkwo
 - C_data, [7](#)
- blight
 - V_data, [38](#)
- blight_off
 - V_data, [38](#)
- boost_ticks
 - V_data, [38](#)
- bound_factor
 - hist_type, [19](#)
- bound_rate
 - hist_type, [19](#)
- bsoc
 - hist_type, [20](#)
- buf
 - ringBufS_t, [26](#)
 - V_data, [38](#)
- button
 - S_data, [28](#)
- bv_full_load
 - C_data, [7](#)
- bv_noload
 - C_data, [8](#)
- bv_one_load
 - C_data, [8](#)
- bv_ror
 - C_data, [8](#)
- bytes2, [5](#)
- bd, [5](#)
- ld, [5](#)
- c_bat
 - C_data, [8](#)
- C_data, [6](#)
 - bank_ah, [7](#)
 - bc_ror, [7](#)
 - bkwi, [7](#)
 - bkwo, [7](#)
 - bv_full_load, [7](#)
 - bv_noload, [8](#)
 - bv_one_load, [8](#)
 - bv_ror, [8](#)
 - c_bat, [8](#)
 - c_load, [8](#)
 - c_mppt, [8](#)
 - c_pv, [8](#)
 - calc, [8](#)
 - day, [9](#)
 - day_end, [9](#)
 - day_start, [9](#)
 - day_update, [9](#)
 - dupdate, [9](#)
 - dynamic_ah, [9](#)
 - dynamic_ah_adj, [9](#)
 - dynamic_ah_adj_daily, [9](#)
 - dynamic_ah_daily, [10](#)
 - esr, [10](#)
 - hist, [10](#)
 - invkw, [10](#)
 - load_i1, [10](#)
 - load_i2, [10](#)
 - loadah, [10](#)
 - loadkw, [10](#)
 - p_bat, [11](#)
 - p_inverter, [11](#)
 - p_load, [11](#)
 - p_mppt, [11](#)
 - p_pv, [11](#)
 - pv_ah, [11](#)
 - pvkw, [11](#)
 - runtime, [11](#)
 - soc, [12](#)

- start_power, 12
- t_comp, 12
- update, 12
- updates, 12
- v_bat, 12
- v_bbat, 12
- v_cbus, 12
- v_cc, 13
- v_cmode, 13
- v_inverter, 13
- v_pv, 13
- v_sensor, 13
- v_temp, 13
- c_load
 - C_data, 8
- c_mppt
 - C_data, 8
- c_pv
 - C_data, 8
- c_scale_cal
 - R_data, 23
- c_zero_cal
 - R_data, 23
- calc
 - C_data, 8
- calib
 - V_data, 38
- calib_p
 - H_data, 17
- cc_active
 - V_data, 38
- cc_state
 - V_data, 38
- cef
 - hist_type, 20
- cef_calc
 - hist_type, 20
- cef_save
 - hist_type, 20
- ceid
 - terminal_type, 31
- checkmark
 - R_data, 24
- checksum_error
 - V_data, 38
- cmdlen
 - terminal_type, 31
- con0
 - spi1_configuration_t, 29
- con1
 - spi1_configuration_t, 29
- con2
 - spi1_configuration_t, 29
- config
 - spi_link_type, 30
- cont
 - D_data, 15
- count
 - rbutton_type, 26
 - ringBufS_t, 26
- crc
 - R_data, 24
- CRC_OBJ, 14
 - dataWidth, 14
 - polyWidth, 14
 - seedDirection, 14
- D_data, 15
 - cont, 15
 - dac0, 15
 - dac1, 15
 - last_info, 15
 - lcd, 15
 - vterm, 15
- dac0
 - D_data, 15
- dac1
 - D_data, 15
- dac_buf_type, 16
 - bd, 16
 - ld, 16
 - map, 17
- dataWidth
 - CRC_OBJ, 14
- day
 - C_data, 9
- day_end
 - C_data, 9
- day_start
 - C_data, 9
- day_update
 - C_data, 9
- debug
 - V_data, 39
- delay
 - spi_link_type, 30
- display
 - V_help, 44
- done
 - R_data, 24
- drate
 - hist_type, 20
- dupdate
 - C_data, 9
- dynamic_ah
 - C_data, 9
- dynamic_ah_adj
 - C_data, 9
- dynamic_ah_adj_daily
 - C_data, 9
- dynamic_ah_daily
 - C_data, 10
- enter
 - V_data, 39
- enter_p
 - H_data, 17

- error
 - V_data, 39
- esr
 - C_data, 10
 - hist_type, 20
- esrp
 - hist_type, 20
- ferr
 - uart1_status_t, 33
 - uart2_status_t, 35
- fixup
 - V_data, 39
- flipper
 - V_data, 39
- float_ticks
 - V_data, 39
- get_time_text
 - V_data, 39
- H
 - R_data, 24
- h
 - hist_type, 20
- H_data, 17
 - calib_p, 17
 - enter_p, 17
 - h_state, 17
 - hid_display, 18
 - select_p, 18
 - sequence, 18
 - wait_calib, 18
 - wait_enter, 18
 - wait_select, 18
- h_state
 - H_data, 17
- head
 - ringBufS_t, 27
- help
 - V_data, 39
- help_id
 - V_data, 40
- help_temp
 - terminal_type, 32
- hid_display
 - H_data, 18
- hist
 - C_data, 10
- hist_save
 - R_data, 24
- hist_type, 19
 - ah, 19
 - bound_factor, 19
 - bound_rate, 19
 - bsoc, 20
 - cef, 20
 - cef_calc, 20
 - cef_save, 20
 - drate, 20
 - esr, 20
 - esrp, 20
 - h, 20
 - pclock, 21
 - peukert, 21
 - peukert_adj, 21
 - pv_eff, 21
 - rate, 21
 - rest_factor, 21
 - rest_rate, 21
 - samplei, 21
 - sampleo, 22
 - tot_eff, 22
 - ttg_t, 22
 - udod, 22
 - updates, 22
 - version, 22
- hist_update
 - R_data, 24
- host_display_ack
 - terminal_type, 32
- in_boost
 - V_data, 40
- in_float
 - V_data, 40
- info
 - terminal_type, 32
 - V_data, 40
- int_count
 - spi_link_type, 30
- invkw
 - C_data, 10
- last_info
 - D_data, 15
- lcd
 - D_data, 15
- LCD_DATA
 - spi_link_type, 30
- LCD_TIMER
 - spi_link_type, 30
- ld
 - bytes2, 5
 - dac_buf_type, 16
- load_i1
 - C_data, 10
- load_i2
 - C_data, 10
- loadah
 - C_data, 10
- loadkw
 - C_data, 10
- log_num
 - terminal_type, 32
- log_seq
 - terminal_type, 32
- lowint_count

- V_data, 40
- map
 - dac_buf_type, 17
- max5322_cmd
 - R_data, 24
- mcc_generated_files/adcc.c, 89
- mcc_generated_files/adcc.h, 93
- mcc_generated_files/clkref.c, 95
- mcc_generated_files/clkref.h, 95
- mcc_generated_files/crc.c, 96
- mcc_generated_files/crc.h, 99
- mcc_generated_files/device_config.c, 100
- mcc_generated_files/device_config.h, 101
- mcc_generated_files/dma1.c, 101
- mcc_generated_files/dma1.h, 104
- mcc_generated_files/dma2.c, 105
- mcc_generated_files/dma2.h, 106
- mcc_generated_files/ext_int.c, 107
- mcc_generated_files/ext_int.h, 108
- mcc_generated_files/fvr.c, 109
- mcc_generated_files/fvr.h, 110
- mcc_generated_files/interrupt_manager.c, 110
- mcc_generated_files/interrupt_manager.h, 111
- mcc_generated_files/mcc.c, 112
- mcc_generated_files/mcc.h, 113
- mcc_generated_files/memory.c, 114
- mcc_generated_files/memory.h, 116
- mcc_generated_files/pin_manager.c, 117
- mcc_generated_files/pin_manager.h, 118
- mcc_generated_files/pwm8.c, 128
- mcc_generated_files/pwm8.h, 128
- mcc_generated_files/spi1.c, 129
- mcc_generated_files/spi1.h, 131
- mcc_generated_files/tmr2.c, 131
- mcc_generated_files/tmr2.h, 133
- mcc_generated_files/tmr3.c, 137
- mcc_generated_files/tmr3.h, 139
- mcc_generated_files/tmr5.c, 140
- mcc_generated_files/tmr5.h, 142
- mcc_generated_files/tmr6.c, 143
- mcc_generated_files/tmr6.h, 145
- mcc_generated_files/uart1.c, 149
- mcc_generated_files/uart1.h, 152
- mcc_generated_files/uart2.c, 153
- mcc_generated_files/uart2.h, 157
- mcode
 - terminal_type, 32
- mesgid
 - terminal_type, 32
- message
 - V_help, 44
- mode_pwm
 - V_data, 40
- mparm
 - terminal_type, 32
- msg_error
 - V_data, 40
- msg_ret
 - V_data, 40
- n_offset
 - R_data, 24
- n_scalar
 - R_data, 25
- oerr
 - uart1_status_t, 33
 - uart2_status_t, 35
- operation
 - spi1_configuration_t, 29
- p_bat
 - C_data, 11
- p_inverter
 - C_data, 11
- p_load
 - C_data, 11
- p_mppt
 - C_data, 11
- p_pv
 - C_data, 11
- pclock
 - hist_type, 21
- perr
 - uart1_status_t, 34
 - uart2_status_t, 35
- peukert
 - hist_type, 21
- peukert_adj
 - hist_type, 21
- polyWidth
 - CRC_OBJ, 14
- pv_ah
 - C_data, 11
- pv_eff
 - hist_type, 21
- pvwk
 - C_data, 11
- r_checksum
 - V_data, 41
- R_data, 23
 - c_scale_cal, 23
 - c_zero_cal, 23
 - checkmark, 24
 - crc, 24
 - done, 24
 - H, 24
 - hist_save, 24
 - hist_update, 24
 - max5322_cmd, 24
 - n_offset, 24
 - n_scalar, 25
 - raw_adc, 25
 - raw_dac, 25
 - scan_index, 25
 - scan_select, 25

- rate
 - hist_type, 21
- raw_adc
 - R_data, 25
- raw_dac
 - R_data, 25
- rbuf
 - V_data, 41
- rbutton_type, 25
 - count, 26
 - sw, 26
- reserved
 - uart1_status_t, 34
 - uart2_status_t, 35
- response
 - V_data, 41
- rest_factor
 - hist_type, 21
- rest_rate
 - hist_type, 21
- ringBufS_t, 26
 - buf, 26
 - count, 26
 - head, 27
 - tail, 27
- runtime
 - C_data, 11
- rx_count
 - V_data, 41
- S_data, 27
 - button, 28
 - sw_bitmap, 28
- samplei
 - hist_type, 21
- sampleo
 - hist_type, 22
- scan_index
 - R_data, 25
- scan_select
 - R_data, 25
- screen
 - V_data, 41
- seedDirection
 - CRC_OBJ, 14
- select_p
 - H_data, 18
- sensor_set
 - V_data, 41
- sequence
 - H_data, 18
- sequences
 - V_data, 41
- set_sequ
 - V_data, 41
- soc
 - C_data, 12
- spi1_configuration_t, 28
 - baud, 29
 - con0, 29
 - con1, 29
 - con2, 29
 - operation, 29
- SPI_AUX
 - spi_link_type, 30
- spi_count
 - V_data, 42
- SPI_LCD
 - spi_link_type, 30
- spi_link_type, 29
 - config, 30
 - delay, 30
 - int_count, 30
 - LCD_DATA, 30
 - LCD_TIMER, 30
 - SPI_AUX, 30
 - SPI_LCD, 30
 - txbuf, 31
- stack
 - V_data, 42
- start_power
 - C_data, 12
- status
 - uart1_status_t, 34
 - uart2_status_t, 35
- sw
 - rbutton_type, 26
- sw_bitmap
 - S_data, 28
- switch_count
 - V_data, 42
- sys_info
 - V_data, 42
- system_stable
 - V_data, 42
- t_checksum
 - V_data, 42
- t_comp
 - C_data, 12
- tail
 - ringBufS_t, 27
- terminal_type, 31
 - ceid, 31
 - cmdlen, 31
 - help_temp, 32
 - host_display_ack, 32
 - info, 32
 - log_num, 32
 - log_seq, 32
 - mcode, 32
 - mesgid, 32
 - mparm, 32
 - TID, 33
- testing
 - V_data, 42
- ticker
 - V_data, 42

- ticks
 - V_data, 43
- TID
 - terminal_type, 33
- time_info
 - V_data, 43
- timerint_count
 - V_data, 43
- tot_eff
 - hist_type, 22
- ttg_t
 - hist_type, 22
- tx_count
 - V_data, 43
- txbuf
 - spi_link_type, 31
- uart1_status_t, 33
 - ferr, 33
 - oerr, 33
 - perr, 34
 - reserved, 34
 - status, 34
- uart2_status_t, 34
 - ferr, 35
 - oerr, 35
 - perr, 35
 - reserved, 35
 - status, 35
- udod
 - hist_type, 22
- ui_state
 - V_data, 43
- ui_sw
 - V_data, 43
- update
 - C_data, 12
- updates
 - C_data, 12
 - hist_type, 22
- v_bat
 - C_data, 12
- v_bbat
 - C_data, 12
- v_cbus
 - C_data, 12
- v_cc
 - C_data, 13
- v_cmode
 - C_data, 13
- V_data, 35
 - abort, 37
 - ac_off, 37
 - ac_on, 37
 - ac_time, 37
 - adc_count, 37
 - alarm, 37
 - all_errors, 37
 - blight, 38
 - blight_off, 38
 - boost_ticks, 38
 - buf, 38
 - calib, 38
 - cc_active, 38
 - cc_state, 38
 - checksum_error, 38
 - debug, 39
 - enter, 39
 - error, 39
 - fixup, 39
 - flipper, 39
 - float_ticks, 39
 - get_time_text, 39
 - help, 39
 - help_id, 40
 - in_boost, 40
 - in_float, 40
 - info, 40
 - lowint_count, 40
 - mode_pwm, 40
 - msg_error, 40
 - msg_ret, 40
 - r_checksum, 41
 - rbuf, 41
 - response, 41
 - rx_count, 41
 - screen, 41
 - sensor_set, 41
 - sequences, 41
 - set_sequ, 41
 - spi_count, 42
 - stack, 42
 - switch_count, 42
 - sys_info, 42
 - system_stable, 42
 - t_checksum, 42
 - testing, 42
 - ticker, 42
 - ticks, 43
 - time_info, 43
 - timerint_count, 43
 - tx_count, 43
 - ui_state, 43
 - ui_sw, 43
 - wdt_ticks, 43
- V_help, 44
 - display, 44
 - message, 44
- v_inverter
 - C_data, 13
- v_pv
 - C_data, 13
- v_sensor
 - C_data, 13
- v_temp
 - C_data, 13

version
 hist_type, [22](#)
vterm
 D_data, [15](#)

wait_calib
 H_data, [18](#)
wait_enter
 H_data, [18](#)
wait_select
 H_data, [18](#)
wdt_ticks
 V_data, [43](#)